

PART 1 — The C Language

1. Foundations

- History, K&R C, and why C exists
- Standards: C89/C90, C95, C99, C11, C17, **C23** — and knowing which one your compiler uses
- Translation phases: preprocess ? compile ? assemble ? link
- Translation units, object files, executables
- `main()` signatures, `argc/argv`, exit codes, `envp`

2. Types & Values

- Integer types, `char` (signedness is implementation-defined), `short/int/long/long long`
- Floating point: `float`, `double`, `long double`, IEEE-754 basics, NaN/Inf, epsilon comparison
- `_Bool/bool`, `void`
- Fixed-width types: `stdint.h` (`int32_t`, `uint8_t`, `intptr_t`, `uintmax_t`)
- `size_t`, `ptrdiff_t`, `ssize_t`, `char16_t/char32_t`, `wchar_t`
- `limits.h`, `float.h`, `inttypes.h` format macros (`PRId64`)
- **Integer promotion, usual arithmetic conversions, implicit conversion traps**
- Signed overflow (UB) vs unsigned wraparound (defined)
- Type qualifiers: `const`, `volatile`, `restrict`, `_Atomic`
- Casting, type punning, strict aliasing rules

3. Variables, Scope, Lifetime

- Declaration vs definition
- Scope: block, file, function, function-prototype
- Linkage: external, internal (`static`), none
- Storage duration: automatic, static, thread (`_Thread_local`), allocated
- `extern`, `static`, `register`, `auto`
- Initialization rules, zero-initialization of statics, indeterminate values

4. Operators & Expressions

- Arithmetic, relational, logical, bitwise, assignment, compound assignment
- Ternary, comma operator
- `sizeof`, `_Alignof`, `_Alignas`
- Precedence and associativity
- Short-circuit evaluation
- **Sequence points / evaluation order, and the UB it causes** (`i = i++ + 1`)
- Bit manipulation: masks, shifts, set/clear/toggle/test, bit tricks, bitfields vs manual bits

5. Control Flow

- `if/else`, `switch` (fallthrough, jump tables, Duff's device)
- `for`, `while`, `do-while`, nested loops
- `break`, `continue`, `goto` (and its *legitimate* use: cleanup chains)

- Labels, unreachable code

6. Functions

- Prototypes, definitions, forward declarations
- Pass-by-value semantics (C has no pass-by-reference)
- Recursion, tail calls, stack depth limits
- Variadic functions: `stdarg.h`, `va_list`, `va_start/arg/end/copy`
- `inline`, `static inline`, `_Noreturn`
- **Function pointers**, callbacks, dispatch tables
- Calling conventions (`cdecl`, `stdcall`, `fastcall`), ABI basics

7. Arrays

- 1D, 2D, multidimensional, row-major layout
- Array-to-pointer decay (and where it *doesn't* happen)
- Array parameters in functions (they're pointers)
- Variable Length Arrays (VLA) — and why many shops ban them
- Designated initializers, compound literals
- Flexible array members (`struct { int n; char data[]; }`)

8. Strings

- Null-terminated convention, `'\0'`
- `string.h`: `strlen`, `strcpy`, `strncpy`, `strcat`, `strcmp`, `strchr`, `strstr`, `strtok`, `memcpy`, `memmove`, `memset`, `memcmp`
- **Why `strcpy`/`strcat`/`sprintf`/`gets` are dangerous**; `snprintf`, `strncpy`/`strlcat`, `strnlen`
- `memcpy` vs `memmove` (overlap)
- Conversions: `atoi` (bad) vs `strtol`/`strtod` (good, with `errno` + `endptr`)
- Wide strings, multibyte, **UTF-8 handling in C**
- Building a string/buffer abstraction

9. Pointers (the core skill)

- Address-of, dereference, pointer declaration syntax
- Pointer arithmetic and its rules
- `void*` and generic programming
- `NULL`, null checks, `nullptr` (C23)
- Pointer to pointer, arrays of pointers, pointer to array
- `const` correctness: `const char *` vs `char * const` vs `const char * const`
- Function pointers, pointer to function returning pointer, `typedef` for readability
- Dangling pointers, wild pointers, uninitialized pointers
- Alignment, `uintptr_t`, pointer-integer conversion
- Reading complex declarations (spiral rule / `cdecl`)

10. Memory Management

- Memory layout: text, rodata, data, BSS, heap, stack
- Stack frames, stack overflow, recursion limits

- `malloc`, `calloc`, `realloc`, `free`, `aligned_alloc`
- Checking allocation failure; `realloc` failure pattern
- **Leaks, double free, use-after-free, buffer overflow, heap corruption**
- Ownership conventions and who-frees-what discipline
- Custom allocators: arena/bump, pool, freelist, slab, stack allocator
- `alloca` (and why to avoid it)
- Zeroing sensitive memory (`memset_s`, `explicit_bzero`)

11. Structs, Unions, Enums

- Struct declaration, initialization, nesting, assignment/copy semantics
- Passing structs by value vs pointer
- **Padding, alignment, `offsetof`, struct packing, cache-friendly layout**
- Bitfields and their portability caveats
- Unions, type punning via union, tagged unions
- Enums, enum scoping, C23 fixed underlying type
- `typedef` — good and bad practice
- **Opaque structs** (`typedef struct Foo Foo;`) for encapsulation
- Self-referential structs, intrusive linked structures

12. The Preprocessor

- `#include` (" " vs `<>`), include guards, `#pragma once`
- Object-like and function-like macros
- Macro pitfalls: multiple evaluation, missing parens, semicolons; `do { } while(0)`
- Stringification `#`, token pasting `##`, variadic macros `__VA_ARGS__`
- Conditional compilation: `#if/#ifdef/#elif/#else/#endif`, `#error`, `#warning`
- Predefined macros: `__FILE__`, `__LINE__`, `__func__`, `__DATE__`, `__STDC_VERSION__`
- **X-Macros**, code generation patterns
- `_Generic` (type-generic dispatch, C11)
- `#embed` (C23)
- When *not* to use macros

13. Input / Output

- `stdio.h`: `printf/scanf` family, full format specifier knowledge, width/precision/flags
- **Why `scanf` is a trap**; `fgets` + `strtoul` pattern
- `FILE*`, `fopen` modes, text vs binary mode
- `fread`, `fwrite`, `fseek`, `ftell`, `rewind`, `fflush`
- Buffering: full/line/none, `setvbuf`
- `EOF`, `feof` misuse, `ferror`, `clearerr`
- `stdin/stdout/stderr`, redirection, pipes
- Low-level POSIX I/O: `open/read/write/close/lseek`, file descriptors, `mmap`

14. Standard Library Tour

`stdio` · `stdlib` · `string` · `math` · `ctype` · `time` · `assert` · `errno` · `signal` · `setjmp` · `locale` · `stdbool` · `stddef` · `stdint` · `inttypes` · `limits` · `float` · `stdarg` · `stdalign` · `stdnoreturn` · `complex` · `tgmath` ·

`threads.h` · `stdatomic.h` · `uchar.h`

- Sorting/searching: `qsort`, `bsearch` (and writing correct comparators)
- Randomness: `rand` limitations, better PRNGs
- Time: `time_t`, `struct tm`, `clock`, monotonic clocks

15. Error Handling Strategies

- Return codes, sentinel values, out-parameters
- `errno`, `perror`, `strerror`, thread-safety of `errno`
- `assert` vs runtime checks; `NDEBUG`
- The `goto cleanup` / single-exit pattern
- `setjmp/longjmp` (and its dangers)
- Designing an error enum + error-string API

16. Undefined, Unspecified, Implementation-Defined Behavior

- The distinction between the three
- Common UB list: signed overflow, OOB access, null deref, uninitialized reads, strict aliasing violations, data races, invalid free, shifting by = width, modifying string literals, misaligned access
- How optimizers exploit UB (why "it worked on my machine" fails)

17. Concurrency in Standard C

- C11 `threads.h`, `mtx_t`, `cnd_t`
- `stdatomic.h`, atomic types, `memory_order_*`
- The C memory model, data races, happens-before
- `volatile` ? thread safety (very common misunderstanding)

18. C23 Highlights

`nullptr`, `typeof`, `constexpr`, `auto`, `bool` as keyword, attributes (`[[nodiscard]]`, `[[deprecated]]`, `[[fallthrough]]`), binary literals, digit separators, `#embed`, `stdbit.h`, enhanced enums, `memset_explicit`, empty initializer `{}`

PART 2 — What Industry Actually Expects (beyond the language)

This is the part that separates "knows C" from "professional C developer."

19. Toolchain & Compilation

- GCC, Clang, MSVC, ICC — differences and extensions
- Warning flags you should always run: `-Wall` `-Wextra` `-Wpedantic` `-Werror` `-Wshadow` `-Wconversion` `-Wstrict-prototypes`
- Optimization levels `-O0..-O3` `-Os` `-Ofast`, `-g`, `-march/-mtune`
- Preprocessor/assembly output (`-E`, `-S`), reading generated assembly (Godbolt)

- Link-Time Optimization, `-ffunction-sections -Wl,--gc-sections`
- Cross-compilation, sysroots, toolchain files
- `objdump`, `nm`, `readelf`, `strip`, `size`, `ldd`, `addr2line`

20. Build Systems & Libraries

- Make (real Makefiles, pattern rules, dependency generation)
- **CMake** (the industry default), Meson, Ninja, Bazel; autotools legacy
- Static libraries `.a` vs shared `.so/.dll/.dylib`
- Position-independent code, symbol visibility, `-fvisibility=hidden`
- Symbol resolution, link order, undefined/duplicate symbols
- **ABI vs API**, soname, versioning, backward compatibility
- `pkg-config`, dependency management (Conan, vcpkg)
- Header design: what goes in `.h` vs `.c`, minimizing includes, forward declarations

21. Debugging & Analysis

- **GDB/LLDB**: breakpoints, watchpoints, backtraces, inspecting memory, TUI, scripting
- Core dumps and post-mortem debugging
- **Valgrind** (memcheck, callgrind, helgrind, massif)
- **Sanitizers**: ASan, UBSan, TSan, MSan, LeakSanitizer — non-negotiable in modern C
- `strace`, `ltrace`, `perf`, `ftrace`, `bpftrace`
- `rr` (record & replay debugging)
- Static analysis: `clang-tidy`, `cppcheck`, `scan-build`, Coverity, PVS-Studio
- Formal-ish tools: Frama-C, CBMC

22. Testing & Quality

- Unit test frameworks: Unity, CMocka, Check, Criterion, greatest, `unit`
- Test doubles/mocks in C (link-time seams, function pointers)
- Code coverage: `gcov/lcov`, branch coverage
- **Fuzzing**: `libFuzzer`, `AFL++`, `honggfuzz` — huge in real C work
- Property-based and differential testing
- Regression suites, golden tests
- CI/CD: GitHub Actions, GitLab CI, building on multiple compilers/platforms

23. Secure Coding

- Buffer overflows (stack & heap), off-by-one, `strcpy` family abuse
- Format string vulnerabilities
- Integer overflow/underflow, signed/unsigned confusion, truncation
- Use-after-free, double free, type confusion
- TOCTOU races, unsafe temp files, `system()`/command injection
- Input validation, canonicalization
- **CERT C**, **MISRA C:2012/2023**, CWE Top 25
- Hardening: `-fstack-protector-strong`, `_FORTIFY_SOURCE=3`, PIE, RELRO, `-D_GLIBCXX_ASSERTIONS`, CFI
- Constant-time code for crypto, avoiding compiler-removed `memset`

- Threat modeling, secure API design (bounds-carrying interfaces)

24. Systems / POSIX Programming

- Syscalls vs library calls
- Processes: `fork`, `exec*`, `wait`, zombies, orphans, process groups
- **Signals**: handlers, async-signal-safety, `sigaction`, masks, `signalfd`
- IPC: pipes, FIFOs, UNIX sockets, shared memory, semaphores, message queues, `mmap`
- File system: paths, permissions, `stat`, directories, symlinks, `inotify`
- **Networking**: TCP/UDP sockets, `bind/listen/accept/connect`, byte order, non-blocking I/O
- Multiplexing: `select`, `poll`, `epoll`, `kqueue`, `io_uring`
- **threads**: create/join/detach, mutexes, condvars, rwlocks, barriers, TLS, thread pools
- Lock-free programming, ABA problem, memory barriers
- Daemons, resource limits, `getopt`, environment variables
- Dynamic loading: `dlopen/dlsym`, plugin architectures
- Windows counterparts if you target it: Win32 API, HANDLE model, IOCP

25. Performance Engineering

- Big-O in practice vs constants and memory
- **CPU caches, cache lines, locality, prefetching, false sharing**
- Branch prediction, branchless techniques
- Data-oriented design, SoA vs AoS
- Profiling: `perf`, `gprof`, `callgrind`, VTune, flamegraphs
- Benchmarking methodology (warmup, variance, avoiding compiler elision)
- SIMD: SSE/AVX/NEON intrinsics, auto-vectorization
- Inline assembly, compiler barriers
- `restrict`, `likely/unlikely`, alignment, memory pools
- Understanding what the optimizer does and doesn't do

26. Portability

- Endianness, word size, `LP64` vs `LLP64`
- Alignment and unaligned access on strict architectures
- Feature test macros (`_POSIX_C_SOURCE`, `_GNU_SOURCE`, `_DEFAULT_SOURCE`)
- Platform detection macros, compiler-specific extensions
- Freestanding vs hosted implementations
- Toolchain differences and abstraction layers

27. Embedded / Bare-Metal C (huge C job market)

- MCU architecture, memory maps, registers
- **Memory-mapped I/O and correct `volatile` usage**
- Interrupts, ISRs, vector tables, latency, reentrancy
- **Linker scripts**, startup code, `.data/.bss` init, stack setup
- No dynamic allocation / static allocation strategies
- Fixed-point arithmetic, no-FPU environments
- Peripherals & protocols: GPIO, UART, SPI, I2C, CAN, ADC, PWM, DMA, timers, watchdog

- RTOS: FreeRTOS, Zephyr, ThreadX — tasks, queues, priorities, priority inversion
- Low-power modes, boot loaders, firmware update / OTA
- Debug hardware: JTAG/SWD, OpenOCD, logic analyzers, oscilloscopes
- MISRA C compliance, safety standards (ISO 26262, DO-178C, IEC 61508)
- CMSIS, HAL vs register-level programming

28. Software Design in C

- Modularity: one module = header + implementation, clear boundaries
- **Opaque handle APIs** (`typedef struct Db Db; Db *db_open(...)`)
- Polymorphism via function-pointer vtables
- Callbacks with context/user-data pointers
- Object lifecycle: `init/deinit`, `create/destroy`, ownership rules
- State machines (table-driven, switch-based)
- Plugin systems, dependency injection
- Generic containers: `void*`, macro templates, intrusive lists
- Layering, avoiding global state, testability
- API design: naming, versioning, deprecation, error reporting, thread-safety documentation

29. Data Structures & Algorithms in C

Dynamic array/vector · singly & doubly linked lists · intrusive lists · stacks/queues · **ring buffers** · hash tables (open addressing, chaining) · binary trees, BST, AVL/red-black · tries · heaps/priority queues · graphs · bitsets · string builders · memory-efficient variants · sorting/searching implementations

30. Interoperability

- `extern "C"` and C++ interop
- FFI: Python (`ctypes`, `cffi`, C extensions), Rust (`bindgen`, `cbindgen`), Go cgo, Java JNI, Lua, Node N-API
- WebAssembly via Emscripten/WASI
- Designing C APIs meant to be bound from other languages

31. Professional Practice

- Git: branching, rebasing, bisect (`git bisect` is gold for C bugs), code review
- Coding standards: Linux kernel style, GNU, Google, MISRA; `clang-format`, `.editorconfig`
- Documentation: Doxygen, README, man pages, architecture docs
- Semantic versioning, changelogs, release engineering
- Licensing: GPL/LGPL/MIT/BSD/Apache and the implications for C libraries
- Reading and contributing to large C codebases
- Debugging production issues, logging, telemetry, crash reporting

32. Study These Real Codebases

SQLite (testing discipline) · **redis** (clean C, data structures) · **curl** (portability) · **musl libc** (minimal, readable stdlib) · **Lua** (small, elegant, embeddable) · **nginx** (event-driven architecture) · **Linux kernel** (drivers/subsystems) · **FFmpeg** (performance) · **zstd/lz4** (optimization) · **stb single-header libs** (API design)

Suggested priority order

Must have before your first job: 1–16, 19, 20 (Make/CMake), 21 (GDB + sanitizers + Valgrind), 22 basics, 23 basics **Makes you strong:** 17, 24, 25, 26, 28, 29, 31 **Specialize by domain:** 27 (embedded/firmware), 24+25 (backend/systems), 30 (library/tooling), 23+MISRA (safety-critical/automotive/medical)

The single biggest differentiator in interviews and real work is not syntax — it's **memory ownership discipline, undefined behavior awareness, and comfort with GDB + sanitizers.**

Module 1 — Foundations

- History, K&R C, and why C exists
- Standards: C89/C90, C95, C99, C11, C17, **C23** — and knowing which one your compiler uses
- Translation phases: preprocess ? compile ? assemble ? link
- Translation units, object files, executables
- `main()` signatures, `argc/argv`, exit codes, `envp`

Everything below I actually compiled and ran on GCC 13.3 (x86-64 Linux), so the outputs are real, not from memory. Open a terminal and follow along — that matters more than reading.

1.1 Why C exists (the history that actually matters)

The lineage:

```
CPL (1963, Cambridge) ? BCPL (1967, Martin Richards)
                        ? B (1969, Ken Thompson)
                        ? C (1972, Dennis Ritchie)
```

The problem C was invented to solve.

In the late 1960s at Bell Labs, Ken Thompson wrote the first Unix in **PDP-7 assembly**. That worked, but it meant Unix was welded to one machine. Port it to new hardware? Rewrite the entire OS by hand.

Thompson made **B** to escape assembly. B had one fatal flaw: it was **typeless** — every value was one machine word. That was fine on the word-addressed PDP-7. But the new **PDP-11 was byte-addressed**, and B had no way to say "this is a single byte" versus "this is a word." You couldn't write efficient byte-manipulating systems code.

So Ritchie added **types** (`char`, `int`, later `float`, `struct`), and **pointer arithmetic that respects the size of the pointed-to type**. That's C.

The payoff moment — 1973: Unix was rewritten in C. Roughly 90% of the kernel became portable source code. This is the single most important event in systems programming history. Porting an OS went from "rewrite everything" to "write a compiler backend + a bit of assembly glue."

Say this in an interview: *C was created to make an operating system portable without giving up the ability to manipulate hardware directly. Every design decision follows from that.*

Design philosophy (from the C rationale document):

Principle	Consequence you'll feel daily
Trust the programmer	No bounds checking. Buffer overflows are <i>your</i> problem.
Don't prevent what needs doing	You can cast anything to anything.
Keep the language small	No strings, no lists, no GC — those are library or your code.
Make it fast	Constructs map ~1:1 to machine instructions.
Portability matters, but so does the machine	Hence "implementation-defined" behavior everywhere.

C is often called "**portable assembly**." That's not an insult — it's the specification.

1.2 K&R C

In 1978 Kernighan & Ritchie published *The C Programming Language*. For ~11 years, **that book was the standard** — there was no committee, no ISO document. "K&R C" means the dialect described in that first edition.

You must be able to recognize old-style (K&R) function syntax, because it still lurks in ancient codebases:

```
/* K&R style – parameter types declared separately, no prototype */
int add(a, b)
int a;
int b;
{
    return a + b;
}

/* Modern (ANSI) style */
int add(int a, int b)
{
    return a + b;
}
```

Why K&R style is dangerous: without a prototype, the compiler cannot check your call. `add(1.5, "x")` compiles silently and corrupts. ANSI C added **function prototypes** (borrowed from C++) specifically to fix this. K&R function definitions were finally **removed in C23**.

Also note: in K&R C, `int f()` meant "*unknown, unchecked parameters*" — **not** "no parameters." That's why you write `int f(void)`.

```
void f();      /* old meaning: takes unspecified args – no checking! */
void f(void); /* correct: takes NOTHING */
```

Rule: always write `(void)`, never empty parentheses. (C23 finally made `()` mean `(void)`, but write `(void)` anyway for compatibility.)

1.3 The Standards

Standard	Year	Also called	Key additions
C89 / C90	1989/90	ANSI C, ISO C90	Prototypes, <code>void*</code> , <code>const/volatile</code> , standard library defined. <i>C89 and C90 are the same language</i> — ANSI ratified in '89, ISO in '90.
C95	1995	NA1 (amendment)	Wide chars (<code>wchar.h</code> , <code>wctype.h</code>), digraphs
C99	1999	—	<code>//</code> comments, declarations anywhere, <code>long long</code> , <code>_Bool</code> , <code>inline</code> , <code>stdint.h</code> , VLAs, designated initializers, compound literals, <code>snprintf</code> , flexible array members, <code>restrict</code>
C11	2011	—	Threads (<code>threads.h</code>), atomics (<code>stdatomic.h</code>), <code>_Generic</code> , <code>_Static_assert</code> , anonymous structs/unions, <code>aligned_alloc</code> , Annex K <code>_s</code> functions, VLAs made optional
C17 / C18	2018	—	Bug-fix release only. Zero new features. Defect reports against C11.
C23	2024	—	<code>nullptr</code> , <code>true/false/bool</code> as keywords, <code>typeof</code> , <code>constexpr</code> , <code>auto</code> , attributes <code>[[nodiscard]]</code> , binary literals <code>0b1010</code> , digit separators <code>1'000'000</code> , <code>#embed</code> , <code>stdbit.h</code> , removed K&R definitions

How to check which standard *your* compiler is using

The predefined macro `__STDC_VERSION__` tells you. Here's the program:

```
#include <stdio.h>

int main(void)
{
#ifdef __STDC_VERSION__
    printf("__STDC_VERSION__ = %ldL\n", __STDC_VERSION__);
#else
    printf("__STDC_VERSION__ not defined -> C89/C90\n");
#endif

#ifdef __STRICT_ANSI__
    printf("__STRICT_ANSI__ = defined (strict -std=cXX mode)\n");
#else
    printf("__STRICT_ANSI__ = NOT defined (GNU extensions enabled)\n");
#endif
    return 0;
}
```

Real output from my runs:

```

=== -std=c89 ===    __STDC_VERSION__ not defined -> C89/C90
=== -std=c99 ===    __STDC_VERSION__ = 199901L
=== -std=c11 ===    __STDC_VERSION__ = 201112L
=== -std=c17 ===    __STDC_VERSION__ = 201710L
=== -std=c2x ===    __STDC_VERSION__ = 202000L
=== default      === __STDC_VERSION__ = 201710L
                   __STRICT_ANSI__ = NOT defined (GNU extensions enabled)

```

Three things to extract from that:

1. **C89 doesn't define `__STDC_VERSION__` at all** — that's how you detect it.
2. **GCC 13's default is `gnu17`**, not `c17`. Plain `gcc file.c` gives you C17 **plus GNU extensions**. Your code may compile for you and fail on someone else's compiler. Always pass `-std=` explicitly.
3. On GCC 13, C23 is spelled `-std=c2x`; `-std=c23` only landed in GCC 14+. Check your version.

Making the standard *actually* enforced

```

gcc -std=c99 file.c           # allows GNU extras silently in some cases
gcc -std=c99 -pedantic file.c # warns on non-standard usage
gcc -std=c99 -pedantic-errors file.c # ? errors. Use this when portability
matters.

```

Live proof of why the standard you pick matters:

```

#include <stdio.h>
int main(void)
{
    printf("start\n");           // C99+
    for (int i = 0; i < 2; i++)   // C99+
        printf("i=%d\n", i);
    int arr[] = { [2] = 99 };    // C99+
    printf("%d\n", arr[2]);
    return 0;
}

```

```

### -std=c89 -pedantic-errors ###
error: C++ style comments are not allowed in ISO C90
error: 'for' loop initial declarations are only allowed in C99 or C11 mode

### -std=c99 -pedantic-errors ###
start

```

```
i=0
i=1
99
```

Which standard should you use professionally?

- **C99** — the safe floor. Universally supported, including old embedded toolchains.
- **C11 / C17** — the current default for most industry work. Pick this unless told otherwise.
- **C23** — early adoption; verify your toolchain.
- **C89** — only for legacy code, MSVC-ancient targets, or maximum-portability libraries.

1.4 Translation phases

The C standard formally defines **8 translation phases** (trigraph replacement, line splicing, tokenization, macro expansion, escape conversion, string concatenation, translation, linking). You should *know they exist*, but day-to-day you work with **4 practical stages**:

🔗 Step-by-Step Meaning

1. **Trigraph replacement**

Rare sequences like `??=` are replaced with their intended characters (here, `#`). This was originally for keyboards that lacked certain symbols.

2. **Line splicing**

A backslash at the end of a line (`\`) tells the compiler to treat the next line as a continuation. This happens before tokenization.

3. **Tokenization**

The source text is broken into meaningful units: identifiers, keywords, operators, literals.

4. **Macro expansion**

The preprocessor replaces macros (`#define`, `#include`) with their definitions or file contents.

5. **Escape conversion**

Escape sequences in strings (`\n`, `\t`, `\x41`) are converted into their actual character values.

6. **String concatenation**

Adjacent string literals like `"Hello, " "world!"` are joined into one: `"Hello, world!"`.

7. **Translation**

The compiler proper translates the preprocessed source into object code for the target machine.

8. **Linking**

The linker combines object files and libraries into a final executable.

```
hello.c  --[1 preprocess]--?  hello.i  (still C, expanded)
          --[2 compile  ]--?  hello.s  (assembly)
```

```
--[3 assemble ]--? hello.o  (machine code + symbol table)
--[4 link      ]--? hello    (executable)
```

GCC flags to stop at each stage: `-E`, `-S`, `-c`, (nothing) = link.

Our test file:

```
#include <stdio.h>

#define SQUARE(x) ((x) * (x))
#define GREETING "hello"

int helper(int n);

int main(void)
{
    int r = helper(SQUARE(4));
    printf("%s %d\n", GREETING, r);
    return 0;
}
```

Stage 1 — Preprocess (`gcc -E demo.c -o demo.i`)

Pure **text substitution**. It knows nothing about C semantics.

```
lines in demo.c : 13
lines in demo.i : 826
```

13 lines became **826** — `#include <stdio.h>` literally pasted the entire header in. Tail of `demo.i`:

```
# 6 "demo.c"
int helper(int n);

int main(void)
{
    int r = helper(((4) * (4)));      ? SQUARE(4) expanded
    printf("%s %d\n", "hello", r);   ? GREETING expanded
    return 0;
}
```

Note: `#include`, `#define`, and the comments are **gone**. The `# 6 "demo.c"` lines are *line markers* so error messages can point back to your original file.

Debugging superpower: when a macro misbehaves, run `gcc -E` and read what it actually expanded to. Solves 90% of macro bugs instantly.

Stage 2 — Compile (`gcc -S demo.c -o demo.s`)

Now real C parsing, type checking, and optimization happen. Output is assembly:

```
.section .rodata
.LC0: .string "hello"
.LC1: .string "%s %d\n"

main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    mov     edi, 16          ? !! 4*4 already folded to 16 at -00
    call    helper@PLT
    ...
```

Two observations worth internalizing:

- Your **string literals live in .rodata** (read-only data). That's why `char *s = "hi"; s[0] = 'H';` crashes.
- `mov edi, 16` — the compiler computed `4*4` at compile time even with optimization off. This is *constant folding*. Your mental model of "C is a thin layer over assembly" is true, but the compiler is smarter than you assume.

Stage 3 — Assemble (`gcc -c demo.c -o demo.o`)

Assembly ? binary machine code, packaged as an **object file**.

```
demo.o: ELF 64-bit LSB relocatable, x86-64, not stripped
```

The key word is **relocatable** — addresses aren't final yet. Inspect the symbol table with `nm`:

```
0000000000000000 U helper
0000000000000000 T main
0000000000000000 U printf
```

Letter	Meaning
T	Text section — defined here, code, globally visible
t	same but static (internal linkage)
U	Undefined — "I use this; linker, go find it"
D / B	initialized data / BSS (zero-init data)

This object file *knows it needs* `helper` and `printf` but doesn't have them.

Stage 4 — Link (`gcc demo.o -o app`)

The linker (`ld`) resolves every `U` against other object files and libraries, assigns final addresses, and produces the executable.

```
$ gcc demo.o -o app
/usr/bin/ld: demo.o: in function `main':
demo.c:(.text+0x12): undefined reference to `helper'
collect2: error: ld returned 1 exit status
```

This is the single most common beginner error, and now you know exactly what it means: a symbol was *declared* (so the compiler was happy) but never *defined* anywhere the linker could see.

Provide it and link both:

```
echo 'int helper(int n) { return n + 1; }' > helper.c
gcc -c helper.c -o helper.o
gcc demo.o helper.o -o app && ./app
```

```
hello 17
```

After linking:

```
000000000000118c T helper      ? resolved, has an address
0000000000001149 T main       ? resolved
                U printf@GLIBC_2.2.5 ? STILL undefined!
```

`printf` is still `U` because **libc is dynamically linked** — it gets resolved by the *dynamic loader* at program startup, not at build time. (`ldd app` shows which shared libraries it will need.)

Diagnosing errors by stage — memorize this:

- **error: 'X' undeclared ? compiler.** Missing `#include` or typo.
- **warning: implicit declaration of function 'X' ? compiler.** Missing prototype.
- **undefined reference to 'X' ? linker.** Missing `.c` file or `-lfoo` library.
- **multiple definition of 'X' ? linker.** Defined a variable in a header, or forgot `static`.

1.5 Translation units, object files, executables

Translation unit (TU) = one `.c` file **after preprocessing** — meaning your source *plus every header it pulled in*. This is the compiler's unit of work. It compiles one TU at a time and **knows nothing about any other TU**.

That single fact explains most of C's structure:

```
main.c -+
      +- each preprocessed independently ? separate TU ? separate .o
math.c -|
io.c -+

      ? linker joins them
      executable
```

Consequence 1 — why headers exist. `main.c` can't see `math.c`. So `math.h` carries the *declarations* (promises), and each `.c` includes it. The compiler trusts the promise; the linker verifies it.

```
/* math.h — DECLARATION: "this exists somewhere" */
int add(int a, int b);

/* math.c — DEFINITION: "here is the actual code" */
int add(int a, int b) { return a + b; }
```

Declaration vs definition — the rule:

You may **declare** a thing many times. You may **define** it exactly once across the whole program. (This is the One Definition Rule.)

Consequence 2 — headers must contain declarations, not definitions. Put `int counter = 0;` in a header included by 3 files and you get *multiple definition of 'counter'* at link time. Correct pattern:

```
/* config.h */
extern int counter; /* declaration */

/* config.c */
int counter = 0; /* the one definition */
```

Consequence 3 — `static` at file scope means "private to this TU." It's C's only encapsulation mechanism. Use it aggressively for helper functions.

What's actually inside the files

	Object file (.o)	Executable
Addresses	relocatable / placeholder	final
Undefined symbols	allowed	only dynamic-library ones
Runnable	no	yes
Entry point	none	<code>_start</code> ? <code>main</code>

Standard ELF sections you should know:

- `.text` — machine code (read+execute)
- `.rodata` — string literals, `const` data (read-only)
- `.data` — initialized globals (`int x = 5;`)
- `.bss` — zero-initialized globals (`int x;`) — takes **no space in the file**, just a size
- `.symtab` / `.rel*` — symbols and relocation entries

Tools to explore: `nm`, `objdump -d`, `readelf -S`, `size`, `strings`, `ldd`, `file`.

1.6 `main()` — signatures, `argc/argv`, exit codes, `envp`

The two standard-sanctioned signatures

```
int main(void)                /* no arguments used */
int main(int argc, char *argv[]) /* command-line arguments */
```

That's it. Those two are guaranteed by the standard. Anything else is implementation-defined.

Never write `void main()`. It's not standard C. Some compilers accept it; it's still wrong, and it means the OS gets a garbage exit status.

`char *argv[]` and `char **argv` are identical in a parameter list (array parameters decay to pointers — we'll cover that properly in Module 7).

`argc` and `argv` in practice

```
#include <stdio.h>

int main(int argc, char *argv[], char *envp[])
{
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++)
        printf("  argv[%d] = \"%s\"\n", i, argv[i]);
    printf("  argv[%d] = %s  <-- always NULL\n",
           argc, argv[argc] ? "not null" : "NULL");
    return 0;
}
```

```
$ ./args --verbose -n 5 "two words"
argc = 5
argv[0] = "./args"
argv[1] = "--verbose"
argv[2] = "-n"
argv[3] = "5"
argv[4] = "two words"
argv[5] = NULL  <-- always NULL
```

Guarantees the standard gives you:

1. `argc >= 0`.
2. `argv[0]` is the **program name** (or an empty string if unavailable). It is *not* the first user argument — user args start at index **1**.
3. `argv[argc]` is **always NULL**. You can iterate either by count or until `NULL`.
4. The strings are **modifiable** and persist for the whole program.
5. Quoted shell arguments arrive as **one** element ("**two words**" above) — quoting is the shell's job, not C's.

Classic bug:

```
if (argc < 2) { /* handle missing arg */ }
printf("%s\n", argv[1]); /* ? reading argv[1] without this check = crash */
```

Exit codes

Three equivalent ways to end the program with status *n*:

```
return n;          /* from main only */
exit(n);           /* from anywhere; flushes stdio, runs atexit handlers */
_exit(n);          /* POSIX: immediate, NO flushing — use after fork() */
```

Use the portable macros from `<stdlib.h>`:

```
#include <stdlib.h>
return EXIT_SUCCESS; /* == 0 everywhere */
return EXIT_FAILURE; /* 1 on Linux; whatever the platform means by "failed" */
```

Convention (Unix, and it's enforced by every shell and CI system): 0 = success, non-zero = failure. Note this is the opposite of C's truthiness — a natural place to slip up.

Working example:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s <file>\n", argv[0]); /* errors ? stderr */
        return EXIT_FAILURE;
    }
    printf("ok: %s\n", argv[1]);
}
```

```
    return EXIT_SUCCESS;
}
```

```
$ ./exitc
usage: ./exitc <file>          ? went to stderr
    exit status = 1

$ ./exitc data.txt
ok: data.txt
    exit status = 0
```

The gotcha — exit status is only 8 bits:

```
int main(void) { return 256; }
```

```
return 256 -> shell sees 0
```

The status is truncated to `status & 0xFF`. **Keep your exit codes in the range 0–125.** (126/127 and 128+ are reserved by the shell for "can't execute" and "killed by signal N".)

Two more details:

- **In C99 and later, falling off the end of `main` implicitly returns 0.** This is a special exception *only* for `main`. Any other non-void function that falls off the end gives you undefined behavior. Write `return 0;` anyway — it's clearer.
- `atexit()` registers cleanup functions that run on `exit()` or normal return from `main` (LIFO order), but **not** on `_exit()` or `abort()`.

`envp` — the third parameter

```
int main(int argc, char *argv[], char *envp[])
```

```
envp[0] = PIP_ROOT_USER_ACTION=ignore
envp[1] = NODE_EXTRA_CA_CERTS=/etc/ssl/certs/ca-certificates.crt
envp[2] = RUST_BACKTRACE=1
```

It's a `NULL`-terminated array of "`KEY=VALUE`" strings.

But: `envp` is not standard C. It's a widely-supported extension (POSIX documents it, Windows has it too). The portable way is:

```
#include <stdlib.h>
char *path = getenv("PATH"); /* NULL if not set */
if (path != NULL) printf("%s\n", path);
```

Use `getenv()` in production code. Only reach for `envp` when you genuinely need to enumerate the entire environment (and then POSIX's `extern char **environ;` is the more common idiom).

?? Security note for later: never trust environment variables in privileged programs. `getenv` returns attacker-controllable data.

Practice — do these before Module 2

1. **Stage tracing.** Write any small program. Produce `.i`, `.s`, `.o`, and the executable. Run `wc -l` on `.i`, read the `.s`, run `nm` on the `.o`. Explain each symbol letter you see.
2. **Break the linker on purpose.** Declare `int mystery(void);`, call it, never define it. Read the error. Now define it in a second `.c` file and link. Then mark it `static` in that second file and observe the linker fail again — explain why.
3. **Standards archaeology.** Write a program using a VLA (`int n = 5; int a[n];`). Compile with `-std=c89 -pedantic-errors`, then `-std=c99`, then `-std=c11 -pedantic-errors`. Explain all three results.
4. **A real CLI tool.** Write `wordcount.c` that takes filenames as `argv`, prints a usage message to `stderr` and returns `EXIT_FAILURE` if given none, and returns `EXIT_SUCCESS` otherwise. Verify with `echo $?` after each run.
5. **Header discipline.** Split a program into `main.c`, `util.c`, `util.h` with an include guard. Deliberately put `int global = 0;` in `util.h`, include it from both `.c` files, and read the linker error. Then fix it with `extern`.
6. **Reading question (answer without compiling):** what does `void f();` mean in C89 vs C23, and why does `void f(void);` exist?

Build everything with:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -g file.c -o prog
```

2. Types & Values

- Integer types, `char` (signedness is implementation-defined), `short/int/long/long long`
- Floating point: `float`, `double`, `long double`, IEEE-754 basics, NaN/Inf, epsilon comparison
- `_Bool/bool`, `void`
- Fixed-width types: `stdint.h` (`int32_t`, `uint8_t`, `intptr_t`, `uintmax_t`)
- `size_t`, `ptrdiff_t`, `ssize_t`, `char16_t/char32_t`, `wchar_t`
- `limits.h`, `float.h`, `inttypes.h` format macros (`PRIu64`)

- **Integer promotion, usual arithmetic conversions, implicit conversion traps**
- Signed overflow (UB) vs unsigned wraparound (defined)
- Type qualifiers: `const`, `volatile`, `restrict`, `_Atomic`
- Casting, type punning, strict aliasing rules

Every output below is from real compilation on GCC 13.3, x86-64 Linux. This module contains more career-saving material than any other in the language section — integer promotion and strict aliasing alone account for a huge share of real-world C bugs.

2.1 Integer types

What the standard actually guarantees

C does **not** fix the size of `int`. It only guarantees *minimum ranges* and an ordering:

```
sizeof(char) == 1 = sizeof(short) = sizeof(int) = sizeof(long) =
sizeof(long long)
```

Type	Minimum guaranteed range	Minimum bits
<code>char</code>	8 bits	8
<code>short</code>	-32,767 ... 32,767	16
<code>int</code>	-32,767 ... 32,767	16
<code>long</code>	-2,147,483,647 ... 2,147,483,647	32
<code>long long</code>	-(2 ⁶³ -1) ... 2 ⁶³ -1	64

Note the minimums are `-32767`, not `-32768` — the standard allowed one's-complement and sign-magnitude machines until C23 mandated two's complement.

What you actually get on x86-64 Linux:

```
TYPE          SIZE  RANGE
char           1    -128 .. 127
short          2    -32768 .. 32767
int            4    -2147483648 .. 2147483647
unsigned int   4     0 .. 4294967295
long           8    -9223372036854775808 .. 9223372036854775807
long long      8    -9223372036854775808 .. 9223372036854775807
void*          8
CHAR_BIT = 8
```

The data model trap

Model	<code>int</code>	<code>long</code>	<code>void*</code>	Where
-------	------------------	-------------------	--------------------	-------

Model	int	long	void*	Where
ILP32	32	32	32	32-bit Linux/Windows
LP64	32	64	64	Linux, macOS, BSD (64-bit)
LLP64	32	32	64	Windows 64-bit

This is the #1 portability bug in real code: `long` is 64-bit on 64-bit Linux but **32-bit on 64-bit Windows**. Code that stores a pointer in a `long` works on Linux and silently truncates on Windows.

Rule: never assume `sizeof(long) == sizeof(void*)`. Use `intptr_t`.

`char` — three distinct types

This is the part people get wrong.

```
char          /* implementation-defined: signed OR unsigned */
signed char   /* definitely signed */
unsigned char  /* definitely unsigned */
```

`char` is a separate type from both. Even where it's signed, `char` and `signed char` are *not* the same type (matters for `_Generic` and pointer compatibility).

Same source file, two compiler flags:

```
char c = 200;
printf("%d\n", c);
```

```
### default (x86-64 Linux: char is SIGNED) ###
char c = 200 -> prints as %d: -56
three types: char=-56 signed char=-56 unsigned char=200

### with -funsigned-char (this is ARM's default!) ###
char c = 200 -> prints as %d: 200
three types: char=200 signed char=-56 unsigned char=200
```

Same code. Different answers. ARM, PowerPC, and s390 default to *unsigned char*; x86 and most others default to *signed*. This bites people porting to Raspberry Pi / embedded ARM constantly.

The professional rule:

Use	Type
Text / characters	<code>char</code>
Raw bytes / binary data	<code>unsigned char</code> (or <code>uint8_t</code>)

Use	Type
Small integer arithmetic	<code>signed char</code> or <code>int8_t</code> — be explicit

`unsigned char` has a special superpower: it's the only type you're allowed to use to inspect the raw bytes of *any* object (it's exempt from strict aliasing — see §2.11), and it has no padding bits or trap representations.

The classic `getchar()` bug

```
char c;
while ((c = getchar()) != EOF) { ... } /* ? BROKEN */
```

`getchar()` returns `int`, not `char`, precisely so it can return 257 distinct values: 0–255 plus `EOF` (-1). Two failure modes:

- If `char` is **unsigned**: `c` can never equal -1 ? **infinite loop**.
- If `char` is **signed**: a valid byte `0xFF` becomes -1 ? **loop ends early** on binary data.

```
int c; /* ? correct */
while ((c = getchar()) != EOF) { ... }
```

2.2 Floating point

The three types

```
sizeof: float=4 double=8 long double=16
```

- **float** — IEEE-754 binary32: 1 sign + 8 exponent + 23 mantissa bits, ~7 decimal digits
- **double** — binary64: 1 + 11 + 52, ~15–16 decimal digits
- **long double** — *not* standardized. On x86-64 Linux it's the **80-bit x87 extended** format padded to 16 bytes. On MSVC it's just `double`. On ARM64 Linux it's true 128-bit quad. **Don't rely on it for portable code.**

double is the default in C. `3.14` is a `double`; `3.14f` is a `float`. Math functions default to `double` (`sin`) with `f/l` variants (`sinf`, `sinl`).

Bit layout, decoded live

$value = (-1)^{sign} \times 1.mantissa \times 2^{(exponent-127)}$

VALUE	BITS	DECODE
1	0x3F800000	s=0 exp=127 (unbiased 0) mantissa=0x000000
2	0x40000000	s=0 exp=128 (unbiased 1) mantissa=0x000000

-2	0xC0000000	s=1 exp=128 (unbiased	1) mantissa=0x000000
0.5	0x3F000000	s=0 exp=126 (unbiased	-1) mantissa=0x000000
0.1	0x3DCCCCCD	s=0 exp=123 (unbiased	-4) mantissa=0x4CCCCD
0	0x00000000	s=0 exp= 0	mantissa=0x000000
inf	0x7F800000	s=0 exp=255	mantissa=0x000000
-nan	0xFFC00000	s=1 exp=255	mantissa=0x400000

Read that table carefully:

- **1.0** is exact. **0.5** is exact. **0.1 is 0xCCCCCD... — a repeating pattern that never terminates.** Binary floating point can only represent sums of powers of two. 0.1 in binary is like 1/3 in decimal.
- **Exponent all-1s (255) is the special marker:** mantissa == 0 ? **Infinity**; mantissa ? 0 ? **NaN**.
- Exponent all-0s ? zero or subnormals.

Why **0.1 + 0.2 != 0.3**

```
0.1 + 0.2 == 0.3 ? FALSE
0.1 + 0.2 = 0.30000000000000004441
0.3      = 0.29999999999999998890
```

Neither operand is exactly representable, so the sum lands on a *different* nearest-representable double than the literal **0.3**.

Never use == on floating point. (Exception: comparing against exact values you know are representable, like **0.0**, or checking a sentinel you yourself assigned.)

NaN and Infinity

```
1.0/0.0 = inf   isinf=1
-1.0/0.0 = -inf isinf=-1
0.0/0.0 = -nan  isnan=1
nan == nan ? FALSE    <-- NaN is not equal to itself!
nan != nan ? true
```

NaN is not equal to itself. This is IEEE-754's defining weirdness and it's genuinely useful — it's the only way to detect NaN without a library:

```
if (x != x) { /* x is NaN */ }
```

But use `<math.h>` in real code: `isnan()`, `isinf()`, `isfinite()`, `isnormal()`, `fpclassify()`. Compile with `-lm`.

Consequences of NaN semantics that will bite you:

- All comparisons with NaN are false — including `<`, `>`, `<=`, `>=`. So `!(a < b)` is **not** the same as `a >= b`.

- A single NaN propagates through every subsequent computation. One bad input poisons a whole pipeline.
- Sorting an array containing NaN with `qsort` gives undefined results — your comparator violates the required ordering.

Signed zero:

```
0.0 == -0.0 ? true  but bit patterns differ
bits(+0.0)=0x0000000000000000  bits(-0.0)=0x8000000000000000
```

They compare equal but `1.0/0.0` is `+inf` while `1.0/-0.0` is `-inf`. Use `signbit()` if you need to distinguish.

Epsilon comparison — done properly

`FLT_EPSILON`/`DBL_EPSILON` is **the gap between 1.0 and the next representable value** — not a universal tolerance:

```
FLT_EPSILON = 1.19209e-07
DBL_EPSILON = 2.22045e-16
```

Naive `fabs(a-b) < DBL_EPSILON` is wrong for anything not near 1.0. Here's why, measured:

```
big1=100000000000000000.0  big2=100000000000000002.0
near_abs(1e-9)  : 0  <-- absolute tolerance FAILS at large magnitude
near_rel(1e-9)  : 1  <-- relative works

tiny 1e-20 vs 2e-20 (100% different):
near_rel(1e-9)  : 0  <-- correctly says different
```

Use a hybrid:

```
#include <math.h>
#include <float.h>
#include <stdbool.h>

bool nearly_equal(double a, double b)
{
    double diff = fabs(a - b);

    /* absolute check: handles values near zero, where relative error blows up */
    if (diff <= DBL_EPSILON * 8) return true;

    /* relative check: scales with magnitude */
    return diff <= fmax(fabs(a), fabs(b)) * 1e-9;
}
```

Tune the tolerances to your domain. **For money, don't use floating point at all** — use integer cents (`int64_t`) or a fixed-point/decimal type.

Practical rules

- Never accumulate floats in a long loop without thinking — error compounds. Look up **Kahan summation** if precision matters.
- `printf("%f", x)` takes a `double`. `float` arguments are **automatically promoted to `double`** by the default argument promotions. There is no `%lf` in `printf` for this reason (in `scanf`, `%f` = float, `%lf` = double — different rules!).
- Avoid `-ffast-math` unless you fully understand it: it lets the compiler assume no NaN/Inf and reassociate operations, breaking IEEE semantics.

2.3 `_Bool` / `bool`, and `void`

`_Bool`

C99 added `_Bool`; `<stdbool.h>` provides the macros `bool`, `true`, `false`. In C23, `bool`/`true`/`false` are real **keywords** and the header is unnecessary.

`_Bool` is not just a small integer — it has a unique conversion rule: **any nonzero value becomes exactly 1**.

```
sizeof(_Bool) = 1
bool b = 5    -> 1
bool b = 0.5  -> 1
bool b = 256  -> 1    <-- NOT truncated to 0 like char would be
char c = 256  -> 0    <-- truncated
```

That difference matters. `char flag = some_int;` can silently become 0 for a "true" value like 256. `bool flag = some_int;` cannot.

Style note: in C, testing pointers and integers directly (`if (ptr)`, `if (count)`) is idiomatic and fine. But **never write `if (x == true)`** — a function returning `int` might return 2, which is truthy but `!= 1`.

`void`

`void` means three different things depending on position:

1. **No return value:** `void f(void);`
2. **No parameters:** `int f(void);` — as covered in Module 1, essential.
3. **`void *` — the generic pointer.** Any object pointer converts to/from `void *` without a cast.

```
void *p = malloc(100);    /* no cast needed in C — casting malloc is a C++ habit */
int *ip = p;              /* implicit conversion is legal and idiomatic */
```

You **cannot** dereference a `void *` or do arithmetic on it (GCC allows `void*` arithmetic as an extension treating it as size 1 — `-pedantic` rejects it). `sizeof(void)` is likewise a GNU extension, not standard.

`void` is an *incomplete type that can never be completed*: you can't declare `void x;` or an array of `void`.

2.4 Fixed-width types — `<stdint.h>`

This is C99's answer to "how big is `int`, really?"

```
exact-width : int8=1 int16=2 int32=4 int64=8
fast        : int_fast8=1 int_fast16=8 int_fast32=8
least       : int_least8=1 int_least16=2
pointer     : intptr_t=8 uintptr_t=8
max         : intmax_t=8 uintmax_t=8
```

The four families

Family	Meaning	Guaranteed to exist?
<code>int32_t</code> , <code>uint8_t</code> ...	Exactly N bits, no padding, two's complement	No — only if the platform has such a type
<code>int_least16_t</code> ...	Smallest type with at least N bits	Yes (8/16/32/64)
<code>int_fast16_t</code> ...	Fastest type with at least N bits	Yes (8/16/32/64)
<code>intmax_t</code> , <code>uintmax_t</code>	Largest supported integer type	Yes
<code>intptr_t</code> , <code>uintptr_t</code>	Integer that can hold a <code>void *</code> round-trip	Optional but universal

Look at the measured output again: **`int_fast16_t` is 8 bytes on x86-64**. The "fast" types trade memory for speed — the CPU prefers full-width registers. Don't use them in structs you care about the size of.

When to use which

```
/* Binary formats, protocols, hardware registers, files on disk ? EXACT width */
struct packet_header {
    uint16_t magic;
    uint8_t  version;
    uint32_t length;
};

/* Storing a pointer as an integer ? intptr_t */
uintptr_t addr = (uintptr_t)&x;
if (addr % 16 == 0) { /* alignment check */ }
```

```
/* Ordinary loop counters and general arithmetic ? just use int */
for (int i = 0; i < n; i++) { }
```

Don't reflexively write `uint32_t` everywhere. `int` is the natural word size and is often faster; over-using unsigned types invites the wraparound traps in §2.8. Use exact-width types where **width is part of the contract**.

`<stdint.h>` also gives you limit macros (`INT32_MAX`, `UINT8_MAX`, `SIZE_MAX`) and constant macros (`UINT64_C(1234)` to give a literal the right type).

2.5 `size_t`, `ptrdiff_t`, and friends

```
size_t=8 ptrdiff_t=8 wchar_t=4 char16_t=2 char32_t=4
```

Type	Header	Meaning
<code>size_t</code>	<code><stddef.h></code>	Unsigned. Result of <code>sizeof</code> . Big enough for any object size.
<code>ptrdiff_t</code>	<code><stddef.h></code>	Signed. Result of subtracting two pointers.
<code>ssize_t</code>	<code><sys/types.h></code>	POSIX, not ISO C. Signed size — used for "count or -1 on error" (<code>read</code> , <code>write</code>).
<code>wchar_t</code>	<code><wchar.h></code>	Wide char. 4 bytes on Linux (UTF-32), 2 on Windows (UTF-16). Avoid for portable code.
<code>char16_t</code> / <code>char32_t</code>	<code><uchar.h></code>	C11. Explicitly UTF-16 / UTF-32 code units.

Use `size_t` for: sizes, array indices, loop counters over arrays, anything returned by `sizeof`/`strlen`/`malloc` arguments.

Print it with `%zu` (C99). Not `%d`, not `%lu` — `%zu` is correct everywhere.

The `size_t` reverse-loop trap

```
for (size_t i = n - 1; i >= 0; i--) { } /* ? INFINITE LOOP */
```

`size_t` is unsigned, so `i >= 0` is **always true**. When `i` is 0 and you decrement, it wraps to `SIZE_MAX`.

Correct patterns:

```
for (size_t i = n; i-- > 0; ) { /* i goes n-1 ... 0 */ }

size_t i = n;
while (i > 0) { i--; /* use i */ }
```

Also note `n - 1` when `n == 0` gives `SIZE_MAX` — check for empty first.

Because `wchar_t` is 2 bytes on Windows and 4 on Linux, professional C code handling text almost always uses UTF-8 in plain `char` arrays instead. We'll cover this properly in Module 8.

2.6 <limits.h>, <float.h>, <inttypes.h>

<limits.h> — integer limits: `CHAR_BIT`, `CHAR_MIN/MAX`, `SCHAR_*`, `UCHAR_MAX`, `SHRT_*`, `INT_*`, `UINT_MAX`, `LONG_*`, `ULONG_MAX`, `LLONG_*`, `ULLONG_MAX`.

<float.h> — `FLT_DIG` (6), `DBL_DIG` (15), `FLT_EPSILON`, `DBL_EPSILON`, `FLT_MAX/MIN`, `DBL_MAX/MIN`, `FLT_RADIX`, `DBL_MANT_DIG`.

<inttypes.h> — includes <stdint.h> and adds the **format macros**, which solve a real problem: how do you `printf` a `uint64_t` portably when it's `unsigned long` on Linux and `unsigned long long` on Windows?

```
#include <inttypes.h>

uint64_t big = 18446744073709551615ULL;
printf("%" PRIu64 "\n", big);           /* NOT %llu, NOT %lu */
printf("%" PRId32 "\n", i32);
printf("0x%" PRIx32 "\n", hex);
printf("%" PRIuPTR "\n", (uintptr_t)ptr);
```

```
PRIu64 : 18446744073709551615
PRId64 : -9223372036854775808
PRIx32 : 0xDEADBEEF
PRIuPTR: 140725905758232
```

The macros expand to string literals ("`lu`" or "`llu`"), and **adjacent string literals concatenate** — that's why the syntax looks odd. Naming: `PRI` (print) or `SCN` (scan) + `d/i/u/o/x/X` + width (`8/16/32/64/MAX/PTR/FAST32/LEAST16`).

Getting a `printf` format wrong is **undefined behavior**, not a cosmetic issue. Always compile with `-Wformat` (included in `-Wall`).

2.7 Integer promotion & usual arithmetic conversions ??

This is the most important section in this module. These rules are applied silently and constantly, and they produce bugs that look impossible.

Rule 1 — Integer promotions

Any type **narrower than `int`** (`char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `_Bool`, bitfields, enums) is **promoted to `int`** whenever used in an expression — if `int` can hold all its values. Otherwise to **`unsigned int`**.

There is essentially **no arithmetic on `char` or `short` in C**. They are storage types only.

```
sizeof('A')=4      ? a character constant is an int in C (not char, unlike C++)
sizeof(c)=1        ? the variable itself
sizeof(c + 0)=4    ? promoted
sizeof(+c)=4       ? even unary + promotes
```

```
TRAP5: (200 + 100) as int = 300, stored back in uchar = 44
```

Two **`unsigned char`**s summing to 300 — the addition happens in **`int`**, so **300** is the true intermediate value. It only truncates when you store it back. This is why **`unsigned char`** "overflow" isn't overflow at all.

```
TRAP7: (signed char)-1 >> 4 = -1, but (unsigned char)-1 >> 4 = 15
```

Both promote to **`int`** first — but **`signed char -1`** promotes to **`int -1`** (0xFFFFFFFF, arithmetic shift preserves sign), while **`unsigned char 255`** promotes to **`int 255`** (shifts to 15). **Sign extension happens during promotion**, before the shift.

Rule 2 — Usual arithmetic conversions

For binary operators with operands of different types, after promotion:

1. If either is **`long double`** ? both to **`long double`**
2. Else if either is **`double`** ? both to **`double`**
3. Else if either is **`float`** ? both to **`float`**
4. **Otherwise (both integer):**
 - Same signedness ? convert to the higher rank
 - **Unsigned has rank = signed ? convert signed to unsigned** ? this is the killer
 - Signed has higher rank and can represent all unsigned values ? convert unsigned to signed
 - Else ? both to the unsigned version of the signed type

Step 4b is where careers go to die. When signed meets unsigned of equal-or-higher rank, **the signed value converts to unsigned**, and negative numbers become enormous.

The traps, measured

```
TRAP1: -1 < sizeof(int) -> NOT taken (-1 became 4294967295!)
TRAP2: 3u - 5u = 4294967294 (as signed: -2)
TRAP3: strlen("abc") - 5 = 18446744073709551614
TRAP6: (-5 > 5u) ? TRUE (!!)
```

Walk through each:

TRAP 1. `sizeof` returns `size_t` (unsigned, 64-bit). `int -1` converts to `18446744073709551615`. The comparison is `huge < 4` ? false. **A negative-index bounds check that silently never fires.** This exact pattern is a real CVE category.

```
if (i < sizeof(buf)) buf[i] = 0;    /* i = -1 passes... no it doesn't, it FAILS, */
/* but if the test were i > 0 && i < len with
*/
/* unsigned len, you'd get the mirror bug */
```

The canonical broken version:

```
int len = get_length();             /* returns -1 on error */
if (len < BUFSIZE) memcpy(buf, src, len); /* len promoted to size_t ? huge copy
*/
```

TRAP 3. `strlen()` returns `size_t`. `strlen(s) - 5` when the string is shorter than 5 wraps to $\sim 1.8 \times 10^{17}$. Then:

```
for (size_t i = 0; i < strlen(s) - 1; i++) /* empty string ? 18 quintillion
iterations */
```

TRAP 6. `-5 > 5u` is **true**. Because `-5` becomes `4294967291`.

How to defend yourself

1. Turn on the warnings. They catch most of this:

```
warning: comparison of integer expressions of different signedness:
'int' and 'long unsigned int' [-Wsign-compare]
```

```
gcc -Wall -Wextra -Wsign-compare -Wsign-conversion -Wconversion
```

`-Wextra` includes `-Wsign-compare`. `-Wconversion` is noisy but catches implicit narrowing — worth enabling on new code.

2. Don't mix signedness in comparisons. Pick one and cast explicitly at the boundary, with a range check:

```
if (len >= 0 && (size_t)len < bufsize) /* check sign FIRST, then convert */
```

3. Restructure to avoid unsigned subtraction:

```
if (strlen(s) > 1) { ... }      /* instead of strlen(s) - 1 > 0 */
if (a > b) diff = a - b;        /* instead of if (a - b > 0) */
```

4. Default to signed `int` for arithmetic. Use unsigned when you need modular arithmetic, bit manipulation, or the full range — not just because "a count can't be negative." (This is the Google C++ style guidance and it applies to C: unsigned types cause more bugs than they prevent.)

2.8 Signed overflow (UB) vs unsigned wraparound (defined)

Unsigned: fully defined

Unsigned arithmetic is defined to be **modulo 2^N** . It wraps. This is guaranteed, portable, and safe to rely on.

```
UINT_MAX + 1 = 0    (defined wraparound)
```

Signed: undefined behavior

Signed overflow is **UB**. Not "implementation-defined," not "wraps on most machines" — **undefined**, which means the compiler may assume it *never happens* and optimize accordingly.

Watch this. Real code, real compiler:

```
int check(int x)
{
    if (x + 1 > x) return 1;
    return 0;
}
```

Mathematically, `check(INT_MAX)` should return 0. Here's what GCC generates at `-O2`:

```
check:
    endbr64
    mov     eax, 1
    ret
```

The entire comparison was deleted. The function is now `return 1;`. The compiler reasoned: "signed overflow is UB, so `x+1` can't wrap, so `x+1 > x` is always true, so the branch is dead."


```
### -O0 ###  check(INT_MAX) = 1
### -O2 ###  check(INT_MAX) = 1
```

This is why "I tested it and it wrapped fine" is not a defense. Your overflow check based on wrapping **will be optimized away**, usually the moment someone bumps the optimization level or upgrades the compiler.

Catching it: UBSan

```
gcc -fsanitize=undefined prog.c -o prog
```

```
ub3.c:6:9: runtime error: signed integer overflow:
      2147483647 + 1 cannot be represented in type 'int'
```

Use **-fsanitize=undefined** in every debug build and every CI run. It is the single highest-value tool in C development.

Writing correct overflow checks

Wrong (relies on UB):

```
if (a + b < a) { /* overflow */ }
```

Right — check before operating:

```
#include <limits.h>
if (b > 0 && a > INT_MAX - b) { /* would overflow */ }
if (b < 0 && a < INT_MIN - b) { /* would underflow */ }
```

Better — use the builtins (GCC/Clang, and now `<stdckdint.h>` in C23):

```
int result;
if (__builtin_add_overflow(a, b, &result)) {
    /* overflow occurred, result is undefined */
}
/* also: __builtin_sub_overflow, __builtin_mul_overflow */
```

```
/* C23 standard version */
#include <stdckdint.h>
if (ckd_add(&result, a, b)) { /* overflow */ }
```

Other UB in this family to memorize:

- `INT_MIN / -1` and `INT_MIN % -1` ? UB (overflows, and traps with SIGFPE on x86)
- Division or modulo by zero ? UB
- `x << n` where `n >= width of x`, or `n` is negative ? UB
- Shifting a negative value left ? UB in C99/C11 (defined in C23)
- `x >> n` on a negative `x` ? implementation-defined (arithmetic shift in practice)
- Converting a float to an integer type when the value is out of range ? UB

Compilers offer `-fwrapv` to *define* signed overflow as wrapping. It's a real escape hatch for legacy code, but it disables useful optimizations and makes your code non-portable. Fix the code instead.

2.9 Type qualifiers

`const`

`const` means "read-only through this name." It does *not* mean "the object is immutable" and it does not mean "compile-time constant."

Reading declarations — read right to left from the identifier:

```
int *p1;           /* p1: pointer to int */
const int *p2;     /* p2: pointer to const int (can't write *p2, CAN repoint) */
int const *p2b;    /* identical to p2 */
int *const p3;     /* p3: const pointer to int (CAN write *p3, can't repoint) */
const int *const p4; /* both locked */
```

Verified:

```
p2: cannot write *p2, CAN repoint
p3: CAN write *p3, cannot repoint

error: assignment of read-only location '*p2'
error: assignment of read-only variable 'p3'
```

Casting away `const` and writing is undefined behavior:

```
const int c = 5;
int *p = (int*)&c;
*p = 99;
printf("c=%d *p=%d\n", c, *p);
```

```
c=5  *p=99
```

Same memory, two different answers — the compiler folded `c` into a literal 5 because it was allowed to assume nobody would modify it. (And if `c` were a global, it'd live in `.rodata` and the write would **segfault**.)

Where `const` earns its keep: function parameters. `void print(const char *s)` documents *and enforces* that the function won't modify your buffer. Const-correctness is expected in professional APIs.

`const` is not a compile-time constant in C (unlike C++). You can't use it for array sizes in C89, or as a `case` label, or in a static initializer. Use `#define`, an `enum`, or C23's `constexpr`.

volatile

volatile means "this object may change in ways the compiler cannot see — reload it every time, never optimize the access away."

```
int flag = 0;
volatile int vflag = 0;

void wait_plain(void)    { while (flag == 0) { } }
void wait_volatile(void) { while (vflag == 0) { } }
```

At -O2:

```
wait_plain:                                wait_volatile:
    mov     eax, DWORD PTR flag              .L6:
    test    eax, eax                        mov     eax, DWORD PTR vflag[rip]
    jne     .L1                             test    eax, eax
    jmp     .L3                             je      .L6
    ? reads ONCE,                          ? re-reads EVERY
iteration                                  ret
                                then spins
```

The non-volatile version **hoisted the load out of the loop** — it will hang forever even if another thread sets `flag`.

Legitimate uses of `volatile`:

1. Memory-mapped hardware registers `*(volatile uint32_t*)0x40021000`
2. Variables modified by a signal handler (must be `volatile sig_atomic_t`)
3. Variables modified by a `setjmp/longjmp` pair
4. Preventing the compiler from optimizing away benchmark code

volatile is NOT for thread safety. This is the most common misconception in C. `volatile` guarantees the *access happens*, but gives you **no atomicity and no memory ordering**. For threads you need `_Atomic` / `<stdatomic.h>` or a mutex. (Java and C# gave `volatile` different, stronger semantics — don't carry that intuition into C.)

restrict

C99. **A promise you make to the compiler:** "for the lifetime of this pointer, the object it points to is accessed *only* through this pointer." No aliasing.

```
void add_plain(int *a, int *b, int *c)           { *a += *c; *b
+= *c; }
void add_restr(int *restrict a, int *restrict b, int *restrict c) { *a += *c; *b
+= *c; }
```

```
add_plain:                                add_restr:
    mov    eax, [rdx]                      mov    eax, [rdx]
    add    [rdi], eax                      add    [rdi], eax
    mov    eax, [rdx]    ? reloads *c      add    [rsi], eax    ? reuses the
register                                     ret
    add    [rsi], eax
    ret
```

Without **restrict**, the compiler must assume **a** might equal **c**, so it reloads. With **restrict**, one load. On tight numeric loops this enables vectorization and can be a multi-x speedup.

restrict is a promise you must keep. Break it and you get UB with no diagnostic. This is exactly why **memcpy** (declares **restrict**) is UB on overlapping buffers while **memmove** (no **restrict**) is safe:

```
void *memcpy(void *restrict dest, const void *restrict src, size_t n);
void *memmove(void *dest, const void *src, size_t n);
```

_Atomic

C11. Makes operations on the object atomic and gives you memory-ordering guarantees.

```
#include <stdatomic.h>
_Atomic int counter = 0;
atomic_fetch_add(&counter, 1);    /* thread-safe */
counter++;                        /* also atomic, seq_cst by default */
```

Full treatment in Module 17. For now: **_Atomic for threads, volatile for hardware.** They solve different problems and are not interchangeable.

2.10 Casting

Implicit conversions happen automatically (§2.7). **Explicit casts** **(type)expr** force one.

```
double d = 3.99;
int i = (int)d;          /* 3 – truncates toward zero, does NOT round */
int j = (int)(d + 0.5); /* 4 – poor man's rounding; use lround() properly */
```

Rules to internalize:

- **Float ? int truncates toward zero.** `(int)-3.7` is `-3`, not `-4`. Use `round()`, `floor()`, `ceil()`, `lround()` from `<math.h>`.
- **Float ? int when out of range is UB.** `(int)1e20` is undefined, not `INT_MAX`.
- **Narrowing integer conversion:** unsigned ? well-defined modulo truncation. Signed ? implementation-defined (in practice, truncation) before C23; C23 mandates two's-complement truncation.
- **Widening keeps the value** (with sign extension for signed, zero extension for unsigned).

When to cast — and when not to:

```
/* GOOD: intentional, documented conversion at a boundary */
size_t n = (size_t)count;      /* after verifying count >= 0 */
double avg = (double)sum / n; /* force float division, avoid integer division */

/* BAD: silencing a warning you don't understand */
foo((int)ptr);                  /* the warning was right */

/* UNNECESSARY: void* converts implicitly in C */
int *p = (int*)malloc(n);      /* the cast can hide a missing <stdlib.h> – omit it */
int *p = malloc(n);            /* idiomatic C */
```

A cast is you overriding the type system. Every cast should have a reason you could defend in code review. Excessive casting is a smell that the types are wrong upstream.

2.11 Type punning & strict aliasing ??

Type punning = reading the same memory as a different type. Extremely common in systems code (serialization, hashing, hardware, float manipulation) — and a minefield.

The strict aliasing rule

An object may only be accessed through an lvalue of:

- its own type (or a qualified/signed-unsigned variant),
- a struct/union containing that type,
- **char, signed char, or unsigned char.**

The compiler uses this to assume that an `int*` and a `float*` **cannot point to the same memory**, which lets it skip reloads and reorder accesses.

Watch it break

```
int bad(float *f, int *i)
{
    *f = 1.0f;
    *i = 42;           /* compiler assumes this can't touch *f */
    return *f == 1.0f;
}

float x;
printf("%d\n", bad(&x, (int*)&x)); /* deliberately aliased */
```

Same binary source, three builds:

```
### -O0 ###                bad() says *f is still 1.0f? 0
### -O2 (strict aliasing on) ### bad() says *f is still 1.0f? 1  ? !!
### -O2 -fno-strict-aliasing ### bad() says *f is still 1.0f? 0
```

GCC even warned:

```
warning: dereferencing type-punned pointer will break strict-aliasing rules
```

The bug appears **only at -O2**. This is the classic "works in debug, fails in release" nightmare.

The safe ways to pun

1. memcpy — always correct, always portable, and optimizes to zero instructions:

```
#include <string.h>
#include <stdint.h>

float f = 1.0f;
uint32_t bits;
memcpy(&bits, &f, sizeof bits); /* ? 0x3F800000 */
```

```
safe memcpy punning: 1.0f -> 0x3F800000
```

Don't worry about the "copy" — every modern compiler turns a fixed-size `memcpy` between registers into a single `mov`. **This is the idiomatic, standard-blessed method.**

2. Union punning — allowed in C (this is a real difference from C++, where it's technically UB):

```
union { float f; uint32_t u; } pun;
pun.f = 1.0f;
printf("%08X\n", pun.u);      /* legal in C99 TC3 and later */
```

Fine in C, but `memcpy` is more portable across the C/C++ boundary.

3. `unsigned char *` — always permitted. Inspecting any object byte-by-byte through `unsigned char*` is explicitly legal:

```
unsigned char *bytes = (unsigned char *)&some_struct;
for (size_t i = 0; i < sizeof some_struct; i++) printf("%02X ", bytes[i]);
```

4. `-fno-strict-aliasing` — the Linux kernel does this. It's a legitimate engineering decision for codebases full of legacy punning, but it costs performance and it's a crutch, not a fix.

What you must never do

```
*(int*)&my_float          /* classic UB */
*(uint32_t*)byte_buffer  /* also an ALIGNMENT bug on ARM/SPARC */
struct foo *f = (struct foo*)buf; /* UB + alignment */
```

That second one is doubly dangerous: on strict-alignment architectures, reading a `uint32_t` from an unaligned address **faults** or silently returns garbage. Use `memcpy` and you get both correct aliasing *and* correct alignment handling for free.

Quick reference — the rules that will save you

#	Rule
1	Use <code>unsigned char/uint8_t</code> for bytes, <code>char</code> only for text
2	<code>getchar()</code> result goes in an <code>int</code> , never a <code>char</code>
3	Never compare floats with <code>==</code> ; use a hybrid absolute+relative tolerance
4	Never store money in floating point
5	Print <code>size_t</code> with <code>%zu</code> , <code>uint64_t</code> with <code>PRIu64</code>
6	Never mix signed and unsigned in a comparison — compile with <code>-Wsign-compare</code>
7	Signed overflow is UB; check <i>before</i> the operation or use <code>__builtin_*_overflow</code>
8	<code>volatile</code> is for hardware and signals, <code>_Atomic</code> is for threads
9	Type-pun with <code>memcpy</code> , never with a pointer cast
10	Run <code>-fsanitize=undefined,address</code> on every debug build

Your standing build command from here on:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wsign-conversion \
    -Wshadow -g -fsanitize=address,undefined prog.c -o prog -lm
```

Practice

1. **Portability audit.** Run the `sizes.c` program from §2.1. Then recompile with `-m32` (install `gcc-multilib` if needed) and diff the output. Which types changed?
2. **Reproduce the char bug.** Write a function that counts how many bytes in a buffer are = 128, using `char`. Compile normally, then with `-funsigned-char`. Explain the difference. Then fix it.
3. **Prove the optimizer removes your check.** Write `int will_overflow(int a, int b) { return a + b < a; }`. Compile at `-O0` and `-O2`, dump the assembly with `-S -masm=intel`, and compare. Then rewrite it correctly and verify with UBSan.
4. **The infinite loop.** Write `for (size_t i = n - 1; i >= 0; i--)` over a 5-element array. Run it under ASan and read the report. Fix it two different ways.
5. **Float forensics.** Write a program that prints the raw bits of a `double` (using `memcpy`) and use it to find: (a) the largest integer where `x` and `x+1` are both exactly representable, (b) the bit pattern of `DBL_MAX`, (c) a subnormal number.
6. **Aliasing bug hunt.** Write a `uint32_t hash(const void *data, size_t len)` that reads 4 bytes at a time. First write it the unsafe way (`*(const uint32_t*)p`), compile at `-O2 -Wall`, and note the warning. Then rewrite with `memcpy` and confirm the generated assembly is identical (`objdump -d`) — proving `memcpy` costs nothing.
7. **Reasoning question, no compiler:** given `unsigned short a = 65535, b = 1;` on a 32-bit-`int` platform, what is the type and value of `a + b`? What if `unsigned short` were 32 bits and `int` were 32 bits?

Ready when you are for **Module 3: Variables, Scope, Lifetime & Linkage** — where `static` finally makes complete sense and you'll understand exactly why `.bss` exists.

3. Variables, Scope, Lifetime

- Declaration vs definition
- Scope: block, file, function, function-prototype
- Linkage: external, internal (`static`), none
- Storage duration: automatic, static, thread (`_Thread_local`), allocated
- `extern`, `static`, `register`, `auto`
- Initialization rules, zero-initialization of statics, indeterminate values

Three independent axes govern every variable in C. Beginners conflate them; professionals keep them separate:


```
SCOPE      - WHERE the name is visible      (a compile-time, textual property)
LINKAGE    - WHETHER other TUs see the same object  (a link-time property)
DURATION   - WHEN the object exists in memory (a runtime property)
```

The word **static** means a *different thing on each axis* — which is exactly why it confuses people. By the end of this module that will be clear.

3.1 Declaration vs Definition

Declaration: introduces a name and its type. "This exists somewhere." **Definition:** allocates storage or provides the body. "Here it is."

A definition is also a declaration. Not every declaration is a definition.

```
/* DECLARATIONS - may repeat freely */
extern int g;
extern int g;
extern int g;
int f(int);
int f(int);

/* DEFINITIONS - exactly one each, program-wide */
int g = 42;
int f(int x) { return x + 1; }
```

```
g=42 f(1)=2      ? compiles clean with -Wall -Wextra
```

Break the rule and the **linker** catches it:

```
/usr/bin/ld: multiple definition of `g'; first defined here
```

How to tell them apart

Construct	Which?
<code>extern int x;</code>	declaration
<code>int x;</code> at file scope	tentative definition (see below)
<code>int x = 5;</code>	definition
<code>int f(void);</code>	declaration (prototype)
<code>int f(void) { ... }</code>	definition

Construct	Which?
<code>struct S;</code>	declaration (incomplete type)
<code>struct S { int a; };</code>	definition of the type
<code>typedef int myint;</code>	declaration — creates no object
<code>int x;</code> inside a function	definition (allocates automatic storage)

Tentative definitions — a C-only quirk

At file scope, `int t;` with no initializer is a **tentative definition**. C lets you write it multiple times in one TU:

```
int t;
int t;          /* legal in C – ERROR in C++ */
int main(void){ printf("t=%d\n", t); }
```

```
t=0
```

At the end of the translation unit, if no real definition appeared, the tentative one becomes `int t = 0;`.

The trap this creates across TUs:

```
/* a.c */ int t;
/* b.c */ int t;      /* two "definitions" – but historically it linked! */
```

```
-- with -fcommon (old GCC default, pre-GCC 10) --
0                               ? linked fine, both refer to ONE object

-- with -fno-common (GCC 10+ default) --
/usr/bin/ld: multiple definition of `t'
```

Old linkers merged these into a "common block." **GCC 10 changed the default to `-fno-common`**, which broke a lot of ancient code — and correctly so, because merging was never guaranteed and hid real bugs.

The rule that avoids all of this: variables are *declared* in headers with `extern`, and *defined* in exactly one `.c` file.

```
/* config.h */
extern int verbose_level;      /* declaration */

/* config.c */
int verbose_level = 0;         /* the one definition */
```

3.2 Scope — four kinds

Scope is **purely textual**. It answers: "at this point in the source, is this name visible?"

1. Block scope

From the declaration to the closing `}` of the enclosing block. Inner declarations **shadow** outer ones.

```
int x = 100;                /* file scope */

void demo(int x)           /* parameter – block scope of the body */
{
    printf("param x = %d\n", x);
    { int x = 2;
      printf("inner  = %d\n", x);
      { int x = 3;
        printf("deeper = %d\n", x); }
      printf("back   = %d\n", x); }
    printf("param   = %d\n", x);
}
```

```
file-scope x      = 100
param x           = 1
inner block x     = 2
deeper x         = 3
back to inner     = 2
back to param     = 1
file-scope x still = 100
```

Function **parameters** have block scope covering the function body — that's why a parameter and a body-local with the same name collide.

Shadowing is legal but almost always a bug. GCC will tell you:

```
warning: declaration of 'x' shadows a global declaration [-Wshadow]
warning: declaration of 'x' shadows a parameter [-Wshadow]
```

Put `-Wshadow` in your standard flags. Shadowing bugs are nasty: you "fix" a variable and nothing changes, because you modified the shadow.

Note: in C89 declarations had to be at the top of a block. **C99 allows declarations anywhere**, and you should declare variables at their point of first use with the narrowest possible scope.

2. File scope

Declared outside any function; visible from the declaration to the end of the translation unit. (Not the whole file — a name declared on line 50 isn't visible on line 10.)

3. Function scope

Only labels have function scope. A label is visible throughout the entire function, across blocks, and even *before* its declaration:

```
void labels(void)
{
    goto deep_inside;
    { { deep_inside:
        printf("jumped into a nested block\n"); } }
}
```

jumped into a nested block via function-scope label

This is why label names must be unique per function, and why `goto` can jump *into* a block (though not past a VLA declaration).

4. Function-prototype scope

Names in a prototype's parameter list exist only within those parentheses:

```
void takes(int n, int arr[n]);           /* n visible only here */
void takes(int count, int data[count]) { ... } /* different names – fine */
```

That's why parameter names in prototypes are optional and purely documentation. **Write them anyway** — `void copy(char *dst, const char *src)` is far more useful in a header than `void copy(char *, const char *)`.

3.3 Linkage — who else can see this object?

Linkage answers: "do declarations of this name in *different* translation units refer to the *same* object?"

Linkage	Meaning	How you get it
External	Same object across the whole program	File-scope, non- <code>static</code> (the default)
Internal	Same object within one TU only	File-scope + <code>static</code>
None	Each declaration is its own object	Block-scope locals, parameters, typedefs, enum constants

Seeing linkage in the symbol table

```
int  ext_var  = 1;           /* external */
static int int_var = 2;      /* internal */
int  ext_func(void) { ... }  /* external */
static int int_func(void) { ... } /* internal */
```

```
$ nm link_a.o
0000000000000000 T ext_func      ? UPPERCASE = external linkage
0000000000000000 D ext_var      ? UPPERCASE
0000000000000010 t int_func     ? lowercase = internal linkage
0000000000000004 d int_var      ? lowercase
0000000000000020 T use
```

nm capitalization is exactly the linkage distinction. Uppercase means the linker exports it; lowercase means it's private to this object file.

Try to reach the internal ones from another TU:

```
undefined reference to `int_func'
undefined reference to `int_var'
```

The object exists in memory — but its name was never published, so the linker can't find it.

static at file scope is C's encapsulation

This is the **most important professional use of static**. Anything not part of your module's public API should be **static**:

```
/* parser.c */
#include "parser.h"

static int  line_number;           /* private state */
static Token next_token(Lexer *lx) { ... } /* private helper */

Token *parse(const char *src) { ... } /* PUBLIC — declared in parser.h */
```

Benefits you get for free:

- **No name collisions.** Two files can both have `static int count;`.
- **Better optimization.** The compiler knows nothing outside can call `next_token`, so it can inline it aggressively or delete it if unused.
- **A readable API.** Anyone can see the public surface by reading the header.
- **Smaller symbol tables and faster linking.**

Rule of thumb: make everything `static` by default, then remove `static` only when a symbol must be shared.

`extern` — the "look elsewhere" keyword

```
extern int g;           /* declaration only, no storage allocated */
extern int g = 5;       /* legal but pointless – the initializer makes it a
definition */
```

For **functions**, `extern` is the default and redundant:

```
extern int f(int);      /* same as: */
int f(int);
```

Nobody writes `extern` on function declarations in modern C.

The global-variable header pattern, done right:

```
/* globals.h */
#ifndef GLOBALS_H
#define GLOBALS_H
extern int      g_verbose;
extern char     g_procname[64];
extern FILE     *g_logfile;
#endif

/* globals.c */
#include "globals.h"
int  g_verbose = 0;
char g_procname[64] = "";
FILE *g_logfile = NULL;
```

(That said — **minimize globals**. They defeat testing, break thread safety, and create hidden coupling. Prefer passing a context struct.)

Linkage of block-scope declarations

Two special cases worth knowing:

```
void f(void)
{
    extern int g;          /* refers to the FILE-SCOPE g – external linkage, block
scope */
    static int s = 0;      /* NO linkage, but static duration */
    int local;             /* no linkage, automatic duration */
}
```

A block-scope `extern` declaration is legal but poor style — put it in a header.

3.4 Storage duration — when does the object exist?

Duration	Lives from ? to	Where	How you get it
Automatic	block entry ? block exit	stack (usually)	plain local variables
Static	program start ? program end	.data / .bss	file-scope, or <code>static</code> anywhere
Thread	thread start ? thread end	TLS block	<code>_Thread_local</code> / <code>thread_local</code>
Allocated	<code>malloc</code> ? <code>free</code>	heap	dynamic allocation

All four, live

```
ADDRESSES (grouped by region):
code   &main      = 0x55cc242fe217   ? .text
.data  &g_init    = 0x55cc24301010   ? initialized globals
.data  &s_file    = 0x55cc24301014
.bss   &g_zero    = 0x55cc2430101c   ? zero-initialized globals
.bss   &g_uninit  = 0x55cc24301020
TLS    &tls       = 0x7fb1aff9173c   ? thread-local block
heap   heap       = 0x55cc2c6fe2a0   ? grows upward
stack  &automatic = 0x7ffef8595f6c   ? way up high, grows downward
```

Read the addresses: code and static data cluster near `0x5500...`, the heap sits above them, and the stack lives near `0x7fff...` at the top. This is the process memory layout.

Automatic vs static, demonstrated

```
void counter(void)
{
    static int calls = 0; /* static duration, block scope, NO linkage */
    int local = 0;        /* automatic — recreated every call */
    calls++; local++;
    printf("call #%d : static calls=%d  automatic local=%d\n", calls, calls,
local);
}
```

```
call #1 : static calls=1  automatic local=1
call #2 : static calls=2  automatic local=1
call #3 : static calls=3  automatic local=1
```

static inside a function means "static duration," not "internal linkage." The variable is still invisible outside the function — that's its *scope*. Its initializer runs **once, before main**, not on each call.

?? Function-local **static** makes a function **non-reentrant and not thread-safe**. This is why **strtok** is dangerous and **strtok_r** exists. Avoid it in library code.

Why **.bss** exists — measured

.data holds initialized globals and must be **stored in the executable file**. **.bss** holds zero-initialized globals and stores only a size — the loader zeroes the memory at startup.

```
int big_zero[1000000];      /* .bss */
int big_init[1000000] = {1}; /* .data */
```

```
### file sizes on disk ###
bss_prog      15808 bytes
data_prog     4015816 bytes      ? 254× larger!

### size command (text data bss dec hex) ###
bss_prog:    1229      544 4000032
data_prog:   1229    4000560      8
```

Same amount of runtime memory, but the **.data** version carries 4 MB of mostly-zeros in the binary. **This is why = {0} on a huge global array is a mistake** — it may push the array from **.bss** into **.data**. (Good compilers detect all-zero initializers and keep them in **.bss**, but don't rely on it, especially on embedded toolchains where binary size is precious.)

Thread storage duration

```
int          shared = 0;      /* ONE copy for all threads */
_Thread_local int per_thread = 0; /* one copy PER THREAD */
```

Four threads each incrementing both 100,000 times:

```
thread 0: per_thread=100000 (correct)
thread 1: per_thread=100000 (correct)
thread 2: per_thread=100000 (correct)
thread 3: per_thread=100000 (correct)

run 1: shared = 300000      ? lost 100,000 increments to a data race
run 2: shared = 400000
run 3: shared = 400000
```


Look at run 1. `shared++` is not atomic — it's load, add, store — so concurrent threads clobber each other. `_Thread_local` sidesteps the race entirely by giving each thread private storage.

Notice also that runs 2 and 3 gave the *correct* answer. **A data race that produces the right output is still a bug.** Never conclude "it works" from a passing run; use `-fsanitize=thread`.

`<threads.h>` provides `thread_local` as a macro for `_Thread_local`; C23 makes `thread_local` a keyword. Common uses: per-thread `errno`, per-thread scratch buffers, per-thread RNG state.

Allocated duration

`malloc/calloc/realloc/free`. Lifetime is entirely under your control — which is why it's the source of leaks, double-frees, and use-after-frees. Full treatment in Module 10.

3.5 The storage-class specifiers

You may use **at most one** per declaration: `extern`, `static`, `auto`, `register`, `_Thread_local` (which may combine with `extern` or `static`).

`static` — the four-in-one keyword

Where written	What it does
File-scope variable	Internal linkage (duration was already static)
File-scope function	Internal linkage
Block-scope variable	Static duration (linkage was already none)
Array parameter: <code>void f(int a[static 10])</code>	C99: "caller guarantees = 10 elements, non-NULL" — an optimization hint, <i>not</i> checked

Same keyword, four unrelated meanings. Now you can explain it in an interview.

`extern`

"Declaration only — the definition is elsewhere." Default and redundant for functions.

`auto`

The default for block-scope variables. **Nobody has written it in decades** — `auto int x;` is identical to `int x;`.

In C23 it was repurposed to mean type inference, like C++:

```
auto x = 3.14;      /* C23: x is double */
```

`register`

A hint that the variable is used heavily. **Modern compilers ignore the hint entirely** — their register allocators are far better than human intuition.

It does have one *real* semantic effect the compiler must honor:

```
register int x;
int *p = &x;      /* ERROR: cannot take the address of a register variable */
```

```
error: address of register variable 'x' requested
```

That's occasionally useful to enforce "this must never be aliased," but in practice: **don't use `register`**. (It's deprecated in C++ and removed in C++17.)

`_Thread_local`

```
_Thread_local int x;          /* C11 */
thread_local int x;          /* with <threads.h>, or C23 keyword */
static _Thread_local int y;  /* thread duration + internal linkage */
```

Only valid at file scope, or at block scope combined with `static/extern`.

3.6 Initialization rules

The single most important rule

Static and thread duration objects are guaranteed zero-initialized. Automatic duration objects are *indeterminate* — reading them is undefined behavior.

"Zero-initialized" means recursively: integers ? 0, floats ? +0.0, pointers ? NULL, and every member/element of aggregates likewise.

```
STATIC storage is always zero-initialized:
s_uninit = 0
s_arr    = 0 0 0 0 0
s_str[0] = 0 (empty string: "")

AUTOMATIC storage is indeterminate:
uninitialized local  = -559038737 (0xDEADBEEF)
uninitialized arr[0] = 0xDEADBEEF arr[1] = 0xDEADBEEF
```

That `0xDEADBEEF` is not random — a function I called *earlier* wrote it into its stack frame, then returned. My uninitialized locals landed on the same bytes. **Stack garbage is leftover data from previous calls**, which is exactly why uninitialized reads can leak secrets (see CVE-2014-0160, Heartbleed).

GCC catches many cases, but **only with optimization on**:

```
warning: 'local' is used uninitialized [-Wuninitialized]
warning: 'arr' is used uninitialized [-Wuninitialized]
```

`-Wuninitialized` requires dataflow analysis, so it needs at least `-O1`. Compile your warning-check builds with `-O2 -Wall -Wextra` even if you ship `-O0` debug builds. For the cases GCC can't see, use **MemorySanitizer** (Clang: `-fsanitize=memory`) or **Valgrind**.

Just initialize everything. `int x = 0;` costs nothing after optimization.

Aggregate initialization

```
int arr1[5] = {1, 2};           /* rest ZERO-filled */
int arr2[5] = {0};             /* classic all-zeros idiom */
int arr3[5] = {};              /* C23 empty initializer */
struct P p1 = {1};             /* b and c zeroed */
struct P p2 = {.c = 9};        /* C99 designated */
int sparse[8] = {[0]=1, [7]=8}; /* C99 designated array */
```

```
arr1 = 1 2 0 0 0
arr2 = 0 0 0 0 0
arr3 = 0 0 0 0 0
p1   = 1 0 0
p2   = 0 0 9
sparse= 1 0 0 0 0 0 0 8
```

The rule: if you supply *any* initializer for an aggregate, every element you don't mention is zero-initialized. `{0}` works because you initialize the first element explicitly and the rest by rule.

?? This applies to **elements**, not padding bytes. `struct S s = {0};` does not guarantee padding is zeroed — so never `memcmp` two structs to compare them, and use `memset(&s, 0, sizeof s)` if you're about to write the struct to a file or socket.

Designated initializers (C99) are a major readability win, especially for config structs and lookup tables:

```
static const struct Config defaults = {
    .timeout_ms   = 5000,
    .max_retries  = 3,
    .verbose      = false,
}; /* everything else zeroed, order-independent, self-documenting */
```

String initialization

```
char s1[6] = "hi";      /* size 6: 'h','i','\0','\0','\0','\0' */
char s2[] = "hi";      /* size 3: compiler counts it */
char s3[2] = "hi";     /* size 2: 'h','i' – NO TERMINATOR. Legal C! */
```

```
sizeof s1=6 s2=3 s3=2
s1 bytes: 68 69 00 00 00 00
s3 bytes: 68 69    <- no 00 terminator
```

That third case is a **special exemption in the standard**: an array exactly the length of the string drops the terminator silently. Pass `s3` to `strlen` and you read off the end. GCC warns with `-Wunterminated-string-initialization` (GCC 15+); until then, always write `char s[] = "..."` and let the compiler size it.

What may appear in an initializer

- **Static/thread duration**: the initializer must be a **constant expression**, evaluated at compile time.

```
static int a = 5;          /* OK */
static int b = 5 * 3;      /* OK */
int n = get_n();
static int c = n;          /* ERROR: not a constant expression */
```

- **Automatic duration**: any expression, evaluated at runtime each time the block is entered.

```
void f(int n) { int arr_size = n * 2; int x = compute(); } /* fine */
```

Also note: a `const int` is **not** a constant expression in C (unlike C++). Use `#define`, `enum`, or C23 `constexpr` for compile-time constants.

3.7 Bugs that come from this material

Returning a pointer to a local

```
int *bad_return(void)
{
    int local = 42;
    return &local;      /* local dies at return */
}
const char *bad_array(void)
{
    char buf[] = "hello"; /* array on the stack */
    return buf;
}
```

```
const char *good_literal(void)
{
    return "hello";    /* string literal has STATIC duration – safe */
}
```

```
warning: function returns address of local variable [-Wreturn-local-addr]
warning: function returns address of local variable [-Wreturn-local-addr]

$ ./dangle
Segmentation fault
```

It didn't even print garbage — GCC saw the UB and returned NULL. Under ASan:

```
ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000
Hint: address points to the zero page.
```

Key distinction: `char buf[] = "hello"` copies the string into a stack array (dies). `return "hello"` returns a pointer into `.rodata` (lives forever, but is read-only).

Safe ways to return data:

1. Caller supplies the buffer: `void get_name(char *out, size_t n);` ? **preferred**
2. Return heap memory and document who frees it
3. Return a pointer to `static` storage — but then you're non-reentrant

The `static` in a header trap

```
/* util.h */
static int counter = 0;    /* ? WRONG */
```

This compiles without error, but **every .c file that includes it gets its own private copy**. Increment it in `a.c` and read it in `b.c` and you'll be baffled. (`-Wunused-variable` sometimes catches it; often it doesn't.)

Headers should contain: declarations, `#defines`, types, `static inline` functions. **Never variable definitions.**

Static initialization order

Within one TU, static objects are initialized in declaration order. **Across TUs, the order is unspecified.** Don't build initialization dependencies between globals in different files — use an explicit `init()` function.

3.8 The unified reference table

Declaration	Scope	Linkage	Duration
<code>int x;</code> (file)	file	external	static
<code>static int x;</code> (file)	file	internal	static
<code>extern int x;</code> (file)	file	external	static (defined elsewhere)
<code>const int x = 5;</code> (file)	file	external*	static
<code>int x;</code> (in function)	block	none	automatic
<code>static int x;</code> (in function)	block	none	static
<code>register int x;</code>	block	none	automatic (no address)
<code>int f(int x)</code> — the <code>x</code>	block	none	automatic
<code>_Thread_local int x;</code> (file)	file	external	thread
<code>static _Thread_local int x;</code>	file	internal	thread
<code>malloc(...)</code> result	—	—	allocated
<code>goto</code> label	function	none	—
prototype parameter name	prototype	none	—
string literal <code>"abc"</code>	—	—	static
compound literal <code>(int[]){1,2}</code> at block scope	block	none	automatic

* Note: in C, a file-scope `const` still has external linkage by default (unlike C++, where it defaults to internal). Add `static` if you want it private.

Practice

1. **Linkage forensics.** Create `mod.c` with one external variable, one static variable, one external function, one static function, and one function-local static. Compile with `-c` and run `nm`. Predict every letter before you look. Then explain why the function-local static appears the way it does.
2. **Reproduce the header bug.** Put `static int counter = 0;` in a header, include it from `a.c` and `b.c`, increment from `a.c`, print from `b.c`. Observe the wrong answer. Fix it with `extern` + one definition and confirm.
3. **.bss vs .data.** Build the two programs from §3.4 yourself and compare with `ls -l` and `size`. Then try `int arr[1000000] = {0};` — does your GCC keep it in `.bss`? Check with `size`.
4. **Stack garbage archaeology.** Write function `A` that fills a local array with a recognizable pattern, and function `B` that declares an uninitialized local array and prints it. Call `A()` then `B()`. Explain what you see. Now run it under `valgrind --track-origins=yes`.
5. **Dangling pointer trio.** Write three functions returning: (a) address of a local `int`, (b) a local `char buf[]`, (c) a string literal. Compile with `-Wall`, note which warn. Run under ASan.

6. **Thread-local counter.** Take the `tls.c` program from §3.4 and run it 20 times in a loop, recording how often `shared` is wrong. Then fix the race two ways: with `_Atomic int shared` and with an `mtx_t`. Compare the runtimes.
7. **Reasoning, no compiler:** what is the scope, linkage, and duration of `n` in each line?

```
static void f(int n) { static int n2 = 0; extern int n3; { int n = 1; } }
```

4. Operators & Expressions

- Arithmetic, relational, logical, bitwise, assignment, compound assignment
- Ternary, comma operator
- `sizeof`, `_Alignof`, `_Alignas`
- Precedence and associativity
- Short-circuit evaluation
- **Sequence points / evaluation order, and the UB it causes** (`i = i++ + 1`)
- Bit manipulation: masks, shifts, set/clear/toggle/test, bit tricks, bitfields vs manual bits

Module 4 — Operators & Expressions

Everything below is real GCC 13.3 output. Two sections here — **sequence points** and **precedence traps** — are where "code that looks right" silently isn't.

4.1 Arithmetic operators

`+` `-` `*` `/` `%` `++` `--` — plus unary `+` and `-`.

Integer division truncates toward zero

```
7 / 2 = 3      7 % 2 = 1
-7 / 2 = -3    -7 % 2 = -1
7 / -2 = -3    7 % -2 = 1
-7 / -2 = 3    -7 % -2 = -1
```

C99 standardized truncation toward zero. (C89 left the direction implementation-defined for negative operands — so ancient code may differ.)

The invariant the standard guarantees:

```
(a / b) * b + (a % b) == a
```

Which forces `%` to take the **sign of the dividend** (the left operand), not the divisor.

% is a remainder, not a mathematical modulo

```
-3 % 3 = 0
-2 % 3 = -2    ? mathematically, -2 mod 3 is 1
-1 % 3 = -1    ? mathematically, -1 mod 3 is 2
```

If you need a genuinely non-negative result (hash tables, circular buffers, ring indices):

```
int mod(int a, int m) { return ((a % m) + m) % m; }    /* -1 mod 3 ? 2 */
```

Also recall from Module 2: `INT_MIN % -1` and `INT_MIN / -1` are undefined behavior (and on x86 they raise SIGFPE), and division by zero is UB.

The integer-division-in-an-expression trap

```
a/b          = 1      (integer division happens first!)
(double)a/b  = 1.50
a/(double)b  = 1.50
(double)(a/b)= 1.00   ? too late, already truncated
```

Casting the *result* doesn't help. Cast an **operand** — that triggers the usual arithmetic conversions and makes the whole division floating-point.

++ and --

```
int a = ++i;    /* PRE: increment, then yield the NEW value */
int b = i++;    /* POST: yield the OLD value, then increment */
```

As a standalone statement, `i++` and `++i` are identical for scalars. In C there's no performance difference (unlike C++ with heavy iterator objects). **The real hazard is using them inside larger expressions** — see §4.7.

4.2 Relational, equality, and logical operators

`<` `>` `<=` `>=` and `==` `!=` yield `int` 0 or 1 — not `_Bool`. (In C++ they yield `bool`; another small difference.)

`&&` `||` `!` also yield `int` 0 or 1, and treat any nonzero operand as true.

Short-circuit evaluation — measured

```
&& : if left is 0, right is NEVER evaluated
0 && side(1) -> side called 0 times
```



```

1 && side(1)  -> side called 1 times

|| : if left is nonzero, right is NEVER evaluated
1 || side(2)  -> side called 0 times
0 || side(2)  -> side called 1 times

&, | (bitwise) DO evaluate both sides:
side(3) CALLED
0 & side(3)  -> side called 1 times  <-- !!

```

&& and **||** are **among the very few operators with guaranteed left-to-right evaluation and a sequence point between operands**. That guarantee is what makes the fundamental C safety idiom work:

```

if (p != NULL && *p == 'x')          /* *p is only reached if p is non-NULL */
if (i < n && arr[i] > 0)              /* arr[i] is only reached if in bounds */
if (fp && (line = fgets(buf, n, fp))) /* chained guards */

```

```

p was NULL, *p never dereferenced -> no crash
bounds check short-circuited, arr[5] never read

```

Order matters and is not commutative here. `*p == 'x' && p != NULL` crashes. Always put the guard first.

Bitwise `&|` do *not* short-circuit and have *no* sequence point between operands. Typing `&` when you meant `&&` compiles fine and evaluates the right side — potentially dereferencing a NULL pointer. Always the correct operator.

! and the double-negation idiom

`!x` yields 1 if `x == 0`, else 0. Hence `!!x` normalizes any value to exactly 0 or 1:

```

int is_set = !(flags & MASK);  /* 0 or 1, not 0 or 0x40 */

```

Useful when you need a canonical boolean from a mask test.

4.3 Assignment and compound assignment

`=` is an **operator**, not a statement. It yields the value assigned, which is why `a = b = c = 0` works (it's right-associative).

Compound assignment: `a op= b` is *not* `a = a op b`

Two real differences.

Difference 1 — the lvalue is evaluated only once:

```
arr[next()] += 10;      /* next() called ONCE */
arr[next()] = arr[next()] + 10; /* next() called TWICE – different elements! */
```

```
next() called
arr = 10 0, idx=1  (next() called once)
```

This makes compound assignment strictly safer with side-effecting subscripts.

Difference 2 — there's an implicit conversion back to the left type:

```
char c = 100; c += 100;      /* ? -56 */
unsigned char u = 200; u *= 2; /* ? 144 */
```

```
char c=100; c += 100; -> -56  (a = a + b would need a cast)
uchar u=200; u *= 2;  -> 144  (400 mod 256)
```

Formally, `a op= b` behaves as `a = (typeof a)(a op b)`. The arithmetic still happens in `int` (integer promotion), then the result is **silently truncated** back. `c = c + 100` would produce a `-Wconversion` warning; `c += 100` typically won't. Compound assignment hides narrowing.

The `=` vs `==` classic

```
if (x = 5) { } /* assigns 5, always true */
if (x == 5) { } /* compares */
```

GCC's `-Wparentheses` (in `-Wall`) warns unless you double-parenthesize to signal intent:

```
while ((c = getchar()) != EOF) { } /* intentional – extra parens */
if ((rc = do_work()) != 0) { }
```

Some codebases use "Yoda conditions" (`if (5 == x)`) so a typo becomes a compile error. Modern practice is to rely on the warning instead — it's less readable to invert everything.

4.4 Ternary and comma

Ternary `?:`

```
result = condition ? if_true : if_false;
```

Exactly one branch is evaluated, and there's a **sequence point after the condition** — so `p ? *p : 0` is safe.

The result type is the usual arithmetic conversion of BOTH branches, regardless of which one runs:

```
sizeof(1 ? 'a' : 'b') = 4  (char promoted to int)
sizeof(1 ? 1 : 2.0)   = 8  (int converts to double)
1 ? 1 : 2.0 prints as 1.0 (it IS a double)
```

Which brings back the Module 2 signedness trap:

```
(1 ? s : u) as %d = -1
(1 ? s : u) as %u = 4294967295
```

```
warning: operand of '?:' changes signedness from 'int' to 'unsigned int'
       due to unsignedness of other operand [-Wsign-compare]
```

Even though only the `s` branch is taken, the *type* of the whole expression is **unsigned int** because `u` is in the other arm.

Use it for: short value selections where an expression is required.

```
int max = a > b ? a : b;
printf("%d item%s\n", n, n == 1 ? "" : "s");
static const int limit = DEBUG ? 10 : 10000; /* if/else can't do this */
```

Don't use it for: side effects, or anything nested more than one level. `a ? b : c ? d : e` is legal (right-associative) but write an **if** chain instead.

The comma operator `,`

Evaluates the left operand, **discards its value**, then evaluates and yields the right. There **is** a sequence point between them.

```
left side runs
x = 42
sizeof((1,2.0)) = 8 (type of the RIGHT operand)
```

Its one genuinely idiomatic use is in **for loops:**

```
for (int i = 0, j = 9; i < j; i++, j--) { ... }
```

```
i=0 j=9 / i=1 j=8 / i=2 j=7 / i=3 j=6 / i=4 j=5
```

?? **Commas in function argument lists and declarations are *separators*, not the comma operator.** `f(a, b)` is two arguments; `f((a, b))` is one. Same character, different grammar.

It's also used inside multi-statement macros, but `do { } while(0)` is clearer (Module 12).

4.5 `sizeof`, `_Alignof`, `_Alignas`

`sizeof`

Yields a `size_t`. Two syntactic forms:

```
sizeof expression      /* no parens needed */
sizeof(type)           /* parens REQUIRED for a type name */
```

`sizeof` does not evaluate its operand — it's a compile-time operation on the *type*:

```
int i = 5;
printf("sizeof(i++) = %zu, i is still %d\n", sizeof(i++), i);
```

```
sizeof(i++) = 4, i is still 5
```

(The one exception: VLAs. `sizeof(vla)` is evaluated at runtime.)

The array-length idiom:

```
int arr[10];
size_t n = sizeof arr / sizeof arr[0];    /* ? 10 */
```

```
sizeof(arr)   = 40  (whole array)
sizeof(p)     = 8   (a pointer)
arr length    = 10
```

?? **This only works on a real array in scope.** Pass the array to a function and it decays to a pointer — `sizeof` gives you 8, silently. This is one of the most common bugs in C. Module 7 covers decay properly; for now: **always pass the length alongside the array.**

Safer macro (GCC/Clang, fails to compile on a pointer):

```
#define ARRAY_LEN(a) (sizeof(a) / sizeof((a)[0]) \
    + 0 * sizeof(struct { _Static_assert(!__builtin_types_compatible_p( \
        typeof(a), typeof(&(a)[0])), "ARRAY_LEN on a pointer"); int _; }))
```

Also prefer `sizeof *ptr` over `sizeof(Type)` in allocations — it survives a type change:

```
p = malloc(n * sizeof *p);      /* good: no type repeated */
p = malloc(n * sizeof(struct Widget)); /* breaks silently if p's type changes */
```

`_Alignof` / `alignof` (C11)

Yields the required alignment of a type. `<stdalign.h>` gives the lowercase spellings; C23 makes `alignof` a keyword.

```
alignof(char)=1 alignof(int)=4 alignof(double)=8 alignof(max_align_t)=16
```

`max_align_t` is the strictest alignment on the platform — what `malloc` guarantees.

`_Alignas` / `alignas` (C11)

Requests stronger alignment for an object or struct member.

```
struct A64 { alignas(64) char buf[8]; };
```

```
alignas(64) struct: sizeof=64 alignof=64
address % 64 = 0
```

Note the size grew to 64 — alignment forces the size to be a multiple of the alignment, so arrays stay correctly aligned.

Why you'd use it: cache-line alignment to avoid **false sharing** between threads, SIMD load requirements (16/32/64-byte), and DMA buffers on embedded targets.

Padding, previewed

```
struct S { char c; int i; char d; };
```

```
sizeof=12 alignof=4
offsetof c=0 i=4 d=8    <- padding inserted
packed version: sizeof=6
```

Three fields totalling 6 bytes occupy 12, because `int` needs 4-byte alignment. `__attribute__((packed))` removes the padding but creates **unaligned members**, which is slow on x86 and a **fault** on some ARM configurations. Full treatment in Module 11.

4.6 Precedence and associativity

Full table, highest to lowest:

Prec	Operators	Assoc
1	<code>() [] . -></code> postfix <code>++ --</code> compound literals	L?R
2	prefix <code>++ -- + - ! ~ * & sizeof _Alignof</code> casts	R?L
3	<code>* / %</code>	L?R
4	<code>+ -</code>	L?R
5	<code><< >></code>	L?R
6	<code>< <= > >=</code>	L?R
7	<code>== !=</code>	L?R
8	<code>&</code>	L?R
9	<code>^</code>	L?R
10	<code>\ </code>	L?R
11	<code>&&</code>	L?R
12	<code>\ \ </code>	L?R
13	<code>?:</code>	R?L
14	<code>= += -= *= ...</code>	R?L
15	<code>,</code>	L?R

The precedence bugs that actually happen

Trap 1 — bitwise operators bind *looser* than comparison. This is the big one.



```
flags & MASK == 0      /* parses as: flags & (MASK == 0) */
```

```
flags & MASK == 0  parses as flags & (MASK==0) = 0
(flags & MASK) == 0                                = 0
warning: suggest parentheses around comparison in operand of '&'
```

This is a historical wart — Ritchie has said he'd fix it if he could, but too much code depended on it by the time the problem was noticed. **Always parenthesize bitmask tests.**

Trap 2 — shifts bind looser than arithmetic:

```
1 << 2 + 3    parses as 1 << (2+3) = 32
(1 << 2) + 3    = 7
warning: suggest parentheses around '+' inside '<<'
```

Trap 3 — ?: binds very low:

```
x = 1 ? 2 : 3;      /* parses as x = (1 ? 2 : 3) ? 2, because = is even lower */
```

But mix it with other operators and it surprises you. Parenthesize.

Trap 4 — unary binds tighter than binary:

```
-3 % 2 = -1    (parses as (-3) % 2)
```

Trap 5 — pointer/increment combinations. Postfix ++ binds tighter than unary *:

```
v = *p++    -> v=10, *p now = 20    /* deref old p, then advance p */
v = (*p)++  -> v=10, a[0] now = 11  /* increment the VALUE */
v = *++p    -> v=20                  /* advance p, then deref */
v = ++*p    -> v=11, a[0] now = 11  /* deref, then increment the value */
```

*p++ is the canonical C string-copy idiom (`while ((*d++ = *s++)) ;`). Learn to read it; the other three are rare and deserve parentheses.

The professional rule: memorize only three tiers — unary > arithmetic > everything else — and parenthesize the rest. Nobody in code review is impressed that you know ^ binds tighter than |. Clarity wins. Compile with `-Wparentheses` (in `-Wall`) and fix every warning.

4.7 Sequence points & evaluation order ??

The single most misunderstood corner of C.

The rule

Between two sequence points, an object may be **modified at most once**, and if it is modified, any read of it must be **only for computing the value to be stored**. Violate this and the behavior is **undefined**.

C11 replaced "sequence points" with the more precise **sequenced-before / unsequenced / indeterminately-sequenced** model, but the practical consequences are identical.

Where sequence points exist

Guaranteed sequence point	Not a sequence point
&& between operands	+ - * / % between operands
\ \ between operands	& \ ^ between operands
?: after the condition	< > == between operands
, operator between operands	, separator in an argument list
End of a full expression (;)	Assignment operands relative to each other
Before a function call executes (after all args are evaluated)	The order the arguments themselves are evaluated

Note the asymmetry: **&&, \|, ?:, and the comma operator are the only operators with defined ordering**. Everything else — including **+** and **=** — leaves order unspecified.

The classics, with real output

```
i = i++ + 1    -> 1
i = ++i + i++ -> 3
i++ + i++     -> 1
a[i]=i++      -> a=0 0 0, i=1
f(i++, i++)   -> f(1, 0)
```

Every single line above is undefined behavior. GCC warned on all of them:

```
warning: operation on 'i' may be undefined [-Wsequence-point]
```

Why each is UB:

- **i = i++ + 1** — **i** is modified twice (by **++** and by **=**) with no sequence point between.
- **i++ + i++** — modified twice.
- **a[i] = i++** — **i** is read to compute the *subscript*, which is not "computing the value to be stored."

- `f(i++, i++)` — arguments are unsequenced relative to each other.

Important: this is not a case where "the compiler picks an order." UB means the compiler may do *anything* — including producing different results for the same expression in the same program, or optimizing based on the assumption it never happens. Don't reason about "what my compiler does."

Unspecified order is a separate problem

Even without UB, **the order of evaluation of function arguments and of most operands is unspecified:**

```
int r = add3(g(), h(), k());    /* g,h,k each print their name */
int s = g() + h() * k();
```

```
### -O0, -O1, -O2, -Os (all identical) ###
argument evaluation order: k h g    -> result 6
in an expression:         g h k    -> result 7
```

GCC evaluates function arguments right-to-left. MSVC also does; some ARM ABIs go left-to-right. There is no guarantee, and it can differ between two calls in the same program.

This is *unspecified behavior*, not UB — the program is valid, you just can't predict which order. But if your functions have side effects, your program is non-portable:

```
printf("%d %d\n", get_next(), get_next());    /* which value comes first? */
buf[pos++] = read_byte(&pos);                 /* ? unsequenced, UB */
```

How to stay safe — four rules

1. **One side effect per expression.** If a statement modifies something, that's all it does.
2. **Never use `++/--` inside a larger expression** unless it's the idiomatic `*p++` / `arr[i++]` where the variable appears exactly once.
3. **Split it up.** Sequence points are free:

```
/* BAD */ result = f(i++, i++);
/* GOOD */ int a = i++; int b = i++; result = f(a, b);
```

4. **Compile with `-Wsequence-point`** (included in `-Wall`) and treat those warnings as errors.

These are fine (each variable modified once, appears once):

```
i++;
arr[i++] = x;
*dst++ = *src++;                /* two different objects */
```

```
while ((c = getchar()) != EOF) /* sequence point at != ? No — but c is modified
once */
sum += arr[i++];
```

These are UB:

```
i = i++;
a[i] = i++;
f(i++, i++);
x = i++ + i;
printf("%d %d", i++, i++);
i = ++i;
```

4.8 Bit manipulation

The daily toolkit of systems and embedded work.

The operators

Op	Name	Note
&	AND	clear bits (with an inverted mask)
	OR	set bits
^	XOR	toggle bits
~	NOT	invert all bits (unary)
<<	left shift	multiply by 2 ⁿ
>>	right shift	divide by 2 ⁿ (see below)

The four core operations — memorize these

```
#define BIT(n)  (1u << (n))

flags |=  BIT(n);          /* SET    */
flags &= ~BIT(n);          /* CLEAR */
flags ^=  BIT(n);          /* TOGGLE */
if (flags & BIT(n))        /* TEST  */
```

```
start                0x00  00000000
SET    f |= (1<<3)   0x08  00001000
SET    f |= (1<<5)   0x28  00101000
TOGGLE f ^= (1<<3)   0x20  00100000
SET again            0x28  00101000
```

```
CLEAR    f &= ~(1<<5)    0x08    00001000
TEST     (f >> 3) & 1    = 1
TEST     !(f & (1<<3))= 1
```

Note the 1u. Use unsigned literals: `1 << 31` on a 32-bit signed `int` is UB (shifting into the sign bit), while `1u << 31` is fine. For 64-bit, use `1ULL << 63` or `UINT64_C(1) << 63`.

Multi-bit fields

```
uint8_t x = 0xA5;
x & 0x0F                                     /* extract low nibble ? 0x05 */
x >> 4                                       /* extract high nibble ? 0x0A */
(x & 0xF0) | 0x03                           /* replace low nibble with 3 ? 0xA3 */
(x >> shift) & ((1u << width) - 1) /* generic field extract */
```

Generic insert:

```
#define FIELD_SET(reg, shift, width, val) \
    ((reg) = ((reg) & ~(((1u << (width)) - 1) << (shift))) \
     | (((val) & ((1u << (width)) - 1)) << (shift)))
```

Shift semantics — the UB list

```
1u << 0..7 : 1 2 4 8 16 32 64 128
-8 >> 1 = -4 (arithmetic shift: sign preserved)
(unsigned)-8 >> 1 = 2147483644 (logical shift)
```

- **Right shift on a signed negative value is implementation-defined** (arithmetic in practice, on every mainstream compiler). Right shift on unsigned is always logical (zero-fill).
- **`x << n` or `x >> n` where `n >= width(x)` is UB.** `int x = 1; x << 32;` is undefined — and on x86 the hardware masks the count to 5 bits, so you get `x << 0 = 1`, not 0.
- **`n` negative is UB.**
- **Left-shifting a negative value is UB** in C99/C11 (defined in C23).
- Remember the **integer promotions**: `uint8_t v = 0xFF; v << 4` computes in `int`, giving `0xFF0`, not `0xF0`.

Classic tricks worth knowing

```
is power of 2?  n && !(n & (n-1))  for 40 -> 0, for 64 -> 1
clear lowest set bit  n & (n-1)    : 40 -> 32
isolate lowest set bit n & -n       : 40 -> 8
round up to next pow2 of 40         : 64
popcount(0xA5)=4  clz(1)=31  ctz(40)=3
```

```
swap without temp: a=7 b=3
abs via branchless: 9
```

```
n && !(n & (n - 1))      /* is n a power of two? */
n & (n - 1)              /* clear lowest set bit (Brian Kernighan's popcount
loop) */
n & -n                  /* isolate lowest set bit */
x & (align - 1)          /* offset within a power-of-2 alignment */
(x + align - 1) & ~(align - 1) /* round up to alignment */
x ^= y, y ^= x, x ^= y   /* XOR swap – a party trick, NOT faster, and broken if x
and y alias */
```

Prefer compiler builtins over hand-rolled loops — they compile to single instructions:

```
__builtin_popcount(x)      /* count set bits          ? POPCNT */
__builtin_clz(x)           /* count leading zeros     ? LZCNT/BSR (UB if x ==
0!) */
__builtin_ctz(x)           /* count trailing zeros     ? TZCNT/BSF (UB if x ==
0!) */
__builtin_bswap32(x)       /* byte swap – endian conversion */
```

C23 standardizes these in `<stdbit.h>`: `stdc_count_ones`, `stdc_leading_zeros`, `stdc_trailing_zeros`, `stdc_bit_ceil`, `stdc_has_single_bit`. Use those when you can target C23.

?? `__builtin_clz(0)` and `__builtin_ctz(0)` are undefined. Guard the zero case.

4.9 Bitfields vs manual bit manipulation

Bitfields

```
struct BF { unsigned a:1; unsigned b:3; unsigned c:4; unsigned d:24; };
```

```
sizeof(struct BF) = 4 (1+3+4+24 = 32 bits)
a=1 b=5 c=0xA d=0x123456 -> raw bytes = 0x123456AB
```

Convenient syntax — but the portability guarantees are almost nonexistent.

Everything the standard leaves unspecified

1. **Bit order within a storage unit.** Whether `a` occupies the low or high bit is implementation-defined.
Your struct will not match a hardware register or wire format across compilers.
2. **Whether a field may straddle a storage-unit boundary.**

- 3. **Alignment and total size** of the containing struct.
- 4. **Signedness of plain `int` bitfields:**

```
struct { int s:1; unsigned u:1; } t;
t.s = 1; t.u = 1;
```

```
int a:1 set to 1 reads as -1 ; unsigned a:1 reads as 1
```

You stored 1 and read back **-1**. A 1-bit signed field holds only 0 and -1. **Always write `unsigned` or `signed` explicitly — never plain `int` in a bitfield.**

- 5. **Overflow silently truncates:**

```
x.b = 9 -> reads back as 1      /* 9 doesn't fit in 3 bits */
warning: unsigned conversion ... changes value from '9' to '1' [-Woverflow]
```

- 6. **You cannot take the address of a bitfield** (`&x.a` is a compile error), which means no pointers, no `sizeof`, and awkward interactions with generic code.
- 7. **Not atomic.** Writing one bitfield may read-modify-write the whole storage unit, so two threads touching *different* bitfields in the same struct can race. (C11 fixed this only for fields separated by `:0`.)

The `unsigned :0;` unnamed zero-width field forces the next field into a new storage unit:

```
sizeof(struct BF2) = 8  (unnamed :0 forces alignment)
```

When to use which

Use bitfields for	Use manual masks for
Internal, single-compiler data compaction	Hardware registers
Readability where layout doesn't matter	Network protocols / file formats
Boolean flag clusters in a struct	Anything crossing a compiler or ABI boundary
	Code that must be portable
	When you need atomicity

The professional pattern for hardware and protocols — explicit shifts and masks, fully defined:

```
#define STATUS_READY_SHIFT    0
#define STATUS_READY_MASK    (1u << STATUS_READY_SHIFT)
#define STATUS_MODE_SHIFT    1
```

```
#define STATUS_MODE_MASK      (0x7u << STATUS_MODE_SHIFT)

static inline unsigned status_mode(uint32_t reg)
{ return (reg & STATUS_MODE_MASK) >> STATUS_MODE_SHIFT; }

static inline uint32_t status_set_mode(uint32_t reg, unsigned mode)
{ return (reg & ~STATUS_MODE_MASK) | ((mode << STATUS_MODE_SHIFT) &
STATUS_MODE_MASK); }
```

Verbose, but it produces identical bits on every compiler and every architecture. That's the trade you want at an interface boundary.

Module 4 rules

#	Rule
1	Cast an operand , not the result, to get float division
2	<code>%</code> follows the sign of the dividend — use <code>((a%m)+m)%m</code> for true modulo
3	Put the guard first in <code>&&</code> chains; never use <code>&</code> where you meant <code>&&</code>
4	Compound assignment silently narrows — watch <code>char/short</code> targets
5	Parenthesize every bitmask test: <code>(flags & MASK) == 0</code>
6	One side effect per expression; never <code>i</code> twice in one statement
7	Argument evaluation order is unspecified — GCC does right-to-left
8	Shift with unsigned types; <code>n >= width</code> is UB; <code>1u << 31</code> , not <code>1 << 31</code>
9	<code>__builtin_clz(0)</code> / <code>ctz(0)</code> are UB
10	Bitfields never for hardware or wire formats — use explicit shifts and masks

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Wsequence-point -Wparentheses -g -fsanitize=address,undefined prog.c -o prog
```

Practice

1. **Modulo table.** Write a program printing `i % 5` and `((i % 5) + 5) % 5` for `i` from -10 to 10. Explain where they diverge and why the second is what a hash table needs.
2. **Short-circuit proof.** Write `int noisy(const char *tag, int v)` that prints and returns `v`. Build expressions using `&&`, `||`, `&`, `|`, and `?:` that demonstrate exactly which operands get evaluated. Predict each before running.

3. **Sequence point hunt.** Take the six UB expressions from §4.7. Compile each with `-Wall` and record which ones GCC catches. Then rewrite each one to be well-defined while preserving the *intended* meaning.
4. **Precedence quiz, no compiler.** Fully parenthesize these, then verify:

```
a & b == c      *p++ = *q++      a << b + c
!x & y          a ? b : c ? d : e  x = y = z & w
~a & b | c      -a % b * c
```

5. **Bit toolkit.** Implement and test: `set_bit`, `clear_bit`, `toggle_bit`, `test_bit`, `extract_field(v, shift, width)`, `insert_field(v, shift, width, val)`, `count_bits` (both a naive loop and the `n & (n-1)` version), `is_power_of_2`, `next_power_of_2`. Compare your `count_bits` to `__builtin_popcount` for all 65536 16-bit values.
6. **Bitfield portability experiment.** Define a 32-bit bitfield struct, set known values, and dump the raw bytes with `memcpy`. Then compile with `-mbig-endian` if you have a cross-compiler, or just reason about what would change. Rewrite the same layout using explicit shifts and confirm the manual version gives identical, *predictable* bytes.
7. **Real-world register model.** Write an API for a hypothetical UART control register: bits 0–1 = parity (0=none,1=even,2=odd), bit 2 = stop bits, bits 3–6 = baud divisor, bit 7 = enable. Provide `static inline` getters and setters using masks and shifts, plus `_Static_asserts` that your masks don't overlap.

5. Control Flow

- `if/else`, `switch` (fallthrough, jump tables, Duff's device)
- `for`, `while`, `do-while`, nested loops
- `break`, `continue`, `goto` (and its *legitimate* use: cleanup chains)
- Labels, unreachable code

Module 5 — Control Flow

Real GCC 13.3 output throughout. This module contains the `goto` cleanup pattern — the single most important idiom in professional C, used everywhere in the Linux kernel, OpenSSL, curl, and SQLite.

5.1 `if / else`

```
if (expr) statement
if (expr) statement else statement
```

The condition is any scalar expression (integer, float, or pointer); nonzero means true. There is no `bool` requirement.

The three classic traps

```
TRAP 1: stray semicolon
    this ALWAYS runs, regardless of x

TRAP 2: missing braces + added line
    B <- runs even though y==0 (not part of the if!)

TRAP 3: dangling else binds to the NEAREST if
    (nothing printed – the else belonged to the inner if)
```

Trap 1 — `if (x > 0);` The `;` is a complete empty statement. The `if` is now a no-op and the "body" always executes.

Trap 2 — missing braces. Someone adds a second line to a braceless `if` and it silently escapes the conditional. This is exactly the [Apple "goto fail" SSL bug \(CVE-2014-1266\)](#) — a duplicated `goto fail;` line after a braceless `if` skipped certificate validation for every Apple device on the planet.

Trap 3 — the dangling `else`. The grammar says an `else` binds to the **nearest unmatched `if`**, regardless of indentation:

```
if (a)
    if (b) printf("both\n");
else printf("...\n");          /* belongs to `if (b)`, NOT `if (a)` */
```

GCC catches all three

```
warning: suggest braces around empty body in an 'if' statement [-Wempty-body]
warning: this 'if' clause does not guard... [-Wmisleading-indentation]
warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
```

`-Wmisleading-indentation` is in `-Wall`; it was added to GCC 6 specifically in response to goto-fail.

The rule that eliminates all three: always use braces, even for one-line bodies. This is mandatory in MISRA C, the Linux kernel style for multi-line bodies, and virtually every professional style guide.

`else if` is not a keyword

```
if (a) { } else if (b) { } else { }
```

is just `else` followed by a nested `if`. C has no `elif`. For more than ~3 branches on the same value, prefer `switch` or a lookup table.

5.2 switch

```
switch (integer_expression) {  
  case constant1:  
    statements  
    break;  
  case constant2:  
    statements  
    break;  
  default:  
    statements  
}
```

The mental model that makes **switch** make sense

switch is a computed goto, not a chain of ifs. It jumps *into* the middle of a block. Everything follows from this:

```
int n = 2;  
switch (n) {  
    printf("never printed\n");    /* no label — unreachable */  
  case 2:  
    printf("case 2 reached\n");  
}
```

```
warning: statement will never be executed [-Wswitch-unreachable]  
case 2 reached
```

Code before the first **case** label is **dead** — nothing can jump to it.

Requirements

- The controlling expression must be an **integer type** (or enum). No floats, no strings, no pointers.
- **case** labels must be **integer constant expressions** — no variables, no function calls.
- **Case labels must be unique** within a switch (a compile error, not a warning).
- Integer promotions apply to the controlling expression.
- **default** may appear anywhere, not just last. Only one per switch.

Fallthrough — the defining feature

```
switch (c) {  
  case 1: case 2: case 3:  
    printf("small ");  
    /* fall through */  
  case 4:
```

```

    printf("<=4 ");
    break;
case 5:
    printf("five ");
    /* NO break */
default:
    printf("other");
    break;
}

```

```

1: small <=4
2: small <=4
3: small <=4
4: <=4
5: five other      ? fell through into default
6: other

```

Without **break**, execution continues into the next case's code. Two distinct kinds:

1. **Grouped labels** (case 1: case 2: case 3:) — no code between them. Universally understood, no warning.
2. **Genuine fallthrough** — code, then no **break**. Almost always a bug.

GCC's **-Wimplicit-fallthrough** (in **-Wextra**) catches type 2:

```

warning: this statement may fall through [-Wimplicit-fallthrough=]

```

Notice it warned on **case 5** but **not** on my **/* fall through */** comment — GCC recognizes that exact comment form as intent. Better options:

```

case 3:
    do_something();
    __attribute__((fallthrough)); /* GCC/Clang */
    /* or, in C23: */
    [[fallthrough]];
case 4:

```

Style rule: every case ends in break, return, continue, goto, or an explicit fallthrough marker. No exceptions. Put **break** after **default** too — so that adding a case below it doesn't create a fallthrough bug.

switch on an enum — free exhaustiveness checking

```
enum Color { RED, GREEN, BLUE };
switch (c) {
case RED:    return "red";
case GREEN:  return "green";
/* BLUE missing */
}
```

```
warning: enumeration value 'BLUE' not handled in switch [-Wswitch]
```

This is a major professional technique. `-Wswitch` (in `-Wall`) tells you every place that needs updating when you add an enum value — but **only if you omit `default`**. Adding `default:` silences it.

```
/* GOOD: compiler tracks exhaustiveness for you */
switch (state) {
case IDLE:    ...; break;
case RUNNING: ...; break;
case DONE:    ...; break;
}
/* handle invalid values after the switch if needed */
```

Combine with `-Werror=switch` to make a missing case a build failure.

Declarations inside `switch` — a real gotcha

```
switch (k) {
case 1:
    ; /* C17 needs a statement before a declaration after a
    label */
    int x = 1;
    printf("x=%d\n", x);
    break;
case 2:
    printf("x is IN SCOPE here but UNINITIALIZED\n");
    break;
}
```

```
x is IN SCOPE here but UNINITIALIZED: skipped its initializer
```

The whole switch body is **one block**, so `x`'s scope covers `case 2` — but jumping directly to `case 2` **skips its initializer**. You get an in-scope, indeterminate variable.

Fix: give each case its own block.

```

case 1: {
    int x = 1;
    printf("x=%d\n", x);
    break;
}

```

(C23 relaxed the "label must precede a statement" rule, so the `;` hack is no longer needed there.)

Jump tables — what the compiler actually generates

Dense case values ? the compiler builds a **jump table**: one indexed indirect jump, $O(1)$ regardless of case count.

```

dense:
    cmp     edi, 7
    ja      .L2          ; range check ? default
    lea     rdx, .L4[rip] ; address of the table
    movsx   rax, DWORD PTR [rdx+rdi*4] ; table[x]
    add     rax, rdx
    jmp     rax          ; ONE indirect jump

.L4:      .long .L11-.L4 ; the table itself, in .rodata
         .long .L10-.L4
         .long .L9-.L4
         ...

```

Sparse case values ? a comparison chain or binary search:

```

sparse:
    cmp     edi, 1000
    je      .L5
    cmp     edi, 999999
    je      .L5
    cmp     edi, 1
    ...

```

Practical implications:

- A dense `switch` over 50 cases is genuinely faster than an `if/else if` chain. Prefer it for opcode dispatch, state machines, and parsers.
- Keep enum values contiguous starting near 0 so the compiler can build a table.
- Compilers are smart: my first dense test had returns of 10, 20, 30... and GCC recognized the arithmetic pattern and emitted `lea eax, 5[rdi+rdi*4]` — **no branch at all**. Always check the actual output before optimizing.

Duff's device

The famous 1983 loop-unrolling trick by Tom Duff, exploiting the fact that `case` labels can appear *inside* a nested loop:

```
void duff_copy(char *to, const char *from, size_t count)
{
    size_t n = (count + 7) / 8;
    switch (count % 8) {
    case 0: do { *to++ = *from++;
    case 7:      *to++ = *from++;
    case 6:      *to++ = *from++;
    case 5:      *to++ = *from++;
    case 4:      *to++ = *from++;
    case 3:      *to++ = *from++;
    case 2:      *to++ = *from++;
    case 1:      *to++ = *from++;
               } while (--n > 0);
    }
}
```

```
copied 21 bytes: "ABCDEFGHJKLMNOPQRSTU"
matches source?  yes
```

It works: the `switch` jumps into the middle of the `do-while` to handle `count % 8` leftovers, then the loop handles the rest 8 at a time. It compiles cleanly with `-Wall`.

Do not use it. It's here so you can *recognize* it in old code and explain it in an interview. Modern compilers unroll loops far better than you can by hand, and `memcpy` is a single vectorized instruction sequence. Duff's device today typically makes code *slower* by defeating the optimizer.

Its real historical significance: it proved `switch` and loops are independent grammatical constructs that can interleave arbitrarily.

5.3 Loops

The three forms

```
for (init; condition; increment) body
while (condition) body
do body while (condition);      /* note the semicolon */
```

`for (;;)` and `while (1)` are both infinite loops. `for (;;)` is slightly preferred — some old compilers warned about a constant condition in `while (1)`.

do-while runs the body at least once, testing at the bottom:

```
while(k<5) with k=100: body ran 0 times
do-while body ran once even though k=100
```

Its main real uses: menu/retry loops, and the `do { } while(0)` macro idiom (Module 12).

Half-open ranges — write loops this way

```
for (size_t i = 0; i < n; i++)      /* [0, n) */
```

```
10 20 30 40 50
count = n - 0 = 5, no off-by-one possible
```

Why half-open `[start, end)` is the correct default:

- The count is exactly `end - start` — no `+1` to forget.
- An empty range is `start == end`, not `end < start`.
- Adjacent ranges join cleanly: `[0,n)` and `[n,m)`.
- It matches how `strlen`, `memcpy`, and every C API works.

Use `<`, never `<=`, with a count. `i <= n` is the single most common off-by-one source.

Reverse iteration with `size_t`

From Module 3: `for (size_t i = n-1; i >= 0; i--)` is an infinite loop. Three correct forms, all verified:

```
for (size_t i = n; i-- > 0; )      printf("%d ", arr[i]);  /* A: idiomatic */
{ size_t i = n; while (i > 0) { i--; printf("%d ", arr[i]); } } /* B: explicit */
for (size_t i = n; i > 0; i--)      printf("%d ", arr[i-1]); /* C: offset */
```

```
A: 50 40 30 20 10
B: 50 40 30 20 10
C: 50 40 30 20 10
```

Form **A** is the standard C idiom. Read `i-- > 0` as "i goes down to zero": it tests the old value, then decrements, so the body sees `n-1` down to `0`.

Loop traps

Stray semicolon:

```
for (size_t i = 0; i < n; i++);
    sum += 1;
```

```
warning: this 'for' clause does not guard... [-Wmisleading-indentation]
    sum = 1 (loop body was empty!)
```

Modifying the loop variable inside:

```
for (int i = 0; i < 3; i++) { printf("i=%d ", i); if (i==1) i++; }
```

```
i=0    i=1        ? i=2 skipped entirely
```

Legal, occasionally intentional, usually a bug. If you need to skip, use `continue` with the condition in the header.

Float loop counters — never do this:

```
for (double x = 0.0; x != 1.0; x += 0.1) /* never terminates (Module 2) */
for (int i = 0; i < 10; i++) { double x = i * 0.1; } /* correct */
```

Recomputing an invariant in the condition:

```
for (size_t i = 0; i < strlen(s); i++) /* O(n²) – strlen runs every iteration */
size_t len = strlen(s);
for (size_t i = 0; i < len; i++)      /* O(n) */
```

(The optimizer *sometimes* hoists it, but only if it can prove `s` isn't modified. Don't rely on it.)

Declaration in the `for` header (C99+)

```
for (int i = 0; i < n; i++) { } /* i's scope ends with the loop */
```

Always do this. It's the tightest possible scope, prevents accidental reuse, and eliminates a whole class of bugs.

5.4 `break` and `continue`

break exits only the innermost enclosing loop or **switch**

```
break only exits the INNERMOST loop:
outer i=0 still runs
outer i=1 still runs
outer i=2 still runs
```

And inside a **switch** inside a loop, **break** exits the **switch**:

```
for (int x=0; x<4; x++) { switch(x) { case 2: break; default: printf("%d ",x); } }
```

```
0 1 3 <- 2 skipped but loop continued
```

This trips people up constantly. If you need to leave the loop from inside a switch, use **goto** or a flag.

continue — different in **for** vs **while**

In a **for** loop, **continue** still runs the increment:

```
0 1 3 4          /* i==2 skipped, loop still advanced */
```

In a **while** loop, **continue** skips everything — including your manual increment:

```
int i = 0;
while (i < 5) {
    if (i == 2) continue; /* i never increments ? INFINITE LOOP */
    printf("%d ", i);
    i++;
}
```

```
0 1 (would spin forever; bailing)
```

Rule: if a **while** loop body contains **continue**, put the increment at the top or convert to a **for** loop.

Breaking out of nested loops

Way 1 — flag variable (verbose, error-prone, adds a condition to every loop header):


```
int found = 0;
for (int i=0; i<3 && !found; i++)
    for (int j=0; j<4; j++)
        if (m[i][j]==target) { found=1; fi=i; fj=j; break; }
```

Way 2 — `goto` (idiomatic C, and what the kernel does):

```
for (int i=0; i<3; i++)
    for (int j=0; j<4; j++)
        if (m[i][j]==target) { fi=i; fj=j; goto found_it; }
printf("not found\n");
goto done;
found_it:
printf("found at [%d][%d]\n", fi, fj);
done:
```

```
found at [1][2] - clean single exit
```

Way 3 — extract the loop into a function and `return`. Often the cleanest of all, and it makes the search testable in isolation.

5.5 `goto` — and when it's actually correct

Dijkstra's 1968 letter "Go To Statement Considered Harmful" left a lasting taboo. In C, that taboo is **wrong in one specific, extremely important case.**

The rules

- `goto` targets a **label**, which has **function scope** (Module 3) — visible anywhere in the function, even before its declaration.
- You **cannot** jump between functions (that's `setjmp/longjmp`).
- You **cannot** jump into the scope of a VLA.
- You **can** jump into a block — which is how Duff's device and the `switch`-into-loop trick work.

The legitimate use: cleanup chains

C has no destructors, no RAII, no `finally`. When a function acquires several resources and any step can fail, you must release everything acquired so far — in reverse order — on every failure path.

Without `goto`: deeply nested `ifs`, or duplicated cleanup code at every early return. Both are where leaks hide.

With `goto`: one cleanup chain, each label unwinding one more level.

```

int process(const char *path, int fail_at)
{
    FILE *fp = NULL; char *buf1 = NULL, *buf2 = NULL;
    int rc = -1;                                /* pessimistic default */

    fp = fopen(path, "r");
    if (!fp) goto out;                          /* nothing acquired yet */
    if (fail_at == 1) goto out_file;

    buf1 = malloc(1024);
    if (!buf1) goto out_file;
    if (fail_at == 2) goto out_buf1;

    buf2 = malloc(4096);
    if (!buf2) goto out_buf1;
    if (fail_at == 3) goto out_buf2;

    /* ... real work ... */
    rc = 0;                                    /* success */

out_buf2: free(buf2);
out_buf1: free(buf1);
out_file: fclose(fp);
out:      return rc;
}

```

Verified across all four paths:

```

--- fail_at=0 ---   acquired: file, buf1, buf2 ? released: buf2, buf1, file ? rc=0
--- fail_at=1 ---   acquired: file                ? released: file                ?
rc=-1
--- fail_at=2 ---   acquired: file, buf1           ? released: buf1, file           ?
rc=-1
--- fail_at=3 ---   acquired: file, buf1, buf2 ? released: buf2, buf1, file ?
rc=-1

$ ASAN_OPTIONS=detect_leaks=1 ./cleanup
ASan/LSan: CLEAN - zero leaks on all 4 paths

```

An honest mistake worth showing you

My *first* version of this had `fail_at == 3` jump to `out_buf1` instead of `out_buf2`. LeakSanitizer caught it immediately:

```

==643==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 4096 byte(s) in 1 object(s)
Direct leak of 1024 byte(s) in 1 object(s)
SUMMARY: AddressSanitizer: 5120 byte(s) leaked in 2 allocation(s).

```

That's the pattern's one real hazard: **jumping to the wrong label leaks the resource you just acquired**. Two things protect you:

1. **Name labels after what they release**, not `err1/err2`. `out_buf2` obviously frees `buf2`.
2. **Run LSan/Valgrind on your error paths**, not just the happy path. Fault injection (like my `fail_at` parameter) is a genuine testing technique — inject failures at every allocation and verify zero leaks.

Why this pattern is correct, not a compromise

- **One exit point.** Easy to add logging, tracing, or a debugger breakpoint.
- **Cleanup written once.** Adding a fifth resource means one new label, not five new `free` calls scattered around.
- **The order is visually enforced.** Labels appear in reverse acquisition order.
- **It's what everyone does.** `grep -c "goto out" linux/` returns tens of thousands of hits. OpenSSL, curl, SQLite, glibc — same pattern.

Supporting conventions

```
FILE *fp = NULL;           /* initialize to a safe sentinel */
int rc = -1;               /* default to failure – success is explicit */

/* free(NULL) and fclose on NULL-checked pointers are safe, so uniform
   labels work even when some resources were never acquired */
out:
    free(buf);              /* free(NULL) is a guaranteed no-op */
    if (fp) fclose(fp);     /* fclose(NULL) is NOT safe – check it */
    return rc;
```

A common simplification is a **single** cleanup label that frees everything, relying on `free(NULL)` being a no-op:

```
int rc = -1;
char *a = NULL, *b = NULL, *c = NULL;
if (!(a = malloc(N))) goto out;
if (!(b = malloc(N))) goto out;
if (!(c = malloc(N))) goto out;
rc = 0;
out:
    free(c); free(b); free(a);
    return rc;
}
```

Simpler, but only works when every cleanup is NULL-safe. Use the chained version for things like `fclose`, `close`, `pthread_mutex_unlock`, or `munmap` that aren't.

Where `goto` is *not* acceptable

- Jumping backwards to build a loop — use a loop
- Jumping into the middle of an unrelated block
- Skipping over initializations
- As a substitute for restructuring genuinely tangled logic

The professional consensus: `goto` forward, to a cleanup label at the end of the function, is correct C. Everything else needs a strong justification.

5.6 Labels and unreachable code

Labels

```
identifier: statement
```

Three kinds: ordinary `goto` labels (function scope), `case` labels, and `default`. A label must be followed by a statement in C17 — hence the `;` hack before a declaration. **C23 removes this restriction**, so `label: }` is now legal.

Unreachable code

```
int f(int x)
{
    if (x > 0) return 1;
    else      return 0;
    printf("unreachable\n");    /* dead */
    return 2;
}
```

Dead code is usually harmless but signals a bug: a misplaced `return`, an inverted condition, or a leftover from a refactor.

Note: GCC's `-Wunreachable-code` is currently a **no-op** (it was removed because the optimizer made it too unreliable). Clang implements it. Static analyzers (`clang-tidy`, `cppcheck`, Coverity) catch it reliably.

Falling off the end of a non-void function

```
int g(int x)
{
    switch (x) { case 1: return 1; case 2: return 2; }
    /* no return */
}
```

```
warning: control reaches end of non-void function [-Wreturn-type]

g(99)=0  <- read a garbage register, UB
```

If the caller uses the value, this is undefined behavior. It "worked" here by accident — `eax` happened to hold 0.

Make this an error: `-Werror=return-type`. It costs nothing and catches a real UB class. (Remember `main` is the one exception — C99+ implicitly returns 0.)

`_Noreturn` / `[[noreturn]]`

Tells the compiler a function never returns, enabling better optimization and suppressing spurious "control reaches end" warnings:

```
_Noreturn void die(const char *msg)          /* C11 */
{ fprintf(stderr, "fatal: %s\n", msg); exit(1); }

[[noreturn]] void die(const char *msg);      /* C23 */
```

Standard library examples: `exit`, `abort`, `_Exit`, `longjmp`, `thrd_exit`. Calling one from inside a function means the code after it is genuinely unreachable, and the compiler knows.

Also useful: `__builtin_unreachable()` (GCC/Clang) and C23's `unreachable()` from `<stddef.h>` — a promise that a path is never taken, which lets the compiler delete the check entirely. **Reaching it is UB**, so use it only where you're certain.

Module 5 rules

#	Rule
1	Always brace <code>if/else</code> /loop bodies, even one-liners
2	Watch for stray <code>;</code> after <code>if (...)</code> , <code>for (...)</code> , <code>while (...)</code>
3	Every <code>case</code> ends in <code>break/return</code> /explicit fallthrough marker — including <code>default</code>
4	Omit <code>default</code> when switching on an enum, so <code>-Wswitch</code> enforces exhaustiveness
5	Give each <code>case</code> its own <code>{ }</code> block if it declares variables
6	Use half-open ranges <code>[0, n)</code> and <code><</code> , never <code><=</code> with a count
7	<code>for (size_t i = n; i-- > 0;)</code> for reverse iteration
8	<code>continue</code> in a <code>while</code> loop skips your increment — convert to <code>for</code>
9	<code>break</code> inside a <code>switch</code> inside a loop exits the switch, not the loop
10	<code>goto</code> forward to a cleanup label is correct C — name labels after what they release

Rule

- 11 `-Werror=return-type` — falling off a non-void function is UB

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
  -Wimplicit-fallthrough -Wempty-body -Wdangling-else \
  -Werror=return-type -Werror=switch \
  -g -fsanitize=address,undefined prog.c -o prog
```

Practice

1. **Reproduce goto-fail.** Write a function that performs three validation checks, then deliberately duplicate a line after a braceless `if` so validation is skipped. Compile with `-Wall` and confirm `-Wmisleading-indentation` catches it. Then fix it with braces.
2. **Jump table forensics.** Write two switches — one with cases 0–15, one with cases 1, 100, 10000. Dump both with `gcc -O2 -S -masm=intel`. Find the jump table in `.rodata` for the dense one. Then add a `case 16:` and see if the table grows.
3. **Enum exhaustiveness.** Define an enum with 5 states and a `switch` handling all 5 with no `default`. Add a 6th enum value and confirm you get a warning at every switch site. Now add `default:` and observe the warning disappear — this is the trade-off.
4. **The switch-declaration trap.** Reproduce the uninitialized-`x` bug from §5.2. Run it under `valgrind` or `MSan`. Then fix it with per-case blocks.
5. **Reverse iteration, three ways.** Implement `reverse_print` on a `size_t`-indexed array using all three idioms from §5.3, plus the broken `i >= 0` version. Run the broken one under `ASan` and read the report.
6. **Build a real cleanup chain.** Write `int copy_file(const char *src, const char *dst)` that: opens `src`, opens `dst`, allocates a buffer, copies in chunks, and cleans up correctly on every failure. Add a `fail_at` parameter to inject failures at each step. Verify with `-fsanitize=address,leak` that **all** paths leak nothing. Then deliberately point one `goto` at the wrong label and confirm `LSan` catches it.
7. **Multi-level break, three ways.** Write a 3D array search that returns the coordinates of a target. Implement it with (a) flag variables, (b) `goto`, (c) extraction into a function with `return`. Compare readability and count the lines.
8. **Reasoning, no compiler:** what does this print, and why?

```
int i = 0;
switch (i) {
default: printf("D");
case 0:  printf("A");
case 1:  printf("B"); break;
```

```
case 2: printf("C");
}
```

6. Functions

- Prototypes, definitions, forward declarations
- Pass-by-value semantics (C has no pass-by-reference)
- Recursion, tail calls, stack depth limits
- Variadic functions: `stdarg.h`, `va_list`, `va_start/arg/end/copy`
- `inline`, `static inline`, `_Noreturn`
- **Function pointers**, callbacks, dispatch tables
- Calling conventions (`cdecl`, `stdcall`, `fastcall`), ABI basics

Module 6 — Functions

Real GCC 13.3 output throughout. **Function pointers** (§6.6) are the mechanism behind every polymorphic design in C — plugin systems, state machines, the kernel's `file_operations`, `qsort`. Learn that section cold.

6.1 Prototypes, definitions, forward declarations

```
int add(int a, int b);           /* declaration (prototype) */
int add(int a, int b) { return a+b; } /* definition */
```

A **prototype** is a declaration that specifies parameter types. It's what lets the compiler type-check calls and convert arguments.

What happens without one — real corruption

```
/* lib.c */ double add(double a, double b) { return a + b; }
/* use.c */ printf("add(2,3) = %d\n", add(2, 3)); /* no prototype */
```

```
### no prototype: compiles, links, CORRUPTS ###
add(2,3) = 0

### with the prototype present ###
add(2,3) = 5.000000
```

Zero. The caller passed two `ints` in integer registers; the callee read two `doubles` from XMM registers. Nothing crashed, nothing complained — the answer was just wrong. This class of bug is why prototypes exist.

The warning that would have caught it:

```
warning: implicit declaration of function 'add' [-Wimplicit-function-declaration]
```

The standards timeline

Standard	Calling an undeclared function
C89	Legal — implicitly declared as <code>int f()</code>
C99	Removed from the language. GCC warns by default.
C23	Hard error

Make it an error today: `-Werror=implicit-function-declaration`. In practice a missing prototype almost always means a missing `#include`.

`()` vs `(void)` — the K&R legacy

```
void f();           /* pre-C23: UNSPECIFIED parameters – NO type checking */
void g(void);       /* takes NOTHING */
f(1, 2, 3);         /* legal in C17! */
```

```
-- C17 -- warning: function declaration isn't a prototype [-Wstrict-prototypes]
-- C23 -- error: too many arguments to function 'f'
```

`void f();` is a **non-prototype declaration** — a K&R holdover meaning "I'm not telling you the parameters." The compiler checks nothing.

C23 finally fixed this: `()` now means `(void)`. Until you can rely on C23:

Always write `(void)` for no-parameter functions, and compile with `-Wstrict-prototypes`.

Forward declarations

C compiles top-to-bottom in a single pass, so a function must be declared before use. Two situations:

Mutual recursion:

```
static bool is_odd(unsigned n);           /* forward declaration */
static bool is_even(unsigned n) { return n == 0 ? true : is_odd(n-1); }
static bool is_odd (unsigned n) { return n == 0 ? false : is_even(n-1); }
```

Public API in a header, definition in the `.c`:


```

/* parser.h */
Token *parse(const char *src);      /* declaration — the promise */

/* parser.c */
static Token next(Lexer*);          /* private forward declaration */
Token *parse(const char *src) { ... }
static Token next(Lexer *lx) { ... }

```

Convention: prototypes in the header (or at the top of the `.c` for `static` helpers), definitions in whatever order reads best. Don't order your file to avoid forward declarations — declare and write in logical order.

6.2 Pass-by-value — C has no pass-by-reference

Every argument in C is copied. There are no exceptions. This one sentence explains a huge amount of C behavior.

```

swap_broken: x=1 y=2  (unchanged)
swap_ok      : x=2 y=1

```

```

void swap_broken(int a, int b) { int t=a; a=b; b=t; } /* swaps the COPIES */
void swap_ok(int *a, int *b)  { int t=*a; *a=*b; *b=t; }

```

The "workaround" is to pass a **pointer by value** — you copy the address, but both copies point at the same object. C++ has real references; C simulates them with pointers, and the simulation is always visible.

Structs are copied wholesale

```

struct Big { int data[256]; };      /* 1 KB */
void by_value(struct Big s) { s.data[0] = 99; }
void by_ptr(struct Big *s)  { s->data[0] = 99; }

```

```

by_value : b.data[0]=0  (copy modified, original untouched)
by_ptr   : b.data[0]=99
          sizeof(struct Big) = 1024 bytes COPIED on every by-value call

```

Guideline: pass structs larger than ~2 pointer-widths (16 bytes on x86-64) by `const` pointer. Small structs by value are fine and often *faster*, since they can travel in registers.

```

void draw(const struct Rect *r);    /* large: pointer, const documents read-only */

```

```
Point midpoint(Point a, Point b); /* small (8-16 bytes): by value is idiomatic
*/
```

Arrays look like an exception — they aren't

```
void takes_array(int arr[100], size_t *sz) { *sz = sizeof arr; arr[0] = 77; }
```

warning: 'sizeof' on array function parameter 'arr' will return size of 'int *'

```
sizeof(arr) in main      = 400
sizeof(arr) in function = 8  <-- it is a POINTER there
arr[0] modified through the pointer = 77
```

An array parameter is silently rewritten as a pointer. These four declarations are *identical*:

```
void f(int arr[100]);
void f(int arr[10]); /* the 100 and 10 are ignored entirely */
void f(int arr[]);
void f(int *arr);
```

So arrays *appear* to be passed by reference, but only because the array **decays** to a pointer at the call site. The pointer itself is still copied. Full treatment in Module 7.

Consequence: `sizeof` on an array parameter is always the pointer size. Always pass the length explicitly:

```
void process(int *arr, size_t n); /* the only correct signature */
void process(int arr[static 10], size_t n); /* C99: hints at least 10 elements */
```

To modify the pointer itself, pass a pointer-to-pointer

```
reassign_ptr_broken: *p=1 (pointer itself was a copy)
reassign_ptr_ok      : *p=5
```

```
void reassign(int **p) { *p = &other; } /* now the caller's pointer changes */
reassign(&p);
```

This is why `realloc`-style APIs take `char **buf`, and why `free_and_null(void **p)` exists.

Return values are copied too

```
struct Point make(int x, int y) { struct Point p = {x,y}; return p; }    /* safe */
int *bad(void) { int local = 5; return &local; }                        /*
dangling! */
```

Returning a **struct by value** is safe (it's copied). Returning a **pointer to a local** is not (Module 3).

6.3 Recursion, tail calls, stack depth

Every call consumes stack

```
stack limit = 8192 KB
depth 50000, stack used 1562 KB
depth 100000, stack used 3124 KB
depth 150000, stack used 4687 KB
depth 200000, stack used 6249 KB
depth 250000, stack used 7812 KB
Segmentation fault
```

~**32 bytes per frame** for a trivial function, and it died at roughly **262,000 frames** with the default 8 MB stack. Add a few local arrays and that number collapses:

```
void f(void) { char buf[4096]; ...; f(); }    /* ~2000 frames before overflow */
```

Stack overflow is not a clean error

A blown stack gives you **SIGSEGV with no message**. Under ASan you at least get a diagnosis:

```
ERROR: AddressSanitizer: stack-overflow on address 0x7ffdb1979ff8
```

Typical stack limits: **8 MB** default on Linux (`ulimit -s`), **1 MB** on Windows, **512 KB** for non-main pthreads by default, and often **just a few KB** on embedded MCUs.

In production C, unbounded recursion is a denial-of-service vulnerability. Recursive descent parsers on untrusted input must have a depth counter. This is a recurring CVE pattern in JSON/XML/protobuf parsers.

Tail calls

A **tail call** is a call in the last position — its result is returned directly, with nothing left to do afterward:

```
unsigned long sum_tail(unsigned long n, unsigned long acc)
{ if (n == 0) return acc; return sum_tail(n-1, acc+n); }    /* tail position */
```

```
unsigned long fact(unsigned long n)
{ if (n <= 1) return 1; return n * fact(n-1); }           /* NOT tail – multiply
pending */
```

The current frame is dead at a tail call, so the compiler can **reuse it**: no stack growth, a `jmp` instead of `call`.

```
### -O0 (no TCO -> 10 million stack frames) ###
Segmentation fault      exit=139

### -O2 (tail call turned into a loop) ###
sum_tail(10,000,000) = 50000005000000      exit=0
```

Same source. One crashes, one works.

Three things to internalize:

1. **The C standard does not require tail-call optimization.** It's an optimization, not a guarantee.
2. **It requires optimization to be on.** `-O0` gives you no TCO, and `-foptimize-sibling-calls` alone wasn't enough here — it needs the surrounding `-O1+` machinery. So debug builds crash where release builds don't.
3. **Compilers are cleverer than the textbook rule suggests.** At `-O2` GCC also converted my *non*-tail factorial into a loop, because multiplication is associative and it could restructure the accumulation.

Practical rule: never rely on TCO in C. If depth could exceed a few thousand, write the loop yourself or use an explicit stack (an array + index). Recursion is for trees and genuinely recursive structures where depth is logarithmic.

```
/* recursive – depth O(n), fine for a balanced tree, dangerous for a list */
size_t count(Node *n) { return n ? 1 + count(n->next) : 0; }

/* iterative – depth O(1), always safe */
size_t count(Node *n) { size_t c=0; for (; n; n = n->next) c++; return c; }
```

6.4 Variadic functions — `<stdarg.h>`

```
#include <stdarg.h>

int sum(int n, ...)           /* at least one named parameter is REQUIRED
*/
{
    va_list ap;
    va_start(ap, n);          /* n = the LAST named parameter */
    int total = 0;
    for (int i = 0; i < n; i++)
```

```
        total += va_arg(ap, int);    /* you must state the type – nothing checks
it */
        va_end(ap);                /* mandatory */
        return total;
    }
```

The macros

Macro	Purpose
<code>va_list ap;</code>	The cursor
<code>va_start(ap, last_named)</code>	Initialize. C23 makes the second argument optional.
<code>va_arg(ap, TYPE)</code>	Fetch the next argument as TYPE . Advances the cursor.
<code>va_end(ap)</code>	Clean up. Required — may be a no-op or may not.
<code>va_copy(dst, src)</code>	C99. Duplicate a cursor for a second pass.

The fundamental problem: there is no type information

`va_arg(ap, int)` doesn't *check* anything — it reinterprets the bytes at the cursor as an `int` and advances by that size. Get the type wrong and you get garbage plus a corrupted cursor. **This is undefined behavior with no diagnostic.**

So a variadic function must be told how many arguments there are, out of band. Two conventions:

```
sum_count(4, 1,2,3,4)    /* explicit count – first arg says how many */
sum_sentinel(1,2,3,4,-1) /* sentinel value terminates the list */
```

```
sum_count(4, 1,2,3,4)    = 10
sum_sentinel(1,2,3,4,-1) = 10
```

`printf` uses a third: the **format string** encodes both the count and the types.

?? **Sentinel pitfall:** `execl(path, "a", "b", NULL)` needs `(char*)NULL`, not bare `NULL`. On many platforms `NULL` is `0` (an `int`, 4 bytes) while the function reads a `char*` (8 bytes) — instant corruption. GCC's `__attribute__((sentinel))` catches this.

Default argument promotions

Arguments in the variadic part are automatically promoted:

- `char, short, _Bool` ? `int`
- `float` ? `double`
- Arrays and functions ? pointers

Consequences:

```
va_arg(ap, char)    /* UNDEFINED – never a char, always promoted to int */
va_arg(ap, float)   /* UNDEFINED – always promoted to double */
va_arg(ap, int)     /* correct for char/short/bool arguments */
va_arg(ap, double)  /* correct for float arguments */
```

This is also why `printf("%f", some_float)` works and why there is no `%lf` in `printf` — every float is already a double by the time `printf` sees it.

`va_copy` — required for a second traversal

A `va_list` is a one-way cursor. Traversing twice requires a copy:

```
va_list ap, ap2;
va_start(ap, n);
va_copy(ap2, ap);          /* independent cursor */
... firstpass with ap ...
va_end(ap);
... second pass with ap2 ...
va_end(ap2);
```

```
max_and_sum(5, 3,9,2,7,4):
    max=9 sum=25
```

Every `va_copy` needs its own `va_end`. And on x86-64, `va_list` is an array type — passing it to a helper function may or may not consume it, so always `va_copy` before delegating.

Writing `printf`-style wrappers

Never re-implement formatting. Use the `v`-prefixed variants that take a `va_list`:

`vprintf`, `vfprintf`, `vsprintf`, `vsnprintf`, `vasprintf` (GNU).

```
void logmsg(const char *level, const char *fmt, ...)
{
    va_list ap; va_start(ap, fmt);
    printf("[%s] ", level);
    vprintf(fmt, ap);          /* NOT printf(fmt, ap) – that passes the va_list
itself */
    va_end(ap);
}
```

```
[INFO] user alice id 42
```

Get compile-time checking with **format**

GCC/Clang can type-check your custom format functions:

```
__attribute__((format(printf, 2, 3)))
void logmsg(const char *level, const char *fmt, ...);
/*          ^fmt is arg 2, varargs start at arg 3 */
```

```
logmsg("X", "%d %s\n", "not_an_int", 42);
```

```
warning: format '%d' expects argument of type 'int', but argument 3 has type 'char
*'
warning: format '%s' expects argument of type 'char *', but argument 4 has type
'int'
```

Put this attribute on every logging function you write. It converts a whole class of runtime crashes into compile-time warnings — and format-string mismatches are both a crash source and a security vulnerability class.

?? **Never pass user input as the format string.** `printf(user_input)` is the classic **format string vulnerability**: `%n` writes to memory, `%s` reads arbitrary pointers. Always `printf("%s", user_input)`.

6.5 **inline**, **static inline**, **_Noreturn**

The C99 **inline** model is genuinely confusing

inline is a **hint**, not a command — the compiler decides. But the *linkage* rules are the surprising part.

```
inline int f(int x) { return x * 2; }    /* "inline definition" – NOT an external
definition */
int main(void) { return f(21); }
```

```
-- -O0 -- undefined reference to `f'
-- -O2 -- (linked fine) 42
```

The same code links at -O2 and fails at -O0. Because: **inline** alone provides only an *inline definition*, which the compiler may use for inlining but which emits **no callable symbol**. At -O0 it doesn't inline, so it needs an

external definition — and there isn't one.

The three forms, and which to use

Form	Meaning	Use it?
<code>static inline</code>	Internal linkage, own copy per TU, always works	? Default choice
<code>inline</code> (alone)	Inline definition only, no external symbol	?? Only with the pattern below
<code>extern inline</code>	Emits the external definition	Exactly once, in one <code>.c</code>

`static inline` always works, at every optimization level:

```
-O0: 42
-O2: 42
```

The correct C99 header pattern if you need one shared out-of-line copy:

```
/* hdr.h */
inline int dbl(int x) { return x * 2; }

/* hdr.c — exactly ONE translation unit */
#include "hdr.h"
extern inline int dbl(int x);      /* emits the external definition */
```

```
-O0: dbl(21)=42
-O2: dbl(21)=42
```

Practical advice: use `static inline` in headers and stop thinking about it. The duplicate-copy cost is negligible (the linker merges identical code, and inlined calls emit nothing at all). Reach for `extern inline` only when you have a measured reason.

Note: **GNU89 `inline` semantics are the opposite** of C99's. Code written for old GCC with `-std=gnu89` behaves differently under `-std=c99`. This bit the Linux kernel during its C99 migration.

Should you use `inline` at all?

Mostly no. Modern compilers inline aggressively based on their own cost model and **ignore your hint** when they disagree. With LTO, they inline across translation units too.

`inline` still matters for:

- **Header-defined functions** (where `static inline` is the mechanism, not the optimization)
- `__attribute__((always_inline))` when you have a measured need
- `__attribute__((noinline))` for debugging or to keep a function in a stack trace

`_Noreturn` / `[[noreturn]]`

```
_Noreturn void die(const char *msg)          /* C11 */
{ fprintf(stderr, "fatal: %s\n", msg); exit(1); }

[[noreturn]] void die(const char *msg);      /* C23 */
```

Benefits: the compiler stops warning about "control reaches end of non-void function" after a call, can delete unreachable code, and can omit saving registers. Standard library: `exit`, `abort`, `_Exit`, `longjmp`, `thrd_exit`.

If a `_Noreturn` function ever returns, that's undefined behavior.

6.6 Function pointers ?? (the important one)

Syntax

```
int add(int, int);

int (*fp)(int, int) = add;    /* pointer to function taking (int,int), returning
int */
```

The parentheses around `*fp` are mandatory — precedence puts `()` above `*`:

```
int (*p)(int);    /* p is a POINTER to a function returning int */
int *p(int);      /* p is a FUNCTION returning int* – completely different */
```

A function name **decays to a pointer** in almost every context, so `&` and `*` are optional and stackable:

```
f(3,4)      = 7
(*f)(3,4)   = 7
(****f)(3,4)= 7  <- deref of a function ptr yields the function again
```

`fp = add;` and `fp = &add;` are identical. `fp(3,4)` and `(*fp)(3,4)` are identical. **Write `fp = add;` and `fp(3,4)`.**

Always `typedef` it

```
typedef int (*BinOp)(int, int);    /* pointer-to-function alias */
typedef int BinOpFn(int, int);     /* function-type alias – sometimes cleaner */

BinOp f = add;
```

```
void install(BinOp op);
BinOp table[3] = { add, sub, mul };
```

Compare to writing it raw: `int (*(*get_handler(int))(int,int))(int)`. Nobody should read that.

Dispatch tables

```
static const struct { char sym; BinOp fn; const char *name; } ops[] = {
    {'+', add, "add"}, {'-', sub, "sub"}, {'*', mul, "mul"},
};

for (size_t i = 0; i < ARRAY_LEN(ops); i++)
    printf("6 %c 3 = %d (%s)\n", ops[i].sym, ops[i].fn(6,3), ops[i].name);
```

```
6 + 3 = 9 (add)
6 - 3 = 3 (sub)
6 * 3 = 18 (mul)
```

This is how you replace a long `if/else` chain with data. Adding an operation means adding a table row — no control flow changes. Used for: interpreters and VMs (opcode dispatch), state machines, command parsers, protocol handlers, plugin registries.

Callbacks — and the `void *` context idiom

```
void for_each(int *a, size_t n, void (*cb)(int, void *), void *ctx)
{ for (size_t i=0; i<n; i++) cb(a[i], ctx); }

static void accumulate(int v, void *ctx) { *(long*)ctx += v; }

long total = 0;
for_each(a, n, accumulate, &total);
```

```
sum via callback = 24
```

C has no closures. The `void *userdata` parameter is how you carry state into a callback. **Every callback API you design should take one** — omitting it is the single most common C API design mistake, because callers can't attach context without resorting to globals.

Look at how the standard library does it — and note `qsort` gets this *wrong*:

```
void qsort(void *base, size_t n, size_t size,
           int (*cmp)(const void *, const void *));    /* no context! */
```

That's why sorting by a runtime-chosen key requires a global or the GNU extension `qsort_r/qsort_s`. Learn from it.

Writing a comparator correctly

```
static int cmp_int_asc(const void *a, const void *b)
{
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);      /* NOT x - y — that overflows */
}
```

```
asc : 1 2 5 7 9
desc: 9 7 5 2 1
```

return x - y; is a real bug: `INT_MIN - 1` overflows (UB), and even without UB it can return the wrong sign. `(x>y)-(x<y)` is branchless, overflow-free, and returns exactly `-1/0/+1`.

Rules and gotchas

- A function pointer is **not** a `void *`. Converting between them is *not* guaranteed by ISO C (POSIX requires it to work, which is what makes `dlsym` possible). Use a union or a POSIX-blessed cast, and know it's a portability compromise.
- **You can't do pointer arithmetic** on function pointers.
- **Calling through a mismatched type is UB** — a common source of crashes in callback registries. Keep signatures exact.
- A NULL function pointer call is UB (usually a segfault at address 0).
- Function pointers are the natural target for **Control Flow Integrity** hardening (`-fcf-protection`, the `endbr64` you see in all our assembly dumps).

The pattern this unlocks: polymorphism in C

```
typedef struct Shape Shape;
struct ShapeVTable {
    double (*area)(const Shape *);
    void (*draw)(const Shape *);
    void (*destroy)(Shape *);
};
struct Shape { const struct ShapeVTable *vt; };

double area(const Shape *s) { return s->vt->area(s); } /* virtual dispatch */
```

That's exactly what C++ generates for virtual functions, and exactly what the Linux kernel's `struct file_operations` is. Full treatment in Module 28.

6.7 Calling conventions & ABI basics

An **ABI** (Application Binary Interface) specifies how compiled code interoperates: argument passing, return values, register preservation, stack alignment, struct layout, name mangling. **The API is what the compiler checks; the ABI is what actually happens at runtime.**

System V AMD64 (Linux, macOS, BSD) — the one you'll debug

Integer/pointer arguments 1–6: *RDI, RSI, RDX, RCX, R8, R9*

```
six:                                ; long six(a,b,c,d,e,f)
    add    rdi, rsi                ; a + b
    add    rdi, rdx                ; + c
    add    rdi, rcx                ; + d
    add    rdi, r8                 ; + e
    lea    rax, [rdi+r9]           ; + f, result in RAX
    ret
```

Arguments 7+ go on the stack, pushed right-to-left:

```
eight:
    ...
    add    rax, QWORD PTR 8[rsi]   ; 7th argument
    add    rax, QWORD PTR 16[rsi]  ; 8th argument
```

Floating point uses a separate register class, *XMM0–XMM7*:

```
dbl6:
    addsd  xmm0, xmm1
    addsd  xmm0, xmm2
    ...    ; result in XMM0
```

Note this is why the missing-prototype bug in §6.1 returned 0: the caller loaded *RDI/RSI*, the callee read *XMM0/XMM1*.

Other SysV AMD64 rules:

Item	Rule
Integer return	<i>RAX</i> (and <i>RDX</i> for 128-bit)
Float return	<i>XMM0</i>
Large struct return	Caller allocates; hidden pointer in <i>RDI</i>
Callee-saved	<i>RBX, RBP, R12–R15</i> — the callee must restore these

Item	Rule
Caller-saved	RAX, RCX, RDX, RSI, RDI, R8–R11, all XMM
Stack alignment	16 bytes at every <code>call</code>
Red zone	128 bytes below <code>RSP</code> , usable without adjusting <code>RSP</code>
Variadic	<code>AL</code> holds the number of vector registers used

The historical x86 (32-bit) conventions

You'll meet these in Windows code, old libraries, and interview questions:

Convention	Args	Who cleans the stack	Notes
<code>cdecl</code>	Stack, right-to-left	Caller	C default. Caller-cleanup is what makes variadics possible.
<code>stdcall</code>	Stack, right-to-left	Callee	Win32 API. Smaller code, but no variadics.
<code>fastcall</code>	First 2 in <code>ECX,EDX</code> , rest on stack	Callee	Compiler-specific, not standardized.
<code>thiscall</code>	<code>this</code> in <code>ECX</code>	Varies	C++ member functions on MSVC.

Why `cdecl` allows variadics: only the caller knows how many arguments it pushed, so only the caller can clean up correctly. `stdcall` functions can't be variadic — which is why `wsprintf` is one of the few `cdecl` functions in the otherwise-`stdcall` Win32 API.

You'll see these as `__cdecl`, `__stdcall`, `WINAPI`, `CALLBACK` in Windows headers. On 64-bit Windows they're all ignored — there's one convention (Microsoft x64: `RCX`, `RDX`, `R8`, `R9`, 32-byte shadow space — **different from `SysV`**, which is why cross-platform C libraries need care).

Stack frames — why this matters for debugging



This is why buffer overflows are exploitable: a local array overflowing upward reaches the saved **RB**P and then the **return address**. Overwrite that and you control where the function returns.

Mitigations you'll see in real builds: **-fstack-protector-strong** (canary between locals and the return address), ASLR, non-executable stack, **-fcf-protection** (the **endbr64** instructions in every dump above).

Where ABI knowledge earns its keep

- **Reading stack traces** when symbols are missing
- **Debugging optimized code** (**-fomit-frame-pointer** breaks naive backtraces)
- **Writing inline assembly** or interfacing with hand-written asm
- **FFI**: calling C from Python/Rust/Go — you're matching the ABI
- **Shared library versioning**: changing a struct's layout or a parameter type breaks the ABI even when the API compiles fine
- **Understanding why long differs on Windows** (Module 2) — it's an ABI decision

Module 6 rules

#	Rule
1	Prototypes for everything; -Werror=implicit-function-declaration
2	Write (void) , never () , for no-parameter functions; -Wstrict-prototypes
3	Everything is copied — to modify a caller's object, pass a pointer to it
4	sizeof on an array parameter gives the pointer size; always pass the length
5	Pass structs > ~16 bytes by const pointer
6	Never rely on tail-call optimization; bound recursion depth on untrusted input
7	va_arg(ap, char) and va_arg(ap, float) are always wrong — promotions apply
8	va_copy before a second traversal; every copy needs its own va_end
9	Put __attribute__((format(printf, m, n))) on every log function
10	Never pass user input as a format string
11	Use static inline in headers; plain inline breaks at -O0
12	typedef every function-pointer type
13	Every callback API takes a void *userdata
14	Comparators return (x>y)-(x<y) , never x-y

Practice

1. **Reproduce the prototype corruption.** Split **double scale(double, double)** into its own **.c**, call it from another TU without a prototype, and observe the wrong answer. Then dump both sides with **-S** and identify exactly which registers disagree.

2. **Swap, four ways.** Implement `swap` for `int`, for a large struct, for two pointers (`swap_ptr(int **a, int **b)`), and a generic `swap_bytes(void *a, void *b, size_t n)`. Test each.
3. **Measure your stack.** Write a recursion that reports depth and stack consumption every 10,000 frames (use unbuffered `stderr` — buffered `printf` output is lost on SIGSEGV). Run it with `ulimit -s 1024` and `ulimit -s unlimited`. Then add `char pad[1024]` to the frame and re-measure.
4. **TCO experiment.** Write a tail-recursive `sum(n, acc)` and call it with `n = 10,000,000`. Build at `-O0`, `-O1`, `-O2`, `-Os`. Record which crash. Dump the `-O2` assembly and find the `jmp` that replaced the `call`.
5. **Build a safe logger.** Write `void log_at(const char *file, int line, const char *level, const char *fmt, ...)` using `vsnprintf` into a fixed buffer, with the `format` attribute. Add a `#define LOG(...) log_at(__FILE__, __LINE__, "INFO", __VA_ARGS__)` wrapper. Deliberately pass a wrong type and confirm the compile-time warning.
6. **va_copy proof.** Write a variadic function that traverses its arguments twice *without* `va_copy` and observe the garbage. Then add `va_copy` and confirm it's correct. Run both under UBSan.
7. **The inline maze.** Reproduce all three linkage cases from §6.5: plain `inline` failing at `-O0`, `static inline` working everywhere, and the `extern inline` header pattern. Use `nm` to inspect which symbols each emits.
8. **Build a calculator with a dispatch table.** Support `+ - * / % ^`, using a table of `{char, BinOp, const char*}`. Add division-by-zero handling. Then extend it with unary operations using a second table — and note how much easier that is than editing a switch.
9. **Generic for_each with context.** Write `map(int *a, size_t n, int (*f)(int, void*), void *ctx)` and use it to implement scale-by-k, clamp-to-range, and running-sum — each carrying its parameters through `ctx`.
10. **ABI archaeology.** Write `long f(long, long, long, long, long, long, long, long, long)` and `double g(double, double, ...)`. Dump the assembly. Identify each argument's location. Then write a struct-returning function and find the hidden return pointer.

7. Arrays

- 1D, 2D, multidimensional, row-major layout
- Array-to-pointer decay (and where it *doesn't* happen)
- Array parameters in functions (they're pointers)
- Variable Length Arrays (VLA) — and why many shops ban them
- Designated initializers, compound literals
- Flexible array members (`struct { int n; char data[]; }`)

Module 7 — Arrays

Real GCC 13.3 output throughout. **Array-to-pointer decay** (§7.2) is the concept that unlocks most of C. If you understand decay completely, `sizeof` bugs, function signatures, 2D arrays, and half of pointer syntax stop being mysterious.

7.1 One-dimensional arrays

```
int a[5] = {10, 20, 30, 40, 50};
```

An array is **N contiguous objects of the same type**. Nothing more — no length field, no bounds checking, no metadata.

```
&a[0] = 0x7ffffafbd7c80 (offset 0 bytes)
&a[1] = 0x7ffffafbd7c84 (offset 4 bytes)
&a[2] = 0x7ffffafbd7c88 (offset 8 bytes)
&a[3] = 0x7ffffafbd7c8c (offset 12 bytes)
&a[4] = 0x7ffffafbd7c90 (offset 16 bytes)
```

$a[i]$ is *defined* as $*(a + i)$

This isn't an analogy — it's the standard's definition. And since addition commutes:

```
a[2]=30  *(a+2)=30  2[a]=30  *(2+a)=30
```

$2[a]$ is legal C. Never write it, but understanding *why* it works means you understand subscripting.

The array name vs **&array** — a critical distinction

```
a      = 0x7ffffafbd7c80 (decays: pointer to first ELEMENT, type int*)
&a[0]  = 0x7ffffafbd7c80 (same value, type int*)
&a     = 0x7ffffafbd7c80 (same value, type int(*)[5] -- DIFFERENT TYPE)
a+1    = 0x7ffffafbd7c84 (+4 bytes: one int)
&a+1   = 0x7ffffafbd7c94 (+20 bytes: one WHOLE ARRAY)
```

Same address, different types, different arithmetic. a decays to $\text{int } *$; $\&a$ is a *pointer to array of 5 ints*, $\text{int } (*)[5]$. Adding 1 to the first advances 4 bytes; adding 1 to the second advances 20.

This is the single cleanest proof that pointer arithmetic is driven entirely by the pointed-to type.

What arrays cannot do

```
int a[5], b[5];
b = a;                /* ERROR: arrays are not assignable */
if (a == b) { }       /* compares ADDRESSES, not contents (always false here) */
int f(void) { int x[5]; return x; } /* ERROR: cannot return an array */
struct S { int a[5]; };
struct S s1, s2; s2 = s1; /* OK! struct assignment DOES copy the array */
/*
```


Arrays are not first-class values — you can't assign, compare, pass, or return them. Wrapping one in a struct makes it copyable. That's a real technique:

```
typedef struct { double v[3]; } Vec3;
Vec3 add(Vec3 a, Vec3 b);      /* by value, copies the array – works fine */
```

7.2 Multidimensional arrays and row-major layout

```
int m[3][4];    /* read as: an array of 3 elements, each of which is int[4] */
```

C has no true 2D arrays. `int m[3][4]` is an *array of arrays*, laid out contiguously.

linear memory: 1 2 3 4 5 6 7 8 9 10 11 12

address map (offset from `&m[0][0]`):

row 0:	0	4	8	12
row 1:	16	20	24	28
row 2:	32	36	40	44

Row-major order: the last index varies fastest. (Fortran and MATLAB are column-major — this matters when interfacing with numerical libraries or BLAS.)

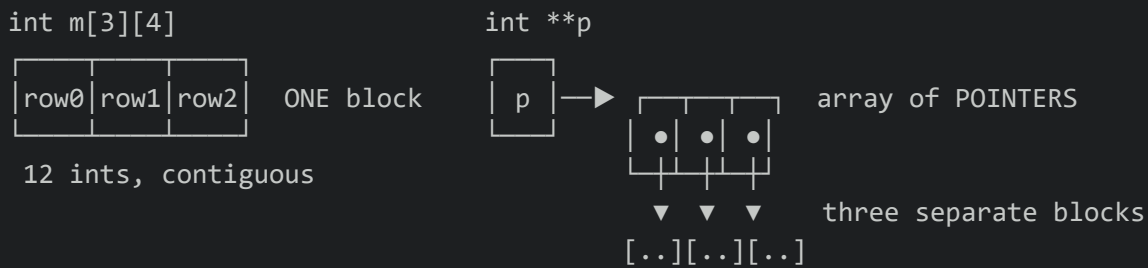
```
m[i][j] == (*(m+i)+j) == flat[i*4 + j]
```

The types at each level

```
sizeof(m)      = 48    type int[3][4]  (whole array)
sizeof(m[0])   = 16    type int[4]     (one row)
sizeof(m[0][0]) = 4    type int
m decays to    : int (*)[4]  -- POINTER TO A ROW, not int**
m+1 advances   : 16 bytes (one whole row)
m[0]+1 advances : 4 bytes (one int)
```

⚠ `int m[3][4]` **does NOT decay to `int **`**. It decays to `int (*)[4]` — a pointer to an array of 4 ints. Passing a 2D array where `int **` is expected is a type error, and forcing it with a cast produces garbage.

The difference is fundamental:



`m[1][2]` on a real 2D array is **one** memory access with computed offset. On an `int **` it's **two** dereferences. Different code, different performance, different allocation.

Why row-major matters for performance

```
/* FAST: sequential memory access, cache-friendly */
for (i = 0; i < rows; i++)
    for (j = 0; j < cols; j++)
        sum += m[i][j];

/* SLOW: strides across rows, cache-hostile */
for (j = 0; j < cols; j++)
    for (i = 0; i < rows; i++)
        sum += m[i][j];
```

On a large matrix this can be a **5–10× difference** with identical results. Iterate in memory order. (Module 25 covers cache behavior properly.)

Practical alternative: flatten it yourself

```
size_t rows = R, cols = C;
int *m = malloc(rows * cols * sizeof *m);
#define M(i,j) m[(i)*cols + (j)]

M(1,2) = 7;
free(m);
```

One allocation, contiguous, works with any runtime dimensions, no VLA. This is what most production code does.

7.3 Array-to-pointer decay ⚠

In almost every expression, an array is implicitly converted to a pointer to its first element.

```
DECAY HAPPENS almost everywhere:
assignment:    int *p = a;    OK
```

```
function call: inside f: sizeof(arr) = 8 (it is int*)
arithmetic:    a+1 = 0x7ffd1c756e24
comparison:    (a == p) = 1
```

The five places decay does *not* happen

- 1. sizeof(a) = 40 (whole array, not 8)
- 2. &a -> type int(*)[10], NOT int**
- 3. _Alignof(a) = 4
- 4. char s[] = "hello" -> COPIES the literal, sizeof(s)=6
- 5. typeof(a) / compound literal keeps the array type

#	Context	Why it matters
1	Operand of sizeof	The array-length idiom depends on this
2	Operand of &	Gives T (*)[N], enabling whole-array pointers
3	Operand of _Alignof	Alignment of the array type
4	String literal initializing a char[]	char s[] = "hi" copies; char *s = "hi" points
5	typeof, _Generic (C23)	Type queries see the real array type

Number 4 is worth dwelling on:

```
char s1[] = "hello"; /* array of 6, on the stack, MODIFIABLE */
char *s2 = "hello"; /* pointer into .rodata, READ-ONLY */

s1[0] = 'H'; /* fine */
s2[0] = 'H'; /* UNDEFINED BEHAVIOR – usually segfault */
```

Same-looking syntax, completely different objects. Always write `const char *s = "hello";` for the pointer form so the compiler enforces it.

Decay is why sizeof fails on parameters

```
void f(int arr[10]) { printf("%zu\n", sizeof arr); } /* prints 8 */
```

```
warning: 'sizeof' on array function parameter 'arr' will return size of 'int *'
[-Wsizeof-array-argument]
```

The most common C bug in this area: `sizeof(arr)/sizeof(arr[0])` works in the function where the array is declared and silently gives `8/4 = 2` in any function it's passed to.

7.4 Array parameters are pointers

These four declarations are identical:

```
void f(int arr[100]);
void f(int arr[3]);      /* the number is IGNORED */
void f(int arr[]);
void f(int *arr);
```

GCC will even tell you they're the same type:

```
warning: argument 1 of type 'int *' declared as a pointer [-Warray-parameter=]
```

The dimension in `int arr[100]` is **pure documentation**. Nothing enforces it.

The correct signature

```
int sum_n(const int *arr, size_t n);      /* pointer + length. Always. */
```

`const` documents that you won't modify it, and `size_t n` is the only thing that actually protects you.

`[static n]` — a C99 contract

```
int sum10(int arr[static 10]); /* "caller guarantees ≥10 elements, and non-NULL"
*/
```

It's a **promise**, not a check — but it lets the compiler diagnose and optimize:

```
int small[3] = {1,2,3};
sum10(small);
```

```
warning: 'sum10' accessing 40 bytes in a region of size 12 [-Wstringop-overflow=]

sum10(small) reads out of bounds -> 34 (UB)
```

And under ASan:

```
ERROR: AddressSanitizer: stack-buffer-overflow
    [32, 44) 'small' (line 32) <== Memory access at offset 44 overflows this
variable
```

GCC caught it at compile time *and* ASan caught it at runtime. `[static n]` is underused — it costs nothing and adds real diagnostics for fixed-size buffers.

2D array parameters

Only the **first** dimension may be omitted, because the compiler needs the row width to compute `i*cols + j`:

```
void f(int m[][4], size_t rows);      /* OK  - width known */
void f(int m[3][4], size_t rows);    /* OK  - 3 ignored */
void f(int (*m)[4], size_t rows);    /* OK  - same thing, explicit */
void f(int m[][], size_t rows);      /* ERROR - width unknown */
void f(int **m, size_t rows);        /* Compiles, but WRONG TYPE for a 2D array
*/
```

For **runtime** dimensions you have three options:

```
/* 1. VLA parameter (C99) - dimensions must be declared first */
void f(size_t rows, size_t cols, int m[rows][cols]);

/* 2. Flat array + manual indexing - most portable */
void f(int *m, size_t rows, size_t cols) { m[i*cols + j] = x; }

/* 3. Array of pointers - needed for jagged data, costs an extra indirection */
void f(int **m, size_t rows, const size_t *cols);
```

7.5 Variable Length Arrays — and why shops ban them

```
void f(size_t n) { int v[n]; }      /* size determined at runtime */
```

C99 added VLAs. **C11 made them optional** — a conforming implementation may define `__STDC_NO_VLA__` and reject them.

What they can do

```
n=3  sizeof(v)=12  (sizeof is EVALUATED at runtime for VLAs)
n=6  sizeof(v)=24
```

VLA function parameters give you genuine runtime-dimensioned 2D arrays:

```
void print_matrix(size_t rows, size_t cols, int m[rows][cols]);
```

```
0  1  2
3  4  5
```

That's the one thing VLAs do that nothing else does cleanly.

sizeof on a VLA is evaluated

```
int k = 0;
int q[(k = 7)];
printf("sizeof q = %zu, k is now %d\n", sizeof q, k);
```

```
int q[(k=7)]; sizeof q = 28, and k is now 7
```

The one exception to "sizeof doesn't evaluate its operand." A VLA's size is a runtime value, so **sizeof** must compute it — with side effects and all.

Why they get banned

1. There is no way to detect failure.

```
size_t n = strtoul(argv[1], 0, 10);
int v[n];          /* if this doesn't fit, you just die */
```

```
$ ./vb 1000
survived: v[0]=1 v[n-1]=2

$ ./vb 1000000000
Segmentation fault    exit=139
```

Silent SIGSEGV. No return value, no **errno**, no chance to recover. **malloc** returns **NULL**; a VLA returns a corpse.

2. It's a security vulnerability. If **n** comes from untrusted input, you have a **stack-clash attack**: a large VLA can skip the guard page and land the stack pointer in the middle of the heap or another thread's stack. Real

CVEs. The Linux kernel removed **every VLA** in 2018 for exactly this reason (Linus: "*VLAs are not only a security problem, they also generate much worse code*").

3. Worse code generation. A fixed-size array is a constant stack offset. A VLA requires runtime **RSP** adjustment, prevents frame-pointer omission, and blocks some optimizations.

4. Optional since C11. MSVC has never supported VLAs. Portable code can't use them.

5. They interact badly with **goto, **longjmp**, and **switch**** — you can't jump into the scope of a VLA.

How to ban them

```
gcc -Wvla -Werror=vla
```

```
error: ISO C90 forbids variable length array 'v' [-Werror=vla]
```

Add this to your build. It's what the Linux kernel, MISRA C, CERT C (rule ARR32-C), and most embedded/safety-critical shops do.

What to use instead

```
/* Heap – checkable failure */
int *v = malloc(n * sizeof *v);
if (!v) return -1;
...
free(v);

/* Fixed maximum with an explicit bound check */
#define MAX_N 1024
if (n > MAX_N) return -1;
int v[MAX_N];

/* Hybrid: stack for the common case, heap for large */
int stackbuf[64];
int *v = (n <= 64) ? stackbuf : malloc(n * sizeof *v);
if (!v) return -1;
...
if (v != stackbuf) free(v);
```

The hybrid pattern is what high-performance C libraries actually do — it avoids **malloc** in the hot path while staying safe.

VLA types in function parameters (**int m[rows][cols]**) are more defensible than VLA *objects*, since no stack allocation occurs — but **-Wvla** flags both. Most codebases just use flat arrays with manual indexing.

7.6 Designated initializers and compound literals

Designated initializers (C99)

```
int sparse[10] = {[0]=1, [5]=5, [9]=9};
struct P p = {.y = 9};
```

```
1 0 0 0 0 5 0 0 0 9
struct P {.y=9} -> (0,9)
```

Everything unmentioned is zero-initialized (Module 3). Benefits: order-independent, self-documenting, gap-tolerant, and refactor-safe — reordering struct members won't silently scramble your initializers.

The killer use case is lookup tables keyed by enum:

```
static const char *const state_names[] = {
    [STATE_IDLE]     = "idle",
    [STATE_RUNNING]  = "running",
    [STATE_DONE]     = "done",
};
/* Array size is inferred, entries stay correct if the enum is reordered,
   and _Static_assert can verify the count. */
_Static_assert(sizeof state_names / sizeof *state_names == STATE_COUNT, "table out
of sync");
```

⚠ **GNU range syntax** `[2 ... 5] = 7` is a **GCC extension, not standard C**. It works:

```
GNU range [2...5]=7: 0 0 7 7 7 7 0 0 0 0
```

...but `-pedantic` rejects it. Avoid in portable code.

Compound literals (C99)

An **unnamed object** created in an expression:

```
(type){ initializers }
```

```
show((struct P){3, 4});           /* pass a temporary struct */
sum((int[]){1,2,3,4,5}, 5);       /* pass a temporary array */
struct P *pp = &(struct P){7, 8}; /* take its address */
```



```
(3,4)
sum=15
&(struct P){7,8} -> (7,8)
```

Compound literals are modifiable (unlike string literals):

```
int *m = (int[]){1,2,3};
m[0] = 99;                /* legal */
```

```
modified: 99 2 3
```

⚠ Compound literal lifetime

At block scope, a compound literal has automatic storage duration — it dies at the end of the enclosing block.

```
const int *make_bad(void) { return (const int[]){1,2,3}; } /* DANGLING */
const int *make_ok(void) { static const int a[]={1,2,3}; return a; }
```

```
make_bad()[0] = 1  <-- UB, dangling
make_ok()[0]  = 1  <-- safe (static)
```

ASan confirms:

```
ERROR: AddressSanitizer: stack-use-after-return
```

It printed **1** — the memory hadn't been reused yet. **That's the worst possible outcome:** it looks correct in testing and corrupts in production.

At file scope, a compound literal has static duration and is safe to return a pointer to. In C23 you can also write `static (int[]){1,2,3}` at block scope to force static duration.

Where compound literals shine

```
/* Passing config structs without a named temporary */
init_widget(&(WidgetConfig){ .width = 100, .height = 50, .title = "Hi" });

/* Returning a struct by value from a helper */
static Point pt(int x, int y) { return (Point){x, y}; }
```

```
/* Array arguments inline */
draw_polygon((Point[]){0,0},{1,0},{1,1}}, 3);

/* Zero-initializing in an assignment */
mystruct = (MyStruct){0};          /* struct assignment, cleaner than memset */
```

7.7 Flexible array members

```
struct Msg {
    size_t len;
    char data[];      /* C99 flexible array member – no size */
};
```

```
sizeof(struct Msg)  = 8    <- FAM contributes ZERO to sizeof
sizeof(struct Hack) = 16   <- the [1] costs a byte (+padding)
sizeof(struct Ptr)  = 16   <- pointer + separate allocation
```

The allocation pattern

```
struct Msg *msg_new(const char *s)
{
    size_t n = strlen(s);
    struct Msg *m = malloc(sizeof *m + n + 1);    /* header + payload, ONE alloc
*/
    if (!m) return NULL;
    m->len = n;
    memcpy(m->data, s, n + 1);
    return m;
}
```

```
len=11 data="hello world"
&m      = 0x55efd987b2b0
&m->data = 0x55efd987b2b8 (offset 8 -> immediately after the header)
ONE allocation, ONE free, ONE cache line
```

Why this beats the alternatives

	Flexible array member	Pointer member
Allocations	1	2

	Flexible array member	Pointer member
free calls	1	2 (easy to leak)
Cache locality	header + data adjacent	two random heap locations
Extra pointer	none	8 bytes
Failure paths	1	2

For network packets, variable-length records, string objects, and anything you allocate millions of, this is a meaningful win. It's why `struct sockaddr`, `struct inotify_event`, and countless kernel structures use it.

The rules

- Must be the **last** member.
- The struct must have **at least one other member**.
- It **cannot** be a member of another struct, or an element of an array.
- `sizeof(struct)` **excludes** the flexible member — that's exactly why the `sizeof *m + n` idiom works.
- **Struct assignment does not copy the flexible part** — `*a = *b`; copies only the header. Use `memcpy` with the real size.

The "struct hack" — its predecessor

Before C99, people wrote `char data[1];` and over-allocated. It was technically UB (accessing past the declared bound), it wastes a byte plus padding, and it makes the size arithmetic error-prone (`sizeof *m + n - 1`). **Use the C99 FAM.**

Get the size arithmetic wrong and it's a heap overflow

```
struct Msg *m = malloc(sizeof *m);      /* forgot the payload! */
memcpy(m->data, "hello", 6);
```

```
ERROR: AddressSanitizer: heap-buffer-overflow
SUMMARY: heap-buffer-overflow in memcpy
```

Always guard against integer overflow in the size computation when the length comes from untrusted input:

```
if (n > SIZE_MAX - sizeof *m - 1) return NULL; /* overflow check FIRST */
struct Msg *m = malloc(sizeof *m + n + 1);
```

A `size_t` overflow here allocates a tiny buffer and then writes a huge amount into it — a classic remote-code-execution primitive.

Module 7 rules

#	Rule
1	<code>a[i]</code> is <code>*(a+i)</code> — subscripting is pointer arithmetic
2	<code>a</code> and <code>&a</code> have the same value but different types and different arithmetic
3	Arrays decay everywhere except <code>sizeof</code> , <code>&</code> , <code>_Alignof</code> , <code>char s[]="..."</code> , and <code>typeof</code>
4	Array parameters are pointers — the dimension is documentation only
5	Always pass (pointer, length); consider <code>[static n]</code> for fixed minimums
6	<code>int[3][4]</code> decays to <code>int (*)[4]</code> , never <code>int **</code>
7	Iterate 2D arrays in row-major order for cache performance
8	Ban VLAs: <code>-Werror=vla</code> . Use <code>malloc</code> (checkable) or a bounded fixed array
9	Designated initializers for enum-keyed tables; add a <code>_Static_assert</code> on the count
10	Block-scope compound literals die with the block — never return pointers to them
11	Flexible array members for variable-length records: one alloc, one free
12	Check for <code>size_t</code> overflow before <code>malloc(sizeof *m + n)</code>

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
  -Werror=vla -Warray-bounds=2 -Wstringop-overflow=4 \
  -Werror=return-type -Werror=implicit-function-declaration \
  -g -fsanitize=address,undefined prog.c -o prog
```

Practice

1. **Prove row-major.** Declare `int m[4][5]`, fill it with `i*10+j`, then print it through an `int *` linear pointer. Verify the order matches your prediction. Then measure the runtime difference between row-major and column-major traversal on a 2000×2000 array.
2. **Decay checklist.** Write one program that demonstrates each of the five non-decay contexts, printing sizes and using `_Generic` (or `__builtin_types_compatible_p`) to prove the types differ.
3. **The `sizeof` trap.** Write `void print_all(int arr[])` that tries `sizeof(arr)/sizeof(arr[0])`. Call it with a 10-element array. Then fix it. Then write an `ARRAY_LEN` macro that fails to *compile* when given a pointer.
4. **2D parameter forms.** Implement matrix-sum four ways: `int m[][4]`, `int (*)[4]`, VLA parameter `int m[r][c]`, and flat `int *m` with manual indexing. Verify all four give the same answer. Which compiles under `-Werror=vla`?

5. **VLA danger demo.** Write a program that takes `n` from `argv` and declares `int v[n]`. Find the largest `n` that works and the smallest that crashes. Then rewrite with `malloc` and confirm you get a clean error instead of a segfault at the same size.
6. **Compound literal lifetime.** Reproduce the dangling `make_bad()` case. Run under ASan. Then add three intervening function calls before printing and observe the value change — proving the memory really was reclaimed.
7. **Build a FAM string type.** Implement:

```
typedef struct { size_t len, cap; char data[]; } Str;
Str *str_new(const char *s);
Str *str_append(Str *s, const char *t); /* may realloc */
void str_free(Str *s);
```

Include integer-overflow checks on every size computation. Verify with ASan and LSan that a workload of 10,000 appends is clean.

8. **Reasoning, no compiler.** Given `int a[3][4]`, what are the types and values of: `a`, `&a`, `a[0]`, `&a[0]`, `*a`, `a+1`, `&a+1`, `a[0]+1`, `sizeof a`, `sizeof a[0]`, `sizeof *a`?

8. Strings

- Null-terminated convention, `'\0'`
- `string.h`: `strlen`, `strcpy`, `strncpy`, `strcat`, `strcmp`, `strchr`, `strstr`, `strtok`, `memcpy`, `memmove`, `memset`, `memcmp`
- **Why `strcpy`/`strcat`/`sprintf`/`gets` are dangerous**; `snprintf`, `strncpy`/`strlcat`, `strnlen`
- `memcpy` vs `memmove` (overlap)
- Conversions: `atoi` (bad) vs `strtol`/`strtod` (good, with `errno` + `endptr`)
- Wide strings, multibyte, **UTF-8 handling in C**
- Building a string/buffer abstraction

Module 8 — Strings

All output is real GCC 13.3 / glibc 2.39. C has no string type — it has a *convention*, and every string bug in the last 50 years traces back to that convention's costs.

8.1 The null-terminated convention

A C "string" is a `char` array whose end is marked by a `'\0'` byte. There is **no length field**.

```
char s[10] = "hi"; bytes: 68 69 00 00 00 00 00 00 00 00
strlen(s)=2 sizeof(s)=10 <- LENGTH != CAPACITY
```

Two numbers you must always track separately:

- **Length** — bytes before the NUL (`strlen`)
- **Capacity** — bytes the buffer can hold (`sizeof` or whatever you `malloced`)

Confusing them is the root of most buffer overflows.

'\0' vs '0' vs "0" vs NULL

```
'\0'    /* the NUL character, value 0, type int (it's a char constant) */
'0'     /* the digit zero, value 48 */
"0"     /* a 2-byte array: '0', '\0' */
NULL    /* a null POINTER */
```

Four different things, routinely mixed up. `if (*s == '\0')` and `if (!*s)` are equivalent and idiomatic; `if (s == NULL)` tests something entirely different.

The three consequences

1. `strlen` is $O(n)$. It scans until it finds the NUL.

```
for (size_t i = 0; i < strlen(s); i++)    /*  $O(n^2)$  */
size_t n = strlen(s);
for (size_t i = 0; i < n; i++)            /*  $O(n)$  */
```

This is not hypothetical — the $O(n^2)$ version is a classic performance bug in string-processing loops.

2. Strings cannot contain embedded NULs.

```
char e[] = "abc\0def";
```

```
sizeof=8 strlen=3
bytes: 61 62 63 00 64 65 66 00
```

The array holds all 8 bytes, but every `str*` function stops at index 3. **You cannot store arbitrary binary data in a C string** — use `mem*` functions with an explicit length.

3. A missing terminator reads off the end.

```
char bad[4] = {'a','b','c','d'};    /* no NUL */
strlen(bad);
```

```
strlen(bad) = 7  <-- read past the array, UB

ASan: stack-buffer-overflow
[48, 52) 'bad' <== Memory access at offset 52 overflows this variable
```

Recall from Module 3 that `char s[3] = "abc"` silently drops the terminator. Always `char s[] = "abc"` and let the compiler count.

8.2 The `<string.h>` toolkit

Length and comparison

Function	Signature	Notes
<code>strlen</code>	<code>size_t strlen(const char *)</code>	$O(n)$. UB if not terminated.
<code>strnlen</code>	<code>size_t strnlen(const char *, size_t n)</code>	POSIX , safer: stops at <code>n</code> . Use for untrusted data.
<code>strcmp</code>	<code>int strcmp(const char *, const char *)</code>	$<0, 0, >0$. Not $-1/0/1$ — never assume.
<code>strncmp</code>	<code>int strncmp(const char *, const char *, size_t)</code>	Compares at most <code>n</code> bytes.
<code>memcmp</code>	<code>int memcmp(const void *, const void *, size_t)</code>	Byte-wise. Not constant time — see below.

⚠️ **`strcmp` returns "some negative/zero/positive value."** Testing `if (strcmp(a,b) == -1)` is a bug. Always `== 0, < 0, > 0`.

⚠️ **Never `memcmp` two structs to compare them** — padding bytes are indeterminate (Module 3).

⚠️ **Never use `memcmp` for secrets** (passwords, HMACs, tokens). It short-circuits on the first differing byte, leaking information via timing. Use a constant-time comparison:

```
static int ct_equal(const void *a, const void *b, size_t n)
{
    const unsigned char *x = a, *y = b;
    unsigned char diff = 0;
    for (size_t i = 0; i < n; i++) diff |= x[i] ^ y[i];
    return diff == 0;
}
```

Copying and concatenation

Function	Safety	Verdict
<code>strcpy</code>	none	✗ Banned

Function	Safety	Verdict
strcat	none	✗ Banned
sprintf	none	✗ Banned
gets	none	✗ Removed from C11
strncpy	partial, misleading	⚠ Not what you think
strncat	partial, confusing n	⚠ Error-prone
snprintf	good	☑ Use this
strlcpy/strlcat	good	☑ Where available
memcpy/memmove	you supply the length	☑ With correct arithmetic

Searching

Function	Finds
strchr(s, c)	First occurrence of c (NUL matches — strchr(s,0) finds the terminator)
strrchr(s, c)	Last occurrence
strstr(hay, needle)	First substring
memchr(p, c, n)	First byte in n bytes — works on binary data
strspn / strcspn	Length of prefix of (not) accepted chars
strpbrk	First occurrence of any char from a set

All return NULL when not found. strchr and strstr return a pointer **into the original string** — don't free it, and beware it dangles if you realloc the buffer.

strtok — avoid it

```
char *tok = strtok(buf, ",");
while (tok) { ...; tok = strtok(NULL, ","); }
```

Three fatal problems:

- 1. **It modifies the input in place** (writes NULs over delimiters). It cannot take a const char * or a string literal.
- 2. **It keeps hidden static state** — not reentrant, not thread-safe, and you cannot tokenize two strings at once (including from a function you call inside the loop).
- 3. It collapses consecutive delimiters, so it can't parse CSV with empty fields.

Use strtok_r (POSIX) / strtok_s (C11 Annex K, Windows), which take an explicit state pointer:


```
char *save;
for (char *t = strtok_r(buf, ",", &save); t; t = strtok_r(NULL, ",", &save)) { ...
}
```

Or write your own splitter with `strcspn` — usually clearer and it handles empty fields.

8.3 Why `strcpy` / `strcat` / `sprintf` / `gets` are banned

```
char dst[8];
strcpy(dst, "this is way too long");
```

Compile time (with `-O2`, thanks to `_FORTIFY_SOURCE`):

```
warning: '__builtin___memcpy_chk' forming offset [8, 20] is out of the bounds
[0, 8] of object 'dst' with type 'char[8]' [-Warray-bounds=]
```

Runtime with ASan:

```
ERROR: AddressSanitizer: stack-buffer-overflow
[32, 40) 'dst' <== Memory access at offset 40 overflows this variable
```

Runtime with `-D_FORTIFY_SOURCE=3`:

```
*** buffer overflow detected ***: terminated
Aborted
```

The problem is structural: **these functions have no way to know the destination size**. They write until they hit a NUL in the source. If the source is longer than the destination, they scribble over whatever is next — locals, saved registers, and eventually the **return address** (Module 6's stack frame diagram). That's arbitrary code execution.

`gets` was removed from the language

```
char b[8]; gets(b);
```

```
warning: implicit declaration of function 'gets'; did you mean 'fgets'?
warning: the `gets' function is dangerous and should not be used.
```

`gets` had **no size parameter at all** — it was unconditionally exploitable. It was deprecated in C99 and **removed in C11**. The Morris Worm (1988) used a `gets` overflow in `fingerd`.

Use `fgets(buf, sizeof buf, stdin)` — but note it **keeps the newline**:

```
if (fgets(buf, sizeof buf, stdin)) {
    buf[strcspn(buf, "\n")] = '\0';    /* strip newline, idiomatically */
}
```

`strncpy` is *not* a safe `strcpy`

This is the single most misunderstood function in the C library. It was designed in 1979 for fixed-width **fields in a record**, not for strings.

Trap 1 — it does not terminate if the source fills the buffer:

```
char a[6]; strncpy(a, "abcdef", 6);
```

```
strncpy(a,"abcdef",6)      61 62 63 64 65 66
-> no 00 at the end. strlen(a) would run off.
```

```
warning: 'strncpy' output truncated before terminating nul [-Wstringop-truncation]
```

Trap 2 — it pads with NULs, always writing n bytes:

```
char b[10]; strncpy(b, "hi", 10);
```

```
strncpy(b,"hi",10)        68 69 00 00 00 00 00 00 00 00
-> wrote 10 bytes to copy 2. strncpy is NOT 'safe strcpy'.
```

So `strncpy` into a 4 KB buffer writes 4 KB regardless of the string length. It's $O(n)$ in the *buffer* size, not the string size.

If you must use it:

```
strncpy(dst, src, sizeof dst - 1);
dst[sizeof dst - 1] = '\0';    /* mandatory manual terminate */
```

strncat's n means something different

`strncat(dst, src, n)` — `n` is the **maximum bytes appended**, not the buffer size. It always adds a NUL, so it writes up to `n+1` bytes past the existing content:

```
strncat(dst, src, sizeof dst - strlen(dst) - 1);    /* the correct computation */
```

```
strncat(d,...,sizeof d - strlen(d) - 1) -> "abcdefghi" (len 9)
```

Get that arithmetic wrong by one and you overflow. It's also $O(n)$ in `strlen(dst)` each call, so repeated `strncat` is **Schlemiel the Painter's algorithm** — $O(n^2)$.

snprintf — the workhorse

```
int snprintf(char *dst, size_t size, const char *fmt, ...);
```

Guarantees:

- Writes at most `size` bytes **including** the NUL.
- **Always NUL-terminates** (when `size > 0`).
- Returns **the number of characters that *would* have been written**, excluding the NUL.

That return value is the key:

```
char e[8];
int r = snprintf(e, sizeof e, "%s-%d", "value", 12345);
```

```
snprintf(e,8,...) -> ret=11, e="value-1" (len 7)
ret >= sizeof(e) means TRUNCATION occurred: YES
```

The truncation check idiom:

```
int r = snprintf(buf, sizeof buf, fmt, ...);
if (r < 0 || (size_t)r >= sizeof buf) {
    /* error or truncated – handle it, don't ignore it */
}
```

The measure-then-allocate idiom:

```
int n = snprintf(NULL, 0, fmt, args);    /* size==0, dst may be NULL */
char *buf = malloc((size_t)n + 1);
snprintf(buf, (size_t)n + 1, fmt, args);
```

⚠ **Never** `snprintf(buf, sizeof buf, buf)` or otherwise let source and destination overlap — that's UB.

⚠ **On old MSVC, `_snprintf` did NOT terminate on truncation and returned `-1`.** If you support ancient Windows toolchains, wrap it.

`strncpy` / `strlcat` — OpenBSD's answer

```
size_t strncpy(char *dst, const char *src, size_t dstsize);
size_t strlcat(char *dst, const char *src, size_t dstsize);
```

- **Always NUL-terminate** (if `dstsize > 0`)
- **Never pad**
- Return `strlen(src)` (or `strlen(dst) + strlen(src)` for `strlcat`) — **the length it tried to create**, so `ret >= dstsize` means truncation
- `strlcat` takes the **full buffer size**, not the remaining space — much harder to get wrong than `strncat`

```
strncpy(d, "toolongstring", 8) -> returns 13, d = "toolong"
```

Availability: BSD, macOS, and **glibc 2.38+** (this box runs 2.39). Not in ISO C, not in older glibc, not in MSVC. If you need portability, ship a 10-line implementation.

Annex K `_s` functions — don't bother

C11 Annex K added `strcpy_s`, `sprintf_s`, etc. It's **optional**, only Microsoft implements it meaningfully, glibc refused it, and WG14 has proposed removing it. Skip it.

8.4 `memcpy` vs `memmove`

The prototypes tell the whole story:

```
void *memcpy (void *restrict dst, const void *restrict src, size_t n);
void *memmove(void *dst,          const void *src,          size_t n);
```

`restrict` (Module 2) is a promise that the regions do not overlap. Break it and you get UB — the compiler is free to copy in any order, use vector loads, or read from both directions.

`memmove` makes no such promise and is guaranteed correct for overlapping regions (as if via a temporary buffer).

The honest test result

```
shift RIGHT by 2 (dst > src) – this is where memcpy breaks:
memcpy(b+2,b,8)           "ABABCDEF GH"
memmove(b+2,b,8)          "ABABCDEF GH"
expected: ABABCDEF GH
```

Both gave the right answer. Even at 4096 bytes with the offset hidden from the optimizer, glibc's `memcpy` produced correct output.

This is exactly why the bug is dangerous. It works on your machine, on your glibc version, at your buffer size — until it doesn't. In 2010 glibc changed `memcpy` on x86-64 to copy backwards for performance, and **Adobe Flash broke worldwide** because it had been relying on overlapping `memcpy`. Linus Torvalds and Ulrich Drepper had a famous public argument about it. The answer was: the code was always wrong.

What actually protects you is the **compiler**, which caught it statically:

```
warning: '__builtin_memcpy' accessing 8 bytes at offsets 0 and 2
overlaps 6 bytes at offset 2 [-Wrestrict]
```

Rule: if the regions might overlap, use `memmove`. The performance difference is negligible; on most platforms `memmove` just checks the pointer order once.

The other `mem*` functions

```
memset(p, 0, n);           /* fill with a BYTE value – memset(a,1,n) is NOT all-
ones ints */
memchr(p, c, n);           /* search binary data */
memcmp(a, b, n);           /* see the constant-time warning above */
```

⚠ **`memset` to zero secrets can be optimized away** — the compiler sees a dead store to a variable about to go out of scope and deletes it. Use `explicit_bzero` (glibc/BSD), `memset_explicit` (C23), or `SecureZeroMemory` (Windows).

8.5 String → number conversion

`atoi` is unusable in production

```
atoi("42") = 42
atoi("abc") = 0
```


- 4. **Range-check separately if your target is narrower than `long`.** `ERANGE` only fires at the `long` boundary.

Bases

```
strtol("0x1F", &e, 0) = 31    (base 0 auto-detects)
strtol("017", &e, 0) = 15    (leading 0 = octal!)
```

`base = 0` accepts C literal syntax including **octal**, which surprises users who type leading zeros. Use an explicit base 10 for user input unless you specifically want C syntax.

The family

Function	Returns
<code>strtol / strtoll</code>	<code>long / long long</code>
<code>strtoul / strtoull</code>	unsigned — ⚠ accepts "-1" and wraps to <code>ULONG_MAX</code>
<code>strtod / strtodf / strtold</code>	floating point
<code>strtoimax / strtoumax</code>	<code>intmax_t</code> (<inttypes.h>)

```
strtod("3.14xyz") = 3.14, stopped at "xyz"
```

- ⚠ `strtoul("-1")` returns `ULONG_MAX`, **not an error**. If negatives are invalid, check for a '-' yourself.
- ⚠ **`strtod` is locale-dependent** — in a `de_DE` locale the decimal separator is `,`. For parsing config files and network protocols, either force `"C"` locale or use `strtod_l`.

`sscanf` — usable but leaky

`sscanf` can't report overflow and `%s` without a width is a buffer overflow. If you use it, always bound `%s` and check the return value:

```
char name[32];
if (sscanf(line, "%31s %d", name, &num) != 2) { /* parse error */ }
```

8.6 Wide strings, multibyte, and UTF-8

The `wchar_t` mess

Platform	<code>sizeof(wchar_t)</code>	Encoding
Linux/macOS	4	UTF-32

Platform	sizeof(wchar_t)	Encoding
Windows	2	UTF-16 (so it can't hold every code point in one unit)

wchar_t is not portable and does not solve Unicode. C11 added `char16_t/char32_t` in `<uchar.h>` with explicit encodings, but almost nobody uses them.

The modern answer: UTF-8 in plain `char`

```
const char *s = "héllø→世界";
```

```
strlen()  = 15    <- BYTES
u8_len()  = 8     <- CODE POINTS

byte dump: 68 C3 A9 6C 6C 6F E2 86 92 E4 B8 96 E7 95 8C
```

UTF-8 was designed by Ken Thompson and Rob Pike specifically to work with existing C string functions. The properties that make it work:

Property	Consequence
ASCII (0x00–0x7F) unchanged	All existing ASCII code keeps working
No byte of a multibyte sequence is ever 0x00	<code>strlen</code> , <code>strcpy</code> , <code>strchr</code> don't break
Lead bytes are <code>11xxxxxx</code> , continuations are <code>10xxxxxx</code>	Self-synchronizing — you can find a boundary from anywhere
Multibyte sequences never contain an ASCII byte	<code>strchr(s, '/')</code> and <code>strstr</code> give no false positives
Byte order matches code point order	<code>strcmp</code> sorts correctly by code point

The encoding

Code points	Bytes	Pattern
U+0000–U+007F	1	<code>0xxxxxxx</code>
U+0080–U+07FF	2	<code>110xxxxx 10xxxxxx</code>
U+0800–U+FFFF	3	<code>1110xxxx 10xxxxxx 10xxxxxx</code>
U+10000–U+10FFFF	4	<code>11110xxx 10xxxxxx 10xxxxxx 10xxxxxx</code>

Counting code points is trivial — count bytes that aren't continuations:


```

size_t u8_len(const char *s)
{
    size_t n = 0;
    for (; *s; s++) if ((*s & 0xC0) != 0x80) n++;
    return n;
}

```

Decoding:

```

U+0068  (1 bytes)  "h"
U+00E9  (2 bytes)  "é"
U+2192  (3 bytes)  "→"
U+4E16  (3 bytes)  "世"
U+754C  (3 bytes)  "界"

```

The practical rules

1. **Store and process everything as UTF-8 in `char` arrays.** Use `wchar_t` only at Windows API boundaries.
2. **Most operations don't need decoding at all.** Concatenation, comparison, searching for ASCII delimiters, hashing, and copying all work byte-wise.
3. **Decode only when you need code points** — cursor movement, character counting, case conversion.
4. **Never split a UTF-8 string at an arbitrary byte offset.** Back up while `(b & 0xC0) == 0x80`.
5. **Validate untrusted UTF-8 on input.** Reject overlong encodings (e.g. `C0 80` for NUL), surrogates (U+D800–DFFF), and values above U+10FFFF. Overlong encodings have been used to bypass path-traversal filters.
6. **Code points ≠ user-perceived characters.** "é" can be one code point or two (e + combining accent); emoji use ZWJ sequences. Real grapheme clustering needs ICU or `utf8proc` — don't roll it yourself.
7. `printf("%s", utf8)` just works — it's byte-transparent. But `%-10s` pads by **bytes**, so column alignment will be wrong for non-ASCII.

8.7 Building a string abstraction

Serious C code stops using raw `char *` and builds a length-carrying type. Here's a complete, verified one.

```

typedef struct { char *data; size_t len, cap; } SB;

static bool sb_reserve(SB *b, size_t need) /* need = extra bytes, excl
NUL */
{
    if (need > SIZE_MAX - 1 - b->len) return false; /* overflow guard */
    size_t want = b->len + need + 1;
    if (want <= b->cap) return true;
    size_t cap = b->cap ? b->cap : 16;
    while (cap < want) { if (cap > SIZE_MAX/2) return false; cap *= 2; }
}

```

```

    char *p = realloc(b->data, cap);
    if (!p) return false; /* b->data still
valid! */
    b->data = p; b->cap = cap;
    return true;
}

static bool sb_append(SB *b, const char *s, size_t n)
{
    if (!sb_reserve(b, n)) return false;
    memcpy(b->data + b->len, s, n);
    b->len += n;
    b->data[b->len] = '\0';
    return true;
}

__attribute__((format(printf, 2, 3)))
static bool sb_printf(SB *b, const char *fmt, ...)
{
    va_list ap, ap2;
    va_start(ap, fmt); va_copy(ap2, ap);
    int n = vsnprintf(NULL, 0, fmt, ap); /* measure */
    va_end(ap);
    bool ok = false;
    if (n >= 0 && sb_reserve(b, (size_t)n)) {
        vsnprintf(b->data + b->len, b->cap - b->len, fmt, ap2);
        b->len += (size_t)n;
        ok = true;
    }
    va_end(ap2);
    return ok;
}

static void sb_free(SB *b) { free(b->data); *b = (SB){0}; }

```

```

result: "Hello, world! You are 42 years old. [raw]"
len=41 cap=64

growth under 10000 appends:
final len=48890 cap=65536 (amortized O(1) per append)

ASan/UBSan/LSan exit=0

```

Every design decision here matters

Decision	Why
Store <code>len</code> explicitly	$O(1)$ length, allows embedded NULs, no repeated scanning
Keep a trailing NUL anyway	<code>b.data</code> works with every C API for free

Decision	Why
Doubling growth	Amortized $O(1)$ append. Linear growth gives $O(n^2)$.
<code>realloc</code> into a temp first	On failure, <code>b->data</code> is still valid — <code>b->data = realloc(b->data,...)</code> leaks on NULL
Overflow guard before arithmetic	<code>len + need + 1</code> can wrap; a wrapped size allocates tiny and then overflows the heap
<code>va_copy</code> before measuring	The measure pass consumes <code>ap</code> (Module 6)
<code>format(printf,2,3)</code> attribute	Compile-time format checking
<code>*b = (SB){0}</code> on free	Prevents use-after-free and double-free

This is the pattern behind every real C string library: SDS (Redis), bstring, sds, `struct Buffer` in curl, `strbuf` in git. Once you have it, `strcpy/strcat/sprintf` never appear in your code again.

Module 8 rules

#	Rule
1	Track length and capacity as separate numbers, always
2	Hoist <code>strlen</code> out of loop conditions
3	<code>strcmp</code> returns a sign, not $-1/0/1$
4	Never <code>memcmp</code> structs (padding) or secrets (timing)
5	Banned: <code>strcpy</code> , <code>strcat</code> , <code>sprintf</code> , <code>gets</code> , <code>atoi</code>
6	<code>strncpy</code> neither terminates reliably nor is efficient — it is not "safe strcpy"
7	<code>snprintf</code> return $\geq$ buffer size means truncation — check it
8	<code>strncpy/strlcat</code> where available; ship your own otherwise
9	Overlapping regions → <code>memmove</code> . "It worked" is not "it's defined."
10	<code>strtol</code> needs all four checks: <code>errno=0</code> , <code>end==s</code> , <code>*end</code> , range
11	UTF-8 in plain <code>char</code> ; decode only when you need code points
12	Validate untrusted UTF-8: reject overlong forms, surrogates, $> U+10FFFF$
13	Build a length-carrying string type; guard every size computation against overflow

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
-Wformat=2 -Wstringop-overflow=4 -Wstringop-truncation -Wrestrict \
```

```
-Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
-D_FORTIFY_SOURCE=3 -O1 -g -fsanitize=address,undefined prog.c -o prog
```

(`_FORTIFY_SOURCE` requires at least `-O1` to work.)

Practice

1. **Break `strcpy` deliberately.** Overflow a 16-byte stack buffer with a 100-byte string. Run it plain, under ASan, and with `-D_FORTIFY_SOURCE=3 -O2`. Compare the three failure modes. Then find where the return address sits relative to the buffer using `&buf` and a backtrace.
2. **`strncpy` audit.** Write a program proving both traps (no terminator, NUL padding). Then implement your own `strcpy` and `strcat` from scratch and test against the edge cases: `dstsize == 0`, source longer than dest, exact fit, empty source.
3. **Truncation detection.** Write `bool build_path(char *out, size_t n, const char *dir, const char *file)` using `snprintf` that returns `false` on truncation. Test with a 8-byte buffer.
4. **The overlap experiment.** Copy an array onto itself shifted right, using `memcpy`, at sizes 8, 64, 512, 4096, 65536. Record where (if anywhere) glibc's `memcpy` diverges from `memmove`. Then compile with `-Wrestrict` and note that the *compiler* is your real defense.
5. **Robust integer parsing.** Write `int parse_u16(const char *s, uint16_t *out)` handling: empty string, leading/trailing whitespace, +/- signs, overflow, trailing garbage, and `NULL` input. Write a test table with 20 cases including "65535", "65536", "-0", " 12 ", "0x10".
6. **UTF-8 toolkit.** Implement `u8_len`, `u8_next` (decode + advance), `u8_prev` (back up one code point), `u8_validate` (rejecting overlong forms, surrogates, and > U+10FFFF), and `u8_truncate(char *s, size_t max_bytes)` that never splits a sequence. Test with a string mixing ASCII, Latin-1, CJK, and a 4-byte emoji.
7. **Finish the string builder.** Extend the `SB` from §8.7 with `sb_insert`, `sb_erase`, `sb_find`, `sb_split` (returning an array of views), and `sb_shrink_to_fit`. Add `_Static_asserts` and run a 100,000-operation fuzz workload under ASan + UBSan + LSan. Then benchmark it against naive `strcat` in a loop and measure the $O(n)$ vs $O(n^2)$ difference.
8. **Constant-time comparison.** Implement `ct_equal` and prove it's timing-independent by measuring 10 million comparisons of strings differing at position 0 vs position 31. Compare against `memcmp`.

9. Pointers (the core skill)

- Address-of, dereference, pointer declaration syntax
- Pointer arithmetic and its rules
- `void*` and generic programming
- `NULL`, null checks, `nullptr` (C23)
- Pointer to pointer, arrays of pointers, pointer to array
- `const` correctness: `const char *` vs `char * const` vs `const char * const`
- Function pointers, pointer to function returning pointer, `typedef` for readability

- Dangling pointers, wild pointers, uninitialized pointers
- Alignment, `uintptr_t`, pointer-integer conversion
- Reading complex declarations (spiral rule / `cdecl`)

Module 9 — Pointers

The core skill. Everything from Module 7 (decay) and Module 8 (strings) rests on this. Real GCC 13.3 output throughout.

9.1 Address-of, dereference, declaration syntax

```
int x = 42;
int *p = &x;    /* & takes an address */
*p = 99;        /* * accesses the pointed-to object */
```

```
x = 42      &x = 0x7ffde852f9d0
p = 0x7ffde852f9d0  *p = 42
&p = 0x7ffde852f9d8  <- the pointer itself has an address

after *p = 99:  x = 99  (same object)
```

A pointer is just a variable whose value is an address. It has its own address, its own size, its own lifetime.

The `*` binds to the variable, not the type

```
int *a, b;    /* a is int*, b is int  !! */
int *c, *d;    /* both pointers */
```

```
int *a, b;    -> a is int*, b is int  (sizeof b = 4)
```

This is why the C convention is `int *p`, not `int* p`. Writing `int* a, b;` reads as if both were pointers — they aren't. The Linux kernel, K&R, and virtually every C style guide use `int *p`.

Better still: **one declaration per line**. It sidesteps the issue entirely.

Sizes

```
char*=8 int*=8 double*=8 void*=8 struct*=8
```

On mainstream platforms all object pointers are the same size, but **the standard does not guarantee this**. Historically, `char *` was larger than `int *` on word-addressed machines. Function pointers may differ from object pointers (they do on some Harvard-architecture MCUs). Never assume; use `sizeof`.

`*` and `&` cancel

`*&x` is `x`. `&*p` is `p` (and it's the one context where a null `p` doesn't get dereferenced — `&*p` on NULL is still UB in C, though).

9.2 Pointer arithmetic and its rules

Arithmetic is scaled by the pointed-to type

```
int*    p+1 advances 4 bytes
double* q+1 advances 8 bytes
char*   advances 1 byte  <- that is why char* is used for byte math
```

`p + n` means "advance by `n` **objects**," not `n` bytes. This is why `a[i] == *(a+i)` works for any element type.

Pointer difference yields `ptrdiff_t`

```
&a[4] - &a[0] = 4 elements (not 16 bytes)
```

Subtracting two pointers gives the **element count** between them, of type `ptrdiff_t` (signed). Print with `%td`.

Both pointers must point into the **same array object** (or one past its end). Subtracting unrelated pointers is UB.

The one-past-the-end rule

```
a+5 = 0x7ffc8fe3e074 (legal address, dereferencing is UB)
loop idiom: for (int *it=a; it!=end; ++it) -> 10 20 30 40 50
a+6 or a-1 -> UNDEFINED even without dereferencing
```

You may form and compare a pointer one past the end of an array. You may not dereference it.

Anything further — `a+6`, `a-1` — is UB **just to compute**, even if you never dereference it.

This is not pedantry. On segmented architectures forming an out-of-range address can trap, and on modern compilers it enables optimizations that break your code.

The rule is *why* the standard iterator idiom works:

```
int *end = arr + n;
for (int *it = arr; it != end; ++it) { ... }
```

Comparison rules

Operation	Defined?
<code>p == q, p != q</code>	Always
<code>p < q, p > q</code>	Only within one array object (or one-past)
<code>p - q</code>	Only within one array object
<code>p + n</code>	Only if the result stays in <code>[array, array+n]</code>

Comparing pointers into two unrelated objects with `<` is **unspecified** — it usually "works" because of flat address spaces, but the compiler isn't required to give a consistent answer.

Also UB

- `NULL + 0, NULL - NULL` — technically UB in C (C++ defines them; C23 relaxed some of this)
- Arithmetic on a `void *` — **not standard C** (GNU treats `void` as size 1):

```
error: pointer of type 'void *' used in arithmetic [-Wpointer-arith]
```

- Arithmetic on function pointers
- Arithmetic on a pointer to an incomplete type

For byte-level arithmetic, cast to `unsigned char *` — it's the one type guaranteed to have size 1 and to be exempt from strict aliasing (Module 2).

9.3 `void *` and generic programming

`void *` is the generic object pointer. In C — unlike C++ — it converts **implicitly** in both directions:

```
void *v = arr;
int *ip = v;           /* legal in C, no cast needed */
void *p = malloc(n);   /* don't cast malloc */
```

What you can't do

```
*v;           /* ERROR: cannot dereference */
v + 1;        /* not standard C */
sizeof(void); /* not standard C (GNU says 1) */
```

And critically: **`void *` does not cover function pointers**. ISO C makes no guarantee that a function pointer survives a round trip through `void *`. POSIX requires it (which is what makes `dlsym` work), but it's a

documented extension, not portable C.

The generic-programming pattern

Since C has no templates, generic code takes `void *` + a size + a comparator:

```
static void swap(void *a, void *b, size_t n)
{
    unsigned char *x = a, *y = b, t;
    while (n--) { t = *x; *x++ = *y; *y++ = t; }
}

static void *find(const void *base, size_t n, size_t sz, const void *key,
                  int (*cmp)(const void *, const void *))
{
    const unsigned char *p = base;
    for (size_t i = 0; i < n; i++, p += sz)
        if (cmp(p, key) == 0) return (void *)p;
    return NULL;
}
```

```
swap ints: a=2 b=1
swap doubles: x=2.5 y=1.5
swap arrays: xyz abc
find(9) -> index 2
```

This is exactly how `qsort` and `bsearch` work. The cost: **no type safety** — pass the wrong `sz` and you silently corrupt memory. Module 28 covers safer alternatives (macro templates, `_Generic`).

9.4 `NULL`, null checks, `nullptr`

```
NULL prints as (nil), comparisons: (p==NULL)=1  (!p)=1
```

`NULL` is a null pointer constant — typically `((void*)0)` or `0`. A null pointer is guaranteed to compare unequal to any pointer to a real object.

Subtleties worth knowing

- **`NULL` is not guaranteed to be all-zero bits.** In practice it's address 0 on every platform you'll meet, but the standard only guarantees that a *zero-valued constant expression* converts to it. Consequence: `memset(&ptr, 0, sizeof ptr)` and `calloc` are not *formally* guaranteed to produce `NULL` pointers. (They do everywhere real.)
- **Dereferencing `NULL` is UB, not "a segfault."** The compiler may assume a pointer you dereferenced is non-null and delete your subsequent null check — a real CVE pattern in the Linux kernel (CVE-2009-

1897).

- **Never pass bare `NULL` as a variadic argument.** `execl(path, "a", NULL)` may pass a 4-byte `int` where an 8-byte pointer is read. Write `(char *)NULL`.

C23 `nullptr`

C23 `nullptr` available, `sizeof(nullptr_t)=8`, `q==NULL: 1`

C23 adds a real `nullptr` keyword with its own type `nullptr_t`. Benefits: it's unambiguously a pointer (fixing the variadic problem) and works cleanly with `_Generic`.

Null-check discipline

```
/* Check what can fail */
FILE *fp = fopen(path, "r");
if (!fp) return -1;

char *buf = malloc(n);
if (!buf) return -1;

/* Document what can't be NULL – and consider enforcing it */
void process(const char *s)
{
    assert(s != NULL);           /* debug builds */
    /* or: */ if (!s) return -1; /* defensive, for public APIs */
}

/* C99: tell the compiler and the reader */
void process(const char s[static 1]); /* "s is never NULL" */
```

Style note: `if (!p)` and `if (p == NULL)` are both idiomatic. Pick one per codebase. **Never write `if (p == 0)`** — it obscures that `p` is a pointer.

9.5 Pointer to pointer, arrays of pointers, pointer to array

The three shapes people confuse

```
int *b[5];           /* ARRAY of 5 pointers to int    – 40 bytes */
int (*c)[5];         /* POINTER to an array of 5 ints    – 8 bytes  */
int **d;             /* POINTER to a pointer to int     – 8 bytes  */
```

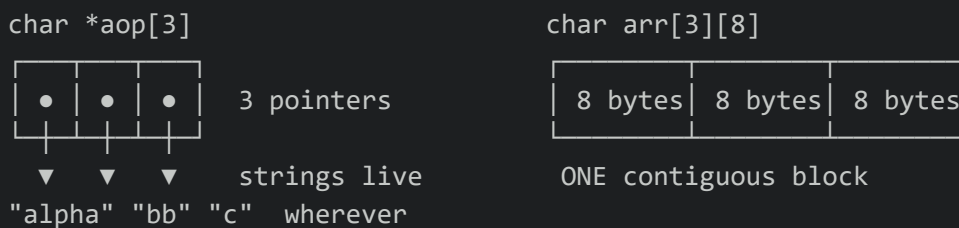
```
sizeof:  int a[5]=20  int *b[5]=40  int(*c)[5]=8
```

`[]` binds tighter than `*`, so `int *b[5]` is "array of pointers." Parentheses are required for "pointer to array."

Array of pointers vs 2D array — the memory difference

```
char *aop[3]    sizeof=24  (3 pointers)
char arr[3][8] sizeof=24  (24 contiguous bytes)
aop[0]=0x...80c0 aop[1]=0x...8100 -> scattered
arr[0]=0x...0080 arr[1]=0x...0088 -> exactly 8 apart
```

Same total size here, completely different structure:



Choose array-of-pointers when rows have different lengths (ragged data), you need to reorder rows cheaply (swap pointers, not data), or elements come from different allocations.

Choose the 2D array when you want one allocation, contiguous memory, cache locality, and a fixed row width.

`argv` is the canonical array of pointers

```
argv      = 0x7fffa22da698  (char**)
argv[0]   = 0x7fffa22dbd24 -> "./pp"
argv[1]   = 0x7fffa22dbd29 -> "one"
argv[2]   = 0x7fffa22dbd2d -> "two"
argv[3]   = NULL
```

Note the string addresses are only 5 and 4 bytes apart — the strings are packed contiguously, but `argv` itself is a separate array of pointers into them.

Pointer to pointer — modifying the caller's pointer

```
char *buf = malloc(8);
char **pbuf = &buf;
*pbuf = realloc(buf, 64);    /* the caller's `buf` is updated */
```

```
before: buf=0x502000000010
after realloc via **: buf=0x506000000020 "hello"
```

Any function that must **change where the caller's pointer points** takes a `T **`:

```
int grow(char **buf, size_t *cap);
void free_and_null(void **p) { free(*p); *p = NULL; }
```

The two ways to allocate a 2D array

```
/* A: flat - 1 malloc, contiguous, best cache behavior */
int *flat = malloc(R * C * sizeof *flat);
flat[i*C + j] = 7;
free(flat);

/* B: array of pointers - R+1 mallocs, ragged-capable, 2 dereferences */
int **rows = malloc(R * sizeof *rows);
for (size_t i = 0; i < R; i++) rows[i] = malloc(C * sizeof **rows);
rows[i][j] = 7;
for (size_t i = 0; i < R; i++) free(rows[i]);
free(rows);
```

```
A flat: 1 malloc, contiguous, flat[i*C+j]=7
B ragged: 4 mallocs, rows[i][j]=7, two derefs, needs 4 frees
```

Prefer A. It's faster, has one failure path, one `free`, and can't leak partially. Use B only when rows genuinely differ in length.

There's a hybrid worth knowing: allocate one flat block *plus* a row-pointer array pointing into it. You get `m[i][j]` syntax with one allocation.

9.6 `const` correctness

```
int *p1           = &x;    /* write OK, repoint OK */
const int *p2     = &x;    /* write NO, repoint OK */
int const *p2b    = &x;    /* IDENTICAL to p2 */
int *const p3     = &x;    /* write OK, repoint NO */
const int *const p4 = &x;  /* write NO, repoint NO */
```

```
p1: write OK, repoint OK
p2: write NO, repoint OK
p3: write OK, repoint NO
p4: write NO, repoint NO
```

The reading rule

const applies to whatever is immediately to its LEFT. If there is nothing to its left, it applies to what's on its RIGHT.

```
const int *p    → nothing left of const → applies right → const int. Pointer to
const int.
int const *p    → int is left of const → const int. Same thing.
int * const p   → * is left of const → const pointer. Pointer (const) to int.
```

Read declarations **right-to-left from the identifier**:

- `const int *p` → "p is a pointer to an int that is const"
- `int *const p` → "p is a const pointer to an int"

The `int const *` spelling makes the rule uniform, which is why some codebases mandate it. `const int *` is more common. Pick one.

Conversions

```
char* -> const char*    : implicit, safe (adding const)
const char* -> char*     : requires a cast, and writing through it may be UB
```

Adding `const` is always safe and implicit. Removing it needs a cast — and if the underlying object really is `const`, writing through the result is UB (Module 2 showed the compiler folding a `const` to a literal and giving two different answers for the same memory).

Catch it with:

```
warning: cast discards 'const' qualifier from pointer target type [-Wcast-qual]
```

Why `char **` → `const char **` is rejected

```
void f(const char **p);
char *q;
f(&q);
```

```
warning: passing argument 1 of 'f' from incompatible pointer type
```

This surprises people, but the rejection is correct. If it were allowed:

```
const char *secret = "const data";
char *q;
f(&q);                /* if this were legal... */
/* inside f: *p = secret; ← stores a const pointer into q */
q[0] = 'X';           /* ...you'd now be writing to const memory, legally-
looking */
```

The conversion $T^* \rightarrow \text{const } T^*$ is safe only at **one** level of indirection. Adding const deeper requires `const char *const *`.

Where const correctness earns its keep

```
size_t strlen(const char *s);                /* won't modify */
void *memcpy(void *dst, const void *src, size_t n);
int process(const Config *cfg, Output *out); /* input vs output is obvious */
```

Benefits: self-documenting APIs, compiler-enforced contracts, allows callers to pass genuinely `const` data, and enables some optimizations. **Apply `const` to every pointer parameter you don't modify.** It costs nothing and it's what reviewers expect.

9.7 Function pointers and complex declarations

(Extending Module 6 §6.6.)

```
int f(int);           /* function */
int *g(int);          /* function returning int* */
int (*d)(int);        /* POINTER to function */
int (*e[3])(int);     /* ARRAY of 3 function ptrs */
int (*h(int))(int);   /* function returning a function pointer */
```

```
sizeof: int(*d)(int)=8 int(*e[3])(int)=24
```

```
h(0)(41) = 42    <- h returns a function pointer, then we call it
e[1](10) = 11
```

`int (*h(int))(int)` is the shape of the real `signal()` prototype — famous for being unreadable. **Always typedef:**

```
typedef void (*sighandler_t)(int);
sighandler_t signal(int sig, sighandler_t handler); /* now readable */
```

Reading complex declarations — the right-left rule

Start at the identifier. Go RIGHT as far as you can, then LEFT. `()` and `[]` bind tighter than `*`. Parentheses group.

Worked example:

```
char *(*(**foo[][8])())[ ]

foo           foo is
foo[]         an array of
foo[][8]      arrays of 8
**           pointers to pointers to
()           functions returning
(*  )[]      pointers to arrays of
char *       pointers to char
```

The "spiral rule" you'll see online is a memorable approximation but **it's wrong for some declarations**. The right-left rule is correct. When in doubt, use `cdecl` (the program) or just add a `typedef`.

Practical advice

```
/* Instead of this: */
void (*(signal_table[16])(int, void (*)(int)))(int);

/* Write this: */
typedef void (*Handler)(int);
typedef Handler (*Installer)(int, Handler);
Installer signal_table[16];
```

If you need the right-left rule to read your own code, you should have used a typedef.

9.8 Dangling, wild, and uninitialized pointers

The complete taxonomy, each verified under ASan:

Bug	Cause	ASan diagnosis
Wild	Never initialized — holds stack garbage	often SEGV / random corruption
Dangling (stack)	Points to a dead local	stack-use-after-return / SEGV
Dangling (heap)	Used after free	heap-use-after-free
Double free	freed twice	attempting double-free

Bug	Cause	ASan diagnosis
Dangling after realloc	Old pointer kept after <code>realloc</code> moved the block	<code>heap-use-after-free</code>
NULL deref	Dereferencing NULL	<code>SEGV on address 0x0</code>

```

=== dangling stack ===
ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000

=== use after free ===
ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010

=== double free ===
ERROR: AddressSanitizer: attempting double-free on 0x502000000010

=== dangling after realloc ===
ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010

```

The `realloc` alias trap

```

char *a = malloc(8);
char *alias = a;           /* second pointer to the same block */
a = realloc(a, 1<<20);     /* may MOVE the block */
printf("%s", alias);       /* alias now dangles */

```

`realloc` may return a different address. Every other pointer into that block becomes invalid — including pointers stored in structs, in other data structures, and returned to callers earlier. This is why growable-array APIs return indices rather than pointers.

Also: `p = realloc(p, n)` **leaks the original block** if `realloc` returns NULL. Always:

```

char *tmp = realloc(p, n);
if (!tmp) { /* p is still valid – clean up properly */ return -1; }
p = tmp;

```

Defensive habits

```

int *p = NULL;           /* 1. Always initialize */

free(p); p = NULL;       /* 2. Null after free – makes double-free a no-op
                           (free(NULL) is a guaranteed no-op) */

/* 3. Have exactly one owner. Document it. */
/* 4. Never return a pointer to a local. -Wreturn-local-addr catches it. */

```

```
/* 5. Invalidate all aliases after realloc. */
/* 6. Run ASan on every debug build and in CI. */
```

⚠ `p = NULL` after `free` only protects **that** pointer. Every alias still dangles. There's no substitute for clear ownership rules — which is Module 10.

9.9 Alignment, `uintptr_t`, pointer↔integer conversion

```
ALIGNMENT REQUIREMENTS:
char=1 short=2 int=4 double=8 max_align_t=16
```

An object of type `T` must live at an address that's a multiple of `alignof(T)`. Violating this is UB.

`uintptr_t` — the only portable pointer↔integer type

```
p=0x7ffe51fb57b4 -> uintptr_t=140730273847220 -> back=0x7ffe51fb57b4 same? 1
alignment check: (u % alignof(int)) == 0
round up to 64: 140730273847232
```

`uintptr_t/intptr_t` (`<stdint.h>`) are guaranteed to round-trip a `void *`. **Never use `long`** — it's 32-bit on 64-bit Windows (Module 2).

Legitimate uses:

```
uintptr_t u = (uintptr_t)p;
if (u % 16 == 0) { /* aligned */ }
void *aligned = (void*)((u + 63) & ~(uintptr_t)63); /* round up */
size_t bucket = ((u >> 4) * 2654435761u) % nbuckets; /* pointer hashing */
uintptr_t tagged = u | 1; /* tagged pointers (low bits are free
                           when alignment > 1) */
```

Misaligned access

```
unsigned char buf[16];
uint32_t *p = (uint32_t*)(buf + 1); /* misaligned */
*p = 0x11223344;
```

```
runtime error: store to misaligned address 0x56248fefa101 for type 'uint32_t',
which requires 4 byte alignment
```


This is doubly undefined — misalignment *and* strict aliasing (Module 2). On x86 it "works" but is slower (and *faults* for SSE instructions). On some ARM and SPARC configurations it faults immediately or silently returns rotated garbage.

The fix is always `memcpy`:

```
uint32_t v;
memcpy(&v, buf + 1, sizeof v);    /* correct at any alignment, any aliasing */
```

```
memcpy round-trip: 0xDEADBEEF  <- always correct, any alignment
```

And it costs nothing — Module 2 showed `memcpy` of a fixed size compiles to a single `mov`.

Aligned allocation

```
malloc      -> 0x55e302aac2b0 (aligned to max_align_t = 16? yes)
aligned_alloc(64,128) -> 0x55e302aac300 (% 64 = 0)
```

`malloc` is guaranteed to return memory suitably aligned for **any** standard type (`alignof(max_align_t)`). For stronger alignment — SIMD, cache lines, DMA — use:

- `aligned_alloc(alignment, size)` — C11. **Size must be a multiple of alignment.** Free with `free`.
- `posix_memalign(&p, align, size)` — POSIX
- `_aligned_malloc` / `_aligned_free` — Windows (needs the matching free!)

Module 9 rules

#	Rule
1	Write <code>int *p</code> , one declaration per line
2	Pointer arithmetic is scaled by the element type; differences yield <code>ptrdiff_t</code> (%td)
3	One-past-the-end is legal to form; two-past and <code>a-1</code> are UB even unread
4	<code></></code> between unrelated objects is unspecified; <code>==/!=</code> is always fine
5	Cast to <code>unsigned char *</code> for byte arithmetic — never <code>void *</code>
6	<code>void *</code> doesn't cover function pointers in ISO C
7	<code>(char*)NULL</code> in variadic calls; C23 gives you <code>nullptr</code>
8	<code>int *b[5]</code> vs <code>int (*c)[5]</code> vs <code>int **d</code> — three different things
9	Prefer a single flat allocation over an array of row pointers

#	Rule
10	<code>const</code> binds to its left; add <code>const</code> to every parameter you don't modify
11	<code>T**</code> → <code>const T**</code> is correctly rejected — it's not a compiler bug
12	<code>typedef</code> every function-pointer type; if you need the right-left rule, use a <code>typedef</code>
13	<code>realloc</code> invalidates every alias; never <code>p = realloc(p, n)</code>
14	Initialize to NULL, null after free, one owner per allocation
15	Type-pun and read unaligned data with <code>memcpy</code> , never a pointer cast
16	<code>uintptr_t</code> for pointer→integer, never <code>long</code>

Standing build command:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
-Wcast-qual -Wcast-align=strict -Wpointer-arith -Wwrite-strings \
-Wnull-dereference -Wstrict-aliasing=2 \
-Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
-O1 -g -D_FORTIFY_SOURCE=3 -fsanitize=address,undefined prog.c -o prog
```

`-Wwrite-strings` makes string literals `const char[]`, catching `char *s = "literal";` — a genuinely useful addition.

Practice

- Arithmetic forensics.** For `int a[5]`, print the byte offsets of `a+i`, `&a[i]`, `(char*)a+i`, and `&a+1`. Then compute `&a[5] - &a[0]` and `(char*)&a[5] - (char*)&a[0]` and explain both numbers.
- One-past-the-end.** Write the iterator loop with `!=` against `arr+n`. Then deliberately compute `arr+n+1` and `arr-1`. Compile with `-fsanitize=address,undefined` and with `-O2` and see whether anything complains. (Spoiler: mostly not — which is the lesson.)
- Generic library.** Implement `void *gmap(const void *src, size_t n, size_t sz, void *dst, void (*fn)(const void*, void*))` and use it to double an `int` array, uppercase a `char` array, and negate a `double` array. Then write `gsort` wrapping `qsort`.
- argv map.** Print `argv`, each `argv[i]`, and each string's bytes, plus the gaps between consecutive strings. Then do the same for `envp`. Draw the memory layout.
- const matrix.** Write a table of all four `const` placements. For each, write the two lines that *should* fail to compile. Verify each error message.
- The `const char**` puzzle.** Reproduce the rejected conversion. Then write the version with `const char *const *` that is accepted, and explain why it's safe.
- Declaration gauntlet.** Without a compiler, describe each, then verify with a test program:

```
int *a[10];          int (*b)[10];          int (*c)(int);
int *(*d)(int);      int (*e[5])(void);    int ((*f)(int))[10];
char *(*g[3])(const char *, int);          void (*h(int, void (*)(int)))
(int);
```

8. **Bug taxonomy lab.** Write one program with six modes reproducing wild, dangling-stack, use-after-free, double-free, realloc-alias, and NULL-deref. Run each under ASan, Valgrind, and plain. Record which tool catches what, and which cases "work" without a tool.

9. **Alignment tool.** Write `bool is_aligned(const void *p, size_t a), void *align_up(void *p, size_t a)`, and a `read_u32_unaligned(const void *p)` using `memcpy`. Verify with UBSan that the naive cast version is flagged and yours isn't. Then `objdump -d` both and compare the generated instructions.

10. **Tagged pointers.** Given that `malloc` returns 16-byte-aligned memory, the low 4 bits of any returned pointer are zero. Build a tagged-pointer type that stores a 4-bit tag in those bits, with `tag_get`, `tag_set`, and `ptr_get`. Verify with ASan that the untagged pointer is always usable.

10. Memory Management

- Memory layout: text, rodata, data, BSS, heap, stack
- Stack frames, stack overflow, recursion limits
- `malloc`, `calloc`, `realloc`, `free`, `aligned_alloc`
- Checking allocation failure; `realloc` failure pattern
- **Leaks, double free, use-after-free, buffer overflow, heap corruption**
- Ownership conventions and who-frees-what discipline
- Custom allocators: arena/bump, pool, freelist, slab, stack allocator
- `alloca` (and why to avoid it)
- Zeroing sensitive memory (`memset_s`, `explicit_bzero`)

Module 10 — Memory Management

This is where C programs live or die in production. Real GCC 13.3 / glibc 2.39 output throughout.

10.1 Process memory layout

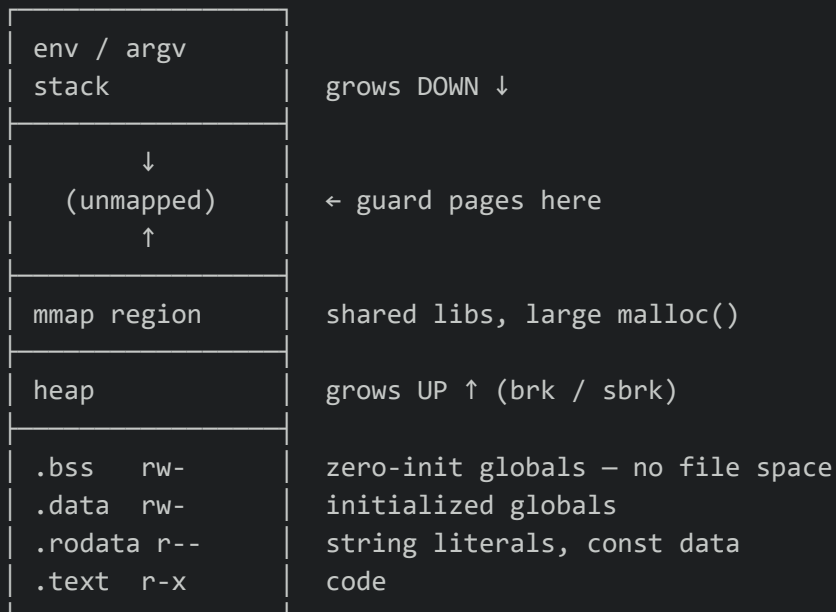
REGION	ADDRESS	SYMBOL
text	0x56239ca9c1f4	main()
text	0x56239ca9c1e9	func()
rodata	0x56239ca9d008	rodata_str
rodata	0x56239ca9d087	string literal
data	0x56239ca9f010	data_var (=42)
bss	0x56239ca9f01c	bss_var
heap	0x5623db22a2a0	malloc(16) #1
heap	0x5623db22a2c0	malloc(16) #2 (+32)

```
mmap      0x7fbb934ff010  malloc(1MB)  <- large allocs use mmap, far away
stack     0x7fffec3fb21c  local variable
```

And the real kernel view:

```
$ cat /proc/self/maps
5579a3c81000-5579a3c82000 r--p /home/claude/c_mem/maps  <- rodata (read-only)
5579a3c82000-5579a3c83000 r-xp /home/claude/c_mem/maps  <- text (execute, NOT
writable)
5579a3c85000-5579a3c86000 rw-p /home/claude/c_mem/maps  <- data + bss (read-
write)
5579c1169000-5579c118a000 rw-p [heap]
7f912ac00000-...      r--p /usr/lib/.../libc.so.6
```

HIGH ADDRESSES



LOW ADDRESSES (page 0 unmapped → NULL deref = SIGSEGV)

The page permissions are enforced by hardware

```
char *s = "literal";
s[0] = 'X';
```

```
about to write to .rodata...
Segmentation fault      exit=139
```

`.rodata` is mapped read-only. This is why `char *s = "hi"; s[0]='H';` crashes but `char s[] = "hi"; s[0]='H';` doesn't — Module 7's decay distinction, enforced by the MMU.

Similarly `.text` is `r-x` — **not writable** — which is why self-modifying code needs `mprotect`, and `[heap]/stack` are `rw-` but not `x` (the **NX bit**), which is why plain stack-smashing shellcode stopped working around 2004.

`malloc` uses two different mechanisms

Note in the output: `malloc(16)` returned `0x5623db22a2a0` (in `[heap]`), but `malloc(1MB)` returned `0x7fbb934ff010` — a completely different region.

glibc switches from `brk` (extending the heap) to `mmap` above `MMAP_THRESHOLD` (default 128 KB).
Consequences:

- Large blocks are returned to the OS immediately on `free`; small ones usually aren't.
- `mmap`d pages arrive **pre-zeroed** by the kernel, so `calloc` of a large block is free.
- Tune with `mallopt(M_MMAP_THRESHOLD, ...)` if you have a measured reason.

10.2 Stack frames and stack overflow

(Building on Module 6 §6.7.)

The stack grows **downward**. Each call pushes a frame containing the return address, saved frame pointer, saved callee-saved registers, and locals.

From Module 6's measurement:

```
stack limit = 8192 KB
depth 50000, stack used 1562 KB    (~32 bytes/frame)
depth 250000, stack used 7812 KB
Segmentation fault
```

Key facts:

Fact	Implication
Default limit is 8 MB on Linux (<code>ulimit -s</code>), 1 MB on Windows, ~512 KB for non-main pthreads	You have far less than you think in threads
Overflow gives a bare SIGSEGV , no message	Use ASan (<code>stack-overflow</code>) or a guard page
Stack memory is not zeroed	Uninitialized locals hold previous frames' data (Module 3's <code>0xDEADBEEF</code>)
Locals are adjacent to the return address	This is <i>why</i> stack buffer overflows are exploitable

Never put large objects on the stack:

```
void f(void) { char buf[1024*1024]; } /* 1 MB – blows a thread stack instantly */
void f(void) { char *buf = malloc(1024*1024); ... free(buf); } /* checkable */
```

And per Module 7: **no VLAs** — `int v[n]` with attacker-controlled `n` is a stack-clash vulnerability.

Mitigations in modern builds:

- `-fstack-protector-strong` — a canary between locals and the return address; corrupted canary → `__stack_chk_fail` → abort
- Guard pages below the stack
- ASLR randomizes the stack base each run
- `-fstack-clash-protection` — probes each page so a huge frame can't skip the guard page

10.3 The `malloc` family

```
void *malloc(size_t size); /* uninitialized */
void *calloc(size_t n, size_t size); /* zeroed, checks n*size overflow */
void *realloc(void *p, size_t size); /* resize, may move */
void free(void *p);
void *aligned_alloc(size_t align, size_t size); /* C11 */
```

`malloc` vs `calloc`

```
malloc: contents INDETERMINATE
bytes: 00 00 00 00 00 00 00 00 ... ← happened to be zero. Don't rely on it.

calloc: ZEROED, and checks n*size for overflow
bytes: 00 00 00 00 00 00 00 00 ...
calloc(SIZE_MAX/2, 4) -> (nil) ← overflow detected, returns NULL
```

The `calloc` overflow check is a real security feature. Compare:

```
p = malloc(n * size); /* n*size can WRAP → tiny allocation → heap overflow */
p = calloc(n, size); /* returns NULL on overflow */
```

An integer overflow in an allocation size is one of the most reliable remote-code-execution primitives. **Use `calloc` for `n * size` computations**, or check explicitly:

```
if (n > SIZE_MAX / size) return NULL;
p = malloc(n * size);
```

GCC even warns:

```
warning: product '9223372036854775807 * 4' of arguments 1 and 2 exceeds 'SIZE_MAX'
[-Walloc-size-larger-than=]
```

Edge cases

```
malloc(0) -> 0x560d97e8c2d0 (may be NULL or a unique freeable ptr – both
legal)
free(NULL) -> guaranteed no-op
realloc(NULL, n) == malloc(n): yes
```

- **malloc(0)** is implementation-defined: NULL or a unique pointer you must still **free**. Never dereference it. Don't test `if (!malloc(0))` for anything.
- **free(NULL)** is a **guaranteed no-op** — this is what makes uniform cleanup labels work (Module 5).
- **realloc(NULL, n)** is exactly **malloc(n)**.
- **realloc(p, 0)** was implementation-defined and is **deprecated in C23**. Don't use it; call **free** explicitly.

realloc semantics

```
before: 0x560d97e8c2d0 "hello"
after:   0x560d97e8c2d0 "hello"  moved? no
```

realloc may grow in place or move the block. **Assume it moves.** Everything from Module 9 §9.8 applies: every alias to the old pointer becomes invalid.

glibc rounds allocations up

```
malloc( 1) usable=24
malloc( 4) usable=24
malloc(16) usable=24
malloc(64) usable=72
```

Every allocation carries per-chunk metadata (size, flags) and is rounded to a 16-byte boundary. **A malloc(1) costs ~32 bytes of real memory.** This is why allocating millions of tiny objects individually is wasteful — and why pool allocators (§10.7) exist.

⚠ `malloc_usable_size` is glibc-specific, and **writing into the extra bytes is UB** even though they're physically there. ASan will catch you.

aligned_alloc

```
malloc          -> aligned to max_align_t (16)? yes
aligned_alloc(64,128) -> 0x55e302aac300 (% 64 = 0)
```

`malloc` guarantees alignment suitable for *any* standard type. For stronger alignment (SIMD, cache lines, DMA):

Function	Notes
<code>aligned_alloc(aligned, size)</code>	C11. size must be a multiple of aligned . Free with <code>free</code> .
<code>posix_memalign(&p, aligned, size)</code>	POSIX. Returns an <code>errno</code> value, doesn't set <code>errno</code> .
<code>_aligned_malloc</code> / <code>_aligned_free</code>	Windows — must use the matching free

10.4 Checking allocation failure

```
char *p = malloc(n);
if (!p) {
    /* handle it */
    return -1;
}
```

Check every allocation. No exceptions.

The `realloc` failure pattern

```
/* WRONG – leaks the original block when realloc fails */
p = realloc(p, newsize);
if (!p) return -1;          /* original block is now unreachable */

/* CORRECT */
char *tmp = realloc(p, newsize);
if (!tmp) {
    /* p is still valid – free it, or return an error and let the caller decide */
    return -1;
}
p = tmp;
```

This is the most common `realloc` bug and it's invisible without LeakSanitizer.

Does `malloc` ever actually fail on Linux?

Not usually, because of **overcommit**: `malloc` returns a pointer to virtual address space, and physical pages are assigned lazily on first touch. If you then touch more than is available, the **OOM killer** terminates your process — you never see a `NULL`.

Check anyway. Because:

- Overcommit can be disabled (`vm.overcommit_memory=2`), and often is on servers and embedded systems
- 32-bit address space exhausts long before RAM does
- `RLIMIT_AS` / cgroups limits cause real failures
- Windows, macOS, and most RTOSes behave differently
- **A huge allocation from an unvalidated length field will fail** — and that's exactly the attacker-controlled path you need to handle

Failure strategy by context

```
/* Library code: propagate. Never exit on someone else's behalf. */
int widget_create(Widget **out) { *out = malloc(...); return *out ? 0 : -ENOMEM; }

/* Application code: an xmalloc wrapper is legitimate and common */
static void *xmalloc(size_t n) {
    void *p = malloc(n);
    if (!p) { fputs("out of memory\n", stderr); abort(); }
    return p;
}

/* Embedded / no-heap: don't allocate dynamically at all (§10.7) */
```

10.5 The heap bug taxonomy

Every one of these verified live:

Bug	ASan	glibc alone
Heap buffer overflow	heap-buffer-overflow	often silent, sometimes <code>malloc()</code> : corrupted top size
Heap underflow (<code>p[-1]</code>)	heap-buffer-overflow	double free or corruption (out)
Use-after-free	heap-use-after-free	usually silent
Double free	attempting double-free	<code>free()</code> : double free detected in tcache 2
Free of non-start pointer	invalid free	<code>free()</code> : invalid pointer

Bug	ASan	glibc alone
Free of stack memory	invalid free	crash or corruption
Leak	LeakSanitizer: detected memory leaks	nothing
Off-by-one read	heap-buffer-overflow	silent

```

=== heap overflow ===
ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000018

=== double free ===
ERROR: AddressSanitizer: attempting double-free on 0x502000000010

=== use after free ===
ERROR: AddressSanitizer: heap-use-after-free /home/claude/c_mem/heapbugs.c:11

=== leak ===
ERROR: LeakSanitizer: detected memory leaks
SUMMARY: AddressSanitizer: 8 byte(s) leaked in 1 allocation(s).

=== off-by-one read ===
ERROR: AddressSanitizer: heap-buffer-overflow /home/claude/c_mem/heapbugs.c:15

```

Without sanitizers, glibc catches some of them at **free** time:

```

case 2 (underflow): double free or corruption (out)
case 3 (double free): free(): double free detected in tcache 2
case 5 (bad pointer): free(): invalid pointer

```

But note what glibc misses entirely: leaks, use-after-free reads, and off-by-one reads. Those are silent, and they're the ones that corrupt data and leak secrets.

Why heap corruption is so dangerous

The allocator stores metadata (chunk size, free-list pointers) **inline, adjacent to your data**. Overflowing a buffer overwrites the *next chunk's header*. Then:

- The next **malloc** returns a pointer computed from corrupted metadata — potentially an attacker-chosen address
- The crash happens in **malloc/free**, **far from the actual bug**, with a useless stack trace
- Classic exploitation primitives: unlink attack, tcache poisoning, House of Force

This is why "it crashed in malloc" almost never means malloc is broken. It means something corrupted the heap earlier. Run ASan.

The discipline

```
p = NULL;                /* 1. initialize */
if (!p) return -1;       /* 2. check every allocation */
free(p); p = NULL;      /* 3. null after free */
/* 4. exactly one owner per allocation */
/* 5. every early return goes through the cleanup chain (Module 5) */
/* 6. ASan + LSan on every debug build and in CI */
```

10.6 Ownership conventions

C has no destructors, no RAII, no GC. **Ownership is a convention you enforce by discipline and documentation.** Get this right and memory bugs mostly disappear.

The four patterns

1. Caller allocates, caller frees (safest — no ambiguity)

```
/* The function never allocates. Caller controls everything. */
int format_name(char *out, size_t outsz, const Person *p);
size_t get_data(void *buf, size_t bufsz); /* returns needed size if too small */
/*
```

This is the pattern in `snprintf`, `read`, `strftime`. **Prefer it.**

2. Callee allocates, caller frees — must be documented

```
/**
 * Returns a newly allocated string. Caller must free() it.
 * Returns NULL on allocation failure.
 */
char *build_message(const Event *e);
```

Every such function needs that comment. Naming conventions help: `*_new`, `*_create`, `*_dup`, `*_strdup`.

3. Paired create/destroy (the standard object pattern)

```
Widget *widget_create(const Config *cfg); /* allocates and initializes */
void    widget_destroy(Widget *w);      /* MUST accept NULL, like free() */
```

`widget_destroy(NULL)` being a no-op makes cleanup chains trivial.

4. Borrowed references — returned pointer is *not* owned

```
const char *widget_name(const Widget *w);    /* points INTO w. Do not free.
                                              Invalid after widget_destroy(w).
*/
```

Document lifetime explicitly. `strchr`, `strstr`, and `getenv` all work this way.

Making ownership visible

```
/* Naming */
Thing *thing_new(void);           /* transfers ownership */
Thing *thing_borrow(Pool *p);    /* does not */

/* Type-level: pass by ** when ownership transfers IN */
void take_ownership(char **s) { free(*s); *s = NULL; }

/* Container ownership: store a destructor with the container */
typedef struct { void **items; size_t n; void (*free_item)(void*); } List;
```

Rules that scale

1. **Exactly one owner.** If two pointers "own" a block, you'll double-free.
2. **Free at the same layer that allocated**, whenever possible.
3. **Document ownership in the header**, not the implementation.
4. **The destroy function accepts NULL.**
5. **Set the pointer to NULL after destroy** — including inside the destroy function via `**`.
6. **On error paths, free everything you acquired** (Module 5's `goto` chain).
7. **Reference counting** when shared ownership is genuinely required — and then it must be atomic if threads are involved.

10.7 Custom allocators

`malloc` is a general-purpose allocator: thread-safe, arbitrary sizes, arbitrary lifetimes. **You pay for generality you often don't need.**

Arena / bump allocator

Allocate by bumping a pointer. Free everything at once.

```
typedef struct { unsigned char *base; size_t cap, off; } Arena;

static void *arena_alloc(Arena *a, size_t n, size_t align)
{
    size_t p = (a->off + align - 1) & ~(align - 1);    /* round up */
    if (n > a->cap - p) return NULL;                    /* overflow-safe check */
    a->off = p + n;
    return a->base + p;
```

```

}

#define ARENA_NEW(a, T)    ((T*)arena_alloc((a), sizeof(T), _Alignof(T)))
#define ARENA_ARR(a, T, n) ((T*)arena_alloc((a), sizeof(T)*(n), _Alignof(T)))

static void arena_reset(Arena *a) { a->off = 0; }      /* free EVERYTHING, O(1) */

/* nested scopes */
static size_t arena_mark(Arena *a)                    { return a->off; }
static void   arena_release(Arena *a, size_t m){ a->off = m; }

```

```

int    at offset 0 (align 4 ok: 1)
double at offset 8 (align 8 ok: 1)    ← note the padding for alignment
values: 42 3.14 "arena string"  used=48/65536
after temp alloc used=1048; after release=48

```

Note the **overflow-safe bounds check**: `if (n > a->cap - p)` rather than `if (p + n > a->cap)`, which could wrap.

Perfect for: per-request/per-frame memory in servers and games, compiler passes, parse trees, any workload with a natural "everything dies at once" boundary.

Cost: you can't free individual objects. That's the trade.

Pool / freelist allocator

Fixed-size objects, O(1) alloc and free, threading the free list **through the free objects themselves** (zero metadata overhead):

```

typedef struct Node { struct Node *next; } Node;
typedef struct { unsigned char *mem; Node *free_list; size_t objsz, count; } Pool;

static void *pool_alloc(Pool *p)
{ Node *n = p->free_list; if (!n) return NULL; p->free_list = n->next; return n; }

static void pool_free(Pool *p, void *ptr)
{ Node *n = ptr; n->next = p->free_list; p->free_list = n; }

```

Perfect for: many same-sized objects with individual lifetimes — network connections, AST nodes, particles, list nodes, task structs.

The measured difference

```

=== BENCHMARK: 2,000,000 allocations of 32 bytes (memory touched) ===
malloc/free :    97.8 ms (1.00x)

```

```
arena      :      7.0 ms  (13.98x faster)
pool       :     18.9 ms  ( 5.17x faster)
```

14x and 5x. That's why every high-performance C codebase has custom allocators.

Where the speedup comes from: no locking, no free-list search, no size-class logic, no per-chunk metadata, and — often the biggest factor — **sequential memory means far better cache behavior.**

The other allocators to know

Allocator	Idea	Used for
Stack allocator	Arena with LIFO free (mark/release)	Nested scratch buffers
Slab	Pools per object type, caching <i>constructed</i> objects	Linux kernel (kmem_cache)
Buddy	Split/merge power-of-two blocks	Kernel page allocator
Ring buffer	Fixed circular region	Streaming, logging, audio
Static pool	Compile-time array, zero heap	Embedded / safety-critical

When to write one

Do when profiling shows allocation is hot, when you have a natural lifetime boundary, when you need deterministic timing (real-time), when fragmentation matters (long-running), or when the heap is banned (MISRA, DO-178C).

Don't as a default. `malloc` is well-optimized, and a custom allocator means ASan can't instrument it — you lose your best bug-detection tool. If you write one, **add ASan manual poisoning**:

```
#include <sanitizer/asan_interface.h>
ASAN_POISON_MEMORY_REGION(ptr, size);    /* on free */
ASAN_UNPOISON_MEMORY_REGION(ptr, size);  /* on alloc */
```

10.8 `alloca` — and why to avoid it

`alloca(n)` allocates on the **current stack frame**; it's released when the function returns.

```
alloca(64) at 0x7ffc36594310
alloca(64) at 0x7ffc36594310    ← two calls, same frame reused across calls
```

Why it's dangerous

1. Failure is a silent crash.



```
alloca(4294967296)...
Segmentation fault   exit=139
```

No NULL, no `errno`, no chance to recover. Attacker-controlled size = stack clash vulnerability.

2. In a loop, memory accumulates until the function returns.

```
iter 0: 0x7ffc36593f50 (offset 0)
iter 1: 0x7ffc36593b40 (offset 1040)
iter 2: 0x7ffc36593730 (offset 2080)
iter 3: 0x7ffc36593320 (offset 3120)
iter 4: 0x7ffc36592f10 (offset 4160)
```

Five `alloca(1024)` calls consumed 5 KB. A loop with a large iteration count blows the stack.

3. It's not standard C — no ISO C mention, no MSVC (`_alloca` differs).

4. It breaks if inlined into a loop, and interacts badly with `longjmp` and exception-like control flow.

5. `alloca` inside a function argument list is UB on some platforms — `f(alloca(n), alloca(m))` can corrupt the argument area.

```
warning: use of 'alloca' [-Walloca]
```

Add `-Walloca -Werror=vla` to your build. Use `malloc` (checkable) or a fixed buffer with a bound check. The hybrid stack/heap pattern from Module 7 gives you the speed without the risk.

10.9 Zeroing sensitive memory

```
void handle_password(void)
{
    char pw[32];
    strcpy(pw, "hunter2-secret!");
    memset(pw, 0, sizeof pw);    /* wipe the secret */
}
```

Here's what GCC generates at `-O2`:

```
handle_password:
    endbr64
    ret
```

The entire function is gone. The compiler saw a dead store to an object about to leave scope and deleted the `memset` — and then the `strcpy` too. Your secret is left sitting on the stack, ready to be read by the next uninitialized local (Module 3), leaked in a core dump, or swapped to disk.

The correct version:

```
explicit_bzero(pw, sizeof pw);
```

```
handle_password_safe:
    sub    rsp, 56
    mov    edx, 32
    movdqa xmm0, XMMWORD PTR .LC0[rip]
    movaps XMMWORD PTR [rsp], xmm0
    call   __explicit_bzero_chk@PLT    ← the wipe actually happens
    ...
```

The portable options

Function	Availability
<code>explicit_bzero(p, n)</code>	glibc 2.25+, OpenBSD, FreeBSD
<code>memset_explicit(p, c, n)</code>	C23 — the standard answer
<code>memset_s(p, smax, c, n)</code>	C11 Annex K — optional, mostly MSVC
<code>SecureZeroMemory(p, n)</code>	Windows

Portable fallback:

```
static void secure_zero(void *p, size_t n)
{
    volatile unsigned char *v = p;    /* volatile forbids eliding the stores */
    while (n--) *v++ = 0;
}
```

Or force a barrier: `memset(p, 0, n); __asm__ __volatile__("" ::: "memory");`

Beyond zeroing

Zeroing is necessary but not sufficient. For real secret handling:

- **mlock** the pages so they're never swapped to disk
- **Disable core dumps** (`setrlimit(RLIMIT_CORE, 0)`) for processes holding secrets
- Minimize copies — every `strcpy` of a key is another location to wipe
- Remember `realloc` may **copy and leave the old block unwiped** — for secrets, allocate the new block, copy, wipe the old, free

- Constant-time comparison (Module 8) — timing leaks defeat all of the above

Module 10 rules

#	Rule
1	<code>.rodata</code> is read-only at the page level — <code>char *s = "x"; s[0]='y';</code> segfaults
2	Nothing large on the stack; thread stacks are much smaller than 8 MB
3	<code>calloc(n, size)</code> for <code>n * size</code> — it checks for overflow
4	Check every allocation, including on platforms where it "never fails"
5	Never <code>p = realloc(p, n)</code> — use a temp, or you leak on failure
6	<code>free(NULL)</code> is a no-op; <code>destroy(NULL)</code> should be too
7	Assume <code>realloc</code> moves; every alias is invalidated
8	Set pointers to NULL after free — and know that aliases still dangle
9	Exactly one owner per allocation, documented in the header
10	"Crashed in malloc" means something corrupted the heap earlier — run ASan
11	Custom allocators only where profiling justifies them; add ASan poisoning
12	Ban <code>alloca</code> and VLAs: <code>-Walloca -Werror=vla</code>
13	<code>memset</code> of a dying secret is deleted — use <code>explicit_bzero</code> / <code>memset_explicit</code>
14	ASan + LSan + UBSan on every debug build and in CI. Non-negotiable.

Standing build command:

```
# Debug / CI
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
  -Wcast-qual -Wcast-align=strict -Wpointer-arith -Wwrite-strings \
  -Walloca -Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
  -O1 -g -D_FORTIFY_SOURCE=3 -fsanitize=address,undefined -fno-omit-frame-
pointer \
  prog.c -o prog

# Release
gcc -std=c17 -O2 -D_FORTIFY_SOURCE=3 -fstack-protector-strong \
  -fstack-clash-protection -fcf-protection -Wl,-z,relro,-z,now,-z,noexecstack \
  prog.c -o prog
```

Run with

`ASAN_OPTIONS=detect_leaks=1:strict_string_checks=1:detect_stack_use_after_return=1.`

Practice

1. **Map your own process.** Print addresses for every region as in §10.1, then dump `/proc/self/maps` and match each address to a mapping. Identify which regions are `r-x`, `rw-`, `r--`, and explain why.
2. **Prove the page protections.** Write four programs that each attempt: writing to `.rodata`, executing the heap, writing to `.text`, and reading page 0. Record which signal each produces.
3. **Overflow arithmetic bug.** Write `Buf *buf_new(size_t n, size_t elemsize)` using `malloc(n * elemsize)` with no check. Call it with `n = SIZE_MAX/2`, `elemsize = 4`, then write `n` elements. Watch ASan. Fix it three ways: explicit check, `calloc`, and `__builtin_mul_overflow`.
4. **The realloc leak.** Write a growable array with `p = realloc(p, n)`. Force `realloc` to fail (use `RLIMIT_AS` via `ulimit -v`, or an LD_PRELOAD failure injector). Confirm LSan reports the leak. Fix with the temp-pointer pattern.
5. **Full bug matrix.** Reproduce all 8 heap bugs from §10.5. For each, record: plain run, glibc-only, ASan, and Valgrind. Build a table of which tool catches what. This is the single most useful table you can have on your desk.
6. **Build an arena.** Implement the arena from §10.7 with alignment, mark/release, and a chained-block version that grows by allocating new blocks instead of failing. Add ASan poisoning. Benchmark it against `malloc` for a realistic workload (parsing 10,000 JSON-ish records).
7. **Build a pool.** Implement a fixed-size pool with a free list, then extend it to grow by chaining new slabs. Add a debug mode that fills freed objects with `0xDD` and detects double-free by checking that pattern.
8. **Ownership audit.** Take any program you've written in this course. For every pointer, write down: who allocates, who frees, what the lifetime is. Find at least one case where it's ambiguous, and fix the API to make it unambiguous.
9. **The secret wipe.** Reproduce the `memset` elimination at `-O2` with `-S`. Then test `explicit_bzero`, `memset_explicit` (with `-std=c2x`), the `volatile` loop, and the memory-barrier trick. Verify in the assembly which ones survive.
10. **alloca in a loop.** Write a function that calls `alloca(4096)` in a loop and prints stack usage each iteration. Find the iteration count that crashes. Then rewrite with `malloc` and confirm you get a clean failure instead.

11. Structs, Unions, Enums

- Struct declaration, initialization, nesting, assignment/copy semantics
- Passing structs by value vs pointer
- **Padding, alignment, offsetof, struct packing, cache-friendly layout**
- Bitfields and their portability caveats
- Unions, type punning via union, tagged unions
- Enums, enum scoping, C23 fixed underlying type
- `typedef` — good and bad practice
- **Opaque structs** (`typedef struct Foo Foo;`) for encapsulation
- Self-referential structs, intrusive linked structures

Module 11 — Structs, Unions, Enums

Real GCC 13.3 output throughout, including measured benchmarks. §11.3 (padding and cache-friendly layout) is where the biggest real-world performance wins in C hide.

11.1 Struct declaration, initialization, copy semantics

```
struct Inner { int a, b; };
struct Outer { struct Inner in; char name[8]; int *ptr; };

struct Outer o1 = { .in = {1,2}, .name = "hello", .ptr = malloc(sizeof(int)) };
```

Access with `.` on an object, `->` on a pointer. `p->x` is exactly `(*p).x`.

Struct assignment is a memberwise copy — **including arrays**

```
struct Outer o2 = o1;
o2.in.a = 99; strcpy(o2.name, "world");
```

after modifying o2: o1.in.a=1 o1.name="hello" (independent)

This is the one place C copies an array. Bare `int a[5]; b = a;` is an error; `struct S { int a[5]; }; s2 = s1;` copies all 5 elements. **Wrapping an array in a struct makes it a first-class value** — that's a real technique for vectors, matrices, fixed IDs, and hashes.

But it's a **shallow** copy

```
o1.ptr=0x5603c979f2a0 o2.ptr=0x5603c979f2a0 same? 1
*o1.ptr is now 555 <- modified through o2
-> double free if BOTH are freed
```

Struct assignment copies the *pointer*, not what it points at. If your struct owns heap memory, you need an explicit deep-copy function:

```
int outer_clone(struct Outer *dst, const struct Outer *src)
{
    *dst = *src;                                /* shallow first */
    dst->ptr = malloc(sizeof *dst->ptr);         /* then fix up owned members */
    if (!dst->ptr) return -1;
    *dst->ptr = *src->ptr;
```

```
    return 0;
}
```

Rule: if a struct contains an owning pointer, it needs a clone function and a destroy function.

Document that plain assignment is a shallow copy.

⚠ Never `memcpy` two structs

```
struct P { char c; int i; };    /* 3 bytes of padding after c */
```

```
same member values: a.c=='x' a.i==42, b.c=='x' b.i==42
memcmp(&a,&b,sizeof) = 255    <-- NOT ZERO
raw bytes a: 78 FF FF FF 2A 00 00 00
raw bytes b: 78 00 00 00 2A 00 00 00
              ^^ the padding bytes differ
```

Padding bytes are indeterminate. Two structs with identical member values can have different bytes. Write a member-by-member comparison:

```
static bool p_equal(const struct P *a, const struct P *b)
{ return a->c == b->c && a->i == b->i; }
```

The same reasoning applies to **hashing** a struct, **writing** it to a file or socket (padding leaks stack contents — a real information-disclosure CVE class), and using `memcpy` on structs containing floats (`-0.0 == 0.0` but the bits differ; `NaN != NaN`).

If you must compare or serialize bytes, `memset(&s, 0, sizeof s)` first — but note that C only guarantees this zeroes padding at the point of the `memset`, not after subsequent member assignments.

Initialization

```
struct P p1 = {1, 2};           /* positional */
struct P p2 = {.i = 42};        /* designated (C99) — c is zeroed */
struct P p3 = {0};              /* all zero */
struct P p4 = {};               /* C23 */
p = (struct P){0};              /* compound literal assignment (Module 7)
*/
memset(&p, 0, sizeof p);         /* zeroes padding too */
```

Prefer designated initializers. They're order-independent, self-documenting, and survive member reordering.

11.2 By value vs by pointer

Measured, 20 million calls, **-O2**, inlining disabled:

```
sizeof Small=16 Large=512
Small by value :    60.9 ms
Small by ptr   :    60.3 ms    ← no difference
Large by value :   386.7 ms
Large by ptr   :    57.6 ms    ← 6.7x faster
```

Small structs by value are free. On SysV AMD64, a 16-byte struct travels in two registers (Module 6 §6.7) — no memory traffic at all. **Point**, **Vec2**, **Complex**, **Range** should all be passed and returned by value; it's idiomatic and often *faster* than a pointer (no aliasing concerns, no indirection).

Large structs by value cost real copies. 512 bytes memcpy'd per call, plus stack space.

The guideline

Size	Pass by
≤ 16 bytes (2 registers)	value
17~64 bytes	either; measure
> ~64 bytes	const pointer
Must modify the caller's object	pointer (always)
Contains an owning pointer	pointer + explicit clone semantics

```
Point midpoint(Point a, Point b);           /* small: by value */
double area(const struct Polygon *p);       /* large: const pointer */
int update(struct State *s, const Event *e); /* modifies s, reads e */
```

Always **const the pointer parameters you don't modify** (Module 9). It documents intent and lets callers pass **const** objects.

Note: returning a large struct by value is *usually* fine — the ABI passes a hidden pointer to caller-allocated space (RVO-like), so there's often no extra copy.

11.3 Padding, alignment, and cache-friendly layout ⚠

The three layout rules

1. **Each member sits at an offset that's a multiple of its alignment.**
2. **The struct's alignment = the maximum member alignment.**
3. **sizeof is rounded up to a multiple of the struct's alignment** — so arrays of the struct keep every element aligned.

Member order changes everything

```
struct Bad { char a; double b; char c; int d; char e; };
struct Good { double b; int d; char a, c, e; }; /* same members! */
```

```
struct Bad
a      offset= 0 size= 1 align=1
b      offset= 8 size= 8 align=8      ← 7 bytes padding before b
c      offset=16 size= 1 align=1
d      offset=20 size= 4 align=4      ← 3 bytes padding before d
e      offset=24 size= 1 align=1
TOTAL sizeof=32 alignof=8 (payload = 15 bytes)
WASTED: 17 bytes of padding (53%)

struct Good
b      offset= 0 size= 8
d      offset= 8 size= 4
a      offset=12 c offset=13 e offset=14
TOTAL sizeof=16 alignof=8
WASTED: 1 byte -> 50% smaller than Bad
```

Identical members, identical semantics, half the memory.

The rule: declare members in decreasing order of alignment (`double`/pointers → `long` → `int` → `short` → `char` → `bool`). This is nearly optimal and costs nothing.

For a struct you allocate a million of, that's 32 MB vs 16 MB — and, more importantly, **half as many cache lines touched**.

Tools: `pahole` (from `dwarves`) analyzes your binary and reports padding holes automatically. Run it on your hot structs.

`__attribute__((packed))` — usually wrong

```
__attribute__((packed)): sizeof=15 alignof=1
-> no padding, but members are MISALIGNED
```

You save 1 byte over `struct Good` and pay:

- Misaligned member access — slower on x86, **faults on some ARM/SPARC configs** (Module 9 §9.9)
- **Taking the address of a packed member yields a misaligned pointer**, which is UB to dereference. GCC warns with `-Waddress-of-packed-member`.
- Blocks vectorization

Use **packed only for**: parsing wire formats and on-disk structures you don't control — and even then, prefer explicit `memcpy` field extraction (Module 8), which is portable *and* endian-explicit.

Cache-friendly layout — the measured impact

```
struct Fat { int key; char blob[60]; }; /* 64 bytes: 1 per cache line */
struct Slim { int key; }; /* 4 bytes: 16 per cache line */
```

Summing 4 million keys — **identical arithmetic**, different memory footprint:

```
AoS with 60-byte blob : 17.8 ms (244 MB touched)
keys only (SoA style) : 2.5 ms (15 MB touched)
speedup: 7.17x
```

7×, from layout alone. The CPU always loads a full 64-byte cache line. Reading a 4-byte key from a 64-byte struct wastes 93% of every line's bandwidth.

Array of Structs vs Struct of Arrays

```
/* AoS — natural, but wasteful when you touch one field */
struct Particle { float x,y,z, vx,vy,vz; float mass; int id; };
struct Particle particles[N];

/* SoA — every field is a dense contiguous array */
struct Particles {
    float x[N], y[N], z[N];
    float vx[N], vy[N], vz[N];
    float mass[N]; int id[N];
};
```

SoA wins when you process one or two fields across many objects (physics updates, filters, aggregations) — perfect locality and it **auto-vectorizes**.

AoS wins when you touch most fields of one object at a time (per-entity logic, tree nodes).

This is the core of **data-oriented design** (Module 25). Layout is not an afterthought.

Two more layout techniques

Hot/cold splitting — put frequently-accessed fields in one struct, rarely-used ones behind a pointer:

```
struct Connection {
    int fd; uint32_t state; uint64_t last_active; /* hot: fits in one cache line */
    struct ConnDetails *cold; /* logs, config, names */
};
```

Cache-line alignment to avoid false sharing (Module 4 §4.5):

```
struct alignas(64) Counter { _Atomic long value; }; /* one per cache line */
struct Counter counters[NTHREADS]; /* threads no longer fight over lines */
```

`offsetof(type, member)` from `<stddef.h>` gives you a member's byte offset — essential for layout analysis, serialization, and `container_of` (§11.9).

11.4 Bitfields (recap + the struct angle)

Covered in Module 4 §4.9. The essentials:

```
struct Flags { unsigned ready:1; unsigned mode:3; unsigned :0; unsigned other:4;
};
```

Portability caveats — all implementation-defined:

- Bit order within a storage unit
- Whether a field may straddle a unit boundary
- Total struct size and alignment
- **Signedness of plain `int x:1`** — Module 4 showed `int s:1` storing 1 and reading back `-1`

Also: you can't take a bitfield's address, overflow silently truncates, and concurrent writes to *different* bitfields in the same unit are a **data race** (C11 fixed this only across `:0` boundaries).

Bitfields: fine for internal flag compaction. Never for hardware registers or wire formats — use explicit shifts and masks.

11.5 Unions

```
union U { uint32_t u; float f; unsigned char b[4]; };
```

```
sizeof(union U) = 4 (largest member), alignof = 4
wrote f=1.0f -> u.u = 0x3F800000 bytes: 00 00 80 3F
```

All members occupy the **same** storage starting at offset 0. `sizeof` is the largest member (rounded up for alignment); alignment is the strictest member's.

Type punning via union

Reading a member other than the last one written is legal in C (clarified in C99 TC3 and explicitly permitted in C11 §6.5.2.3). The value is reinterpreted from the object representation.

⚠ **It's undefined in C++**. For code that must compile as both, or for maximum portability, use `memcpy` (Module 2 §2.11) — it compiles to identical instructions.

⚠ Reading a member with a **trap representation**, or a larger member than was written, reads indeterminate bytes.

Tagged unions — the safe pattern

A bare union is unsafe: nothing records which member is active. Pair it with a discriminant:

```
typedef enum { V_INT, V_FLOAT, V_STR } Kind;

typedef struct {
    Kind kind;
    union { long i; double f; char *s; } as;
} Value;

static void print_value(const Value *v)
{
    switch (v->kind) {                /* omit default → -Wswitch enforces
coverage */
    case V_INT:    printf("int   %ld\n", v->as.i); break;
    case V_FLOAT:  printf("float %g\n",  v->as.f); break;
    case V_STR:    printf("str   \"%s\"\n", v->as.s); break;
    }
}
```

```
int    42
float  3.14
str    "hello"
sizeof(Value) = 16  (enum 4 + union 8 + padding)
```

16 bytes instead of 4+8+8 = 20+. This is how interpreters represent values, how ASTs represent nodes, and how protocol messages represent variants.

The discipline: always set the tag and the payload **together**, ideally through a constructor:

```
static Value v_int(long x)    { return (Value){ .kind = V_INT,   .as.i = x }; }
static Value v_float(double x){ return (Value){ .kind = V_FLOAT, .as.f = x }; }
```

Anonymous unions and structs (C11)

```
struct { int tag; union { int i; float f; }; } an = { .tag = 1, .i = 7 };
printf("%d\n", an.i);          /* no ".u." needed */
```

```
an.i = 7    (no member name needed)
```

Cleaner syntax for tagged unions and for protocol headers with overlapping fields.

Other legitimate uses

```
/* Guaranteeing alignment/size */
union Aligned { max_align_t _a; char bytes[256]; };

/* Compact variant storage in a container */
/* Endian inspection */
union { uint32_t u; unsigned char b[4]; } e = { .u = 1 };
bool little_endian = (e.b[0] == 1);
```

11.6 Enums

```
enum Color { RED, GREEN, BLUE };          /* 0, 1, 2 */
enum Mixed { M_X=10, M_Y, M_Z=20, M_W };  /* 10, 11, 20, 21 */
```

```
RED=0 GREEN=1 BLUE=2
M_X=10 M_Y=11 M_Z=20 M_W=21  (auto-increment from the last value)
```

The three things people get wrong

1. There is no scoping. Enum constants live in the ordinary identifier namespace at file (or block) scope:

```
enum A { OK }; enum B { OK };    /* ERROR: redefinition */
```

This is why prefixes are mandatory: `COLOR_RED`, `STATE_IDLE`, `TOKEN_PLUS`.

2. Enums are not type-safe.

```
enum Color c = 42;    /* no error, no warning by default */
```

```
enum Color c = 42; -> c=42 (any int assigns)
```

The enum *type* is compatible with some integer type; the *constants* are `int`. Arithmetic on enums yields `int`. You get documentation and `-Wswitch`, not enforcement. (`-Wassign-enum` in Clang helps; GCC's `-Wswitch` is the main win.)

3. The underlying type is implementation-defined (before C23):

```
sizeof(enum Color)=4 sizeof(enum Big)=4
```

The compiler picks *some* integer type that fits all values. **Never put a bare enum in a struct that maps to a wire format or an ABI boundary** — use `uint8_t/uint32_t` and convert.

C23 fixes the underlying type

```
enum Small : unsigned char { S_A, S_B, S_C };
enum Wide  : long long      { W_A = 1LL << 40 };
```

```
enum Small : unsigned char -> sizeof=1
enum Wide  : long long      -> sizeof=8, W_A=1099511627776
```

C23 also allows values exceeding `int` range and permits a trailing comma. This makes enums genuinely usable in packed structures.

The two idioms worth memorizing

Flag enums:

```
enum Flags { F_A = 1u<<0, F_B = 1u<<1, F_C = 1u<<2, F_D = 1u<<3 };
unsigned f = F_A | F_C;
if (f & F_C) { ... }
```

```
f = F_A|F_C = 5   has F_C? 1   has F_B? 0
```

Store the combination in an `unsigned`, not the enum type — the combined value isn't a valid enumerator.

The COUNT idiom — keeps parallel tables in sync:

```
enum State { STATE_IDLE, STATE_RUN, STATE_DONE, STATE_COUNT };

static const char *const state_names[] = {
    [STATE_IDLE] = "idle",
    [STATE_RUN]  = "run",
    [STATE_DONE] = "done",
};
_Static_assert(sizeof state_names / sizeof *state_names == STATE_COUNT,
               "state_names out of sync with enum State");
```

Add a state, forget the name, and the **build fails**. This pattern plus `-Werror=switch` gives you most of the safety a real sum type would.

Anonymous enums as compile-time constants:

```
enum { MAX_USERS = 1024, BUFFER_SIZE = 4096 };
int buf[BUFFER_SIZE];          /* works – enum constants ARE constant expressions,
                                unlike `const int` in C */
```

11.7 typedef — good and bad practice

`typedef` creates an **alias**, not a new type. No type checking is added.

Good uses

```
/* 1. Function pointers – always (Module 9) */
typedef int (*Comparator)(const void *, const void *);

/* 2. Opaque handles (§11.8) */
typedef struct Database Database;

/* 3. Platform abstraction */
typedef uint32_t pixel_t;
typedef int64_t  timestamp_ms_t;

/* 4. Genuinely awkward declarations */
typedef char (*ArrayOfTenCharPtrs[10]);
```

Bad uses

```
typedef struct { int x, y; } *PointPtr;    /* HIDING A POINTER – the worst
offender */
typedef unsigned int uint;                 /* saves 4 characters, adds a lookup
*/
typedef struct Foo FooStruct;              /* redundant */
```

Hiding pointers behind a typedef is the cardinal sin. The reader can't tell that `PointPtr p;` needs freeing, can't apply `const` correctly (`const PointPtr` is a *const pointer*, not a pointer to `const`), and can't see the indirection.

The struct-typedef debate

```
/* Style A – Linux kernel: no typedef for visible structs */
struct point { int x, y; };
struct point p;

/* Style B – common in application code */
typedef struct { int x, y; } Point;
Point p;
```

The kernel argues the explicit `struct` keyword tells you it's a struct at every use site. Application code argues brevity. **Both are defensible; be consistent.** The one universal rule: **use a typedef for opaque handles** (where the point is to hide the layout) and **never for pointers**.

Note the self-referential form:

```
typedef struct Node Node;      /* declare the tag AND the alias */
struct Node { int v; Node *next; };
```

11.8 Opaque structs — encapsulation in C

The header declares the type without defining it. Only the `.c` file knows the layout.

```
/* counter.h */
typedef struct Counter Counter;      /* incomplete type */

Counter *counter_create(const char *name);
void      counter_destroy(Counter *c);      /* accepts NULL */
void      counter_inc(Counter *c);
long      counter_value(const Counter *c);
const char *counter_name(const Counter *c);
```

```
/* counter.c */
struct Counter { char *name; long value; int magic; };      /* PRIVATE */
#define MAGIC 0xC0FFEE

Counter *counter_create(const char *name)
{
```

```
Counter *c = calloc(1, sizeof *c);
if (!c) return NULL;
c->name = strdup(name ? name : "");
if (!c->name) { free(c); return NULL; }
c->magic = MAGIC;
return c;
}
void counter_destroy(Counter *c)
{ if (!c) return; c->magic = 0; free(c->name); free(c); }
```

```
hits = 5
ASan exit=0
```

The compiler enforces the encapsulation:

```
error: invalid application of 'sizeof' to incomplete type 'Counter'
error: invalid use of incomplete typedef 'Counter'
```

What callers cannot do

- sizeof(Counter) — incomplete type
- Counter x; — can't declare an object
- c->value — unknown member
- Copy it by assignment
- Depend on the layout

What you get

Benefit	Why it matters
ABI stability	You can add/reorder/remove fields without recompiling callers
Real invariants	State can only change through your functions
Smaller headers	No need to #include the types your fields use
Faster builds	Changing the layout doesn't trigger a rebuild of the world

Note the magic field — a cheap debug-time sanity check that catches use-after-free and type-confusion. Real libraries do this (FILE, pthread_mutex_t internals).

This is the single most important API design pattern in C. FILE*, sqlite3*, pthread_t, SSL_CTX*, png_structp — every serious C library works this way.

The trade: an allocation per object (no stack-allocated instances) and a function call per field access. When that matters, expose the struct in the header but document that only the API may touch it — a weaker guarantee, and the reason pthread_mutex_t is a visible-but-opaque blob.

11.9 Self-referential structs and intrusive data structures

Self-reference

```
struct Node { int value; struct Node *next; };    /* pointer to incomplete self:
OK */
struct Bad  { int value; struct Bad  self; };    /* ERROR: infinite size */
```

Mutually recursive types need a forward declaration:

```
struct B;
struct A { struct B *b; };
struct B { struct A *a; };
```

Intrusive lists — the Linux kernel pattern

The list node lives *inside* the object, not the other way round:

```
struct list_head { struct list_head *next, *prev; };

#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

#define list_entry(ptr, type, member) container_of(ptr, type, member)

#define list_for_each(pos, head) \
    for (pos = (head)->next; pos != (head); pos = pos->next)
```

`container_of` is the trick: given a pointer to an embedded member, subtract the member's offset to recover the enclosing object.

```
struct Task {
    char name[16];
    int prio;
    struct list_head all;    /* membership in "all tasks" */
    struct list_head ready; /* membership in "ready queue" */
};
```

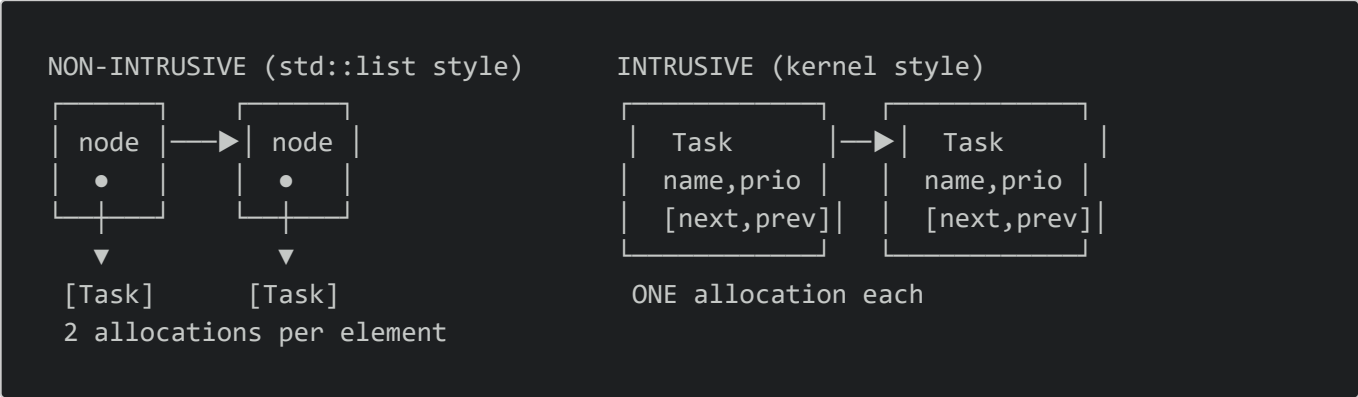
```
offsetof(Task, all)    = 24
offsetof(Task, ready) = 40
```

ALL tasks (via <code>.all</code>):	READY tasks (via <code>.ready</code>):
disk prio=20	disk prio=20

```
net    prio=10      init    prio=0
init   prio=0      ← SAME objects, DIFFERENT list
```

Verified clean under ASan + UBSan.

Intrusive vs non-intrusive



	Intrusive	Non-intrusive
Allocations per element	1	2
Cache locality	node adjacent to data	two hops
Membership in N lists	trivial — N embedded nodes	needs N node allocations
Removal given only the object	O(1)	O(n) search
Type genericity	list code is fully generic	needs <code>void*</code> or macros
Coupling	the type must know about lists	none

This is why the Linux kernel uses it everywhere — `struct list_head`, `struct hlist_node`, `struct rb_node`. It's also in FreeBSD (`sys/queue.h`), Redis, and most high-performance C.

⚠ **container_of caveats:** it's technically UB by a strict reading of the standard (pointer arithmetic outside an array), but it's universally supported and the kernel depends on it. Use `offsetof`, never a hand-computed constant. And never `container_of` with the wrong type — nothing checks you.

The safer variant (GCC/Clang) type-checks the member:

```
#define container_of(ptr, type, member) __extension__({      \
    const __typeof__(((type *)0)->member) *__mptr = (ptr);  \
    (type *)((char *)__mptr - offsetof(type, member)); })
```

Module 11 rules

#	Rule
---	------

#	Rule
1	Struct assignment copies arrays but is shallow for pointers
2	Never <code>memcpy</code> , hash, or write structs byte-wise — padding is indeterminate
3	Declare members in decreasing alignment order — often halves the size
4	Avoid <code>packed</code> ; use explicit <code>memcpy</code> field extraction for wire formats
5	Layout is performance: 7× measured from cache footprint alone
6	≤16 bytes by value, >64 bytes by <code>const</code> pointer
7	Bitfields never for hardware or protocols; always spell <code>unsigned/signed</code>
8	Unions need a tag; set tag and payload together via a constructor
9	Union punning is legal in C, UB in C++ — <code>memcpy</code> if portability matters
10	Enums have no scoping — prefix every constant
11	Use the COUNT idiom + <code>_Static_assert</code> + <code>-Werror=switch</code>
12	Never <code>typedef</code> a pointer
13	Opaque structs for every non-trivial type you expose
14	Intrusive containers where allocation count and locality matter

Standing build command:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
  -Wpadded -Waddress-of-packed-member -Wcast-align=strict \
  -Werror=switch -Werror=vla -Werror=return-type \
  -O1 -g -D_FORTIFY_SOURCE=3 -fsanitize=address,undefined prog.c -o prog
```

`-Wpadded` reports every padding hole — noisy, but run it once on your hot structs.

Practice

1. **Layout analysis.** Write a `FIELD()` macro using `offsetof` and `_Alignof`, and use it to dump the layout of 5 structs from a real codebase (or invent them). For each, compute the wasted bytes and reorder for minimum size. Verify with `sizeof`. Then install `pahole` and compare its output to yours.
2. **The `memcpy` trap.** Build a struct with padding, `memset` two instances to different values, set identical member values, and prove `memcpy` disagrees. Then write a correct `equal()` and a correct `hash()`.
3. **Cache benchmark.** Reproduce the AoS/SoA test at 1M, 4M, and 16M elements. Plot the speedup against your L1/L2/L3 sizes (`lscpu`). Find the array size where the two converge and explain why.
4. **Hot/cold split.** Take a 200-byte struct where only 3 fields are read in a hot loop. Split it into hot and cold parts, benchmark before and after, and report the speedup.

5. **Tagged union interpreter.** Build a `Value` type supporting int, double, string, and array. Write constructors, a `value_print`, a `value_free` (recursive for arrays), and a `value_clone`. Verify with ASan + LSan that a 10,000-node tree round-trips cleanly.
6. **Enum table discipline.** Define an 8-state enum with a `_COUNT`, a designated-initializer name table, and a `_Static_assert`. Then add a 9th state without updating the table and confirm the build fails. Add a `switch` without `default` and confirm `-Werror=switch` fires.
7. **Build an opaque type.** Write `stack.h/stack.c` implementing an opaque `Stack` of `long`, with create/destroy/push/pop/peek/size, automatic growth, and a magic field. Prove from a test file that `sizeof(Stack)` and member access are compile errors. Verify with ASan.
8. **Intrusive list library.** Implement `list_head` with `list_add`, `list_add_tail`, `list_del`, `list_empty`, `list_for_each_safe` (safe against deletion during iteration), and `container_of`. Then build a task struct that lives in three lists simultaneously and write a test that moves tasks between them. Verify with ASan.
9. **Reasoning, no compiler.** Given `struct S { char a; short b; char c; long d; char e; };` on x86-64: give every member's offset, the total `sizeof`, the `alignof`, and the number of padding bytes. Then give the optimal reordering and its size.

12. The Preprocessor

- `#include` (" vs <>), include guards, `#pragma once`
- Object-like and function-like macros
- Macro pitfalls: multiple evaluation, missing parens, semicolons; `do { } while(0)`
- Stringification `#`, token pasting `##`, variadic macros `__VA_ARGS__`
- Conditional compilation: `#if/#ifdef/#elif/#else/#endif`, `#error`, `#warning`
- Predefined macros: `__FILE__`, `__LINE__`, `__func__`, `__DATE__`, `__STDC_VERSION__`
- **X-Macros**, code generation patterns
- `_Generic` (type-generic dispatch, C11)
- `#embed` (C23)
- When *not* to use macros

Module 12 — The Preprocessor

The preprocessor runs **before** the compiler and knows nothing about C. It's pure token manipulation — which is exactly why it's powerful and why it produces the most baffling error messages in the language. Real GCC 13.3 output throughout.

12.1 `#include`

```
#include <stdio.h>      /* system headers: search the standard paths */
#include "myheader.h"   /* project headers: search the current dir FIRST, then the
rest */
```

The actual search order on this box:

```
#include "..." search starts here:
#include <...> search starts here:
 /usr/lib/gcc/x86_64-linux-gnu/13/include
 /usr/local/include
 /usr/include/x86_64-linux-gnu
 /usr/include
End of search list.
```

`-I dir` adds to the `<>` path (and comes before the system dirs); `-iquote dir` adds only to the `"` path.

Convention: `<>` for system and third-party headers, `"` for your own. Some codebases use `<>` for everything with `-I` set to the project root, which makes include paths explicit and location-independent — a defensible choice for large projects.

`#include` is literal text substitution

```
stdio.h      812 lines
stdlib.h    1298 lines
string.h     370 lines
```

Every `#include <stdio.h>` pastes 812 lines into your translation unit. This is why **build times are dominated by header parsing** and why "include what you use" matters:

- Include the minimum in headers; push includes down into `.c` files
- Use **forward declarations** in headers when you only need a pointer: `struct Widget;` instead of `#include "widget.h"`
- Opaque types (Module 11 §11.8) exist partly for this reason

Include guards

```
#ifndef GUARD_H
#define GUARD_H
struct Thing { int x; };
#endif /* GUARD_H */
```

Without one, a double include is a hard error:

```
error: redefinition of 'struct Other'
```

Every header needs a guard. Not because *you* would include it twice, but because header A includes B and C, both of which include D.

Naming: derive it from the path — `PROJECT_MODULE_FILE_H`. Avoid leading underscores (`_MYHEADER_H` is in the reserved namespace).

`#pragma once`

```
#pragma once
struct P1 { int z; };

compiled clean
```

	#ifndef guards	#pragma once
Standard C	☑	✗ (but universally supported)
Name collisions	possible	impossible
Speed	fast (compilers optimize it)	fast
Symlinks / hard links / same file via two paths	works	may fail — the compiler dedupes by file identity

Both are fine in practice. `#ifndef` is safer for maximum portability and for build systems that copy headers around; `#pragma once` is cleaner. Some projects use both.

12.2 Object-like and function-like macros

```
#define PI 3.14159 /* object-like */
#define SQ(x) ((x) * (x)) /* function-like */
```

⚠ **No space before `(` in a function-like macro.** `#define F (x) x` defines `F` as the object-like text `(x) x`.

Multi-line macros use backslash continuation — with **nothing after the backslash**, not even a space.

12.3 The five macro pitfalls

Every one of these measured live:

1. Missing parens around parameters

```
#define SQ_BAD(x) x * x
SQ_BAD(2+3) /* expands to: 2+3 * 2+3 */
```

```
SQ_BAD(2+3) = 11    (expands to 2+3*2+3 = 11)
SQ_OK(2+3)  = 25
```

2. Missing parens around the whole expansion

```
100 / SQ_BAD(5) /* expands to: 100 / 5 * 5 */
```

```
100/SQ_BAD(5) = 100
100/SQ_OK(5)  = 4
```

3. Precedence inside the body

```
#define MAX_BAD(a,b) a > b ? a : b
1 + MAX_BAD(2,3)    /* expands to: 1 + 2 > 3 ? 2 : 3 */
```

```
1 + MAX_BAD(2,3) = 3
1 + MAX_OK(2,3)  = 4
```

Rule 1: parenthesize every parameter AND the entire body. `#define SQ(x) ((x) * (x))`

4. Multiple evaluation — the one parens can't fix

```
#define SQ_OK(x) ((x) * (x))
int r = SQ_OK(f());
```

```
SQ_OK(f()) = 9, f() called 2 times (!!)
```

```
MAX_OK(a++,b++) = 3, a=2 b=4 (b incremented twice)
```

The argument text is substituted, so side effects happen once per appearance. This is the pitfall that survives correct parenthesization and produces genuinely mysterious bugs — off-by-one counters, doubled I/O, corrupted iterators.

Three defenses:

```
/* a. Use a static inline function (best — see §12.9) */
static inline int max_i(int a, int b) { return a > b ? a : b; }
```

```
/* b. GNU statement expression – evaluates each argument once */
#define MAX_SAFE(a,b) __extension__({ __auto_type _a=(a); __auto_type _b=(b);
_a>_b?_a:_b; })

/* c. Name it so callers know */
#define MAX_EVAL_TWICE(a,b) ((a) > (b) ? (a) : (b))
```

```
MAX_MACRO(f(),3) = 5, f() called 2 times
max_fn(f(),3)    = 5, f() called 1 time
MAX_SAFE(f(),3)  = 5, f() called 1 time  (GNU stmt-expr)
```

5. Statement macros and the semicolon

```
#define SWAP_BAD(a,b) int t=a; a=b; b=t;
if (x<y) SWAP_BAD(x,y); else printf("no\n");
```

```
error: expected expression before 'int'
error: 't' undeclared
```

The multi-statement body escapes the `if`, and the `else` becomes orphaned — the same class of bug as Module 5's dangling-else.

The fix: `do { ... } while (0)`

```
#define SWAP_OK(a,b) do { int t_ = (a); (a) = (b); (b) = t_; } while (0)

if (x < y) SWAP_OK(x, y); else printf("no swap\n");    /* works */
```

```
do{}while(0) version works with if/else: x=2 y=1
```

Why it works: `do{...}while(0)` is a **single statement** that *requires* a trailing semicolon, so `MACRO(a,b);` parses exactly like a function call. A bare `{ ... }` block wouldn't — the `;` after it would be an empty statement, breaking `else`.

Note the `t_` naming: **name macro-local variables with a distinctive suffix** so `SWAP_OK(t, x)` doesn't shadow the caller's `t`. GNU `__COUNTER__` can generate unique names.

12.4 Stringification, token pasting, variadics

— stringify

```
#define STR(x)  #x
#define XSTR(x) STR(x)      /* expand first, THEN stringify */
#define VERSION 42
```

```
STR(VERSION)  = "VERSION"    <- literal text, NOT expanded
XSTR(VERSION) = "42"         <- expanded first
STR(a + b)    = "a + b"
STR("quoted") = "quoted"     <- quotes/backslashes are escaped
```

The two-level idiom is essential. # and ## suppress expansion of their operand, so you need an extra layer to force it. Same for ##:

```
#define CAT(a,b)  a##b
#define XCAT(a,b) CAT(a,b)
```

— token pasting

```
#define CAT(a,b) a##b
int id_var = 7;
CAT(id_, var)      /* → id_var */
```

```
CAT(id_, var) = 7    (built the identifier id_var)
```

The result must be a **valid single token**. `CAT(1, +)` is UB.

Common uses: generating unique identifiers (`XCAT(tmp_, __LINE__)`), building function names in code-generation macros, and X-Macros (§12.7).

Variadic macros

```
#define LOG(fmt, ...)    printf("[LOG] " fmt "\n", __VA_ARGS__)
#define LOG_GNU(fmt, ...) printf("[GNU] "  fmt "\n", ##__VA_ARGS__)    /* GNU */
#define LOG_C23(fmt, ...) printf("[C23] "  fmt "\n" __VA_OPT__(,) __VA_ARGS__)
```

```
[LOG] value = 42
[GNU] no args at all
[C23] also no args
```

The zero-argument problem: plain `__VA_ARGS__` with no variadic arguments leaves a trailing comma:

```
warning: ISO C99 requires at least one argument for the "..." in a variadic macro
error: expected expression before ')' token
```

Three solutions:

- `, ##__VA_ARGS__` — GNU extension, eats the preceding comma. Widely supported (GCC, Clang, and MSVC with `/Zc:preprocessor`).
- `__VA_OPT__(,)` — **C23 standard**. Emits the comma only when `__VA_ARGS__` is non-empty. Use this when you can.
- **Require at least one argument:** `#define LOG(...) printf("[LOG] " __VA_ARGS__)` — the format string goes into the variadic part.

Counting arguments:

```
#define COUNT_ARGS(...) (sizeof((int[]){__VA_ARGS__}) / sizeof(int))
```

```
COUNT_ARGS(1,2,3,4,5) = 5
```

⚠ **Commas inside macro arguments must be protected.** `FOO(struct {int a, b;})` passes two arguments. Wrap in parens, or use a `COMMA` macro.

12.5 Conditional compilation

```
#if defined(FEATURE_A) && FEATURE_A
#elif __STDC_VERSION__ >= 202000L
#else
#endif

#ifdef NAME      /* == #if defined(NAME) */
#ifndef NAME     /* == #if !defined(NAME) */
```

```
FEATURE_A: enabled   FEATURE_B: disabled
standard : C11/C17
note      : undefined macros are 0 in #if
```

The rules of `#if` expressions

- Only **integer constant expressions**. No `sizeof`, no floats, no variables, no function calls.
- **Undefined identifiers evaluate to 0** — so `#if FOO` is false when `FOO` is undefined, and `#if FOO == 0` is *true*. That asymmetry causes real bugs. **Prefer `#ifdef` / `defined()` when you mean "is it defined."**
- Arithmetic is done in `intmax_t/uintmax_t`.
- `defined(X)` and `defined X` both work.

`#error` and `#warning`

```
#if __STDC_VERSION__ < 201112L
# error "this code requires C11 or later"
#endif
#warning "build note: experimental feature enabled"
```

```
-- as C99 -- error: #error "this code requires C11 or later"
-- as C17 -- warning: #warning "build note: experimental feature enabled" [-Wcpp]
```

`#error` is standard; `#warning` is a widely-supported extension (standardized in C23). Use `#error` to fail fast on unsupported configurations — far better than a mysterious failure 200 errors later.

Style

Indent the `#` directives after the `#`, not before:

```
#if A
# if B
#   define X
# endif
#endif
```

(Some old preprocessors required `#` in column 1.)

Comment your `#endif` when the block is long: `#endif /* FEATURE_A */`.

Prefer runtime `if` to `#ifdef` when you can:

```
/* BAD: code inside #ifdef isn't compiled, so it rots */
#ifdef DEBUG
    validate(x);
#endif

/* BETTER: always compiled and type-checked, optimized away when DEBUG is 0 */
if (DEBUG) validate(x);
```

The second version keeps every configuration compiling. Excessive `#ifdef` is a known maintainability disaster — you effectively have 2ⁿ programs and test one.

12.6 Predefined macros

```
__FILE__      = cond.c
__LINE__      = 40
__func__      = main  (NOT a macro – a static const char[])
__DATE__ __TIME__ = Jul 28 2026 06:03:10
__STDC__      = 1
__STDC_VERSION__ = 201710L
__GNUC__      = 13.3.0
__x86_64__    = defined
__COUNTER__   = 0 1 2  (GNU: increments each use)
```

Macro	Notes
<code>__FILE__</code>	Source path as given to the compiler
<code>__LINE__</code>	Current line number
<code>__func__</code>	C99, not a macro — a <code>static const char[]</code> inside the function, so it's unavailable to the preprocessor
<code>__DATE__</code> , <code>__TIME__</code>	Compilation timestamp — breaks reproducible builds , avoid
<code>__STDC__</code> , <code>__STDC_VERSION__</code>	Conformance and standard version (Module 1)
<code>__COUNTER__</code>	GNU; unique increment per use — for generating unique identifiers
<code>__GNUC__</code> , <code>_MSC_VER</code> , <code>__clang__</code>	Compiler detection
<code>__x86_64__</code> , <code>__aarch64__</code> , <code>_WIN32</code> , <code>__linux__</code>	Platform detection
<code>__STDC_NO_VLA__</code> , <code>__STDC_NO_THREADS__</code> , <code>__STDC_NO_ATOMICS__</code>	Optional-feature detection (C11)

The classic use — a debug/logging macro:

```
#define LOG(fmt, ...) \
    fprintf(stderr, "[%s:%d %s] " fmt "\n", __FILE__, __LINE__, __func__ \
    __VA_OPT__(,) __VA_ARGS__)

#define CHECK(cond) do { \
    if (!(cond)) { \
        fprintf(stderr, "%s:%d: CHECK failed: %s\n", __FILE__, __LINE__, \
        #cond); \
```

```

        abort(); \
    } \
} while (0)

```

`#cond` stringifies the *source text* of the condition — that's why `assert` can print `assert(x > 0)`. This is the single most useful thing `#` does.

12.7 X-Macros — code generation

The problem: an enum, a name table, a value table, and a parser must all stay in sync. Add one entry and you must edit four places. Miss one and you get a silent bug.

The solution: define the data once, expand it multiple ways.

```

/* ---- THE SINGLE SOURCE OF TRUTH ---- */
#define COLOR_LIST
    X(RED,    0xFF0000, "red")
    X(GREEN,  0x00FF00, "green")
    X(BLUE,   0x0000FF, "blue")
    X(WHITE,  0xFFFFFFFF, "white")

/* 1. the enum */
#define X(name, rgb, str) COLOR_##name,
typedef enum { COLOR_LIST COLOR_COUNT } Color;
#undef X

/* 2. the name table */
#define X(name, rgb, str) [COLOR_##name] = str,
static const char *const color_names[] = { COLOR_LIST };
#undef X

/* 3. the RGB table */
#define X(name, rgb, str) [COLOR_##name] = rgb,
static const unsigned color_rgb[] = { COLOR_LIST };
#undef X

/* 4. a parser */
#define X(name, rgb, str) if (strcmp(s, str) == 0) return COLOR_##name;
static int color_parse(const char *s) { COLOR_LIST return -1; }
#undef X

_Static_assert(sizeof color_names / sizeof *color_names == COLOR_COUNT, "out of
sync");

```

```

generated from ONE list (4 colors):
red    = #FF0000 (enum value 0)
green  = #00FF00 (enum value 1)
blue   = #0000FF (enum value 2)

```

```
white = #FFFFFF (enum value 3)

parse("blue")    -> 2 (blue)
parse("purple") -> -1
```

What the preprocessor actually produced:

```
typedef enum { COLOR_RED, COLOR_GREEN, COLOR_BLUE, COLOR_WHITE, COLOR_COUNT }
Color;
static const char *const color_names[] = { [COLOR_RED] = "red", [COLOR_GREEN] =
"green",
                                           [COLOR_BLUE] = "blue", [COLOR_WHITE] =
"white", };
```

Add one `X(...)` line and the enum, both tables, and the parser all update. Combined with `_Static_assert` and `-Werror=switch`, it's impossible for them to drift.

Where X-Macros shine

- Enum ↔ string conversion (both directions)
- Error code tables
- Serialization: generating pack/unpack for every field of a struct
- Register maps in embedded code
- Opcode dispatch tables for VMs
- Test registries
- Command tables for CLIs

The costs

- **Debugging is painful** — errors point at the macro definition. Use `gcc -E` to see the expansion (Module 1).
- **Unusual-looking code.** Comment the pattern.
- **IDE tooling struggles** with the generated names.

For large-scale generation, a real code generator (Python script + build rule) is often clearer. But X-Macros need no build-system changes and can't get out of sync with the source.

12.8 `_Generic` — type-based dispatch (C11)

```
_Generic(controlling_expression,
         type1: expr1,
         type2: expr2,
         default: expr_default)
```

Resolved at **compile time** based on the *type* of the controlling expression.

```
#define type_name(x) _Generic((x), \
    char: "char", int: "int", long: "long", \
    float: "float", double: "double", \
    char *: "char *", default: "unknown")
```

```
1      -> int
1.0    -> double
1.0f   -> float
'c'    -> int    <- character constants are int in C
"lit"  -> char *
```

Type-generic function dispatch

```
#define my_abs(x) _Generic((x), \
    int: abs, long: labs, float: fabsf, double: fabs, default: fabs)(x)

#define print(x) _Generic((x), \
    int: print_int, double: print_dbl, char*: print_str, const char*: print_str)
(x)
```

```
int:      1
double: 2.5
string: hi
my_abs(-5)=5   my_abs(-5.5)=5.5
```

This is exactly how `<tgmath.h>` works, and it's the closest C gets to function overloading.

The rules that trip people up

1. **The controlling expression is not evaluated** — only its type is used. (Similar to `sizeof`.)
2. **Non-selected branches must still be valid syntax and type-check**. My first version failed to compile because `abs` wasn't declared, even though the `int` branch wasn't the one being selected:

```
error: 'abs' undeclared (first use in this function)
```

3. **Arrays and functions decay** before selection — you can't match `int[10]`, only `int*`.
4. **Top-level qualifiers are stripped** — you can't distinguish `int` from `const int`. (But `const char *` and `char *` are different types and can both appear.)
5. **Every type must be distinct**; duplicates are an error.
6. `_Generic` is an **expression**, not a statement.

Real uses

```
/* Type-safe max without multiple-evaluation risk (GNU) */
#define max(a,b) __extension__({ __auto_type _a=(a); __auto_type _b=(b); _a>_b?
_a:_b; })

/* Format-string selection */
#define FMT(x) _Generic((x), int:"%d", long:"%ld", double:"%g", char*:"%s")

/* Compile-time type assertions */
#define IS_POINTER(x) _Generic((x), default:0, int*:1, char*:1, void*:1)
```

```
IS_POINTER(i)=0 IS_POINTER(s)=1 IS_POINTER(&i)=1
```

12.9 #embed (C23)

Embeds a file's bytes directly as an initializer list — no `xxd -i`, no build step:

```
static const unsigned char logo[] = {
    #embed "logo.png"
};
static const char shader[] = {
    #embed "shader.glsl" suffix(, 0)    /* NUL-terminate */
};

#if __has_embed("config.bin")
    /* ... */
#endif
```

Parameters: `limit(n)`, `prefix(...)`, `suffix(...)`, `if_empty(...)`.

Availability check on this box:

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

error: invalid preprocessing directive #embed
```

GCC gained #embed in version 15; Clang in 19. It's genuinely useful (embedding assets, firmware blobs, default configs, test fixtures) and dramatically faster than the giant-initializer-list workarounds, but you'll need a recent toolchain. Guard with `#if __has_embed(...)` or `#ifdef __STDC_EMBED_....`

12.10 When *not* to use macros

`static inline` beats a macro almost every time

```
MAX_MACRO(f(),3) = 5, f() called 2 times    ← bug
max_fn(f(),3)    = 5, f() called 1 time     ← correct
```

And the generated code is identical at `-O2`:

```
use_macro:                                use_inline:
    cmp     esi, edi                        cmp     edi, esi
    mov     eax, edi                        mov     eax, esi
    cmovge  eax, esi                        cmovge  eax, edi
    ret                                     ret
```

Same instructions. One is correct, one has a latent bug.

	Macro	<code>static inline</code>
Type checking	✗ none	☑ full
Multiple evaluation	✗ risk	☑ impossible
Debugger visibility	✗ invisible	☑ steppable, breakpointable
Error messages	✗ point at the expansion	☑ point at the call
Scope	✗ everything after <code>#define</code>	☑ normal C scoping
Address-of	✗	☑
Recursion	✗	☑
Generated code at <code>-O2</code>	same	same

Use a macro only when you genuinely can't use a function

Legitimate	Why
Needs <code>__FILE__</code> / <code>__LINE__</code> at the call site	<code>assert</code> , logging, <code>CHECK</code>
Needs the source text (<code>#cond</code>)	assertions, test frameworks
Conditional compilation	<code>#ifdef</code> blocks
Type-generic code	X-Macros, <code>_Generic</code> wrappers, container templates
Compile-time constants	<code>#define MAX_USERS 1024</code> (or better: <code>enum</code>)
Token construction	building identifiers with <code>##</code>

Constants: prefer `enum` or `static const`

```

#define MAX 1024                /* no type, no scope, no debugger symbol */
enum { MAX = 1024 };            /* typed int, scoped, constant expression */
static const int MAX = 1024;    /* typed, scoped – but NOT a constant
expression in C */
constexpr int MAX = 1024;       /* C23: typed, scoped, AND a constant
expression */

```

In C (unlike C++), `const int` is **not** a constant expression — it can't size an array or be a `case` label. `enum` can. C23's `constexpr` finally gives you both.

Macro hygiene checklist

```

#define MACRO_NAME(a, b)        /* 1. UPPER_CASE names */
    ((a) > (b) ? (a) : (b))    /* 2. parens on every parameter and the whole
body */

#define STMT(x) do { ... } while (0) /* 3. do-while for statement macros */
/* 4. name locals with a trailing _ */
/* 5. document multiple-evaluation risk */

#undef TEMP_MACRO               /* 6. #undef helper macros */
/* 7. never define macros in public headers
with generic names (min, max, DEBUG)

*/

```

That last one matters: `#define max(a,b) ...` in a public header will silently break any caller that has a variable or method called `max`. Windows headers defining `min/max` is a legendary source of pain. **Prefix macros in public headers: `MYLIB_MAX`.**

Module 12 rules

#	Rule
1	Every header gets a guard; <code>#pragma once</code> is fine but non-standard
2	Minimize includes in headers; forward-declare where possible
3	Parenthesize every parameter and the whole macro body
4	Assume every argument is evaluated multiple times — document or avoid
5	<code>do { } while (0)</code> for every statement macro
6	Two-level <code>STR/CAT</code> when the argument is itself a macro
7	<code>__VA_OPT__(,)</code> (C23) or <code>, ##__VA_ARGS__</code> (GNU) for zero-argument variadics
8	Undefined identifiers are 0 in <code>#if</code> — use <code>#ifdef</code> when you mean "defined"
9	Prefer runtime <code>if (DEBUG)</code> to <code>#ifdef DEBUG</code> so all configs stay compiled

#	Rule
10	X-Macros + <code>_Static_assert</code> + <code>-Werror=switch</code> for tables that must not drift
11	<code>_Generic</code> 's unselected branches must still compile
12	<code>static inline</code> unless you need <code>__FILE__</code> / <code>#x</code> / <code>##</code> /conditional compilation
13	<code>enum</code> (or C23 <code>constexpr</code>) for constants, not <code>#define</code>
14	Prefix macros in public headers
15	<code>gcc -E</code> is your debugger for macro problems

Practice

1. **Header hygiene.** Take a 3-file project and remove every unnecessary `#include` from the headers, replacing with forward declarations. Measure the build time before and after with `time make -B`. Then run `gcc -E file.c | wc -l` before and after.
2. **Reproduce all five pitfalls.** Write one program demonstrating each, predict each output before running, then fix each one. For pitfall 4, find a fix that works without GNU extensions.
3. **Build a logging framework.** Implement `LOG_ERROR`, `LOG_WARN`, `LOG_INFO`, `LOG_DEBUG` with `__FILE__`, `__LINE__`, `__func__`, a compile-time level filter, and `__VA_OPT__`. Add `__attribute__((format(printf,...)))` to the underlying function (Module 6). Verify a format mismatch is caught at compile time.
4. **CHECK and assert.** Implement `CHECK(cond)` that prints the *source text* of the condition on failure. Then implement `CHECK_EQ(a,b)` that prints both expressions and both values — and make sure each argument is evaluated exactly once.
5. **X-Macro suite.** Define an error-code list with `X(name, number, message)` and generate: an enum, a `const char *err_string(int)`, an `int err_from_string(const char*)`, and a `_Static_assert` on the count. Then add one entry and confirm everything updates.
6. **X-Macro serialization.** Define a struct's fields as an X list and generate the struct definition, a `to_json`, a `from_json` skeleton, and a `print`. Verify round-tripping.
7. **_Generic toolkit.** Build `type_name(x)`, `FMT(x)` (format-string selection), and a `PRINT(x)` that works for `int`, `long`, `double`, `char*`, and pointers. Then discover a limitation: try to distinguish `int[10]` from `int*` and explain why you can't.
8. **Macro vs inline codegen.** Write five common macros (`MIN`, `MAX`, `ABS`, `CLAMP`, `SWAP`) and their `static inline` equivalents. Diff the `-O2` assembly for each. Then find a case where the macro is genuinely necessary.
9. **Expansion archaeology.** Take the most complex macro you can find in a real header (`assert.h`, `container_of`, `offsetof`, a kernel macro) and run `gcc -E` on a file using it. Explain the expansion line by line.

10. **#embed if you can.** If you have GCC 15+ or Clang 19+, embed a binary file and compare compile time against the `xxd -i` approach for a 1 MB file.
13. Input / Output

- `stdio.h`: `printf`/`scanf` family, full format specifier knowledge, width/precision/flags
- **Why `scanf` is a trap**; `fgets` + `strtol` pattern
- `FILE*`, `fopen` modes, text vs binary mode
- `fread`, `fwrite`, `fseek`, `ftell`, `rewind`, `fflush`
- Buffering: full/line/none, `setvbuf`
- `EOF`, `feof` misuse, `ferror`, `clearerr`
- `stdin`/`stdout`/`stderr`, redirection, pipes
- Low-level POSIX I/O: `open`/`read`/`write`/`close`/`lseek`, file descriptors, `mmap`

Module 13 — Input / Output

Real GCC 13.3 / glibc 2.39 output throughout. Two things here cause more confusion than the rest of `stdio` combined: **buffering** (§13.6) and **`scanf`** (§13.2).

13.1 The `printf` family

```
int printf (const char *fmt, ...);           /* to stdout */
int fprintf(FILE *fp, const char *fmt, ...); /* to a stream */
int sprintf(char *buf, const char *fmt, ...); /* ✗ BANNED – no bound */
int snprintf(char *buf, size_t n, const char *fmt, ...); /* ☑ use this */
int vprintf / vfprintf / vsnprintf(..., va_list ap); /* for wrappers (Module 6)
*/
```

Return value = number of characters written (excluding the NUL), or negative on error. For `snprintf`, the number that *would* have been written — the truncation check from Module 8.

The specifier anatomy

```
%[flags][width][.precision][length]conversion
```

Conversions

<code>%d %i</code>	signed decimal	<code>%F %f</code>	fixed-point
<code>%u</code>	unsigned decimal	<code>%e %E</code>	scientific
<code>%o</code>	octal	<code>%g %G</code>	shorter of <code>%e/%f</code>
<code>%x %X</code>	hex	<code>%a %A</code>	hex float (exact)

<code>%c</code>	character	<code>%s</code>	string
<code>%p</code>	pointer (cast to <code>void*</code> !)	<code>%%</code>	literal %
<code>%n</code>	writes a count — security hazard, never use		

Length modifiers — get these wrong and it's UB

Modifier	Type
<code>hh / h</code>	<code>char</code> / <code>short</code> (promoted to <code>int</code> , so the modifier tells <code>printf</code> to truncate)
<code>l / ll</code>	<code>long</code> / <code>long long</code>
<code>z</code>	<code>size_t</code>
<code>t</code>	<code>ptrdiff_t</code>
<code>j</code>	<code>intmax_t</code>
<code>L</code>	<code>long double</code>

```
%d -42      %i -42
%u 42       %o 52      %x ff   %X FF
%hhd -1     %hd -1     %ld -1  %lld -1
%zu 4       %td -5     %jd -9
PRId64 -1   PRIx32 abcd
```

Use `%zu` for `size_t`, `%td` for `ptrdiff_t`, and the `<inttypes.h>` macros for fixed-width types (Module 2). `printf("%d", sizeof x)` is UB on 64-bit.

There is no `%lf` in `printf` — `float` is promoted to `double` by the default argument promotions (Module 6). `%f` handles both. (In `scanf` it's different: `%f` = `float*`, `%lf` = `double*`.)

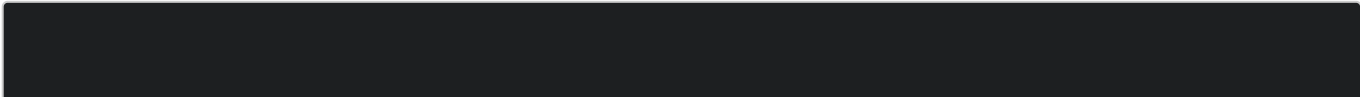
Floats

```
%f 3.141590      %e 3.141590e+04
%g 31415.9       %g 3.1415e-05  (shortest of %e/%f)
%a 0x1.999999999999ap-4  (hex float – exact, round-trips)
%.17g 0.100000000000000001 (enough digits to round-trip a double)
```

`%g` drops trailing zeros and picks the shorter representation — usually what you want for human output.

`%.17g` round-trips a `double` exactly in decimal; `%a` round-trips exactly and compactly. Use `%a` for debugging float bugs — it shows you the true value (Module 2).

Flags, width, precision



%10d		42	
%-10d	42		left-justify
%010d	0000000042		zero-pad
%+d	+42	-42	always show sign
% d	42		space for positive
%#x	0xff	%#o	010 alternate form
%.3f	3.142		
%.3s	abc		truncate string
%10.3f		3.142	
%*d		42	width from an argument
%-*.*s	abcd		both from arguments

* takes the width/precision from an `int` argument — useful for runtime-computed table columns.

%.*s is the safest way to print a non-NUL-terminated buffer:

```
printf("%.s", (int)len, ptr);    /* prints exactly len bytes */
```

Security

⚠ **Never pass user input as a format string.**

```
printf(user_input);             /* FORMAT STRING VULNERABILITY */
printf("%s", user_input);       /* correct */
```

`%n` writes to memory through a pointer argument — with a crafted format string it's an arbitrary-write primitive. `%s` with no matching argument reads a garbage pointer. This is a classic RCE class (CVE-2000-0573 and hundreds since).

Compile with `-Wformat=2` (includes `-Wformat-security` and `-Wformat-nonliteral`), and put `__attribute__((format(printf,m,n)))` on every wrapper (Module 6).

13.2 Why `scanf` is a trap

Trap 1 — the newline stays in the buffer

```
scanf("%d", &n);
fgets(name, sizeof name, stdin);
```

```
input "42\nhello\n":
got n=42
fgets got: "\n"    <- the leftover newline!
```

`%d` skips leading whitespace and stops at the first non-digit — leaving `\n` in the stream. The next `fgets` or `%c` reads it immediately. Beginners "fix" this with `getchar()` or `while(getchar() != '\n')`, which mostly works and occasionally doesn't.

Trap 2 — on failure, the input is *not consumed*

```
now reading %d from "abc":
scanf returned 0 (0 = matched nothing)
trying again returns 0 - SAME input still there -> infinite loop
```

This is the classic "my program hung" bug. A retry loop around `scanf` spins forever because the offending characters never leave the buffer. You must drain them manually.

Trap 3 — `%s` has no bound

```
char b[8];
scanf("%s", b);
```

```
ERROR: AddressSanitizer: stack-buffer-overflow
[32, 40) 'b' <== Memory access at offset 40 overflows this variable
```

`scanf("%s")` is as dangerous as `gets` — unbounded write. The bounded form works:

```
scanf("%7s", b);          /* 7 = sizeof b - 1; the NUL is added on top */
```

```
got: AAAAAAA
```

But the width must be a **literal** — you can't write `scanf("%*ds")` with a runtime bound, so it can't track `sizeof buf` automatically. Every change to the buffer size requires editing the format string.

Trap 4 — no error detail, no overflow detection

`scanf` returns the number of items **successfully assigned**, or `EOF`. It can't tell you *why* something failed, and `%d` on `"99999999999999999999"` is **undefined behavior** (same as `atoi`).

The other subtleties

- Whitespace in the format matches *any amount* of whitespace, including none.
- Non-whitespace characters must match literally, or the scan stops.
- `%c` does **not** skip leading whitespace (unlike everything else). Use `" %c"` with a leading space.

- `%[^\n]` (scanset) reads a line — but fails and returns 0 on an *empty* line.
- Suppression: `scanf` reads and discards.

The verdict: `scanf` from `stdin` is unusable for robust programs. `sscanf` on a string you already have is defensible (the input is bounded), but even then you must bound every `%s` and check the return value.

13.3 The correct input pattern: `fgets` + `strtol`

Read a whole line, then parse it. The two concerns are separated, both are checkable.

```
static int read_line(char *buf, size_t n, FILE *fp)
{
    if (!fgets(buf, n, fp)) return -1;           /* EOF or error */
    size_t len = strcspn(buf, "\n");
    if (buf[len] != '\n') {                      /* line was truncated */
        int c;
        while ((c = fgetc(fp)) != '\n' && c != EOF) { } /* drain the rest */
        return -2;
    }
    buf[len] = '\0';                            /* strip the newline */
    return 0;
}
```

Combined with the `parse_long` from Module 8:

```
"42" -> 42
"abc" -> not a number
"99xyz" -> trailing garbage
[line too long, discarded]
"-7" -> -7
```

Every failure mode is detected and reported distinctly. That's what you can't get from `scanf`.

Key points:

- `fgets` keeps the newline. `buf[strcspn(buf, "\n")] = '\0'` is the idiomatic strip.
- **A missing newline means the line was truncated** — you must drain the remainder or the next read gets the tail.
- `fgets` returns `NULL` on EOF or error — distinguish with `feof/ferror`.
- POSIX `getline()` allocates and grows the buffer for you, handling arbitrary line lengths:

```
char *line = NULL; size_t cap = 0;
ssize_t len;
while ((len = getline(&line, &cap, stdin)) != -1) { ... }
free(line);
```

Not ISO C, but available on Linux/macOS/BSD and much nicer. ⚠ Note `getline` also keeps the newline, and returns `-1` at EOF.

13.4 FILE* and fopen modes

```
FILE *fp = fopen(path, mode);
if (!fp) { perror(path); return -1; }      /* ALWAYS check */
```

Mode	Read	Write	Truncate	Create	Position
"r"	☒				start
"w"		☒	☒	☒	start
"a"		☒		☒	end, always
"r+"	☒	☒			start
"w+"	☒	☒	☒	☒	start
"a+"	☒	☒		☒	reads anywhere, writes always append

Add `b` for binary ("`rb`", "`wb`") and `x` (C11) to fail if the file exists — "`wx`" is the atomic-create mode you want to avoid TOCTOU races.

Text vs binary mode

On Windows text mode translates `\n` ↔ `\r\n` on the way in and out, and historically treated Ctrl-Z as EOF.
On Unix there is no difference.

Always use "`rb`"/"`wb`" for binary data. Forgetting the `b` is a Windows-only corruption bug that Unix developers never see. There is no downside to using `b` on Unix.

Also: with an update mode ("`r+`", "`w+`", "`a+`"), you **must** call `fflush`, `fseek`, `fsetpos`, or `rewind` between a read and a write, or the behavior is undefined.

Cleanup

`fclose` flushes and releases the `FILE*`. **Check its return value** — a write error may only surface at flush time:

```
if (fclose(fp) != 0) { perror("fclose"); /* data may not have been written */ }
```

For durability you need more than `fclose`: `fflush(fp)` pushes stdio buffers to the OS, then `fsync(fileno(fp))` pushes OS buffers to the disk.

13.5 fread, fwrite, fseek, ftell, rewind

```
size_t fread (void *ptr, size_t size, size_t nmemb, FILE *fp);
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *fp);
```

Both return the **number of items** (not bytes) transferred. A short count means EOF or error — check with `fEOF/ferror`.

```
fwrite wrote 3 of 3 records (24 bytes each)
file size = 72 bytes (3 * 24 = 72)

random access with fseek:
  record[1]: id=2 name=bob score=87.25 (ftell now 48)

fread read 3 records after rewind:
  1 alice  95.5
  2 bob    87.25
  3 carol  99
round-trip identical? 1
```

Positioning

```
fseek(fp, offset, SEEK_SET); /* from the start */
fseek(fp, offset, SEEK_CUR); /* from the current position */
fseek(fp, 0, SEEK_END); /* to the end */
long pos = ftell(fp); /* current position */
rewind(fp); /* = fseek(fp,0,SEEK_SET) + clearerr() */
```

⚠ **ftell returns long** — that's 32-bit on Windows, so it breaks past 2 GB. Use `fgetpos/fsetpos` (portable, opaque `fpos_t`) or POSIX `ftello/fseeko` with `off_t` and `-D_FILE_OFFSET_BITS=64`.

⚠ **fseek on a text-mode stream** is only well-defined with offsets obtained from `ftell` (or 0 with `SEEK_SET`).

⚠ **fseek(fp,0,SEEK_END) + ftell to get a file size is not portable** for text streams and doesn't work on pipes. Use `stat/fstat`.

Writing raw structs is not portable

```
WARNING: writing raw structs is NOT portable:
- padding bytes leak stack data (Module 11)
- endianness differs between machines
- sizeof/alignment differ between ABIs
-> serialize field by field, in a fixed byte order
```


`fwrite(&mystruct, sizeof mystruct, 1, fp)` is fine for a scratch file your own program reads back on the same machine. **For any file another program or machine will read, serialize explicitly:**

```
static void put_u32le(FILE *fp, uint32_t v)
{ fputc(v & 0xFF, fp); fputc((v>>8)&0xFF, fp); fputc((v>>16)&0xFF, fp);
  fputc((v>>24)&0xFF, fp); }
```

The padding-leak issue is a real information-disclosure CVE class — you write your struct to a network socket and 3 bytes of uninitialized stack go with it.

13.6 Buffering ⚠

`stdio` buffers writes to reduce syscalls. **The buffering mode depends on what the stream is connected to**, which is why the same program behaves differently in a terminal and in a pipe.

Stream	Default mode
<code>stdout</code> to a terminal	Line buffered — flushes on <code>\n</code>
<code>stdout</code> to a pipe or file	Full buffered — flushes when the buffer fills
<code>stderr</code>	Unbuffered — always immediate

The ordering surprise

```
printf("this printf may sit in the buffer...\n");
fprintf(stderr, "this fprintf(stderr) appears IMMEDIATELY\n");
write(1, "this raw write(2) bypasses stdio entirely\n", 41);
printf("...and only now does the earlier printf flush\n");
```

Piped to `cat`:

```
this fprintf(stderr) appears IMMEDIATELY
this raw write(2) bypasses stdio entirely
stdout is a pipe/file -> FULL buffered
stderr is ALWAYS unbuffered
this printf may sit in the buffer...
...and only now does the earlier printf flush
```

The output is completely out of source order. `stderr` and `write(2)` went out immediately; the `printf`s all appeared at the end when the buffer flushed at exit. Two lessons:

1. **Interleaving `stdout` and `stderr` gives scrambled logs.** Pick one stream for a given output, or make `stdout` unbuffered.
2. **Never mix `printf` and `write(2)` on the same fd.**

The "my output disappeared" bug

```
printf("step 1 done\n");
printf("step 2 done\n");
abort();
```

```
-- output to a pipe (buffered -> LOST): --
    (nothing above = both printf's were lost)

-- with setvbuf(stdout, NULL, _IONBF, 0): --
step 1 done
step 2 done
```

abort(), _exit(), and a segfault do NOT flush stdio buffers. Your last N lines of debug output — the ones right before the crash — are exactly the ones you lose.

This is why "the program crashes but the printf before the crash didn't print" is so common, and why beginners conclude the crash is on the wrong line.

Fixes:

```
setvbuf(stdout, NULL, _IONBF, 0);          /* unbuffered – do this while debugging */
/*
setvbuf(stdout, NULL, _IOLBF, 0);          /* line buffered even when piped */
fflush(stdout);                            /* explicit flush at critical points */
fprintf(stderr, ...);                       /* stderr is always unbuffered */
stdbuf -o0 ./prog                          /* from the shell, no recompile */
```

exit() and returning from **main** **do** flush (via **atexit** handlers). **_exit()** and **abort()** don't.

setvbuf

```
int setvbuf(FILE *fp, char *buf, int mode, size_t size);
/* mode: _IOFBF (full), _IOLBF (line), _IONBF (none) */
```

Must be called after fopen and before any I/O on the stream. Pass **NULL** for **buf** to let stdio allocate.

Performance note: full buffering is dramatically faster for bulk output. Unbuffered **printf** in a tight loop is one syscall per call.

13.7 EOF, feof, ferror, clearerr

The while (!feof(fp)) bug

```
while (!feof(f)) { c = fgetc(f); ... }
```

```
iter 1: c=97 'a'
iter 2: c=98 'b'
iter 3: c=99 'c'
iter 4: c=-1 '?'      ← one iteration too many
```

feof reports that a read has *already* failed — it's set *after* you hit the end, not before. So the loop runs one extra time with garbage.

The correct form tests the read's return value:

```
while ((c = fgetc(f)) != EOF) { ... }
while (fgets(buf, sizeof buf, fp)) { ... }
while ((n = fread(buf, 1, sizeof buf, fp)) > 0) { ... }
```

```
iter 1: 'a'   iter 2: 'b'   iter 3: 'c'      ← exactly 3
```

feof/ferror are for *after* the loop

```
while ((c = fgetc(f)) != EOF) { ... }
if (ferror(f))      { perror("read error"); }
else if (feof(f))  { /* clean end of file */ }
```

```
feof(f)=1 ferror(f)=0 -> clean end of file
```

EOF alone doesn't tell you *why* the read stopped — end of file or a real I/O error. **Always distinguish after a loop that processes real data.**

clearerr(fp) resets both flags — needed if you want to keep reading a stream after a transient error, or after EOF on a terminal.

EOF must go in an **int**

From Module 2: **fgetc/getchar** return **int** so **EOF** (−1) is distinguishable from the valid byte **0xFF**. **char c = getchar()** is broken both ways (infinite loop if **char** is unsigned, early exit on binary data if signed).

13.8 **stdin**, **stdout**, **stderr**, redirection

Three **FILE*** streams open before **main**, backed by file descriptors 0, 1, 2.

```
STANDARD FILE DESCRIPTORS: stdin=0 stdout=1 stderr=2
```

The conventions that make Unix tools compose

Stream	Purpose
stdout	Program output — the data. Redirectable, pipeable.
stderr	Diagnostics — errors, progress, warnings. Stays on the terminal even when stdout is piped.
stdin	Input, possibly a pipe or file

```
./prog > out.txt           # stdout to a file
./prog 2> err.txt          # stderr to a file
./prog > out 2>&1          # both to the same file
./prog 2>/dev/null         # discard errors
./prog < in.txt            # stdin from a file
a | b | c                  # a's stdout → b's stdin → c's stdin
```

Rule: errors, warnings, and usage messages go to **stderr; data goes to **stdout**.** A tool that prints errors to stdout corrupts every pipeline it's used in. This is why the Module 1 example wrote **usage:** to **stderr**.

Also: **check whether stdout is a terminal** before emitting colors or progress bars:

```
#include <unistd.h>
if (isatty(STDOUT_FILENO)) { /* colors, spinners */ }
```

13.9 POSIX low-level I/O

```
#include <fcntl.h>
#include <unistd.h>

int      open(const char *path, int flags, mode_t mode);
ssize_t  read(int fd, void *buf, size_t count);
ssize_t  write(int fd, const void *buf, size_t count);
int       close(int fd);
off_t    lseek(int fd, off_t offset, int whence);
```

```
open() returned fd=3 (lowest available)
write() returned 15 (may be SHORT - always loop!)
read() got 15 bytes: "hello posix io"
lseek to 6: 6, then read: "posix"
```

Flags

`O_RDONLY` / `O_WRONLY` / `O_RDWR` (pick exactly one), OR'd with `O_CREAT`, `O_TRUNC`, `O_APPEND`, `O_EXCL`, `O_NONBLOCK`, `O_CLOEXEC`, `O_SYNC`. The `mode` argument (permissions) is only used with `O_CREAT`.

`O_CREAT|O_EXCL` is the atomic "create only if absent" — the fix for TOCTOU races.

⚠ Short reads and writes are normal

read and write may transfer fewer bytes than requested, and it is not an error. This happens on pipes, sockets, terminals, and when interrupted by a signal. **You must loop:**

```
static ssize_t write_all(int fd, const void *buf, size_t n)
{
    const char *p = buf;
    size_t left = n;
    while (left > 0) {
        ssize_t w = write(fd, p, left);
        if (w < 0) {
            if (errno == EINTR) continue;      /* interrupted - retry */
            return -1;
        }
        p += w; left -= (size_t)w;
    }
    return (ssize_t)n;
}
```

Forgetting this is one of the most common bugs in networking code — it works in testing (small messages) and truncates in production (large ones).

`mmap` — the file *is* memory

```
#include <sys/mman.h>
void *map = mmap(NULL, len, PROT_READ, MAP_PRIVATE, fd, 0);
```

```
mmap: file is now MEMORY at 0x7fc4681b1000, size 15
map[0..4] = "hello" (no read() syscall, just a page fault)
```

Pages are faulted in on demand by the kernel — **zero copies, zero syscalls per access.**

mmap wins	read/write wins
Large files, random access	Sequential streaming
Multiple processes sharing a file	Small files (setup cost dominates)
Avoiding a copy into your buffer	Pipes, sockets (can't mmap them)
Treating a file as an array	Files that may be truncated (→ SIGBUS)

⚠ **If the file shrinks while mapped, touching the vanished pages raises SIGBUS**, not a return code. And `mmap` doesn't work on non-seekable streams.

Always `munmap` when done. Use `MAP_SHARED` + `msync` for writes visible to other processes.

FILE* vs raw fd

	FILE*	file descriptor
Buffering	yes (stdio)	none (each call is a syscall)
Portability	ISO C, everywhere	POSIX
Formatted I/O	<code>printf/scanf</code>	none
Needed for	ordinary files	sockets, pipes, <code>select/poll/epoll</code> , <code>fcntl</code> , <code>mmap</code> , <code>ioctl</code>
Bridge	<code>fileno(fp) → fd</code>	<code>fdopen(fd, mode) → FILE*</code>

⚠ **Never mix buffered and unbuffered I/O on the same file.** `printf` then `write(1,...)` gives you the scrambled output demonstrated in §13.6. If you must, `fflush` first.

Also worth knowing: `pread/pwrite` (positioned I/O without changing the offset — essential for threaded code), `readv/writev` (scatter/gather), `sendfile/splice` (zero-copy file→socket), and `O_CLOEXEC` (so descriptors don't leak across `exec`).

Module 13 rules

#	Rule
1	<code>%zu</code> for <code>size_t</code> , <code>%td</code> for <code>ptrdiff_t</code> , <code>PRId64</code> for <code>uint64_t</code> ; cast to <code>void*</code> for <code>%p</code>
2	Never pass user input as a format string; compile with <code>-Wformat=2</code>
3	<code>%.17g</code> or <code>%a</code> to see a float's true value
4	Don't use <code>scanf</code> on <code>stdin</code> — use <code>fgets</code> + <code>strtoul</code>
5	If you must, bound every <code>%s</code> (<code>%15s</code>) and check the return count
6	Always check <code>fopen</code> 's return, and <code>fclose</code> 's too
7	Always use <code>"rb"/"wb"</code> for binary — it's free on Unix and required on Windows
8	Never <code>fwrite</code> raw structs to a file another program reads

#	Rule
9	<code>while (!feof(fp))</code> is always wrong — test the read's return value
10	Check <code>ferror</code> after the loop to distinguish EOF from an I/O error
11	<code>abort()</code> and <code>_exit()</code> don't flush — <code>setvbuf(stdout, NULL, _IONBF, 0)</code> while debugging
12	Errors and diagnostics to <code>stderr</code> , data to <code>stdout</code>
13	<code>read/write</code> return short — always loop, and handle <code>EINTR</code>
14	Never mix <code>FILE*</code> and raw fd I/O on the same file

Standing build command:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wformat=2 -Wshadow -Wconversion \
-Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
-D_FILE_OFFSET_BITS=64 -O1 -g -D_FORTIFY_SOURCE=3 \
-fsanitize=address,undefined prog.c -o prog
```

Practice

- Format specifier drill.** Write a program printing the same value with 15 different flag/width/precision combinations. Then deliberately mismatch types (`%d` with a `long`, `%s` with an `int`) and confirm `-Wformat=2` catches each. Run the mismatched version under UBSan.
- Break `scanf` four ways.** Reproduce all four traps from §13.2 in one program. For the infinite-loop trap, write a correct retry loop that drains the bad input.
- Build a robust reader.** Implement `read_line` and `parse_long` from §13.3 plus `parse_double` and `parse_bool`. Feed it a file with 20 pathological inputs (empty lines, whitespace-only, huge numbers, embedded NULs, a 10 KB line, no trailing newline) and verify every case reports correctly. Then rewrite using POSIX `getline` and compare.
- Buffering forensics.** Write a program that prints 5 lines to `stdout` and 5 to `stderr`, alternating. Run it: (a) to a terminal, (b) piped to `cat`, (c) redirected to a file, (d) under `stdbuf -o0`. Record the ordering in each case and explain it.
- Lost output.** Reproduce the `abort()` buffer-loss bug. Then fix it four ways: `setvbuf` unbuffered, `setvbuf` line-buffered, explicit `fflush`, and switching to `stderr`. Which would you use in production and why?
- `feof` off-by-one.** Write both loop forms over a 3-byte file and a 0-byte file. Then write a version that correctly distinguishes clean EOF from a read error — simulate the error by reading from a directory (`fopen("/tmp", "r")`).
- Portable serialization.** Write `rec_write(FILE*, const struct Rec*)` and `rec_read(FILE*, struct Rec*)` that serialize field by field in little-endian with explicit sizes. Verify the file is byte-

identical regardless of struct padding. Then write the naive `fwrite(&rec,...)` version and dump both files with `xxd` to compare.

- 8. **write_all / read_all**. Implement both with `EINTR` handling and short-transfer looping. Test `write_all` by writing 10 MB to a pipe with a slow reader on the other end, and prove a naive single `write` would have truncated.
- 9. **mmap vs read**. Write two versions of "count the newlines in a file" — one with `read` into a 64 KB buffer, one with `mmap`. Benchmark on a 1 GB file. Then repeat on a 1 KB file and explain the reversal.
- 10. **A real tool**. Write `wc`-alike: reads from files named in `argv` or from `stdin` if none, counts lines/words/bytes, writes counts to `stdout`, errors to `stderr`, returns `EXIT_FAILURE` if any file failed. Verify it composes: `cat big.txt | ./mywc` and `./mywc *.c | sort -n`.

14. Standard Library Tour

`stdio` · `stdlib` · `string` · `math` · `ctype` · `time` · `assert` · `errno` · `signal` · `setjmp` · `locale` · `stdbool` · `stddef` · `stdint` · `inttypes` · `limits` · `float` · `stdarg` · `stdalign` · `stdnoreturn` · `complex` · `tgmath` · `threads.h` · `stdatomic.h` · `uchar.h`

- Sorting/searching: `qsort`, `bsearch` (and writing correct comparators)
- Randomness: `rand` limitations, better PRNGs
- Time: `time_t`, `struct tm`, `clock`, monotonic clocks

Module 14 — The Standard Library Tour

A working map of every standard header: what's in it, what's safe, what's a trap. Real GCC 13.3 / glibc 2.39 output throughout.

14.1 The header map

Core

Header	Contents	Notes
<stdio.h>	<code>FILE*</code> , <code>printf/scanf</code> , <code>fopen</code> , <code>fread</code>	Module 13. Banned: <code>gets</code> , <code>sprintf</code> , unbounded <code>scanf("%s")</code>
<stdlib.h>	<code>malloc/free</code> , <code>exit</code> , <code>atexit</code> , <code>qsort/bsearch</code> , <code>rand</code> , <code>strtol</code> , <code>abs</code> , <code>div</code> , <code>getenv</code> , <code>system</code>	Banned: <code>atoi</code> family. ⚠ <code>system()</code> is a command-injection vector
<string.h>	<code>str*</code> , <code>mem*</code>	Module 8. Banned: <code>strcpy</code> , <code>strcat</code> , <code>strtok</code>
<ctype.h>	<code>isalpha</code> , <code>toupper</code> , ...	⚠ See §14.2 — the argument must be an <code>unsigned char</code> value
<math.h>	Everything numeric	Link with <code>-lm</code> . See §14.6
<time.h>	<code>time_t</code> , <code>struct tm</code> , <code>clock</code> , <code>strftime</code>	§14.5

Header	Contents	Notes
<errno.h>	errno, E* codes	Thread-local. §14.6
<assert.h>	assert, static_assert	§14.6

Types and limits (Module 2)

Header	Contents
<stddef.h>	size_t, ptrdiff_t, NULL, offsetof, max_align_t, C23 nullptr_t, unreachable
<stdint.h>	int32_t, uintptr_t, intmax_t, SIZE_MAX, INT64_C
<inttypes.h>	Includes stdint.h + PRIu64/SCNd32 format macros + strtoumax
<limits.h>	CHAR_BIT, INT_MAX, LONG_MIN, ...
<float.h>	DBL_EPSILON, FLT_MAX, DBL_DIG, ...
<stdbool.h>	bool, true, false — obsolete in C23 (they're keywords)
<stdalign.h>	alignas, alignof — obsolete in C23
<stdnoreturn.h>	noreturn — deprecated in C23 , use [[noreturn]]

Specialized

Header	Contents	Verdict
<stdarg.h>	va_list, va_start/va_arg/va_end/va_copy	Module 6
<signal.h>	signal, raise, sig_atomic_t	ISO C's version is nearly useless — use POSIX sigaction
<setjmp.h>	setjmp, longjmp	§14.7 — dangerous, rarely correct
<locale.h>	setlocale, localeconv	§14.6 — a hidden global that changes parsing
<complex.h>	double complex, cabs, creal	Optional since C11 (__STDC_NO_COMPLEX__). Rarely used
<tgmath.h>	Type-generic math via _Generic	Include instead of <math.h> to get sqrt(x) working for float/double/long double
<threads.h>	thr_t, mtx_t, cnd_t	Optional (__STDC_NO_THREADS__). Module 17. Not in MSVC
<stdatomic.h>	_Atomic, atomic_fetch_add, memory orders	Optional (__STDC_NO_ATOMICS__). Module 17
<uchar.h>	char16_t, char32_t, mbrotoc32	Module 8 — UTF-8 in char is usually better
<wchar.h> / <wctype.h>	Wide-character everything	Avoid; wchar_t is 2 bytes on Windows, 4 on Unix

Header	Contents	Verdict
<code><iso646.h></code>	<code>and, or, not</code> macros	Historical curiosity for terminals lacking <code>&&/\ \\</code>
<code><stdckdint.h></code>	C23 <code>ckd_add/ckd_sub/ckd_mul</code>	Checked arithmetic — Module 2
<code><stdbit.h></code>	C23 <code>stdc_count_ones,</code> <code>stdc_bit_ceil</code>	Module 4

Optional-feature detection:

```
#ifdef __STDC_NO_THREADS__ /* also: _VLA, _ATOMICS, _COMPLEX */
```

14.2 `<ctype.h>` — the argument trap

Every `ctype` function takes an `int` whose value must be representable as an `unsigned char`, or `EOF`. Anything else is undefined behavior.

```
char c = (char)0xE9; /* 'é' in Latin-1 */
isalpha(c);          /* passes -23 → UB: indexes before the lookup table */
```

```
char c = 0xE9 -> as int: -23
isalpha(c)          -> UB! index -23 into the table
isalpha((unsigned char)c) -> 0 <- CORRECT
```

These functions are typically implemented as a lookup table indexed by the argument. A negative index reads memory before the table — silently wrong values, or a crash.

The idiom, every time:

```
for (const char *p = s; *p; p++)
    putchar(toupper((unsigned char)*p));
```

```
input : "Hello, World! 42"
upper : HELLO, WORLD! 42
```

This bites specifically on **non-ASCII input** with a signed `char` (Module 2), so it passes every test with English text and breaks on the first accented character.

The full set: `isalpha isdigit isalnum isspace ispunct isupper islower isxdigit iscntrl isgraph isprint isblank (C99), toupper tolower`.

⚠ All are **locale-dependent**, and none of them work on UTF-8 multibyte sequences — `isalpha` on a continuation byte is meaningless.

14.3 `qsort` and `bsearch`

```
void qsort (void *base, size_t n, size_t size, int (*cmp)(const void*, const void*));
void *bsearch(const void *key, const void *base, size_t n, size_t size,
              int (*cmp)(const void*, const void*));
```

The comparator contract

Return **negative** if `a < b`, **zero** if equal, **positive** if `a > b`. And it must be:

- **Consistent** — same inputs, same answer, every time
- **Antisymmetric** — `cmp(a,b) > 0 ⇔ cmp(b,a) < 0`
- **Transitive** — `a < b` and `b < c` ⇒ `a < c`, and the same for equality

Violating this is undefined behavior, and glibc's introsort will happily run off the end of the array and segfault.

⚠ `return a - b;` is a real bug

```
static int cmp_bad(const void *a, const void *b)
{ return *(const int*)a - *(const int*)b; }
```

```
cmp_bad(INT_MIN, INT_MAX) = 1    <- WRONG SIGN (overflowed)
cmp_int(INT_MIN, INT_MAX) = -1   <- correct
```

And it genuinely fails to sort:

```
cmp_bad result: 0 5 2147483646 2147483647 -2147483648 -2147483647 -1000000000
1000000000
sorted? NO
cmp_ok  result: -2147483648 -2147483647 -1000000000 0 5 1000000000 2147483646
2147483647
sorted? yes
```

UBSan confirms:

```
runtime error: signed integer overflow: 2147483647 - -2147483648 cannot be
represented in type 'int'
```

The correct form — branchless, overflow-free, returns exactly $-1/0/1$:

```
static int cmp_int(const void *a, const void *b)
{ int x = *(const int*)a, y = *(const int*)b; return (x > y) - (x < y); }
```

Comparators for other types

```
/* array of strings – note the DOUBLE indirection */
static int cmp_str(const void *a, const void *b)
{ return strcmp(*(const char *const *)a, *(const char *const *)b); }

/* multi-key: sort by dept, then name */
static int cmp_emp(const void *a, const void *b)
{
    const struct Emp *x = a, *y = b;
    if (x->dept != y->dept) return (x->dept > y->dept) - (x->dept < y->dept);
    return strcmp(x->name, y->name);
}

/* doubles – must handle NaN or you break transitivity */
static int cmp_dbl(const void *a, const void *b)
{
    double x = *(const double*)a, y = *(const double*)b;
    if (isnan(x)) return isnan(y) ? 0 : 1;      /* NaNs sort last */
    if (isnan(y)) return -1;
    return (x > y) - (x < y);
}
```

```
sorted: 1 3 3 5 7 9
bsearch(7) -> index 4
bsearch(4) -> NULL (not present)
alpha bravo charlie delta
dept=1 amy / dept=1 cal / dept=2 bob / dept=2 zoe
```

The gotchas

Gotcha	Detail
Not stable	Equal elements may be reordered. Add a tiebreaker key if order matters.
No worst-case guarantee	Typically introsort ($O(n \log n)$) in glibc, but the standard promises nothing.

Gotcha	Detail
No user-data parameter	You need a global (not thread-safe) or the non-standard <code>qsort_r/qsort_s</code> . A real API design flaw (Module 6).
bsearch needs a sorted array	Sorted <i>by the same comparator</i> . Otherwise it returns garbage or NULL.
bsearch returns any match	With duplicates, you don't know which one.
Comparator called on the same element	Some implementations call <code>cmp(x, x)</code> — must return 0.
NaN breaks everything	See above.

14.4 Randomness

`rand()` limitations

```
RAND_MAX = 2147483647 (the standard only guarantees >= 32767)
```

That "only guarantees 32767" is important: on some platforms `rand()` returns at most 15 bits of entropy. Code that assumes a large range breaks silently.

Modulo bias, made visible with a small generator:

```
a generator returning 0..9 uniformly, mod 3
values 0..9 mapped by %3 -> counts: 4 3 3
0 and 1 get 4 slots, 2 gets 3 -> 33% more likely!

Same effect for rand()%n whenever (RAND_MAX+1) % n != 0:
2147483648 % 3 = 2 -> bias present
2147483648 % 4 = 0 -> no bias (power of two)
```

With a 31-bit `RAND_MAX` the bias is tiny (~1 part in 10^9) but it's real, and with a 15-bit `RAND_MAX` and a large modulus it's substantial. **The fix is rejection sampling:**

```
static int rand_below(int n)                /* uniform in [0, n) */
{
    int lim = RAND_MAX - (RAND_MAX % n);    /* largest usable multiple */
    int r;
    do { r = rand(); } while (r >= lim);
    return r % n;
}
```

Other `rand()` problems:

- **Not thread-safe** — shared hidden state (POSIX `rand_r` exists but is deprecated and poor)
- **Quality varies wildly** — glibc's is decent; some libc implementations fail basic statistical tests badly
- **Not cryptographically secure** — the state is recoverable from the output
- `srand(time(NULL))` gives the same seed for processes started in the same second

Reproducibility is a *feature*

```
srand(42): 66 40 81 41 12
srand(42): 66 40 81 41 12
```

Fixed seeds make tests, simulations, and bug reports reproducible. Never seed from the clock in a test suite.

A better PRNG

For simulation, games, hashing, and procedural generation, use a modern small PRNG. **xoshiro256**** (Blackman & Vigna, public domain) is a good default:

```
static uint64_t s[4];
static uint64_t rotl(uint64_t x, int k) { return (x << k) | (x >> (64 - k)); }

static uint64_t xoshiro(void) {
    uint64_t r = rotl(s[1] * 5, 7) * 9, t = s[1] << 17;
    s[2] ^= s[0]; s[3] ^= s[1]; s[1] ^= s[2]; s[0] ^= s[3];
    s[2] ^= t;    s[3] = rotl(s[3], 45);
    return r;
}

/* seed with splitmix64 – never seed xoshiro with all zeros */
static uint64_t sm(uint64_t *x) {
    uint64_t z = (*x += 0x9E3779B97F4A7C15ULL);
    z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9ULL;
    z = (z ^ (z >> 27)) * 0x94D049BB133111EBULL;
    return z ^ (z >> 31);
}
```

```
raw:      E01D6FAFC557F1B9 BD627EBE4406B404 2C23132B578B57DB 2E8B319D4D1F276A
unbiased bounded(6) x12: 1 6 3 5 5 5 6 3 2 6 5 3
```

Advantages: explicit state (thread-safe by construction), 2^{56} period, faster than `rand()`, passes BigCrush.

Other good choices: **PCG**, **SplitMix64** (trivially small), **Mersenne Twister** (large state, statistically fine but slow and not cryptographic).

For anything security-related, never use a PRNG

Platform	Use
Linux/BSD	<code>getrandom(2), getentropy(3)</code>
Anywhere Unix	<code>/dev/urandom</code>
BSD/macOS/glibc 2.36+	<code>arc4random_buf()</code>
Windows	<code>BCryptGenRandom</code>

Session tokens, keys, nonces, salts, password reset links — **all of these must come from the OS CSPRNG.**
Predictable-token vulnerabilities are a recurring CVE class.

14.5 Time

```
time(NULL) = 1785221286, sizeof(time_t)=8
32-bit time_t overflows 2038-01-19 -> the Y2038 problem
```

The three time concepts

```
clock()           = 0.1409 s CPU time (CLOCKS_PER_SEC=1000000)
CLOCK_MONOTONIC   = 0.1436 s elapsed <- USE THIS for durations
CLOCK_REALTIME    = wall-clock date/time; can JUMP (NTP, DST, admin)
```

Clock	Measures	Use for
<code>time()</code> / <code>CLOCK_REALTIME</code>	Wall-clock date/time	Timestamps, dates, "when did this happen"
<code>CLOCK_MONOTONIC</code>	Elapsed time since some fixed point	Durations, timeouts, benchmarks
<code>clock()</code>	CPU time consumed by this process	Profiling actual compute

⚠ **Never measure elapsed time with `time()` or `CLOCK_REALTIME`.** NTP can step the clock backwards, DST changes it, an admin can set it. You get negative durations, hung timeouts, and 1-hour-long "milliseconds." Every production timeout bug in this category traces to using the wrong clock.

```
#include <time.h>
struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);
/* ... work ... */
clock_gettime(CLOCK_MONOTONIC, &t1);
double secs = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
```

(`clock_gettime` is POSIX; C11 has `timespec_get(&ts, TIME_UTC)` but no monotonic option.)

Note in the measurement above that CPU time (0.1409 s) is slightly less than wall time (0.1436 s) — the difference is scheduling. On a multithreaded program `clock()` can exceed wall time.

`struct tm` — three different bases

```
gmtime_r -> 2026-07-28 06:48:06 UTC
NOTE: tm_year is years since 1900, tm_mon is 0-11, tm_mday is 1-31
      (three different bases in one struct – a classic bug source)
```

Field	Range
<code>tm_year</code>	years since 1900
<code>tm_mon</code>	0 –11
<code>tm_mday</code>	1 –31
<code>tm_hour</code>	0–23
<code>tm_wday</code>	0–6 (Sunday = 0)
<code>tm_yday</code>	0–365
<code>tm_isdst</code>	>0 DST, 0 no, <0 = unknown , let <code>mktime</code> figure it out

⚠ The static buffer trap

```
struct tm *a = gmtime(&t1);
struct tm *b = gmtime(&t2);      /* OVERWRITES what a points to */
```

```
a->tm_year+1900 = 2001  <- should be 1970!
b->tm_year+1900 = 2001
a == b ? 1    (same static buffer)
```

`gmtime`, `localtime`, `ctime`, and `asctime` all return a pointer to the same static buffer. Not thread-safe, and broken even single-threaded if you hold two results at once.

Use the `_r` variants: `gmtime_r`, `localtime_r`, `ctime_r`, `asctime_r` (POSIX), or `gmtime_s/localtime_s` (C11 Annex K / Windows).

Formatting and parsing

```
char buf[64];
strftime(buf, sizeof buf, "%Y-%m-%d %H:%M:%S %Z", &tm);  /* → 2026-07-28 06:48:06
```



```
GMT */
strftime(buf, sizeof buf, "%A, %d %B %Y", &tm);          /* → Tuesday, 28 July
2026 */
```

`strftime` returns 0 if the result doesn't fit — **check it**. `%A/%B` are locale-dependent.

Parsing: `strptime` (POSIX, not ISO C). `mktime` converts a `struct tm` back to `time_t` in **local time** and normalizes out-of-range fields — which makes it a handy date calculator:

```
tm.tm_mday += 45;          /* "45 days from now" */
time_t future = mktime(&tm); /* normalizes month/year automatically */
```

Other traps

- `difftime(t2, t1)` is the portable way to subtract `time_ts` — `time_t` isn't guaranteed to be an arithmetic count of seconds (it is everywhere real, but `difftime` costs nothing).
- **Leap seconds** don't exist in `time_t` (POSIX time skips them).
- **localtime reads the TZ environment variable** and may call `tzset()` — a hidden global dependency, and a performance trap in loops.
- **Y2038**: 32-bit `time_t` overflows on 2026-01-19 03:14:07 UTC + 12 years. Build with `-D_TIME_BITS=64 -D_FILE_OFFSET_BITS=64` on 32-bit systems.

14.6 assert, errno, math, locale

assert and NDEBUG

```
assert(f() == 1);
```

```
### normal build ###      assert(f()==1): f called 1 time(s)
### -DNDEBUG build ###   assert(f()==1): f called 0 time(s)
```

With `-DNDEBUG`, `assert(expr)` expands to nothing — the expression is not evaluated at all.

Rule: never put a side effect inside an `assert`.

```
assert(do_the_work() == 0);          /* the work vanishes in release! */
int rc = do_the_work(); assert(rc == 0); /* correct */
```

assert is for programmer errors (broken invariants, impossible states) — not for runtime conditions like a failed `malloc`, a bad user input, or a missing file. Those need real error handling that survives `NDEBUG`.

`_Static_assert(cond, "msg")` (C11; `static_assert` in `<assert.h>`, a keyword in C23) checks at **compile time** and costs nothing:

```
_Static_assert(sizeof(int) >= 4, "int must be at least 32 bits");
_Static_assert(sizeof table / sizeof *table == COUNT, "table out of sync");
```

Use it liberally — it's the cheapest correctness tool in C.

errno

```
fopen failed: errno=2  strerror="No such file or directory"
perror form : fopen: No such file or directory
```

Rule	Why
errno is thread-local	Safe per-thread; it's a macro expanding to a function call in practice
Clear it before the call (<code>errno = 0</code>)	Functions only <i>set</i> it; they never clear it
Only check it after a failure indication	A successful call may set <code>errno</code> to anything
strerror is not thread-safe	Use <code>strerror_r</code> (POSIX) or <code>strerror_s</code>
<code>perror(msg)</code> prints <code>msg: description</code> to <code>stderr</code>	Convenient, but no formatting

⚠ `strerror_r` has **two incompatible versions**: the XSI one returns `int` and fills your buffer; the GNU one returns `char*` and may not use your buffer at all. My demo above hit exactly this — it printed the wrong message because I used the return value incorrectly. Check which one you're getting (`_POSIX_C_SOURCE` vs `_GNU_SOURCE`).

Only functions **documented** to set `errno` do so. `strtol` does; `printf` mostly doesn't.

<math.h>

```
sqrt(-1)  = -nan, errno=33 (Numerical argument out of domain)    EDOM
log(0)    = -inf, errno=34 (Numerical result out of range)       ERANGE
math_errhandling = 3 (1=errno, 2=fp exceptions)
```

Link with `-lm` on Linux (not needed on macOS or Windows).

Error reporting is split: `math_errhandling` tells you whether the implementation uses `errno` (`MATH_ERRNO`), floating-point exception flags (`MATH_ERREXCEPT`), or both. **Check results with `isnan()/isinf()`** rather than relying on `errno` alone.

Useful pieces beyond the obvious: `fabs`, `fmod`, `remainder`, `round`/`lround`/`floor`/`ceil`/`trunc`, `fma` (fused multiply-add, single rounding), `hypot` (overflow-safe $\sqrt{x^2+y^2}$), `copysign`, `nextafter`, `fmin`/`fmax`, `signbit`, `fpclassify`, `isfinite`.

`<tgmath.h>` gives you type-generic versions via `_Generic` (Module 12) — `sqrt(x)` dispatches to `sqrtf/sqrt/sqrtl` automatically.

`<locale.h>` — a hidden global that changes parsing

```
default locale: C
strtod("3.14") = 3.14
```

Under a German locale, the decimal separator is `,`:

```
setlocale(LC_NUMERIC, "de_DE.UTF-8");
strtod("3.14", NULL);      /* → 3.0, stops at the dot! */
strtod("3,14", NULL);      /* → 3.14 */
```

This silently corrupts config-file parsing, network protocols, and CSV handling on non-English systems.

The program works perfectly in testing and produces wrong numbers for European users.

```
/* For anything machine-readable, force the C locale */
setlocale(LC_ALL, "");      /* user's locale for messages/dates... */
setlocale(LC_NUMERIC, "C"); /* ...but C for number parsing */
```

Or use `strtod_l/newlocale` (POSIX) for per-call locale control. Locale also affects `printf("%f")` output, `isalpha`, `strcoll`, and `strftime`.

A program that never calls `setlocale` runs in the "C" locale — which is why this bug only appears after someone adds internationalization.

14.7 `<setjmp.h>` — non-local jumps

```
static jmp_buf env;

int rc = setjmp(env);      /* returns 0 first time, the longjmp value later */
if (rc == 0) { deep(5); }  /* deep() calls longjmp(env, 42) */
else      { /* rc == 42 */ }
```

```
setjmp returned 0 (first pass)
  reached depth 0, jumping out...
setjmp returned 42 (came back from longjmp)
```

It's a non-local `goto`: unwind an arbitrary number of stack frames in one step.

⚠ The `volatile` requirement

```
int plain = 1;
volatile int vol = 1;
if (setjmp(e) == 0) { plain = 2; vol = 2; longjmp(e, 1); }
printf("plain=%d vol=%d\n", plain, vol);
```

after longjmp: plain=1 (INDETERMINATE) vol=2 (guaranteed 2)

Local, non-`volatile`, automatic variables modified between `setjmp` and `longjmp` have indeterminate values afterward. The compiler may have kept them in registers that `longjmp` restored. `plain` reverted to 1.

GCC's `-Wclobbered` catches some cases, but not reliably. **Mark every local you need to survive as `volatile`.**

The other dangers

- **It does not unwind.** No `free`, no `fclose`, no mutex unlock. Every resource acquired between `setjmp` and `longjmp` leaks.
- **`longjmp` into a function that has already returned is UB** — and a common bug.
- **`longjmp(env, 0)` is silently converted to 1**, so you can't use 0 as a signal value.
- **Not safe across threads.** Each thread needs its own `jmp_buf`.
- **From a signal handler you need `sigsetjmp/siglongjmp`** to save/restore the signal mask.

When it's legitimate

Fatal-error escape in interpreters and parsers, and libraries that adopted it before better options existed (libpng, libjpeg, Lua's error handling). If you use it, pair it with an arena allocator (Module 10) so cleanup is a single `arena_reset`.

Otherwise: return an error code and use the `goto` cleanup chain from Module 5. It's explicit, it releases resources, and it survives code review.

14.8 `<signal.h>` — why ISO C's version is nearly useless

```
void (*signal(int sig, void (*handler)(int)))(int);
```

ISO C guarantees only six signals (`SIGABRT`, `SIGFPE`, `SIGILL`, `SIGINT`, `SIGSEGV`, `SIGTERM`) and leaves almost everything else unspecified: whether the handler is reset after firing, whether interrupted syscalls restart, what you may safely do inside a handler.

Inside a signal handler, ISO C lets you do essentially two things:

- Assign to a `volatile sig_atomic_t`
- Call `abort`, `_Exit`, `quick_exit`, or `signal`

No `printf`, no `malloc`, no `free` — those aren't **async-signal-safe** and calling them can deadlock or corrupt the heap.

The standard pattern:

```
static volatile sig_atomic_t g_stop = 0;
static void on_sigint(int sig) { (void)sig; g_stop = 1; } /* that's ALL */

/* main loop */
while (!g_stop) { do_work(); }
```

For real work use POSIX **sigaction** — it has defined reset behavior, a signal mask during the handler, `SA_RESTART`, and `siginfo_t`. Or `signalfd/pselect` to turn signals into ordinary file-descriptor events. Full treatment in Module 24.

Module 14 rules

#	Rule
1	Always cast to <code>(unsigned char)</code> before any <code>ctype</code> function
2	Comparators: <code>(x>y)-(x<y)</code> , never <code>x-y</code>
3	<code>qsort</code> isn't stable and takes no user data; <code>bsearch</code> needs the same comparator
4	Handle NaN explicitly in float comparators
5	<code>rand() % n</code> is biased — use rejection sampling, or a real PRNG
6	Never use <code>rand()</code> for anything security-related — use the OS CSPRNG
7	<code>CLOCK_MONOTONIC</code> for durations, <code>CLOCK_REALTIME</code> only for dates
8	<code>gmtime/localtime/ctime</code> share one static buffer — use <code>_r</code> variants
9	<code>tm_year</code> is since 1900, <code>tm_mon</code> is 0-based
10	Never put side effects in <code>assert</code> ; use <code>_Static_assert</code> liberally
11	<code>errno = 0</code> before, check only after a documented failure
12	Force <code>LC_NUMERIC="C"</code> for machine-readable number parsing
13	<code>setjmp</code> locals need <code>volatile</code> ; <code>longjmp</code> leaks every resource
14	Signal handlers: set a <code>volatile sig_atomic_t</code> flag and nothing else
15	Link <code>-lm</code> for <code><math.h></code> ; check <code>isnan/isinf</code> , not just <code>errno</code>

Practice

1. **ctype audit.** Write a word-counting function using `isspace` without the cast. Feed it a UTF-8 file with accented characters and run under ASan with a hand-built lookup table so the out-of-bounds index is detectable. Then fix it.
2. **Comparator suite.** Write correct comparators for `int`, `unsigned`, `double` (NaN-safe), `char*`, a multi-key struct, and case-insensitive strings. Test each against a shuffled 10,000-element array and verify with an `is_sorted` check. Then run the `a-b` version under UBSan.
3. **Stability test.** Sort an array of `{key, original_index}` pairs by `key` alone with `qsort` and check whether original order is preserved among equals. Then make it stable by adding `original_index` as a tiebreaker.
4. **Bias measurement.** Write a generator returning 0–255 uniformly and measure the distribution of `x % 100` over 10 million samples. Plot or tabulate the counts. Then implement rejection sampling and re-measure.
5. **PRNG comparison.** Implement `rand()`, `xoshiro256**`, and `SplitMix64`. Benchmark 100 million calls each. Then run a simple statistical check (chi-squared on byte frequencies, or count runs) and compare.
6. **Time forensics.** Write a benchmark harness using `clock()`, `CLOCK_REALTIME`, and `CLOCK_MONOTONIC`. Run it while manually changing the system clock (`date -s`, in a container) and observe which measurements break.
7. **gmtime trap.** Reproduce the static-buffer aliasing bug. Then write a `format_range(time_t start, time_t end)` that needs both broken-down times simultaneously — first incorrectly, then with `gmtime_r`.
8. **Date arithmetic.** Using `mktime` normalization, write `time_t add_days(time_t t, int days)` and `int days_between(time_t a, time_t b)`. Test across a DST boundary and across a leap day.
9. **assert and NDEBUG.** Build a program with an `assert` containing a side effect. Compile with and without `-DNDEBUG` and show the behavior difference. Then convert every runtime check to proper error handling and every invariant to `_Static_assert` where possible.
10. **Locale bug.** Write a config parser reading `"threshold = 3.14"` with `strtod`. Run it under `LC_ALL=de_DE.UTF-8` (install the locale with `locale-gen`) and observe the corruption. Fix it three ways: `setlocale(LC_NUMERIC, "C")`, `strtod_l`, and a hand-written parser.

15. Error Handling Strategies

- Return codes, sentinel values, out-parameters
- `errno`, `perror`, `strerror`, thread-safety of `errno`
- `assert` vs runtime checks; `NDEBUG`
- The `goto cleanup` / single-exit pattern
- `setjmp/longjmp` (and its dangers)
- Designing an error enum + error-string API

Module 15 — Error Handling Strategies

15.1 The four conventions

```
enum { E_OK = 0, E_NOMEM = -1, E_RANGE = -2, E_INVALID = -3 };

static int parse(const char *s, int *out)
{
    if (!s || !out) return E_INVALID;
    ...
    *out = (int)v;
    return E_OK;
}
```

```
static long dbl(long v)
{ if (v > LONG_MAX/2) { errno = ERANGE; return LONG_MAX; } return v*2; }
```

```
double(100)=200 errno=0
double(LONG_MAX)=9223372036854775807 errno=34 (Numerical result out of range)
```

The caller must clear `errno` before and check after — the discipline from Module 14. **Use it only for compatibility with existing libc conventions.** For new code it's error-prone: forgetting `errno = 0` gives you a stale error, and the error is easy to lose across intervening calls.

Style 4 — `bool` + out-parameter

```
static bool divide(int a, int b, int *q)
{ if (b == 0) return false; *q = a/b; return true; }
```

Clean and readable when there are exactly **two** outcomes and the caller doesn't need to know why. Escalate to Style 1 the moment you have a second failure mode.

⚠ The out-parameter evaluation-order trap

```
printf("%d q=%d\n", divide(10, 2, &q), q);
```

```
-> 1 q=-999    <- q read BEFORE divide ran (unspecified order, Module 4)
```

Argument evaluation order is **unspecified** (Module 4 §4.7). GCC evaluates right-to-left, so `q` was read before `divide` wrote to it.

```
bool ok = divide(10, 2, &q);      /* sequence the call first */
printf("%d q=%d\n", ok, q);      /* -> 1 q=5 */
```

Rule: never read an out-parameter in the same expression as the call that fills it.

Choosing

Situation	Convention
Library with multiple failure modes	Error code + out-param
A truly impossible value exists	Sentinel (<code>NULL</code> , <code>-1</code> , <code>(size_t)-1</code>)

Situation	Convention
Two outcomes, reason irrelevant	<code>bool</code> + out-param
Matching POSIX/libc	<code>-1</code> + <code>errno</code>
Application <code>main</code> -level	Whatever the layers below use, plus a top-level report

Pick one and use it everywhere. A codebase where some functions return 0-on-success and others return 1-on-success is where bugs live. Note also the polarity clash: `return 0` means *success* in Unix exit codes and error conventions, but *false* in C truthiness. Never write `if (parse(s, &v)) { /* success */ }` against a 0-on-success API.

15.2 `errno` in depth

```
errno is THREAD-LOCAL:
thread 3: errno=103 (unchanged - thread-local!)
thread 2: errno=102 (unchanged - thread-local!)
thread 1: errno=101 (unchanged - thread-local!)
thread 0: errno=100 (unchanged - thread-local!)
```

`errno` is a macro, not a variable. On glibc:

```
#define errno (*__errno_location())
```

It expands to a function call returning a pointer to thread-local storage. Consequences:

- ☒ **It is thread-safe** — each thread has its own.
- ☒ **You cannot write `extern int errno;`** — that's pre-C89 code that will not link.
- ⚠ Taking `&errno` is fine but the address is per-thread.

The four rules

```
errno = 0; /* 1. clear before */
long v = strtol(s, &end, 10);
if (end == s) { ... } /* 2. check the primary failure indication first */
if (errno == ERANGE) { ... } /* 3. only then consult errno */
```

- Only functions documented to set `errno` do so.** A *successful* call may set it to anything — `printf` internally calls `stat`, which may fail harmlessly and leave `ENOTTY` behind.

Reporting

```
fopen failed: errno=2  strerror="No such file or directory"
perror form : fopen: No such file or directory
```

Function	Notes
<code>perror(msg)</code>	Prints <code>msg: description</code> to <code>stderr</code> . Convenient, no formatting.
<code>strerror(e)</code>	Not thread-safe — returns a pointer to a static buffer.
<code>strerror_r(e, buf, n)</code>	⚠ Two incompatible versions. XSI returns <code>int</code> and fills your buffer; GNU returns <code>char*</code> and may ignore your buffer entirely. Which you get depends on <code>_GNU_SOURCE</code> .
<code>strerror_l(e, locale)</code>	POSIX 2008, thread-safe, no ambiguity — prefer it.

I hit the `strerror_r` ambiguity live in Module 14 and printed the wrong message. **Check which variant your feature-test macros select.**

Preserving `errno` across cleanup

Cleanup code can clobber it:

```
int saved = errno;      /* save it FIRST */
close(fd);              /* may set errno */
free(buf);
errno = saved;          /* restore before returning */
return -1;
```

15.3 `assert` vs runtime checks

The decision rule

Can this happen in a correct program given valid input?

- **No** → `assert()` — it documents an invariant and is free in release.
- **Yes** → a runtime check that survives `-DNDEBUG`.

```
/* ASSERT: a caller bug if violated */
static int array_get(const int *a, size_t n, size_t i)
{
    assert(a != NULL);
    assert(i < n);
    return a[i];
}

/* RUNTIME CHECK: user input can be anything */
static int parse_age(const char *s, int *out)
```

```
{
    if (!s || !out) return -1;
    char *end; long v = strtol(s, &end, 10);
    if (end == s || *end || v < 0 || v > 150) return -1;
    *out = (int)v; return 0;
}
```

```
parse_age("30")    -> ok
parse_age("-5")    -> rejected (runtime check, survives NDEBUG)
parse_age("abc")   -> rejected
parse_age("200")   -> rejected
```

Use assert for	Use a runtime check for
Internal invariants	malloc / fopen / read failure
Unreachable branches	User input, config files, CLI arguments
Preconditions of static functions	Preconditions of public API functions
"This can't be NULL here" after you just checked	Network data, file contents
Post-conditions during development	Anything an attacker controls

What **NDEBUG** does

```
### debug build ###
af: af.c:2: main: Assertion `i<n' failed.
Aborted

### -DNDEBUG build ###
(silently gone, exit=0)
```

assert(expr) expands to nothing under **-DNDEBUG** — **the expression is not evaluated at all** (Module 14 measured **f()** being called 1 time vs 0). Never put a side effect inside one.

The debate about shipping with assertions on

Arguments for -DNDEBUG in release: performance, and a crash is worse than degraded behavior for user-facing software.

Arguments against: an assertion failure means your program is already in an undefined state; continuing is how a bug becomes a *vulnerability*. Google, and much of the security community, argue for keeping assertions enabled in production.

A middle path used by many production systems: a separate always-on check macro.



```

#define CHECK(cond) do {
    if (!(cond)) {
        fprintf(stderr, "%s:%d: CHECK failed: %s\n",
            __FILE__, __LINE__, #cond);
        abort();
    }
} while (0)

```

`assert` for cheap dev-time invariants, `CHECK` for the ones that must never be violated in production. (Note `#cond` — Module 12's stringification.)

`_Static_assert` costs nothing

```

_Static_assert(sizeof err_msgs / sizeof *err_msgs == ERR_COUNT, "err table out of
sync");
_Static_assert(sizeof(void*) == 8, "64-bit only");

```

Every invariant you can check at compile time, you should. It's the cheapest correctness tool in C.

15.4 Designing an error API

Here is a complete, working design. It's ~60 lines and it scales.

The header

```

/* ---- SINGLE SOURCE OF TRUTH (X-Macro, Module 12) ---- */
#define ERR_LIST
    X(OK,          0, "success")
    X(NOMEM,       1, "out of memory")
    X(INVAL,       2, "invalid argument")
    X(RANGE,       3, "value out of range")
    X(NOTFOUND,    4, "not found")
    X(IO,          5, "I/O error")
    X(PARSE,       6, "parse error")
    X(PERM,        7, "permission denied")
    X(AGAIN,       8, "temporarily unavailable")

#define X(name, val, msg) ERR_##name = val,
typedef enum { ERR_LIST ERR_COUNT } err_t;
#undef X

const char *err_str(err_t e);          /* stateless code → message */
const char *err_detail(void);          /* rich per-thread context */
void err_clear(void);

err_t err_set(err_t code, const char *file, int line, const char *fmt, ...)
    __attribute__((format(printf, 4, 5)));

```

```

#define ERR(code, ...) err_set((code), __FILE__, __LINE__, __VA_ARGS__)

#define TRY(expr)          do { err_t e_ = (expr); if (e_ != ERR_OK) return e_; }
while (0)
#define TRY_GOTO(expr,l) do { rc = (expr); if (rc != ERR_OK) goto l; } while (0)

```

The implementation

```

#define X(name, val, msg) [ERR_##name] = msg,
static const char *const err_msgs[] = { ERR_LIST };
#undef X
_Static_assert(sizeof err_msgs / sizeof *err_msgs == ERR_COUNT, "err table out of
sync");

const char *err_str(err_t e)
{ return (e >= 0 && e < ERR_COUNT) ? err_msgs[e] : "unknown error"; }

static _Thread_local char g_detail[256];          /* per-thread, like errno */

const char *err_detail(void) { return g_detail[0] ? g_detail : "(no detail)"; }
void          err_clear(void) { g_detail[0] = '\0'; }

err_t err_set(err_t code, const char *file, int line, const char *fmt, ...)
{
    if (code == ERR_OK) { err_clear(); return code; }
    int n = snprintf(g_detail, sizeof g_detail, "%s:%d: %s: ", file, line,
err_str(code));
    if (n < 0 || (size_t)n >= sizeof g_detail) return code;
    va_list ap; va_start(ap, fmt);
    vsnprintf(g_detail + n, sizeof g_detail - (size_t)n, fmt, ap);
    va_end(ap);
    return code;
}

```

Using it

```

static err_t parse_port(const char *s, int *out)
{
    if (!s || !out) return ERR(ERR_INVAL, "null argument");
    char *end; errno = 0;
    long v = strtol(s, &end, 10);
    if (end == s)          return ERR(ERR_PARSE, "no digits in \"%s\"", s);
    if (*end)              return ERR(ERR_PARSE, "trailing garbage \"%s\"", end);
    if (errno == ERANGE)   return ERR(ERR_RANGE, "\"%s\" overflows long", s);
    if (v < 1 || v > 65535) return ERR(ERR_RANGE, "port %ld not in 1..65535", v);
    *out = (int)v;
    return ERR_OK;
}

```

```
static err_t configure(const char *port_s, const char *backlog_s, int *port, int
*backlog)
{
    TRY(parse_port(port_s, port));          /* propagates, preserving the detail */
    TRY(parse_port(backlog_s, backlog));
    return ERR_OK;
}
```

```
configure(8080    ,128    ) -> success      port=8080 backlog=128
configure(abc     ,128    ) -> parse error  | emain.c:13: parse error: no digits
in "abc"
configure(8080    ,99999 ) -> range         | emain.c:16: range: port 99999 not in
1..65535
configure(70000   ,1      ) -> range         | emain.c:16: range: port 70000 not in
1..65535
configure(80x     ,1      ) -> parse error  | emain.c:14: parse error: trailing
garbage "x"

ASan/UBSan exit=0
```

Why each piece is there

Decision	Reason
X-Macro table	Codes, names, and messages can never drift (Module 12)
<code>_Static_assert</code> on the count	Adding a code without a message fails the build
<code>ERR_OK == 0</code>	<code>if (rc)</code> reads naturally as "if error"; matches libc conventions
Both a code and a detail string	The code is for branching, the string is for humans
<code>_Thread_local</code> detail	Thread-safe by construction, exactly like <code>errno</code>
<code>__FILE__</code> / <code>__LINE__</code> in the macro	Free provenance, no manual bookkeeping
<code>format printf,4,5</code>	Compile-time checking of every detail message (Module 6)
TRY macro	Propagation without four lines of boilerplate per call
Fixed-size detail buffer	No allocation — so it works when the error is <code>ERR_NOMEM</code>

That last point is easy to miss: **an error-reporting path must not allocate**, because the most important error to report is often out-of-memory.

Alternative designs and their trade-offs

Design	Pros	Cons
--------	------	------

Design	Pros	Cons
This one (code + thread-local detail)	Rich, no allocation, thread-safe	Detail is global-ish; easy to forget <code>err_clear</code>
Error object out-param (<code>err_t f(..., Error **e)</code>)	Explicit lifetime, chainable causes	Verbose; needs allocation or a pool
Bare integer codes	Simplest, zero overhead	No context — "parse error" without saying where
errno-style negative errno values	Interops with POSIX directly	Limited to the standard code set
Result struct (<code>struct { err_t e; T v; }</code>)	Value and error travel together	Only works for small <code>T</code> ; verbose in C

For error chaining (a low-level `EIO` becoming a high-level "config load failed"), extend the detail buffer to append rather than overwrite, or move to the error-object design with a `cause` pointer.

15.5 The `goto cleanup` chain with error propagation

Module 5 established the pattern. Here it is carrying real error information:

```
static err_t process(const char *inpath, const char *outpath, size_t bufsz, int fail_at)
{
    FILE  *in = NULL, *out = NULL;
    char  *buf = NULL;
    err_t  rc = ERR_OK;

    in = fopen(inpath, "rb");
    if (!in) { rc = ERR(ERR_IO, "cannot open input \"%s\"", inpath); goto out; }

    out = fopen(outpath, "wb");
    if (!out) { rc = ERR(ERR_IO, "cannot open output \"%s\"", outpath); goto out_in; }

    buf = malloc(bufsz);
    if (!buf) { rc = ERR(ERR_NOMEM, "buffer of %zu bytes", bufsz); goto out_out; }

    size_t n;
    while ((n = fread(buf, 1, bufsz, in)) > 0)
        if (fwrite(buf, 1, n, out) != n) { rc = ERR(ERR_IO, "short write"); goto out_buf; }
    if (ferror(in)) { rc = ERR(ERR_IO, "read error"); goto out_buf; }

out_buf: free(buf);
out_out: if (fclose(out) != 0 && rc == ERR_OK) rc = ERR(ERR_IO, "fclose failed");
out_in:  fclose(in);
}
```

```
out:    return rc;
}
```

```
happy path          -> success
input open fails    -> I/O error   cleanup.c:14: I/O error: cannot open input
"missing"
output open fails    -> I/O error   cleanup.c:17: I/O error: cannot open output
"/no/dir/x"
mid-work failure     -> parse error cleanup.c:22: parse error: simulated mid-work
failure
```

```
ASan/LSan exit=0 (no leaks on ANY error path)
```

The three refinements over Module 5

1. **rc is set before the goto**, not at the label. Each failure site records its own specific cause.
2. **fclose errors are checked** — and only overwrite **rc** if nothing has failed yet. **Don't clobber the first error with a cleanup error**; the original cause is the useful one.
3. **Every label unwinds exactly one more resource** than the one above it. Name labels after what they release.

Testing error paths

Notice the **fail_at** parameter. **Fault injection is a real technique**, not a toy:

- Add a test-only parameter or a global that forces failure at a chosen step
- **LD_PRELOAD** a **malloc** that fails after N calls
- **ulimit -v** to force real allocation failure
- Point paths at **/dev/full**, unreadable files, or full filesystems

Then run the whole matrix under LSan. The happy path is easy; the error paths are where the leaks are, and they're the paths that never get exercised in normal testing.

15.6 setjmp / longjmp

Covered in Module 14 §14.7. The summary for error handling purposes:

```
after longjmp: plain=1 (INDETERMINATE) vol=2 (guaranteed 2)
```

Danger	Consequence
Non- volatile locals are indeterminate after longjmp	Silent wrong values; -Wclobbered catches only some

Danger	Consequence
No unwinding	Every <code>malloc</code> , <code>fopen</code> , and locked mutex between <code>setjmp</code> and <code>longjmp</code> leaks
<code>longjmp</code> to a returned function	Undefined behavior, usually a crash
<code>longjmp(env, 0)</code> becomes 1	You can't use 0 as a signal value
Not thread-safe across <code>jmp_buf</code> sharing	Each thread needs its own
Signal handlers need <code>sigsetjmp/siglongjmp</code>	Otherwise the signal mask isn't restored

When it's defensible: you have an **arena allocator** (Module 10), so all cleanup is one `arena_reset`, and the error is genuinely fatal to the current operation. That's how Lua and some parsers use it.

Otherwise: error codes plus the `goto` chain. It's explicit, it releases resources, and it survives code review.

15.7 Putting it together — a checklist

At the API boundary

```
/**
 * Load configuration from `path`.
 *
 * @param path  Config file path. Must not be NULL.
 * @param out   On success, receives a newly allocated Config.
 *              Caller must free with config_free(). Untouched on failure.
 * @return ERR_OK on success.
 *         ERR_INVALID if path or out is NULL.
 *         ERR_IO      if the file cannot be read.
 *         ERR_PARSE   if the contents are malformed.
 *         ERR_NOMEM   on allocation failure.
 *         On error, err_detail() describes the specific problem.
 */
err_t config_load(const char *path, Config **out);
```

Document every error code a function can return. An undocumented failure mode is a failure mode nobody handles.

The rules

1. **One convention per codebase.** Enforce it in review.
2. **Never ignore a return value.** Mark functions `__attribute__((warn_unused_result))` (or C23 `[[nodiscard]]`) where ignoring is always a bug.
3. **Fail fast at the boundary, propagate inside.** Validate untrusted input once, at the edge; internal functions can then `assert` their preconditions.

4. **Leave out-parameters untouched on failure.** The caller shouldn't have to guess whether a partial result was written.
 5. **Don't lose the original error.** Cleanup failures don't overwrite the real cause.
 6. **The error path must not allocate.**
 7. **Errors and diagnostics go to `stderr`** (Module 13); the process exits non-zero.
 8. **Test the error paths.** Fault injection + LSan on every failure branch.
 9. **`assert` for bugs, checks for conditions.** Ask: *could this happen with valid input?*
 10. **Never `exit()` from a library.** Return the error and let the application decide.
-

Module 15 rules

#	Rule
1	Error code return + out-param is the default for library code
2	Sentinels only when a value is genuinely impossible
3	Never read an out-param in the same expression as the call that fills it
4	<code>errno</code> is thread-local and is a macro — never <code>extern int errno</code>
5	<code>errno = 0</code> before; check the primary failure indication first
6	Save/restore <code>errno</code> around cleanup
7	<code>assert</code> for "impossible with valid input"; runtime checks for everything else
8	No side effects in <code>assert</code> ; <code>_Static_assert</code> everything you can
9	X-Macro your error table + <code>_Static_assert</code> the count
10	Carry both a machine code and a human detail string
11	The error path must not allocate
12	Cleanup errors must not overwrite the original cause
13	Document every error code each function can return
14	Test error paths with fault injection under LSan
15	Libraries return errors; only applications call <code>exit</code>

Practice

1. **Convention comparison.** Implement the same `parse_ipv4(const char *, uint32_t *)` four times — sentinel, `bool`+out, error code, and `errno` style. Write a caller for each. Which reads best? Which catches the most bugs?
2. **Evaluation-order bug.** Reproduce the `printf("%d %d", f(&x), x)` trap. Then find one in real code by grepping for out-parameters passed inside a larger expression.
3. **`errno` discipline.** Write a function that calls three libc functions in sequence and correctly reports which one failed and why — including saving `errno` across cleanup. Test it by making each of the three

fail in turn.

- 4. **strerror_r archaeology.** Write a program using `strerror_r` under `-D_GNU_SOURCE` and under `-D_POSIX_C_SOURCE=200809L`. Observe the two different signatures and the two different behaviors. Then write a wrapper that works correctly under both.
- 5. **Build the error library.** Implement `err.h/err.c` from §15.4 in full. Add: `err_code()` returning the last code, error *chaining* (append a cause rather than overwrite), and an `err_print()` that writes the full chain to `stderr`. Verify with ASan under 4 concurrent threads that the thread-local storage is correct.
- 6. **TRY macro variants.** Implement `TRY`, `TRY_GOTO`, and `TRY_CLEANUP(expr, label)`. Then write a 6-step function using them and verify with LSan that every one of the 6 failure points leaks nothing.
- 7. **Fault injection harness.** Add a `#ifdef FAULT_INJECT` global counter to an allocating function so the Nth allocation fails. Write a loop that runs the function with `N = 1..20` and asserts LSan reports zero leaks each time. This is how real projects test OOM paths.
- 8. **assert audit.** Take any program you've written in this course. For every `assert`, decide whether it should be an assert or a runtime check using the §15.3 rule. Convert the ones that are wrong. Then build with `-DNDEBUG` and confirm nothing that matters disappeared.
- 9. **Error-path leak hunt.** Write a function acquiring 5 resources with 5 possible failure points. Deliberately point one `goto` at the wrong label. Confirm LSan finds it. Then write a small script that runs all 6 paths automatically.
- 10. **API documentation.** Take the `config_load` doc comment from §15.7 and write the full implementation matching it exactly — including "out is untouched on failure." Then write a test that verifies each documented error code is actually returned in the documented situation.

16. Undefined, Unspecified, Implementation-Defined Behavior

- The distinction between the three
- Common UB list: signed overflow, OOB access, null deref, uninitialized reads, strict aliasing violations, data races, invalid free, shifting by = width, modifying string literals, misaligned access
- How optimizers exploit UB (why "it worked on my machine" fails)

Module 16 — Undefined, Unspecified, and Implementation-Defined Behavior

This module changes how you read C. Everything you've met across Modules 2–15 comes together into one framework. Real GCC 13.3 output throughout.

16.1 The three categories (four, actually)

The standard defines four levels of "the language doesn't fully pin this down." They are **not** interchangeable, and the difference matters enormously.

Category	Definition	Documented?	Consistent?	Portable?
----------	------------	-------------	-------------	-----------

Category	Definition	Documented?	Consistent?	Portable?
Implementation-defined	Multiple behaviors allowed; the implementation must choose one and document it	☑ yes	☑ yes	⚠ per-platform
Unspecified	Multiple behaviors allowed; the implementation need not choose consistently or document	✗ no	✗ no	✗ no
Undefined	No requirements whatsoever. The standard imposes nothing.	—	—	✗ the program is invalid
Locale-specific	Behavior depends on the current locale	☑	☑	⚠ per-locale

Implementation-defined — measured

```
sizeof(int)      = 4
char is signed?  = yes
-7 >> 1          = -4   (arithmetic vs logical shift)
(int)3000000000u = -1294967296 (out-of-range conversion)
&main prints as  = 0x56530819f169 (pointer→integer representation)
```

Your compiler's manual documents each of these. Your program is **valid and well-defined** — it just gives a different answer on a different platform. You can write portable code by not depending on the choice, or by testing which choice you have (`#if CHAR_MIN < 0`).

Other examples: number of bits in a byte (`CHAR_BIT`), whether bitfields straddle storage units, the underlying type of an enum (pre-C23), whether `>>` on a negative value is arithmetic.

Unspecified — measured

```
int i = 0;
printf("%d %d\n", i++, i++);
```

```
argument eval order: 1 0
^ which i++ ran first? unspecified. GCC does right-to-left.
```

The compiler picks, need not tell you, and **may pick differently for two calls in the same program**. Your program is still valid — you just can't predict which of the allowed outcomes you get.

(Note: the example above is actually *undefined*, not merely unspecified, because `i` is modified twice — Module 4 §4.7. The *order* is unspecified; the *double modification* is UB.)

Other examples: the order of evaluation in `f() + g()`, the values of padding bytes, whether identical string literals share storage, the order struct members are initialized in.

Undefined — the important one

Undefined behavior: behavior for which this document imposes no requirements.

A note in the standard adds: *"Possible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message)."*

The crucial reframing: UB is not "the result is unpredictable." It is **"the standard does not define what this program means."** A program containing UB on an executed path has no defined behavior *at all* — not just at that point, but for the entire execution.

That's why the optimizer is allowed to do what §16.3 shows.

16.2 The undefined behavior catalogue

The C standard's Annex J.2 lists over 200 cases. Here are the ones that actually happen, organized by the module where you met them.

Arithmetic

UB	Module	Note
Signed integer overflow	2	Unsigned wraps and is <i>defined</i> ; signed does not
<code>INT_MIN / -1, INT_MIN % -1</code>	2	Also raises SIGFPE on x86
Division or modulo by zero	2	Integer only; float division by zero gives $\pm\text{Inf}$
<code>x << n</code> or <code>x >> n</code> with <code>n >= width(x)</code> or <code>n < 0</code>	4	x86 masks the count to 5 bits, so you get <code>x << 0</code>
Left-shifting a negative value	4	Defined in C23
Float→int conversion when out of range	2	<code>(int)1e20</code>

Memory

UB	Module	Note
Out-of-bounds array access	7	Read or write
Null pointer dereference	9	Not "a segfault" — see §16.3
Reading an uninitialized automatic variable	3	Statics are guaranteed zeroed
Use after free, double free , freeing a non- <code>malloc</code> pointer	10	

UB	Module	Note
Dangling pointer to a dead local	3, 9	
Computing <code>arr - 1</code> or <code>arr + n + 1</code>	9	Even without dereferencing
Comparing <code></></code> between unrelated objects	9	Unspecified, technically
Misaligned access	9	Faults on some ARM/SPARC
Modifying a string literal	8, 10	<code>char *s = "hi"; s[0]='H';</code>
Casting away <code>const</code> and writing	2, 9	
Overlapping <code>memcpy</code> (<code>restrict</code> violation)	8	

Types

UB	Module	Note
Strict aliasing violation	2	Accessing an object through an incompatible type
Calling a function through a mismatched function-pointer type	6	
<code>va_arg</code> with the wrong type	6	Including <code>char</code> or <code>float</code>
Reading a union member with a trap representation	11	Plain union punning <i>is</i> legal in C

Sequencing and control flow

UB	Module	Note
Two modifications between sequence points (<code>i = i++</code>)	4	
Falling off the end of a non- <code>void</code> function <i>and using the value</i>	5	
<code>longjmp</code> to a function that already returned	14	
Non- <code>volatile</code> locals after <code>longjmp</code>	14	
Infinite loop with no side effects	16	See §16.3

Library and concurrency

UB	Module	Note
Wrong <code>printf/scanf</code> format specifier	13	
<code>ctype</code> function with a negative argument	14	
<code>str*</code> on a non-NUL-terminated buffer	8	
Inconsistent <code>qsort</code> comparator	14	glibc can crash

UB	Module	Note
Data race (unsynchronized concurrent access, at least one write)	17	
Non-async-signal-safe function in a signal handler	14, 24	

The ones that surprise people

- **NULL + 0 and NULL - NULL** are UB in C (defined in C++; C23 relaxed some).
- Declaring `main` as `void main()`.
- Using an identifier from the **reserved namespace** (leading underscore at file scope, `str*`, `is*`, `E[0-9A-Z]*`, `__anything`).
- Modifying an object between the two evaluations of a `_Generic` — no wait, `_Generic` doesn't evaluate. But `sizeof` on a VLA *does* (Module 7).
- Recursive `main`? Legal in C, UB in C++.

16.3 How optimizers exploit UB ⚠

The compiler's reasoning is always the same:

"This program contains an operation that would be undefined if condition X held. A correct program never invokes undefined behavior. Therefore I may assume X is false, everywhere, and optimize accordingly."

This is not compiler writers being hostile. It's the only way to make `a[i]` fast without a bounds check, `a + b` fast without an overflow check, and pointer arithmetic fast without validity checks.

Case 1 — Null check deleted after a dereference

This is **CVE-2009-1897**, a real Linux kernel privilege-escalation bug.

```
int use_after_deref(struct S *p)
{
    int v = p->x;                /* deref: compiler now ASSUMES p != NULL */
    if (p == NULL) {             /* ...so this branch is unreachable */
        log_msg("null pointer!");
        return -1;
    }
    return v;
}
```

At `-00` the check is there:

```
use_after_deref:
    mov     rax, QWORD PTR -24[rbp]
    mov     eax, DWORD PTR [rax]    ; p->x
    mov     DWORD PTR -4[rbp], eax
    cmp     QWORD PTR -24[rbp], 0    ; the null check
```

```

jne    .L2
lea    rax, .LC0[rip]
call   log_msg@PLT          ; the error handler
mov    eax, -1

```

At **-02** it is gone:

```

use_after_deref:
    endbr64
    mov    eax, DWORD PTR [rdi]
    ret

```

The entire null check and error path vanished. The function is now a bare load.

In the kernel case, the dereference was in a macro, page 0 was mappable via `mmap`, and an attacker could arrange for the "impossible" NULL to point at controlled memory. The check that would have caught it had been optimized away.

Lesson: check *before* you dereference, not after. And a null check after a dereference isn't just redundant — it's *deleted*.

Case 2 — A non-terminating loop turned into **return**

C11 §6.8.5p6: "An iteration statement whose controlling expression is not a constant expression, that performs no input/output operations, does not access volatile objects, and performs no synchronization or atomic operations in its body... may be assumed by the implementation to terminate."

```

static int find_never(int target)
{
    int i = 0;
    while (i != target) i += 2;    /* never terminates for odd target */
    return i;
}
int caller(int t) { return find_never(t); }

```

At **-02**:

```

caller:
    endbr64
    mov    eax, edi
    ret

```

caller(3) returns 3 immediately. The loop — which mathematically runs forever — was deleted entirely. The compiler reasoned: "this loop may be assumed to terminate; if it terminates, `i == target`; so return `target`."

Add a side effect to the body and the loop is preserved:

```
caller2:
    ...
    call    side@PLT    ; the loop is real now
```

Case 3 — Signed overflow (Module 2, recap)

```
int check(int x) { if (x + 1 > x) return 1; return 0; }
```

```
check:
    mov     eax, 1      ; the entire comparison deleted
    ret
```

Case 4 — Strict aliasing (Module 2, recap)

Same source, three builds, three different answers:

```
-O0                → 0
-O2 (strict aliasing on) → 1
-O2 -fno-strict-aliasing → 0
```

Case 5 — Dead-store elimination of a secret wipe (Module 10, recap)

```
handle_password:
    endbr64
    ret          ; strcpy AND memset both deleted
```

"UB time travel"

Because UB makes the *whole execution* undefined, a compiler may eliminate code that appears **before** the UB — including observable side effects like I/O.

The canonical demonstration is Clang's handling of an uninitialized function pointer: if the only value ever assigned to a **static** function pointer is **evil**, and calling it while NULL would be UB, Clang may emit a direct call to **evil** — running code that no reachable path ever assigns.

Honest reporting: GCC 13 did *not* perform this transformation in my test, nor did it delete code before a divide-by-zero or an out-of-range array access:

```
call:
    jmp    [QWORD PTR fp[rip]]    ; GCC kept the indirect call
```

Compilers differ in how aggressively they exploit UB, and they get more aggressive over time. "GCC 13 doesn't do this" is not a guarantee about GCC 14, or Clang, or ICC, or the compiler your code ships under in three years. The standard permits it; that's the only thing that matters.

Why "it worked on my machine" fails

```
Your build:    -O0, glibc 2.39, x86-64, this GCC, small inputs
Their build:   -O3 + LTO, musl, ARM64, Clang 19, adversarial inputs
```

Every one of those axes can change whether a UB program behaves as you intended. The specific ways it breaks:

- 1. **Optimization level.** Nearly every demo in this module behaves differently at `-O0` and `-O2`.
- 2. **Compiler and version.** Optimizers get smarter. Code that "worked" for a decade breaks on upgrade.
- 3. **Architecture.** Misalignment faults on ARM, is slow on x86. `char` signedness flips. Unaligned loads that x86 tolerates trap elsewhere.
- 4. **Inlining.** UB inside a function may only be exploitable once the compiler sees the call site — so it appears when an unrelated caller changes.
- 5. **LTO.** Whole-program analysis exposes UB across translation-unit boundaries.
- 6. **Timing.** Data races and uninitialized reads are load-dependent.
- 7. **Input.** The UB path may only be reachable with input an attacker supplies.

"It works" is not evidence of correctness in C. It's evidence that this compilation, on this machine, with these inputs, produced acceptable output.

16.4 The tooling matrix

Measured on this box, one program with 12 distinct UB cases, sanitizers at `-O0`:

#	UB	plain run	UBSan	ASan
1	signed overflow	silent	☒	—
2	stack OOB write	crash	☒	☒
3	heap OOB write	silent	—	☒
4	null deref	crash	☒	☒
5	uninitialized read	silent	—	—
6	strict aliasing	silent	—	—
7	double free	silent	—	☒

#	UB	plain run	UBSan	ASan
8	shift ≥ width	silent	☒	—
9	modify string literal	crash	—	☒
10	misaligned access	silent	☒	—
11	use after free	silent	—	☒
12	division by zero	silent	☒	—

Three things to take from this table:

- 1. **8 of 12 are completely silent without tooling.** They don't crash. They produce wrong answers, or right answers today and wrong answers after a recompile.
- 2. **UBSan and ASan are complementary, not redundant.** UBSan covers arithmetic, alignment, and type UB; ASan covers memory UB. You need both.
- 3. **Two are caught by neither.** Uninitialized reads need **MemorySanitizer** (Clang only) or **Valgrind**. Strict aliasing violations are caught by **no runtime tool at all** — only `-Wstrict-aliasing` at compile time, and it misses plenty.

⚠ Optimization interacts with instrumentation

At `-O1`, ASan **missed** the heap OOB and double free — the optimizer had deleted the dead code before the sanitizer could instrument it. At `-O0` both were caught.

Run sanitizers at `-O0` or `-O1`. At higher levels the optimizer may remove the very code you're testing. (Conversely, run *warnings* at `-O2` — `-Wuninitialized` needs dataflow analysis.)

The complete toolchain

Compile-time:

```
-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion
-Wcast-qual -Wcast-align=strict -Wpointer-arith -Wwrite-strings
-Wstrict-aliasing=2 -Wnull-dereference -Warray-bounds=2
-Wstringop-overflow=4 -Wformat=2 -Wsequence-point -Walloca
-Werror=vla -Werror=return-type -Werror=implicit-function-declaration
```

Runtime:

Tool	Catches	Cost
UBSan (<code>-fsanitize=undefined</code>)	Arithmetic, shifts, alignment, null, bounds, enum/bool range	~20%
ASan (<code>-fsanitize=address</code>)	Heap/stack/global overflow, UAF, double free, leaks	~2×
MSan (Clang, <code>-fsanitize=memory</code>)	Uninitialized reads	~3×

Tool	Catches	Cost
TSan (<code>-fsanitize=thread</code>)	Data races	~5–15×
Valgrind memcheck	Memory + uninit, no recompile needed	~20–50×
<code>-D_FORTIFY_SOURCE=3</code>	Buffer overflows in libc calls	~free

⚠ **ASan and TSan/MSan cannot be combined.** Run separate builds.

Static analysis: `clang-tidy`, `cppcheck`, `scan-build`, Coverity, PVS-Studio. **Formal:** Frama-C, CBMC.

Fuzzing: libFuzzer, AFL++ — the most effective way to *reach* UB paths that tests never exercise. Fuzzing + ASan + UBSan together is the industry standard for finding memory-safety bugs.

The practical CI setup

```
# Build 1: maximum warnings, treated as errors
gcc -std=c17 -O2 $WARNINGS -Werror -c *.c

# Build 2: ASan + UBSan + LSan
gcc -std=c17 -O1 -g -fsanitize=address,undefined -fno-omit-frame-pointer ...
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1 \
UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1 ./tests

# Build 3: TSan (if threaded)
gcc -std=c17 -O1 -g -fsanitize=thread ... && ./tests

# Build 4: Valgrind (catches what ASan doesn't)
valgrind --error-exitcode=1 --leak-check=full --track-origins=yes ./tests

# Build 5: a second compiler
clang -std=c17 -O2 $WARNINGS -Werror -c *.c
```

Building with two compilers is genuinely valuable — they exploit different UB and warn about different things.

16.5 How to think about this

The mental shift

Before: "UB means the result is unpredictable, so I'll check what my compiler does."

After: "UB means my program is not a valid C program. The compiler is under no obligation to compile it into anything resembling what I wrote."

There is no "my compiler's UB." There is only *this compilation of an invalid program*.

Practices that follow

- 1. **Never reason from observed behavior.** "I tested it and it wraps" is not a fact about your program; it's a fact about one build.
- 2. **Never `-fwrapv`/`-fno-strict-aliasing` as a fix.** They're legitimate escape hatches for legacy code you can't audit, but they hide bugs and cost performance. Fix the code.
- 3. **Learn the list.** The catalogue in §16.2 is maybe 30 items you'll actually meet. That's a weekend of study for a career of avoided bugs.
- 4. **Assume every warning is a real bug** until you've proven otherwise.
- 5. **Sanitizers are not optional.** They are the only reason C can be written safely at all in 2026.
- 6. **Fuzz.** Sanitizers only catch UB on paths you execute. Fuzzing finds the paths.
- 7. **When you can't avoid it, isolate it.** If you must do something the standard doesn't bless (`container_of`, union punning across languages, `dlsym` casts), put it in one documented, commented function.

The counterargument, fairly stated

Some experienced engineers — notably in the Linux kernel and embedded worlds — argue that aggressive UB exploitation was a mistake: that compilers should have treated UB as "do the obvious thing on this hardware" rather than "assume it never happens," and that the performance gains rarely justify the security cost.

There's real substance to this. Signed-overflow exploitation in particular buys little on modern hardware and has caused genuine CVEs. That's why `-fwrapv` and `-fno-strict-aliasing` exist and why the kernel uses them.

But the standard says what it says, and your compiler implements the standard. **Whatever you think the rules *should* be, the rules you must program against are the ones in §16.1.**

Module 16 rules

#	Rule
1	Implementation-defined = documented and consistent; unspecified = neither; undefined = your program is invalid
2	UB makes the <i>entire execution</i> undefined, not just one operation
3	The compiler assumes UB never happens and deletes code accordingly
4	Check before you dereference — a null check after a deref is <i>deleted</i>
5	A loop with no side effects may be assumed to terminate
6	8 of 12 common UBs are completely silent without tooling
7	UBSan and ASan are complementary — run both
8	Uninit reads need MSan or Valgrind; strict aliasing has no runtime tool
9	Run sanitizers at <code>-O0</code> / <code>-O1</code> ; run warnings at <code>-O2</code>
10	Build with two compilers; fuzz with sanitizers on

#	Rule
11	<code>-fwrapv/-fno-strict-aliasing</code> are escape hatches, not fixes
12	"It worked on my machine" is not evidence of correctness

Practice

1. **Category sorting.** Classify each as UB, unspecified, implementation-defined, or well-defined — then verify with the standard or a compiler: `i = i++; · printf("%d %d", f(), g()); · -5 >> 1 · (char)200 · 1u - 2u · 1 - 2` (as `int`, at `INT_MIN`) · `int a[5]; &a[5] · &a[6] · sizeof(void) · free(p); free(p); · struct S s1, s2; memcmp(&s1,&s2,sizeof s1)`
2. **Reproduce the null-check deletion.** Build the §16.3 example, dump assembly at `-O0`, `-O1`, `-O2`, `-Os`, `-O3`. Find the optimization level where the check disappears. Then reorder the check before the dereference and confirm it survives.
3. **The vanishing loop.** Build `find_never` and call it with an odd argument at `-O0` (hangs) and `-O2` (returns immediately). Then add `volatile` to `i` and re-check. Explain the difference using C11 6.8.5p6.
4. **Build your own tool matrix.** Take the 12-case program from §16.4 and add: `alloca` overflow, `qsort` with an inconsistent comparator, `ctype` with a negative char, a `printf` format mismatch, and an overlapping `memcpy`. Run under plain, UBSan, ASan, MSan (if you have Clang), and Valgrind. Publish the table.
5. **The -O1 blind spot.** Reproduce the finding that ASan missed the heap OOB at `-O1` but caught it at `-O0`. Explain why, and decide what optimization level your CI should use.
6. **Two-compiler diff.** Take any nontrivial program you've written in this course. Build with `gcc -Wall -Wextra -Werror` and `clang -Wall -Wextra -Weverything`. Collect the warnings each finds that the other doesn't.
7. **Fuzz something.** Write a small parser (the `parse_port` from Module 15 will do). Wrap it in a libFuzzer harness, build with `-fsanitize=fuzzer,address,undefined`, and run for 60 seconds. Fix whatever it finds.
8. **UB archaeology.** Find a real CVE caused by UB — CVE-2009-1897 (null check deletion), CVE-2012-2100 (signed overflow), or the 2014 "goto fail" bug. Read the patch. Explain in your own words which UB was involved and which tool would have caught it.
9. **Audit your own code.** Take the largest program you've written so far. Run it under every tool in §16.4. Fix everything. Then add the full warning set to your Makefile with `-Werror` and fix that too.
10. **The counterargument, seriously.** Read Linus Torvalds' and John Regehr's positions on UB exploitation. Write a paragraph arguing each side. Then decide what *your* team's policy should be: `-fwrapv` or not? `-fno-strict-aliasing` or not? Be able to defend it.

17. Concurrency in Standard C

- C11 `threads.h`, `mtx_t`, `cnd_t`

- `stdatomic.h`, atomic types, `memory_order_*`
- The C memory model, data races, happens-before
- `volatile` ? thread safety (very common misunderstanding)

Module 17 — Concurrency in Standard C

The final module of the core language section. Real GCC 13.3 / glibc 2.39 output throughout, including TSan verification. §17.5 (`volatile` is not thread safety) is the most-misunderstood topic in C.

17.1 <threads.h> — C11 threads

```
#include <threads.h>

static int worker(void *arg) { ... return exit_code; }

thrd_t th;
thrd_create(&th, worker, &task);
int result;
thrd_join(th, &result);
```

```
thread 0 finished, result=500000
joined thread 0, returned 0
...
```

The API surface

Category	Functions
Threads	<code>thrd_create</code> , <code>thrd_join</code> , <code>thrd_detach</code> , <code>thrd_exit</code> , <code>thrd_sleep</code> , <code>thrd_yield</code> , <code>thrd_current</code> , <code>thrd_equal</code>
Mutexes	<code>mtx_init</code> (<code>mtx_plain</code> <code>mtx_recursive</code> <code>mtx_timed</code>), <code>mtx_lock</code> , <code>mtx_trylock</code> , <code>mtx_timedlock</code> , <code>mtx_unlock</code> , <code>mtx_destroy</code>
Condition variables	<code>cnd_init</code> , <code>cnd_wait</code> , <code>cnd_timedwait</code> , <code>cnd_signal</code> , <code>cnd_broadcast</code> , <code>cnd_destroy</code>
One-time init	<code>once_flag</code> , <code>call_once</code>
Thread-local with destructor	<code>tss_t</code> , <code>tss_create</code> , <code>tss_get</code> , <code>tss_set</code> , <code>tss_delete</code>
Return codes	<code>thrd_success</code> , <code>thrd_error</code> , <code>thrd_busy</code> , <code>thrd_nomem</code> , <code>thrd_timedout</code>

Plus `_Thread_local` / `thread_local` storage duration (Module 3).

⚠ The portability reality

`<threads.h>` is optional (`__STDC_NO_THREADS__`) and **MSVC has never shipped it**. glibc only added it in 2.28 (2018). Most real C code uses **POSIX pthreads** on Unix and Win32 threads on Windows, often behind a thin abstraction layer.

Also note: on Linux, `<threads.h>` is a thin wrapper over pthreads, and **ThreadSanitizer's interceptors don't handle it reliably** — my `thrd_create` test crashed TSan outright, while the identical pthread version worked perfectly. **For TSan-verified code, use pthreads directly.**

The concepts below are identical either way.

The mutex + condition variable pattern

```
static int    buf[CAP], head = 0, tail = 0, count = 0;
static bool   done = false;
static mtx_t  m;
static cnd_t   not_full, not_empty;

/* producer */
mtx_lock(&m);
while (count == CAP) cnd_wait(&not_full, &m);    /* WHILE, not if */
buf[tail] = v; tail = (tail+1) % CAP; count++;
cnd_signal(&not_empty);
mtx_unlock(&m);

/* consumer */
mtx_lock(&m);
while (count == 0 && !done) cnd_wait(&not_empty, &m);
if (count == 0 && done) { mtx_unlock(&m); break; }
int v = buf[head]; head = (head+1) % CAP; count--;
cnd_signal(&not_full);
mtx_unlock(&m);
```

```
produced  1 (count=1)
  consumer 1 got  1 (count=0)
produced  2 (count=1)
produced  3 (count=2)
  consumer 2 got  2 (count=3)
...
```

The four rules of condition variables

1. **Always wait in a while loop, never an if.** `cnd_wait` may return **spuriously** (POSIX explicitly allows it), and even without spurious wakeups another thread may consume the item between the signal and your reacquisition of the mutex.
2. **`cnd_wait` atomically releases the mutex and sleeps**, then reacquires it before returning. You must hold the mutex when calling it.

3. **The predicate is protected by the mutex.** The condition variable carries no state — it's just a wakeup mechanism.
4. **cond_signal wakes one waiter; cond_broadcast wakes all.** Use **broadcast** when waiters are waiting on *different* predicates, or on shutdown.

Also: **mtx_destroy** on a locked mutex is UB, recursive locking of an **mtx_plain** is UB, and unlocking a mutex you don't own is UB.

17.2 Data races

A data race occurs when two threads access the same memory location concurrently, at least one access is a write, and the accesses are not synchronized.

A data race is undefined behavior. Not "you get a torn value" — undefined, with everything Module 16 implies.

```
static int counter = 0;
void *worker(void *a) { for (int i=0;i<100000;i++) counter++; return NULL; }
```

counter++ is three operations — load, add, store. Two threads interleaving lose updates.

TSan finds it immediately:

```
WARNING: ThreadSanitizer: data race (pid=698)
  Write of size 4 at 0x55c43103f014 by thread T2:
    Previous write of size 4 at 0x55c43103f014 by thread T3:
    Location is global 'counter' of size 4 at 0x55c43103f014
SUMMARY: ThreadSanitizer: data race racep.c:4 in worker
```

TSan is the only reliable way to find these. Data races are timing-dependent — they pass a thousand test runs and fail in production under load.

```
gcc -fsanitize=thread -O1 -g prog.c -o prog -lpthread
```

Cost: ~5–15× slowdown and ~5–10× memory. **Cannot be combined with ASan** — run separate builds.

17.3 **volatile** is NOT thread safety ⚠

This is the single most common concurrency misconception in C, and I can prove it three ways.

Experiment: 8 threads × 500,000 increments, three variables

```
static int      plain = 0;
static volatile int vol  = 0;
static _Atomic int atom  = 0;
```

```
--- run 1 ---
expected          4000000
plain int++        4000000    lost 0
volatile int++     3358953    lost 641047    ← volatile LOST 641,047 updates
atomic_fetch_add   4000000    lost 0

--- run 2 ---
volatile int++     2673045    lost 1326955    ← lost a third of them

--- run 3 ---
volatile int++     4000000    lost 0          ← and sometimes it's fine!
```

Why **volatile** performed *worse* than plain **int** — the assembly

```
w_plain:
    add     DWORD PTR plain[rip], 500000    ← the optimizer collapsed the
whole loop
    ret

w_vol:
    mov     eax, DWORD PTR vol[rip]         ← load
    add     eax, 1                          ← modify          3 separate steps,
    mov     DWORD PTR vol[rip], eax         ← store            500,000 times
    sub     edx, 1
    jne     .L4

w_atom:
    lock add DWORD PTR atom[rip], 1         ← the LOCK prefix. THIS is
atomicity.
```

Three lessons in one dump:

1. **plain "won" by accident.** The optimizer turned 500,000 increments into one `add [plain], 500000`. That's still a non-atomic read-modify-write and still a data race — it just has a tiny window, so it rarely loses. **A correct answer is not evidence of correctness.**
2. **volatile made the race worse** by forcing a real load-modify-store every iteration — a wide window, hit constantly. **volatile** prevented an optimization that was accidentally hiding the bug.
3. **Only _Atomic emits lock.** That prefix is what makes the read-modify-write indivisible on x86.

What **volatile** actually guarantees

volatile gives you	volatile does NOT give you
Each access in the source produces an actual memory access	Atomicity — a volatile RMW is still three operations
The compiler won't cache the value in a register	Ordering — no barriers are emitted against <i>non-volatile</i> accesses
The compiler won't elide or duplicate accesses	Any protection from CPU reordering or store buffers
	Cache-coherence beyond what the hardware already does

volatile is a compiler directive about *observable accesses*. **_Atomic** is a contract about *the memory model*. They solve different problems.

The confusion's origin

Java and C# gave volatile acquire/release semantics. C did not. Programmers carrying that intuition into C write broken code. Likewise, pre-C11 Linux kernel code used **volatile** + hand-written barriers because atomics didn't exist — that's a historical workaround, not a pattern to copy.

volatile is for memory-mapped hardware registers, **sig_atomic_t** flags set by signal handlers, and **setjmp** locals. **_Atomic** (or a mutex) is for threads.

17.4 <stdatomic.h>

```
#include <stdatomic.h>

_Atomic int      counter = 0;
atomic_int       counter2 = 0;      /* same thing, typedef'd */
_Atomic(struct Foo) big;           /* any type can be atomic */
atomic_flag      f = ATOMIC_FLAG_INIT; /* guaranteed lock-free */
```

Operations

```
atomic_store(&x, v);          atomic_load(&x);
atomic_exchange(&x, v);
atomic_fetch_add(&x, n);      atomic_fetch_sub / _or / _xor / _and
atomic_compare_exchange_strong(&x, &expected, desired);
atomic_compare_exchange_weak(&x, &expected, desired); /* may fail spuriously –
for loops */
atomic_thread_fence(order);
```

Plain operators work too — **counter++** on an **_Atomic int** is **atomic_fetch_add** with **seq_cst**.

Lock-freedom: `ATOMIC_INT_LOCK_FREE` etc. are compile-time; `atomic_is_lock_free(&x)` is runtime. **An `_Atomic` type that isn't lock-free is implemented with a hidden mutex** — correct, but not usable in a signal handler and not fast. Large structs are the usual culprits.

The memory orders

Order	Guarantee
<code>memory_order_relaxed</code>	Atomic, but no ordering with any other operation
<code>memory_order_consume</code>	Data-dependency ordering — deprecated in practice , compilers promote it to acquire
<code>memory_order_acquire</code>	On a load : nothing after it in program order may move before it
<code>memory_order_release</code>	On a store : nothing before it in program order may move after it
<code>memory_order_acq_rel</code>	Both, for read-modify-write operations
<code>memory_order_seq_cst</code>	Default. All <code>seq_cst</code> operations share a single global total order

How they map to x86-64

```
load_relaxed    : mov eax, DWORD PTR x[rip]
load_acquire   : mov eax, DWORD PTR x[rip]      ← identical
load_seqcst     : mov eax, DWORD PTR x[rip]      ← identical
store_relaxed   : mov DWORD PTR x[rip], edi
store_release   : mov DWORD PTR x[rip], edi      ← identical
store_seqcst    : xchg edi, DWORD PTR x[rip]     ← the ONLY one that costs extra
rmw_relaxed     : lock xadd DWORD PTR x[rip], eax
```

x86-64 has a strong memory model: loads are acquire and stores are release *for free*. Only a `seq_cst` store needs a barrier.

And the measured cost of `fetch_add`:

```
seq_cst  : 46.3 ms
relaxed  : 47.0 ms (0.98x) ← no difference
```

Because every x86 RMW uses `lock` regardless of the order requested.

⚠ **This is exactly why you must not tune memory orders by benchmarking on x86.** On ARM64, POWER, or RISC-V — weak memory models — each order maps to genuinely different instructions with genuinely different costs. Code that "works" with `relaxed` on x86 can break on ARM.

Use the default `seq_cst` unless you have measured a problem on the actual target architecture and can prove the weaker order is correct.

The one exception where **relaxed** is obviously right and obviously safe: a counter whose value is only read at the end, with no other data depending on it.

17.5 The C memory model and happens-before

The formal machinery

- **Sequenced-before** — ordering within a single thread (Module 4's sequence points).
- **Synchronizes-with** — a release operation synchronizes with an acquire operation that *reads the value it wrote*.
- **Happens-before** — the transitive closure of sequenced-before and synchronizes-with.

If two conflicting accesses (same location, at least one a write) are not ordered by happens-before, the program has a data race, and the behavior is undefined.

That's the entire rule. Everything else is machinery for establishing happens-before edges.

The canonical release/acquire pattern

```
static int      data = 0;          /* plain, NON-atomic */
static _Atomic bool ready = false;

/* producer */
data = 42;                               /* 1 */
atomic_store_explicit(&ready, true, memory_order_release); /* 2 */

/* consumer */
while (!atomic_load_explicit(&ready, memory_order_acquire)) /* 3 */
    ;
printf("%d\n", data);                     /* 4 – guaranteed 42 */
/*
```

consumer saw data = 42

The chain:

```
(1) data = 42          sequenced-before      (2) store-release(ready)
(2) store-release      SYNCHRONIZES-WITH     (3) load-acquire(ready) that reads it
(3) load-acquire       sequenced-before      (4) read of data
⇒ (1) HAPPENS-BEFORE (4) ⇒ no data race, value guaranteed
```

Note that `data` is a plain `int`. It doesn't need to be atomic — the happens-before edge established by the atomic flag makes the non-atomic access safe. This is the entire point of release/acquire: **publish arbitrary data with one atomic operation.**

TSan proves it

```
### release/acquire version ###
warnings: 0

### same code with memory_order_relaxed on both ###
WARNING: ThreadSanitizer: data race (pid=803)
  Location is global 'data' of size 4
```

Change two enum values and a correct program becomes undefined behavior. **TSan understands the memory model** and reports exactly this.

What else establishes happens-before

Mechanism	Edge
<code>mtx_unlock</code> → <code>mtx_lock</code> (same mutex)	release → acquire
<code>thrd_create</code>	everything before the call happens-before the thread body
<code>thrd_join</code>	everything in the thread happens-before the join returns
<code>cnd_signal</code> → <code>cnd_wait</code> return	via the mutex
<code>call_once</code>	the initialization happens-before every subsequent call
Release store → acquire load of the same atomic	as above
<code>atomic_thread_fence</code> pairs	explicit barriers

This is why mutexes work. `mtx_unlock` is a release, `mtx_lock` is an acquire — everything the first thread did before unlocking is visible to the second after locking.

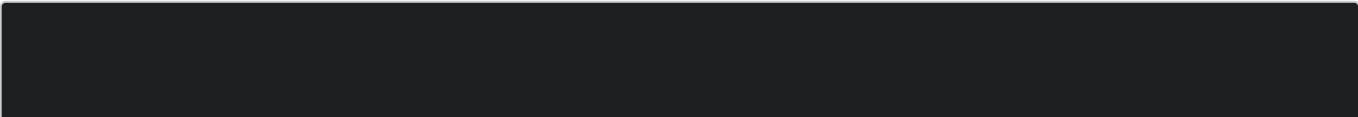
Two things the memory model does *not* guarantee

1. Adjacent variables can still be separate. C11 defines a **memory location** such that distinct scalar objects (and bitfields separated by `:0`) can be updated concurrently without a race — even if they share a cache line. This was a real problem pre-C11.

But **false sharing** remains a *performance* problem: two threads updating adjacent variables in the same cache line ping-pong the line between cores. Fix with `alignas(64)` (Module 11).

2. Nothing about fairness or progress. A spin loop on a relaxed atomic may never observe another thread's write in bounded time.

Lock-free programming — a warning



```
int expected = old;
while (!atomic_compare_exchange_weak(&x, &expected, compute(expected)))
    ; /* expected is updated with the current value on failure */
```

CAS loops are the foundation of lock-free data structures. They are also where the **ABA problem** lives: x changes $A \rightarrow B \rightarrow A$ between your read and your CAS, the CAS succeeds, and your reasoning about intervening state was wrong. Solutions (tagged pointers, hazard pointers, epoch reclamation) are genuinely hard.

Practical advice: use a mutex. Uncontended mutex lock/unlock is ~20 ns and correct. Lock-free algorithms are for measured hot paths, written by people who have read the papers, and verified with TSan plus model checking. The number of published, peer-reviewed lock-free queues that later turned out to be wrong should be sobering.

17.6 Practical guidance

The hierarchy — use the simplest thing that works

1. **Don't share.** Thread-local state, message passing, immutable data. No sharing, no races.
2. **Mutex.** Simple, correct, fast enough for almost everything.
3. **Atomics with `seq_cst`.** For counters and flags.
4. **Atomics with explicit weaker orders.** Only with a measured need on the real target.
5. **Lock-free data structures.** Only if you can prove correctness.

The checklist

```
/* One-time initialization */
static once_flag init_flag = ONCE_FLAG_INIT;
call_once(&init_flag, do_init); /* NOT double-checked locking by hand */

/* Always release what you lock — the goto chain from Module 5 applies */
mtx_lock(&m);
if (bad) { rc = -1; goto out_unlock; }
...
out_unlock: mtx_unlock(&m);

/* Consistent lock ordering prevents deadlock */
/* Always lock A before B, everywhere in the program */
```

- **Deadlock:** enforce a global lock ordering; never call user callbacks while holding a lock.
- **errno is thread-local** (Module 15) — safe.
- **Not thread-safe:** `strtok`, `gmtime/localtime`, `strerror`, `rand`, `asctime`. Use the `_r` variants (Module 14).
- **Function-local `static` makes a function non-reentrant** (Module 3).
- **Signal handlers in a threaded program** need `volatile sig_atomic_t` or lock-free atomics — a mutex in a handler can deadlock.

- **Only lock-free atomics are safe in signal handlers.** Check `atomic_is_lock_free`.

The CI setup

```
# TSan build (separate – cannot combine with ASan)
gcc -std=c17 -O1 -g -fsanitize=thread -fno-omit-frame-pointer prog.c -o prog -lpthread
TSAN_OPTIONS=halt_on_error=1:second_deadlock_stack=1 ./tests

# Run the test suite many times – races are timing-dependent
for i in $(seq 1 100); do ./tests || break; done
```

Run threaded tests repeatedly and under load. A race that fires 1 in 1000 runs will fire constantly in production.

Module 17 rules

#	Rule
1	A data race is undefined behavior , not "a torn value"
2	<code>volatile</code> provides no atomicity and no ordering — it is not thread safety
3	Only <code>_Atomic</code> emits the <code>lock</code> prefix; only that is atomic
4	A correct-looking result proves nothing — run TSan
5	<code>cnd_wait</code> always goes in a <code>while</code> loop
6	The mutex protects the predicate; the condvar carries no state
7	Default to <code>memory_order_seq_cst</code> ; weaken only with measurement on the real target
8	Never tune memory orders by benchmarking on x86 — it hides the cost
9	Release/acquire publishes arbitrary non-atomic data with one atomic op
10	<code>mtx_unlock</code> → <code>mtx_lock</code> , <code>thrd_create</code> , and <code>thrd_join</code> all create happens-before edges
11	<code>alignas(64)</code> on per-thread counters to avoid false sharing
12	Prefer no sharing > mutex > atomics > lock-free, in that order
13	<code><threads.h></code> is optional and absent on MSVC; pthreads is the portable reality
14	TSan cannot combine with ASan — separate builds, and run tests repeatedly

Practice

1. **Reproduce the three-counter experiment.** Build `plain/volatile/_Atomic` with 8 threads. Run 20 times and tabulate the losses. Then dump the assembly for all three and explain each instruction sequence.

2. **Make plain lose.** The optimizer collapsed the plain loop. Defeat that (an opaque function call in the loop, or `-O0`) and show that a plain `int++` loses updates too.
3. **TSan a real program.** Take any multithreaded program (or write a work queue) and run it under TSan. Fix every race. Then run the test suite 200 times and confirm it's clean.
4. **Producer-consumer.** Implement a bounded queue with `mtx_t` + two `cnd_ts`, supporting multiple producers and consumers plus clean shutdown. Verify with TSan. Then deliberately change a `while` to an `if` and find the failure.
5. **Release/acquire proof.** Reproduce the §17.5 pattern. Verify TSan reports 0 warnings. Change both orders to `relaxed` and confirm TSan reports the race on `data`. Explain the missing happens-before edge.
6. **Memory-order codegen.** Dump the assembly for all six memory orders on load, store, and RMW. If you have access to an ARM64 machine (or `aarch64-linux-gnu-gcc`), do the same and compare. This is the clearest way to internalize why x86 hides the cost.
7. **False sharing.** Write 4 threads each incrementing its own counter in a `long counters[4]`. Benchmark. Then change to `struct { alignas(64) long v; } counters[4]` and benchmark again. Measure the speedup.
8. **CAS loop.** Implement `atomic_max(_Atomic long *x, long v)` using `compare_exchange_weak`. Test it with 8 threads pushing random values and verify the result equals the true maximum. Explain why you need `weak` in a loop and `strong` outside one.
9. **Lock-free stack, and its ABA problem.** Implement a Treiber stack with CAS. Then construct a scenario where ABA corrupts it. Fix it with a tagged pointer (using the low bits of a 16-byte-aligned pointer, Module 9).
10. **Portability layer.** Write `thread.h` abstracting over `<threads.h>`, pthreads, and Win32 threads, selected with `#ifdef` (Module 12). Include thread creation, mutexes, condvars, and once-init. Build the same test program against all available backends.

That completes **the core C language section (Modules 1–17)**. You now have the language itself.

Next: **Module 18 — C23 Highlights**, a short module consolidating what's new and what it changes about the idioms you've learned — and then we move into **Part 2: the industry toolchain**, starting with Module 19 (compilers, warnings, and reading generated assembly), which is where "knows C" becomes "professional C developer."

18. C23 Highlights

`nullptr`, `typeof`, `constexpr`, `auto`, `bool` as keyword, attributes (`[[nodiscard]]`, `[[deprecated]]`, `[[fallthrough]]`), binary literals, digit separators, `#embed`, `stdint.h`, enhanced enums, `memset_explicit`, empty initializer `{}`

Module 18 — C23 Highlights

C23 (ISO/IEC 9899:2024) is the biggest language update since C99. Everything below I verified on GCC 13.3 with `-std=c2x` — I'll note explicitly where support is missing.

```
__STDC_VERSION__ = 202000L      ← GCC 13's c2x mode (final C23 is 202311L)
attributes: nodiscard=202003 deprecated=201904 fallthrough=201910
          maybe_unused=202106 noreturn=202202
<stdbit.h>=1 <stdckdint.h>=0
```

18.1 Cleanups — things that stop being awkward

`bool`, `true`, `false` are keywords

```
bool b = true;      /* no #include <stdbool.h> */
```

```
bool b = true -> 1, sizeof(bool)=1
```

`<stdbool.h>` still exists but is now empty-ish and **deprecated**. Same for `<stdalign.h>` (`alignas/alignof` are keywords) and `<stdnoreturn.h>`.

`nullptr` and `nullptr_t`

```
int *p = nullptr;
```

```
int *p = nullptr;  p==NULL: 1  sizeof(nullptr_t)=8
```

Why it matters: `NULL` may be `0` (an `int`) or `((void*)0)`, which is why Module 6 warned you to write `(char*)NULL` in variadic calls. `nullptr` is unambiguously a pointer, so:

```
execl(path, "arg", nullptr);      /* correct — no cast needed */
```

It also works cleanly with `_Generic` (Module 12), where `NULL` was ambiguous.

Binary literals and digit separators

```
int mask    = 0b1010'1100;
long big    = 1'000'000'000;
unsigned h   = 0xDEAD'BEEF;
```

```
0b1010'1100 = 172 (0xAC)
1'000'000'000 = 1000000000
0xDEAD'BEEF = 0xDEADBEEF
```

`0b` literals were a GCC/Clang extension for decades — now standard. The `'` separator is purely visual and works in any base, including floats (`3.141'592`).

And `printf` gained matching `%b` / `%B` conversions:

```
%b test: 172 in binary = 10101100
```

Empty initializer `{}`

```
struct S s = {};
int arr[5] = {};
```

```
struct S s = {}; -> a=0 b=0 buf[0]=0
int arr[5] = {}; -> 0 0 0 0 0
```

Replaces the `= {0}` idiom from Module 3. It's clearer (you're not initializing "the first member to 0"), and it works on structs whose first member isn't a scalar.

⚠ Same caveat as before: **it zeroes members, not necessarily padding**. For byte-exact zeroing use `memset`.

`()` finally means `(void)`

```
void f();
f(1,2,3);
```

```
error: too many arguments to function 'f'
```

The K&R non-prototype declaration from Module 1 §1.3 is gone. `void f()` now means "takes nothing," matching C++ and matching everyone's intuition for 45 years.

K&R-style function *definitions* are also removed. (GCC 13 still only warns with `-Wold-style-definition`; a conforming C23 compiler must reject them.)

Two's complement is mandatory

```
INT_MIN = -2147483648, INT_MAX = 2147483647, -INT_MAX-1 == INT_MIN: 1
no more one's-complement or sign-magnitude implementations
```

This retires a pile of Module 2 caveats. `INT_MIN` is now guaranteed to be `-INT_MAX - 1`, `~0` is `-1`, and **left-shifting a negative value is defined**. Signed *overflow* is still UB — that hasn't changed.

Labels may end a block

```
goto end;
x = 2;
end:          /* legal now – no `;` hack needed */
}
```

This removes the awkward `case 1: ;` workaround from Module 5 §5.2 when a label precedes a declaration.

18.2 New language features

`typeof` and `typeof_unqual`

```
const int ci = 7;
typeof(ci)      same = 8;  /* const int */
typeof_unqual(ci) mut  = 9; /* int – qualifier stripped */
mut = 10;       /* OK */
```

`typeof(ci)` keeps `const`; `typeof_unqual(ci)` strips it -> `mut=10`

A GCC extension since forever, now standard. It makes **safe macros** possible without GNU statement expressions:

```
#define SWAP(a, b) do { typeof(a) tmp_ = (a); (a) = (b); (b) = tmp_; } while (0)
```

`typeof_unqual` is the one you usually want in macros — otherwise a `const` argument makes your temporary unassignable.

`auto` — type inference

```
auto x = 3.14;    /* double */
auto y = 42;      /* int */
```

```
auto x = 3.14 -> sizeof=8 (double)
auto y = 42    -> sizeof=4 (int)
```

The old meaning of `auto` (the default storage class nobody wrote) is repurposed. **This is the most controversial C23 change** — C has always been explicit about types, and `auto` hides them.

Where it's genuinely useful: macro internals, where you need "the type of this expression" and don't know it.

```
#define MAX(a, b) ({ auto a_ = (a); auto b_ = (b); a_ > b_ ? a_ : b_; })
```

Where it's not: ordinary variable declarations. Use it sparingly.

constexpr

```
constexpr int SIZE = 16;
int fixed[SIZE];           /* a real constant expression */
```

```
constexpr int SIZE = 16; int fixed[SIZE]; sizeof=64
```

This closes a real C wart. From Module 3: `const int` is *not* a constant expression in C — you can't use it for array sizes, `case` labels, bitfield widths, or static initializers. `enum` worked but only for `int` values.

```
constexpr double PI = 3.14159265358979;    /* enum can't do this */
constexpr size_t CAP = 1024;               /* typed, scoped, usable as an array
size */
```

C23 `constexpr` applies to **objects only** — not functions, unlike C++.

unreachable()

```
#include <stddef.h>

static const char *name(enum State s)
{
    switch (s) { case S_A: return "A"; case S_B: return "B"; }
    unreachable();
}
```

Standardizes `__builtin_unreachable()`. Suppresses "control reaches end of non-void function," lets the compiler delete the dead path, and documents intent.

⚠ **Reaching it is undefined behavior.** Only use it where you're certain — and combine with `-Werror=switch` so the compiler proves it for you.

18.3 Attributes

```
[[nodiscard]] int must_use(void);
[[nodiscard("the return value carries the error code")]] int must_use2(void);
[[deprecated("use new_api()")]] void old_api(void);
[[maybe_unused]] static int helper(void);
[[noreturn]] void die(void);
[[fallthrough]];
```

All firing correctly:

```
warning: ignoring return value of 'must_use', declared with attribute 'nodiscard'
warning: ignoring return value of 'must_use2', declared with attribute
'nodiscard':
    "the return value carries the error code"
warning: 'old_api' is deprecated: use new_api()
```

Attribute	Use
[[nodiscard]]	Put this on every fallible function — it's the Module 15 rule "never ignore a return value," enforced by the compiler
[[deprecated("msg")]]	Migrate APIs without breaking builds
[[maybe_unused]]	Silence unused warnings on conditionally-used things
[[noreturn]]	Replaces <code>_Noreturn</code> (Module 6)
[[fallthrough]]	Replaces the <code>/* fall through */</code> comment (Module 5)
[[reproducible]], [[unsequenced]]	Purity hints, enabling CSE and reordering

[[fallthrough]] in action — the marked case is silent, the unmarked one warns:

```
ft.c:6:13: warning: this statement may fall through [-Wimplicit-fallthrough=]
one two
three four
```

Detect availability with `__has_c_attribute(nodiscard)` — as shown at the top of this module.

[[nodiscard]] on your error-returning functions is the single highest-value C23 change for reliability. Combined with Module 15's error API, it makes ignoring an error a compile-time warning.

18.4 Enhanced enums

```
enum Small : unsigned char { S_A, S_B, S_C, };    /* fixed type + trailing comma
*/
enum Wide  : long long      { W_BIG = 1LL << 40 };
enum Flags : uint32_t       { F_A = 1u<<0, F_C = 1u<<31 };

enum Fwd : int;                                /* forward declaration – impossible before C23 */
int use(enum Fwd f);
enum Fwd : int { FWD_X = 1, FWD_Y = 2 };
```

```
enum Small : unsigned char -> sizeof=1
enum Wide  : long long      -> sizeof=8 W_BIG=1099511627776
enum Flags : uint32_t       -> sizeof=4 F_C=0x80000000
forward-declared enum: use(FWD_Y)=2
```

Four fixes to Module 11 §11.6's complaints:

1. **Fixed underlying type** — enums are now usable in packed structs, wire formats, and ABI boundaries.
2. **Values beyond `int` range** are allowed.
3. **Forward declaration** works, so enums can appear in headers without their definition.
4. **Trailing comma** is legal (helps X-Macros and generated code).

18.5 Library additions

`<stdbit.h>` — standardized bit operations

```
x = 0xAC
count_ones      = 4
count_zeros     = 28
leading_zeros   = 24
trailing_zeros  = 2
first_leading_one = 25
bit_width       = 8
bit_floor       = 128
bit_ceil        = 256
has_single_bit(64) = 1
__STDC_ENDIAN_NATIVE__ == LITTLE? 1
```

Everything from Module 4 §4.8, now portable: `stdc_count_ones`, `stdc_count_zeros`, `stdc_leading_zeros/ones`, `stdc_trailing_zeros/ones`, `stdc_first_leading_one`, `stdc_bit_width`, `stdc_bit_floor`, `stdc_bit_ceil`, `stdc_has_single_bit`, plus `_uc/_us/_ui/_ul/_ull` variants and the generic macros.

Plus endianness macros: `__STDC_ENDIAN_NATIVE__`, `__STDC_ENDIAN_LITTLE__`, `__STDC_ENDIAN_BIG__` — finally a standard way to test byte order at compile time.

⚠ **`stdc_leading_zeros(0)` is defined** (returns the width), unlike `__builtin_clz(0)` which is UB. A real improvement.

`<stdckdint.h>` — checked arithmetic

```
int r;
if (ckd_add(&r, a, b)) { /* overflow */ }
ckd_sub(&r, a, b);
ckd_mul(&r, a, b);
```

Returns **true** on overflow. Standardizes `__builtin_add_overflow` (Module 2 §2.8), and the operations are performed as if in infinite precision then converted.

Not yet in glibc 2.39:

```
<stdckdint.h> not available in this libc.
GCC builtin equivalent: __builtin_add_overflow(INT_MAX,1,&r) = 1
```

`memset_explicit`

```
memset_explicit(pw, 0, sizeof pw);
```

The standard answer to Module 10 §10.9 — a `memset` the compiler is forbidden to optimize away. **Not in glibc 2.39 yet**; use `explicit_bzero` meanwhile.

Now-standard functions

Previously POSIX or GNU extensions, now ISO C:

`strdup`, `strndup`, `memccpy`, `gmtime_r`, `localtime_r`, `timespec_getres`, `memalignment`, `free_sized`, `free_aligned_sized`, `asctime_r`/`ctime_r`.

```
strdup (C23 standard): "now standard in C23"
```

This is a real portability win. Module 14's advice to use `gmtime_r` no longer requires a feature-test macro.

`#embed` (preprocessor)

```
static const unsigned char logo[] = {
    #embed "logo.png"
```



```
};
```

Covered in Module 12 §12.9. **Not in GCC 13:**

```
error: missing binary operator before token "("      ← __has_embed unsupported
```

Landed in GCC 15 and Clang 19.

`__VA_OPT__`

```
#define LOG(fmt, ...) printf("[log] " fmt "\n" __VA_OPT__(,) __VA_ARGS__)
```

```
[log] no args
[log] x=7
```

Works with `-pedantic` — the standard replacement for the GNU `##__VA_ARGS__` hack (Module 12 §12.4).

Also standardized: `#warning`, `#elifdef`/`#elifndef`, and `__has_include`/`__has_c_attribute`/`__has_embed`.

18.6 Support and adoption

Feature	GCC 13 (<code>-std=c2x</code>)	Notes
<code>bool/true/false</code> keywords	☑ verified	
<code>nullptr</code> / <code>nullptr_t</code>	☑ verified	
Binary literals, digit separators	☑ verified	
Empty initializer <code>{}</code>	☑ verified	
<code>typeof</code> / <code>typeof_unqual</code>	☑ verified	
<code>auto</code> type inference	☑ verified	
<code>constexpr</code>	☑ verified	
<code>unreachable()</code>	☑ verified	
All six attributes	☑ verified	
Enum fixed type + forward decl	☑ verified	
<code>()</code> means <code>(void)</code>	☑ verified	
<code>__VA_OPT__</code> , <code>#warning</code>	☑ verified	

Feature	GCC 13 (-std=c2x)	Notes
<code>%b/%B</code> in <code>printf</code>	☑ verified	
<code><stdbit.h></code>	☑ verified	
<code><stdckdint.h></code>	✗ not in glibc 2.39	use <code>__builtin*_overflow</code>
<code>memset_explicit</code>	✗ not in glibc 2.39	use <code>explicit_bzero</code>
<code>#embed</code>	✗ GCC 15+	Clang 19+
K&R definitions rejected	⚠ warns only	

Two moving parts: the **compiler** (GCC 13/14 have most of the language; GCC 15+ adds `#embed`) and the **C library** (glibc 2.38+ for `strncpy`, 2.39 still missing `stdckdint.h/memset_explicit`).

MSVC's C23 support is partial and lags well behind. For cross-platform code, plan for a subset.

How to adopt safely:

```
#if __STDC_VERSION__ >= 202311L
# define MY_NODISCARD [[nodiscard]]
#elif defined(__GNUC__)
# define MY_NODISCARD __attribute__((warn_unused_result))
#else
# define MY_NODISCARD
#endif

#if __has_include(<stdckdint.h>)
# include <stdckdint.h>
#elif defined(__GNUC__)
# define ckd_add(r,a,b) __builtin_add_overflow((a),(b),(r))
#endif
```

Use `__has_include`, `__has_c_attribute`, and `__STDC_VERSION__` rather than compiler-version checks.

18.7 What changes about the idioms you've learned

Module	Old idiom	C23
1	<code>int f(void)</code> mandatory	<code>int f()</code> now means the same
2	<code>char</code> signedness, <code>~0</code> caveats	Two's complement mandatory
3	<code>= {0}</code>	<code>= {}</code>
3	<code>#define MAX 1024</code> or <code>enum</code>	<code>constexpr size_t MAX = 1024;</code>
4	<code>__builtin_popcount</code>	<code>stdc_count_ones</code>
4	<code>__builtin_clz(0)</code> is UB	<code>stdc_leading_zeros(0)</code> is defined

Module	Old idiom	C23
5	<code>/* fall through */</code>	<code>[[fallthrough]]</code>
5	<code>case 1: ;</code> before a declaration	Label may precede anything
6	<code>_Noreturn</code>	<code>[[noreturn]]</code>
6	<code>__attribute__((warn_unused_result))</code>	<code>[[nodiscard]]</code>
8	<code>explicit_bzero</code>	<code>memset_explicit</code>
8	<code>strdup</code> needs <code>_POSIX_C_SOURCE</code>	Now ISO C
9	<code>(char*)NULL</code> in variadics	<code>nullptr</code>
11	Enum underlying type is impl-defined	<code>enum E : uint8_t</code>
12	<code>, ##__VA_ARGS__</code> (GNU)	<code>__VA_OPT__(,)</code>
12	<code>xxd -i</code> for embedded assets	<code>#embed</code>
14	<code>gmtime_r</code> needs POSIX	Now ISO C
14	<code>__builtin_add_overflow</code>	<code>ckd_add</code>

Module 18 rules

#	Rule
1	Put <code>[[nodiscard]]</code> on every function whose return value must be checked
2	<code>constexpr</code> replaces <code>#define</code> and <code>enum</code> for typed compile-time constants
3	<code>typeof_unqual</code> (not <code>typeof</code>) inside macros, so <code>const</code> arguments still work
4	<code>nullptr</code> in variadic calls and <code>_Generic</code> — never bare <code>NULL</code>
5	Use <code>auto</code> in macro internals, not in ordinary declarations
6	<code>enum E : uint8_t</code> makes enums safe in packed structs and ABI boundaries
7	<code>unreachable()</code> only where <code>-Werror=switch</code> proves the case is impossible
8	Feature-test with <code>__has_include</code> / <code>__has_c_attribute</code> , not compiler versions
9	Compiler support ≠ library support — check both
10	For cross-platform code, assume MSVC has a smaller C23 subset

Practice

1. **Probe your toolchain.** Run the §18 probe program on every compiler you have access to. Build a support table for the features you'd actually want to use.
2. `[[nodiscard]]` **retrofit.** Take the error library from Module 15 and add `[[nodiscard]]` to every function returning `err_t`. Then deliberately ignore a return and confirm the warning. Add -

`Werror=unused-result.`

3. **Safe macros with `typeof_unqual`.** Rewrite `MIN`, `MAX`, `SWAP`, and `CLAMP` from Module 12 using `typeof_unqual` so each argument is evaluated exactly once. Verify with the side-effect counter that they call `f()` once. Then test with `const` arguments.
4. **`constexpr` migration.** Take a file full of `#define` constants and convert them to `constexpr`. Note which ones now get type checking they didn't have. Find one where the `#define` was silently the wrong type.
5. **`<stdbit.h>` vs builtins.** Reimplement the Module 4 bit toolkit using `<stdbit.h>`. Compare the generated assembly to the `__builtin_*` versions. Then test the zero case for `leading_zeros` and confirm it's defined where `__builtin_clz` was UB.
6. **Enum ABI fix.** Take a struct with a bare enum field that's written to a file. Show its size on two different compilers (or with `-fshort-enums`). Then fix it with `enum E : uint8_t` and confirm the size is now guaranteed.
7. **Compatibility shim header.** Write `c23compat.h` providing `MY_NODISCARD`, `MY_FALLTHROUGH`, `MY_MAYBE_UNUSED`, `ckd_add`, and `secure_zero` that work on C23, GCC/Clang pre-C23, and MSVC. Test by building the same source under `-std=c17` and `-std=c2x`.
8. **`#embed` if you have GCC 15+/Clang 19+.** Embed a 1 MB binary and compare compile time against `xxd -i`. If you don't have it, write the `xxd -i` build rule and note what `#embed` would replace.

That completes **Part 1 — the C language (Modules 1–18)**.

You now have the language itself: types, memory, pointers, the standard library, undefined behavior, and concurrency, plus what C23 changes about all of it.

Part 2 begins with Module 19 — Toolchain & Compilation: GCC/Clang/MSVC differences, the complete warning set and why each flag exists, optimization levels and what they actually do, reading generated assembly (you've been doing this all along — now systematically), LTO, cross-compilation, and the binutils suite (`objdump`, `nm`, `readelf`, `addr2line`) that you'll live in when debugging production binaries.

PART 2 — What Industry Actually Expects (beyond the language)

This is the part that separates "knows C" from "professional C developer."

19. Toolchain & Compilation

- GCC, Clang, MSVC, ICC — differences and extensions
- Warning flags you should always run: `-Wall -Wextra -Wpedantic -Werror -Wshadow -Wconversion -Wstrict-prototypes`
- Optimization levels `-O0..-O3 -Os -Ofast, -g, -march/-mtune`
- Preprocessor/assembly output (`-E, -S`), reading generated assembly (Godbolt)
- Link-Time Optimization, `-ffunction-sections -Wl,--gc-sections`

- Cross-compilation, sysroots, toolchain files
- `objdump`, `nm`, `readelf`, `strip`, `size`, `ldd`, `addr2line`

Part 2 — The Industry Toolchain

You have the language. This is where "knows C" becomes "professional C developer." Everything from here is about the ecosystem: building, debugging, testing, securing, and shipping C.

Module 19 — Toolchain & Compilation

Real GCC 13.3 / binutils output throughout.

19.1 The compilers

Compiler	Notes
GCC	The Linux/embedded default. Best target coverage (60+ architectures). Excellent optimizer.
Clang/LLVM	Best diagnostics, best sanitizers (MSan is Clang-only), fast builds. Default on macOS/FreeBSD. <code>clang-tidy</code> , <code>clang-format</code> , <code>scan-build</code> .
MSVC	Windows. Historically weak C support — C99 only completed around VS2019, C11/C17 partial, C23 minimal, and <code><threads.h></code> has never shipped. Its C++ compiler is excellent; its C compiler is an afterthought.
ICC / ICX	Intel. Classic ICC is retired; ICX is LLVM-based. Best autovectorization for Intel CPUs and HPC.
tcc	Tiny C Compiler. Compiles ~10× faster than GCC, no optimizer. Good for quick scripts.

What to know practically

Build with two compilers. GCC and Clang warn about different things, exploit different UB (Module 16), and catch different bugs. This is the single cheapest quality improvement available.

Extensions to be aware of:

Extension	GCC	Clang	MSVC
<code>__attribute__((...))</code>	✓	✓	✗ (<code>__declspec</code>)
Statement expressions <code>{ { ... } }</code>	✓	✓	✗
<code>typeof</code> (pre-C23)	✓	✓	✗
<code>__builtin_*</code>	✓	mostly	✗ (different intrinsics)
Nested functions	✓	✗	✗
VLAs	✓	✓	✗

Extension	GCC	Clang	MSVC
<code>#pragma once</code>	✓	✓	✓
Zero-length arrays	✓	✓	partial

Detect with feature tests, not compiler versions:

```
#if defined(__GNUC__) || defined(__clang__)
# define MY_UNUSED __attribute__((unused))
#elif defined(_MSC_VER)
# define MY_UNUSED
#endif
```

`__clang__` is defined *and so is* `__GNUC__` — Clang masquerades as GCC. Check `__clang__` first.

The compiler is a driver

```
$ gcc -v t.c -o t
/usr/libexec/gcc/x86_64-linux-gnu/13/cc1 -quiet -v -imultiarch ... t.c ...
/usr/libexec/gcc/x86_64-linux-gnu/13/collect2 -plugin .../liblto_plugin.so ...
```

`gcc` isn't the compiler — it's a **driver** that runs `cc1` (the actual C compiler), `as` (assembler), and `collect2/ld` (linker). `gcc -v` shows you exactly what it invoked, which is invaluable when a build mysteriously fails.

```
-dumpmachine      : x86_64-linux-gnu
-print-multiarch  : x86_64-linux-gnu
libc.so path     : /usr/lib/gcc/.../libc.so
```

19.2 Warning flags

Measured on one deliberately-bad file:

```
### -Wall only ###      warnings: 8
### -Wall -Wextra ###   warnings: 9
### full recommended set ### warnings: 16
```

Nearly double. `-Wall` is not "all warnings" — it's "warnings Richard Stallman agreed to in 1988." Here's what the extra flags actually caught:

```
[-Warray-bounds=]  [-Wfloat-conversion]  [-Wformat=]         [-Woverflow]
[-Wparentheses]    [-Wshadow]          [-Wsign-compare]    [-Wsign-conversion]
```

```
[-Wstrict-prototypes] [-Wunused-function] [-Wunused-variable]
```

The recommended set, and why each flag exists

```
# The baseline – non-negotiable
-Wall -Wextra -Wpedantic

# Bugs from earlier modules
-Wshadow           # M3: shadowing a variable you meant to modify
-Wconversion       # M2: implicit narrowing
-Wsign-conversion  # M2: the signed/unsigned trap
-Wstrict-prototypes # M6: `void f()` non-prototypes
-Wold-style-definition # M1: K&R definitions
-Wmissing-prototypes # M3: a non-static function with no header declaration
-Wcast-qual        # M9: casting away const
-Wcast-align=strict # M9: producing a misaligned pointer
-Wpointer-arith     # M9: void* arithmetic
-Wwrite-strings     # M8: makes string literals const char[]
-Wformat=2          # M13: format-string bugs and security
-Wnull-dereference  # M16: the CVE-2009-1897 pattern
-Wstrict-aliasing=2 # M2: aliasing violations
-Warray-bounds=2    # M7: out-of-bounds indexing
-Wstringop-overflow=4 # M8: strcpy/memcpy overflows
-Wimplicit-fallthrough # M5: switch fallthrough
-Wsequence-point    # M4: i = i++ family
-Walloca           # M10: alloca
-Wvla              # M7: variable length arrays
-Wdouble-promotion # M2: accidental float→double in embedded code
-Wfloat-equal      # M2: == on floats
-Wswitch-enum      # M11: unhandled enum values
-Wundef            # M12: #if on an undefined macro
-Wredundant-decls  # duplicate declarations

# Make the dangerous ones fatal
-Werror=implicit-function-declaration # M6: silent ABI corruption
-Werror=return-type                    # M5: UB
-Werror=incompatible-pointer-types
-Werror=int-conversion
-Werror=vla
```

-Werror — yes or no?

In CI: absolutely yes. A warning that nobody fixes is noise, and noise hides real bugs.

In local development: often no. A new compiler version adds warnings and suddenly nobody can build. The common compromise:

```
CFLAGS += $(WARNINGS)
ifdef CI
```

```
CFLAGS += -Werror
endif
```

Plus `-Werror=` on the specific warnings that are *always* bugs (the list above).

Noise management

`-Wconversion` and `-Wsign-conversion` are genuinely noisy on existing code. Two strategies:

- 1. **New code only** — enable them per-directory, or on new files.
- 2. **Fix incrementally** — enable, count, fix 10 a day.

Suppress locally when you're sure:

```
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wconversion"
...deliberate narrowing, explained in a comment...
#pragma GCC diagnostic pop
```

Never suppress globally.

19.3 Optimization levels

Measured on a 400×400 matrix multiply:

Level	Time	Binary	.text
-O0	217.6 ms	16248	2798
-O1	48.3 ms	16224	2459
-O2	30.6 ms	16224	2848
-O3	26.1 ms	16192	2881
-Os	43.7 ms	16224	2368
-Ofast	25.1 ms	17736	2981

`-O0` → `-O3` is **8.3×**. That's the single largest performance lever you have, and it costs nothing.

What each level means

Level	Purpose
-O0	No optimization. Fast compiles, exact debugging — variables live in memory, statements execute in source order. Never ship this.
-O1	Cheap optimizations. Already 4.5× faster here.
-O2	The production default. Everything that doesn't dramatically increase code size.

Level	Purpose
-O3	-O2 + aggressive inlining, loop unrolling, vectorization. Sometimes slower (code bloat, i-cache pressure). Measure.
-Os	-O2 minus size-increasing passes. Smallest .text here (2368 vs 2848) . The right default for embedded and for i-cache-bound code.
-Oz	Clang: even more aggressive size reduction.
-Ofast	 -O3 -ffast-math and it breaks IEEE 754 — assumes no NaN/Inf, allows reassociation. Not standards-conforming . Only for numerics where you've verified the result.
-Og	-O1-ish but optimized for a good debugging experience. Better than -O0 for debug builds of large programs.

-g is orthogonal

-g **does not disable optimization**. -O2 -g is a legitimate and common production configuration — you get full speed and debug info, at the cost of a larger binary (which you then split out, \$19.7).

Levels: -g1 (line tables only), -g = -g2, -g3 (adds macro definitions — lets GDB expand your macros). -gsplit-dwarf puts debug info in .dwo files for faster links.

-march and -mtune

```
-O3                26.6 ms
-O3 -march=native   24.3 ms
-O3 -march=x86-64-v3 23.3 ms
-O3 -march=native -ffast-math 22.0 ms
```

-march=native enabled on this machine:

```
-mavx2 [enabled]  -mavx512f [enabled]  -mfma [enabled]  -msse4 [enabled]
```

And the vectorizer reported:

```
bench.c:10:22: optimized: loop vectorized using 32 byte vectors
```

Flag	Meaning
-march=X	Generate instructions for X. The binary will not run on older CPUs.
-mtune=X	Schedule for X but stay compatible with everything. Free.
-march=native	The machine you're compiling on. Never for distributed binaries .

Flag	Meaning
<code>-march=x86-64-v2/v3/v4</code>	Portable feature levels: v2 $\approx$ SSE4.2 (2009+), v3 $\approx$ AVX2 (2013+), v4 $\approx$ AVX-512.

For shipped software: `-mtune=generic` (default) or a conservative `-march=x86-64-v2`, plus runtime dispatch (`__builtin_cpu_supports`) for hot paths. `-march=native` in a distributed binary produces **SIGILL** on customer machines.

Useful diagnostics: `-fopt-info-vec`, `-fopt-info-vec-missed` (why a loop *didn't* vectorize), `-fopt-info-inline`.

19.4 Reading generated assembly

`-E`, `-S`, `-c` were covered in Module 1. Here's the systematic view.

```
gcc -S -masm=intel -fno-asynchronous-unwind-tables -fverbose-asm -O2 f.c -o -
```

- `-masm=intel` — Intel syntax (`mov dst, src`). AT&T is the GCC default (`mov src, dst` — reversed, with % and \$ sigils).
- `-fno-asynchronous-unwind-tables` — removes `.cfi_*` noise
- `-fverbose-asm` — annotates with variable names

The same function at two levels

```
int sum_array(const int *a, int n)
{ int s = 0; for (int i = 0; i < n; i++) s += a[i]; return s; }
```

`-O0` — every variable lives in memory, one operation at a time:

```
sum_array:
    push    rbp
    mov     rbp, rsp
    mov     QWORD PTR -24[rbp], rdi        ; spill the parameter
    mov     DWORD PTR -8[rbp], 0          ; s = 0
    mov     DWORD PTR -4[rbp], 0          ; i = 0
.L3:    mov     eax, DWORD PTR -4[rbp]      ; load i
    cdqe
    lea     rdx, 0[0+rax*4]                ; i*4
    mov     rax, QWORD PTR -24[rbp]        ; load a
    add     rax, rdx
    mov     eax, DWORD PTR [rax]           ; load a[i]
    add     DWORD PTR -8[rbp], eax         ; s += ...
    add     DWORD PTR -4[rbp], 1           ; i++
```

-O3 -march=native — vectorized:

```
vpaddq ymm1, ymm1, YMMWORD PTR [rax] ; 8 ints added per instruction
vextracti128 xmm1, ymm1, 0x1 ; horizontal reduction
vpaddq xmm0, xmm0, xmm1
```

This is what -O0 vs -O3 looks like. The **-O0** version does ~9 memory operations per element; the **-O3** version does one 32-byte load and one **vpaddq** per 8 elements.

Why read assembly

1. **Verify an optimization happened** — was that loop vectorized? was that function inlined?
2. **Understand UB exploitation** — every demo in Module 16.
3. **Debug optimized code** where the source doesn't match execution.
4. **Confirm a security property** — did that **memset** survive? (Module 10)
5. **Micro-optimization** — but only after profiling.

Godbolt Compiler Explorer

godbolt.org is the tool for this. Paste C, pick any of ~50 compilers and versions, see color-matched assembly side by side, diff two compilers or two flag sets, and share a permalink.

Use it constantly. It turns "I think the compiler does X" into "I know the compiler does X."

19.5 Link-Time Optimization

Normally the compiler optimizes one translation unit at a time (Module 1 §1.5) — it cannot inline across **.c** files. **LTO defers code generation to link time** so the optimizer sees the whole program.

```
gcc -O2 -flto -c a.c -o a.o
gcc -O2 -flto -c b.c -o b.o
gcc -O2 -flto a.o b.o -o prog # the SAME flags must appear at link time
```

Measured

Without LTO — a real cross-TU call:

```
1075: call 1190 <helper>
1091: call 1050 <__printf_chk@plt>
```

With LTO — **helper** is gone entirely; the loop was inlined *and* constant-folded:

```
107b: call 1050 <__printf_chk@plt>
```

```

no LTO    : 16128 bytes, text=1621
with LTO  : 15936 bytes, text=1412

dead code (unused_a/b/c symbols):
no LTO    : 3
with LTO  : 0

```

Practical notes

- ☒ Cross-TU inlining, whole-program dead-code elimination, better devirtualization, smaller binaries.
- ⚠ **Much slower links** and high memory use. Use `-flto=auto` or `-flto=jobserver` to parallelize.
- ⚠ **Exposes latent UB** — the optimizer now sees across boundaries where bugs were hiding (Module 16).
- ⚠ **The same `-flto` and optimization flags must be on both compile and link.**
- ⚠ Debugging is harder; symbols may vanish.
- **Thin LTO** (`-flto=thin`, Clang) is much faster with most of the benefit — the standard choice for large projects.

Verdict: enable it for release builds, keep it off for development builds, and make sure your CI tests the LTO build.

19.6 Section garbage collection

The linker's unit of inclusion is a **section**, and by default all functions in a `.o` share one `.text`. So linking against an object file pulls in *everything* in it.

```

gcc -ffunction-sections -fdata-sections -c *.c
gcc -Wl,--gc-sections *.o -o prog

```

This puts each function in its own `.text.funcname` and each object in its own `.data.name`, so the linker can discard unreferenced ones.

Measured — 20 functions, one used

```

plain build:                                size=16616  text=2512  fn* symbols=20
-ffunction-sections --gc-sections:  size=15920  text=1461  fn* symbols=1

```

`.text` almost halved, and 19 of 20 symbols removed.

```

/usr/bin/ld: removing unused section '.text.fn2' in file 'bigs.o'
/usr/bin/ld: removing unused section '.text.fn3' in file 'bigs.o'
...

```

```
plain .o          : 1 .text section
-ffunction-sections .o : 20 .text.* sections
```

Essential for embedded (where flash is measured in KB) and for static linking against large libraries. Use `-Wl,--print-gc-sections` to see what's being removed, and `-Wl,-Map=out.map` for a full link map.

⚠ Slightly slower links, and it can break code that relies on section adjacency (linker-script-placed tables — Module 27). Mark those `KEEP()` in the linker script or `__attribute__((used))`.

19.7 The binutils suite

```
$ file tools
ELF 64-bit LSB pie executable, x86-64, dynamically linked, interpreter /lib64/ld-
linux-x86-64.so.2
```

size — section sizes

text	data	bss	dec	hex filename
1578	616	8	2202	89a tools

The Module 3 **.bss** distinction, on any binary. First thing to check when a binary is unexpectedly large.

nm — symbols

```
00000000000001149 T api      ← T = text, global
00000000000004024 B g_bss   ← B = bss, global
00000000000004010 D g_init  ← D = data, global
00000000000001168 T main
```

Module 1's letter table applies. `nm -C` demangles C++; `nm -D` shows dynamic symbols; `nm -u` shows undefined ones. **This is how you debug "undefined reference" and "multiple definition."**

readelf — ELF structure

```
$ readelf -S tools      # sections
$ readelf -h tools      # header (type, machine, entry point)
$ readelf -d tools      # dynamic section (NEEDED libraries, RPATH, soname)
$ readelf -l tools      # program headers (what the loader maps)
$ readelf --relocs tools # relocations
```

`readelf -d` is how you diagnose "wrong library version loaded" — it shows **NEEDED**, **SONAME**, **RUNPATH**.

objdump — disassembly

```
$ objdump -d --no-show-raw-insn -M intel tools

0000000000001149 <api>:
   1149:  endbr64
   114d:  mov     eax,DWORD PTR [rip+0x2ebd]    # 4010 <g_init>
   1153:  lea     eax,[rax+rdi*2+0x7]
   1157:  ret
```

objdump -S interleaves source (requires **-g**):

```
int api(int x){ return helper(x) + g_init + s_priv; }
   1149:  endbr64
   114d:  mov     eax,DWORD PTR [rip+0x2ebd]
   1153:  lea     eax,[rax+rdi*2+0x7]
```

Note how `helper(x)` (a **static** function) and `s_priv` were both inlined and constant-folded into a single `lea`. This is why debugging optimized code is confusing.

Also: **objdump -t** (symbols), **-r** (relocations), **-x** (everything), **-h** (sections).

addr2line — crash address → source location

```
$ ./tools crash
Segmentation fault

$ addr2line -e tools -f -C -i 0x115c
crasher
/home/claude/c_tool/tools.c:10
```

This is the tool that turns a production crash log into a bug report. **-f** shows the function, **-C** demangles, **-i** expands inlined frames.

Given a stack trace with raw addresses (from **backtrace_symbols**, a core dump, or a crash reporter), **addr2line** maps each one back to source — **provided you kept the unstripped binary or the separate debug file**.

⚠ With PIE (position-independent executables, the default), runtime addresses are randomized by ASLR. Subtract the load base (from `/proc/self/maps`) before calling **addr2line**.

strip — and the right way to do it

```
before strip: 18472 bytes, 35 symbols
after strip: 14488 bytes, 1 symbol
```

But stripping destroys your ability to debug crashes. **The professional pattern is to split the debug info out:**

```
objcopy --only-keep-debug prog prog.debug
strip --strip-debug --strip-unneeded prog
objcopy --add-gnu-debuglink=prog.debug prog
```

```
binary: 14592 bytes   debug file: 7088 bytes
```

Ship the small binary; archive `prog.debug` alongside the release. GDB finds it automatically via the debuglink, and `addr2line -e prog.debug` works on production crash addresses. **Every serious project does this.**

ldd and strings

```
$ ldd tools
    linux-vdso.so.1
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
    /lib64/ld-linux-x86-64.so.2

$ strings tools | grep distinctive
a distinctive string constant
```

⚠ **ldd runs the dynamic loader** — never run it on an untrusted binary. Use `objdump -p | grep NEEDED` or `readelf -d` instead.

The others

`ar` / `ranlib` (static libraries), `objcopy` (format conversion, section surgery, `-O binary` for embedded flashing), `c++filt` (demangling), `nm --size-sort` (find the biggest functions), `bloaty` (not `binutils`, but the best binary size analyzer).

19.8 Cross-compilation

The toolchain naming convention

```
<arch>-<vendor>-<os>-<abi>

aarch64-linux-gnu-gcc      64-bit ARM, Linux, glibc
arm-none-eabi-gcc          32-bit ARM, bare metal (no OS)
```

```
riscv64-unknown-elf-gcc    RISC-V, bare metal
x86_64-w64-mingw32-gcc    Windows, from Linux
```

none/unknown in the OS field means **bare metal** — no operating system, freestanding implementation (Module 26).

Each cross toolchain has its own `cpp`, `cc1`, `as`, `ld`, `libc`, and `headers`. They are not interchangeable.

The sysroot

```
aarch64-linux-gnu-gcc --sysroot=/opt/arm-rootfs prog.c -o prog
```

The **sysroot** is a directory tree containing the *target's* `/usr/include` and `/usr/lib`. Without it, the compiler would use your host's headers and libraries — which would compile but produce a binary linked against the wrong ABI.

Related: `-isysroot`, `-B` (where to find the tools), `-L/-I` for extra paths.

Useful driver queries

```
gcc -dumpmachine           # x86_64-linux-gnu – the target triple
gcc -print-search-dirs     # where it looks for tools and libraries
gcc -print-file-name=libc.so # resolve a library to a full path
gcc -print-multiarch       # what the driver actually runs
gcc -v
```

`-m32` needs multilib

```
$ gcc -m32 t.c -o t32
/usr/bin/ld: cannot find Scrt1.o: No such file or directory
/usr/bin/ld: cannot find crt1.o: No such file or directory
```

Even for the "same" architecture you need the 32-bit C runtime and libraries (`gcc-multilib`, `libc6-dev-i386`).

Build-system integration

CMake toolchain file:

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR aarch64)
set(CMAKE_C_COMPILER aarch64-linux-gnu-gcc)
set(CMAKE_SYSROOT /opt/arm-rootfs)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
```



```
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
```

```
cmake -DCMAKE_TOOLCHAIN_FILE=arm.cmake ..
```

Autotools: `./configure --host=aarch64-linux-gnu --build=x86_64-linux-gnu`

Note the terminology: **build** = where you compile, **host** = where the binary runs, **target** = what the *compiler you're building* will emit for (only relevant when building a compiler).

Toolchain sources: distro packages (`gcc-aarch64-linux-gnu`), ARM's official GNU toolchain, crosstool-NG, Buildroot, Yocto, Zig (`zig cc` is a remarkably good drop-in cross-compiler with bundled sysroots), Docker images.

Testing: `qemu-aarch64` runs foreign binaries directly, and `binfmt_misc` makes it transparent.

Module 19 rules

#	Rule
1	Build with two compilers — GCC and Clang catch different bugs
2	<code>-Wall</code> is not all warnings; the full set nearly doubles coverage
3	<code>-Werror</code> in CI, <code>-Werror=</code> on always-bug warnings locally
4	<code>-O0</code> → <code>-O3</code> is $\sim 8\times$ — measured. Never ship <code>-O0</code>
5	<code>-O2</code> for production, <code>-Os</code> for embedded, <code>-O3</code> only if measured faster
6	<code>-Ofast</code> breaks IEEE 754 — verify your numerics before using it
7	<code>-g</code> is orthogonal to <code>-O</code> ; <code>-O2 -g</code> is a valid production build
8	Never ship <code>-march=native</code> ; use <code>-mtune=generic</code> or a v2/v3 baseline
9	Read the assembly. Godbolt makes it trivial
10	LTO for release builds; expect it to expose latent UB
11	<code>-ffunction-sections -fdata-sections -Wl,--gc-sections</code> for size
12	Split debug info (<code>objcopy --only-keep-debug</code>), don't just <code>strip</code>
13	<code>addr2line</code> + the archived debug file turns crash logs into bug reports
14	Never <code>ldd</code> an untrusted binary — use <code>readelf -d</code>
15	Cross-compiling needs a sysroot, not just a differently-named <code>gcc</code>

Practice

1. **Build your warning set.** Write a file that triggers 20 distinct warnings. Compile with `-Wall`, then `-Wall -Wextra`, then your full set. Record which flag caught which. Then run it through Clang and compare.
2. **Optimization benchmark.** Take a compute-heavy function of your own. Measure `-O0/-O1/-O2/-O3/-Os/-Ofast`, and record both time and `.text` size. Find a case where `-O3` is *slower* than `-O2` (hint: something i-cache bound, or with a hot loop the unroller bloats).
3. **-march portability failure.** Build with `-march=native`. Then check `/proc/cpuinfo`, pick an instruction set your CPU has, and confirm with `objdump -d | grep vpaddd` (or similar) that it's in the binary. Explain why shipping this would break on older hardware.
4. **Vectorization hunt.** Write three loops: one that vectorizes, one that doesn't because of a data dependency, one that doesn't because of possible aliasing. Use `-fopt-info-vec-missed` to see why. Fix the aliasing one with `restrict` (Module 2) and confirm it now vectorizes.
5. **Assembly reading drill.** For each of `if/else`, `switch` (dense and sparse), a `for` loop, a function call, and a struct return — dump `-O0` and `-O2` assembly and annotate every instruction. Then do the same on Godbolt for ARM64 and compare.
6. **LTO experiment.** Split a program into 5 `.c` files with cross-file calls. Measure binary size, `.text` size, symbol count, link time, and runtime with and without `-flto`. Then introduce a strict-aliasing violation and see whether LTO exposes it.
7. **Section GC.** Build a static library with 50 functions, link a program using 3. Measure with and without `-ffunction-sections -Wl,--gc-sections`. Use `-Wl,-Map=out.map` to see exactly what got included.
8. **Full binutils tour.** Take one binary and run every tool: `file`, `size`, `nm`, `readelf -h/-S/-d/-l`, `objdump -d/-S/-t`, `strings`, `ldd`. Write one sentence explaining what each told you.
9. **Crash → source.** Write a program that segfaults in a nested function. Enable core dumps (`ulimit -c unlimited`), get the faulting address from the core or from a signal handler with `backtrace()`, and use `addr2line` to find the exact line. Then repeat after splitting the debug info out, proving the workflow survives stripping.
10. **Cross-compile something.** Install `gcc-aarch64-linux-gnu` and `qemu-user-static`. Build a hello-world for ARM64, verify with `file` that it's an ARM binary, and run it under QEMU. Then write a CMake toolchain file and cross-build a small project with it.

Next: **Module 20 — Build Systems & Libraries**, covering real Makefiles (pattern rules and automatic dependency generation), CMake as the industry default, static vs shared libraries, symbol visibility, the ABI-vs-API distinction and versioning, and header design — the practical skills for making a codebase that more than one person can build.

20. Build Systems & Libraries

- Make (real Makefiles, pattern rules, dependency generation)
- **CMake** (the industry default), Meson, Ninja, Bazel; autotools legacy
- Static libraries `.a` vs shared `.so/.dll/.dylib`
- Position-independent code, symbol visibility, `-fvisibility=hidden`
- Symbol resolution, link order, undefined/duplicate symbols

- **ABI vs API**, soname, versioning, backward compatibility
- **pkg-config**, dependency management (Conan, vcpkg)
- Header design: what goes in **.h** vs **.c**, minimizing includes, forward declarations

Module 20 — Build Systems & Libraries

Real GNU Make 4.3 / CMake 4.4 / binutils output throughout. This module is about making a codebase that more than one person can build.

20.1 Make

A production Makefile

```
CC      ?= gcc
CSTD    := -std=c17
WARN    := -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wstrict-prototypes \
          -Wcast-qual -Wpointer-arith -Wwrite-strings -Wformat=2 \
          -Werror=implicit-function-declaration -Werror=return-type

OPT     ?= -O2 -g
CPPFLAGS := -Iinclude -MMD -MP           # ← automatic dependency generation
CFLAGS  := $(CSTD) $(WARN) $(OPT)

BUILD   := build
SRCS    := $(wildcard src/*.c)
OBJS    := $(SRCS:src/%.c=$(BUILD)/%.o)
DEPS    := $(OBJS:.o=.d)
TARGET  := $(BUILD)/app

.PHONY: all clean test debug release
all: $(TARGET)

$(TARGET): $(OBJS)
    @mkdir -p $(@D)
    $(CC) $(LDFLAGS) $^ $(LDLIBS) -o $@

$(BUILD)/%.o: src/%.c           # PATTERN RULE — one rule for every .c
    @mkdir -p $(@D)
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

debug: OPT := -O0 -g3 -fsanitize=address,undefined
debug: LDFLAGS += -fsanitize=address,undefined
debug: clean all

release: OPT := -O3 -DNDEBUG -flto
release: LDFLAGS += -flto
release: clean all

test: $(TARGET); ./$$(TARGET)
clean: ; $(RM) -r $(BUILD)
```

```
-include $(DEPS)           # ← pull in the generated dependencies
```

Automatic dependency generation — the part most Makefiles get wrong

`-MMD -MP` makes the compiler emit a `.d` file alongside each `.o`:

```
$ cat build/main.d
build/main.o: src/main.c include/mathlib.h include/strlib.h
include/mathlib.h:
include/strlib.h:
```

The first line is a real Make rule declaring the header dependencies. `-include $(DEPS)` pulls them in, and now Make knows the full dependency graph **without you maintaining it**.

Proof it works:

```
-- touch a .c: only that file rebuilds --
rebuild: src/strlib.c

-- touch a HEADER: everything that includes it rebuilds --
rebuild: src/main.c
rebuild: src/mathlib.c

-- touch nothing --
make: Nothing to be done for 'all'.
```

-MP matters too. It emits those empty `include/mathlib.h:` targets. Without them, *deleting* a header makes the build fail with **No rule to make target 'include/foo.h'** instead of just rebuilding. Every developer has hit this; the fix is one flag.

Use `-MMD` (not `-MD`) to skip system headers — they don't change and tracking them bloats the `.d` files.

Automatic variables and syntax

<code>\$@</code>	the target
<code>\$<</code>	the first prerequisite
<code>\$^</code>	all prerequisites (deduplicated)
<code>\$?</code>	prerequisites newer than the target
<code>\$(@D) / \$(@F)</code>	directory / file part of the target
<code>:=</code>	immediate assignment (evaluate now)
<code>=</code>	recursive (evaluate at use — beware)

<code>?=</code>	only if not already set — lets <code>CC=clang</code> <code>make</code> work
<code>+=</code>	append

Rules that save pain:

- **.PHONY:** for every target that isn't a file (`all`, `clean`, `test`, `install`).
 - **Recipes need real tabs**, not spaces. This is Make's most infamous wart.
 - Each recipe line runs in its own shell — use `\` or **.ONESHELL:** for multi-line logic.
 - `$(shell ...)` runs at parse time; `$$` escapes a shell `$`.
 - **Avoid recursive Make** (`make -C subdir`). It breaks parallelism and dependency tracking. Read "*Recursive Make Considered Harmful*" (Miller, 1997); use non-recursive Make or a real build system.
 - `make -j$(nproc)` for parallel builds; `make -n` to dry-run; `make -p` to dump the database; `make --debug=b` to see why something rebuilt.
-

20.2 CMake and the alternatives

CMake is the industry default — not because it's elegant, but because it's what everyone else uses and every IDE, package manager, and CI system supports it.

The same project in CMake

```
cmake_minimum_required(VERSION 3.20)
project(mathapp VERSION 1.2.3 LANGUAGES C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)           # -std=c17, not -std=gnu17
set(CMAKE_EXPORT_COMPILE_COMMANDS ON) # compile_commands.json for
clangd/clang-tidy

add_library(mathlib src/mathlib.c src/strlib.c)

target_include_directories(mathlib
    PUBLIC $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
           $<INSTALL_INTERFACE:include>)

target_compile_options(mathlib PRIVATE -Wall -Wextra -Wpedantic -Wshadow -
Wconversion)

set_target_properties(mathlib PROPERTIES
    VERSION    ${PROJECT_VERSION}      # libmathlib.so.1.2.3
    SOVERSION  ${PROJECT_VERSION_MAJOR} # libmathlib.so.1
    C_VISIBILITY_PRESET hidden
    POSITION_INDEPENDENT_CODE ON)

add_executable(app src/main.c)
target_link_libraries(app PRIVATE mathlib) # inherits includes and flags
transitively
```

```
include(GNUInstallDirs)
install(TARGETS mathlib app EXPORT mathlibTargets
        LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
        ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
        RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR})
install(DIRECTORY include/ DESTINATION ${CMAKE_INSTALL_INCLUDEDIR})

enable_testing()
add_test(NAME smoke COMMAND app)
```

```
$ cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
-- Build files have been written to: ../build

$ cmake --build build
[3/5] Linking C static library libmathlib.a
[5/5] Linking C executable app

$ ctest --test-dir build --output-on-failure
100% tests passed out of 1
```

Modern ("target-based") CMake — the only style worth learning

Everything is a property of a target, with a visibility keyword:

Keyword	Applies to
PRIVATE	this target only
INTERFACE	consumers only
PUBLIC	both

That's why `target_link_libraries(app PRIVATE mathlib)` is enough — `app` automatically gets `mathlib`'s `PUBLIC` include directories and compile definitions. No manual `include_directories()` anywhere.

Do not use the old commands: `include_directories()`, `link_directories()`, `add_definitions()`, `set(CMAKE_CXX_FLAGS ...)`. They're global and they don't compose. Every tutorial older than ~2018 will show you these. Use the `target_*` family exclusively.

Other essentials: out-of-source builds always (`-B build`), `CMAKE_BUILD_TYPE` (Debug/Release/RelWithDebInfo/MinSizeRel), generator expressions (`$<CONFIG:Debug>:...>`), `FetchContent` for dependencies, `find_package()` with config files, and `CMAKE_EXPORT_COMPILE_COMMANDS` (which powers clangd, clang-tidy, and every modern editor).

The landscape

System	Verdict
--------	---------

System	Verdict
CMake	The default. Ugly language, universal support. Learn it.
Ninja	Not a build system — a <i>backend</i> . Extremely fast, files are machine-generated. Use as CMake's generator.
Meson	Much nicer syntax, generates Ninja, excellent cross-compilation. Used by GNOME, systemd, Mesa. Genuinely better than CMake if you control the whole project.
Bazel	Google. Hermetic, reproducible, remote caching. Huge win at monorepo scale, heavy for anything smaller.
Autotools	<code>./configure && make && make install</code> . Legacy . Slow, opaque, M4 macros. You will encounter it; you shouldn't start new projects with it.
Plain Make	Fine for small projects. Portability across platforms is the limit.
xmake, premake, SCons, Waf	Niche.

Practical advice: use CMake with the Ninja generator. Learn Meson if you get the choice. Know how to build an autotools project because half of Linux still ships that way.

20.3 Static vs shared libraries

```
# static
gcc -c mathlib.c strlib.c
ar rcs libmath.a mathlib.o strlib.o

# shared
gcc -fPIC -c mathlib.c strlib.c
gcc -shared -Wl,-soname,libmath.so.1 mathlib.o strlib.o -o libmath.so.1.2.3
ln -sf libmath.so.1.2.3 libmath.so.1
ln -sf libmath.so.1      libmath.so

static binary : 16328 bytes
shared binary : 16176 bytes
libmath.so    : 15688 bytes

$ ar t libmath.a
mathlib.o
strlib.o      ← a .a is literally just an archive of .o files
```

	Static <code>.a</code>	Shared <code>.so</code> / <code>.dll</code> / <code>.dylib</code>
Linked at	build time (copied in)	load time (referenced)

	Static .a	Shared .so / .dll / .dylib
Binary size	larger	smaller
Deployment	single self-contained file	must ship the library too
Security fixes	rebuild everything	replace one file
Memory across processes	one copy each	shared
Startup	faster	dynamic-linker work
Cross-TU optimization	☒ (with LTO)	✗ across the boundary
Version conflicts	impossible	"DLL hell"

A .a pulls in only the .o files that are actually needed — which is the granularity problem `-ffunction-sections` (Module 19) solves.

Platform notes: macOS uses `.dylib` and `install_name` instead of `soname`; Windows uses `.dll` plus a `.lib` import library and requires explicit `__declspec(dllexport)/dllimport`.

20.4 PIC and symbol visibility

Position-independent code

Shared libraries can be loaded at any address, so all their code must be **position-independent** — hence `-fPIC` (or `-fPIE` for executables, which is the default on modern distros, giving you ASLR).

PIC accesses globals through the **GOT** (Global Offset Table) and calls through the **PLT** (Procedure Linkage Table), which costs an indirection. `-fno-plt` and `-Bsymbolic` reduce it for intra-library calls.

`-fvisibility=hidden` — measured

```
int public_a(int x);           /* intended API */
int public_b(int x);
int internal_helper1(int x);  /* implementation detail */
int internal_helper2(int x);
static int truly_private(int x);
```

```
default visibility           : 5 exported symbols
  internal_helper1 internal_helper2 public_a public_b use_all

-fvisibility=hidden          : 0 exported symbols

hidden + explicit API markers : 2 exported
  public_a public_b

sizes: default=15424 hidden+API=15256
```


By default every non-`static` symbol is exported. That means: your internal helpers are part of your ABI whether you meant them to be, users can call them, name collisions with other libraries become possible, the dynamic symbol table is bloated, and every call goes through the PLT.

The correct pattern:

```
/* mylib_export.h */
#ifdef _WIN32
#   ifdef MYLIB_BUILDING
#       define MYLIB_API __declspec(dllexport)
#   else
#       define MYLIB_API __declspec(dllimport)
#   endif
#elif defined(__GNUC__)
#   define MYLIB_API __attribute__((visibility("default")))
#else
#   define MYLIB_API
#endif
```

```
MYLIB_API int public_a(int x);    /* in the header */
```

Build with `-fvisibility=hidden` and mark only the API. CMake does this for you with `C_VISIBILITY_PRESET hidden` plus `generate_export_header()`.

Benefits: smaller binaries, faster load times, better optimization (the compiler knows nothing external calls a hidden function), no accidental ABI surface, no symbol collisions.

The alternative is a **version script** (`-Wl,--version-script=lib.map`), which also gives you symbol versioning — more powerful, more work.

20.5 Symbol resolution and link order

⚠ Static library link order matters

```
libA depends on libB.

-- WRONG order (-lB before -lA) --
la.c:(.text+0x9): undefined reference to `b_func'

-- RIGHT order (-lA then -lB) --
42
```

The linker processes arguments left to right, once each. When it reaches a static library it takes only the members that resolve currently-undefined symbols, then moves on. If `libA` introduces a new undefined symbol and `libB` was already processed, it's too late.

Rule: a library must appear *after* everything that uses it.

For circular dependencies:

```
gcc main.o -Wl,--start-group -lA -lB -Wl,--end-group -o prog
```

```
--start-group works: 42
```

(Or just list the library twice. `--start-group` is cleaner.)

Shared libraries don't have this problem — the dynamic linker resolves everything at load time.

Duplicate symbols

```
/usr/bin/ld: multiple definition of `shared_thing'; first defined here
/usr/bin/ld: multiple definition of `dup'; first defined here
```

The One Definition Rule from Module 1, enforced by the linker. Causes: a variable defined in a header (Module 3), a missing `static`, or the same file compiled twice.

The three errors you'll see, and what they mean

Error	Cause
undefined reference to 'X'	Missing <code>.c</code> / <code>-lfoo</code> , wrong link order, or a typo — check <code>nm -u</code>
multiple definition of 'X'	Defined twice — check <code>nm</code> on each <code>.o</code>
cannot find -lfoo	Library not on the <code>-L</code> path

`nm -u prog.o` lists what a file needs; `nm --defined-only lib.a` lists what a library provides. That pair solves almost every link error.

20.6 ABI vs API ⚠

- API** — what the *source code* sees: function names, signatures, types, macros. Break it and code fails to **compile**.
- ABI** — what the *machine code* sees: struct layout and size, calling conventions, symbol names, enum widths, alignment, vtable layout. Break it and already-compiled code **silently corrupts**.

A real ABI break, measured

```
/* v1.h — what the application compiled against */
struct Config { int width; int height; };
```

```
void show(const struct Config *c);

/* v2.c – a new library version. A field was INSERTED at the front. */
struct Config { int depth; int width; int height; };
```

The header change is API-compatible in the sense that old source still compiles. But:

```
-- compiled against v1, running against v1 --
lib sees: w=800 h=600

-- SAME BINARY, library replaced with v2 --
lib sees: d=800 w=600 h=-2058889728
```

The application was never recompiled. Its struct layout is baked into the machine code. width became depth, height became width, and height reads past the object.

No error, no crash, just wrong data — potentially a security bug.

What breaks the ABI

Breaks ABI	Safe
Adding, removing, or reordering struct members	Adding a new function
Changing a member's type or size	Adding members to a struct only ever allocated by the library (opaque!)
Changing a function's parameters or return type	Changing the <i>implementation</i>
Changing an enum's underlying type	Adding a value to the end of an enum (usually)
Changing alignment or packing	
Removing a symbol, or changing visibility	
Changing the calling convention	

This is the strongest argument for opaque structs (Module 11 §11.8). If callers can't see the layout, you can change it freely.

Soname and versioning

```
libmath.so      -> libmath.so.1      (dev symlink, used at build time)
libmath.so.1    -> libmath.so.1.2.3  (soname symlink, used at runtime)
libmath.so.1.2.3      (the real file)
```

```
$ readelf -d libmath.so.1.2.3 | grep SONAME
0x000000000000000e (SONAME) Library soname: [libmath.so.1]

$ readelf -d app_shared | grep NEEDED
0x0000000000000001 (NEEDED) Shared library: [libmath.so.1]
```

The application records the SONAME, not the filename. So:

- Bump the **soname** (**.so.1** → **.so.2**) on any ABI break. Both versions can then coexist and old binaries keep working.
- Bump only the **minor/patch** for compatible changes — old binaries pick up the fix automatically.

Semantic versioning for C libraries: MAJOR = ABI break, MINOR = API addition (ABI compatible), PATCH = bug fix.

Tools: **abi-compliance-checker** and **abidiff** (libabigail) compare two builds and report ABI differences — run them in CI if you ship a library.

Symbol versioning (**--version-script**) lets you keep multiple ABI versions of the same function in one **.so**. That's how glibc has maintained backward compatibility since 1997.

20.7 pkg-config and dependency management

```
Name: mathlib
Version: 1.2.3
Libs: -L${libdir} -lmath
Libs.private: -lm
Cflags: -I${includedir}
```

```
--cflags      : -I/usr/local/include
--libs        : -L/usr/local/lib -lmath
--libs --static : -L/usr/local/lib -lmath -lm
--modversion   : 1.2.3
--atleast-version=1.0 : yes
--atleast-version=2.0 : no

zlib here: -lz
```

```
gcc $(pkg-config --cflags --libs zlib) prog.c -o prog
```

Ship a .pc file with every library you publish. It's ~10 lines and it makes your library usable from Make, CMake, Meson, and autotools without anyone hardcoding paths.

In CMake:

```
find_package(PkgConfig REQUIRED)
pkg_check_modules(ZLIB REQUIRED IMPORTED_TARGET zlib)
target_link_libraries(app PRIVATE PkgConfig::ZLIB)
```

`Libs.private` and `Requires.private` are only emitted with `--static` — that's the distinction between "what my consumers need to link" and "what I need internally."

The package managers

	Notes
System packages (<code>apt</code> , <code>dnf</code> , <code>brew</code>)	Simplest, but versions are whatever the distro ships
Conan	Decentralized, binary caching, good CMake integration. The most C/C++-native option.
vcpkg	Microsoft, source-based, huge catalog, excellent CMake integration
CMake FetchContent	Zero infrastructure — clone and build a dependency at configure time. Great for small dep counts.
Git submodules / vendoring	Full control, reproducible, painful to update
Meson subprojects / wraps	Meson's built-in equivalent

C has no universal package manager, and that's a real weakness compared to Rust/Go/Python. Most projects vendor their dependencies or use system packages.

20.8 Header design

What goes where

Header (<code>.h</code>)	Implementation (<code>.c</code>)
Include guard	The <code>#include</code> of its own header first
Public type declarations, opaque typedefs	Private struct definitions
Function declarations	Function definitions
<code>#define</code> constants, public macros	<code>static</code> helpers
<code>static inline</code> for tiny hot functions	Global variable definitions
<code>extern</code> declarations of globals	

Never in a header: variable definitions (Module 3's `multiple definition`), `static` variables (each TU gets its own copy — the Module 3 trap), function definitions other than `static inline`, `using`-style pollution, or

unprefixed generic macros like `min/max/DEBUG`.

The rules

1. Self-contained. A header must compile on its own:

```
/* widget.h */
#ifndef WIDGET_H
#define WIDGET_H
#include <stddef.h>          /* it uses size_t, so it includes stddef */
...
#endif
```

Enforce it with a test that compiles each header alone.

2. Include what you use, and no more. Module 12 measured `<stdio.h>` at 812 lines. Every unnecessary include multiplies across every TU.

3. Forward-declare instead of including. If you only need a pointer or a reference:

```
/* GOOD */
struct Database;           /* incomplete type is enough */
int query(struct Database *db, const char *sql);

/* BAD */
#include "database.h"      /* pulls in everything it includes */
```

4. Include your own header first in the .c. This proves the header is self-contained.

5. Prefix everything. C has no namespaces: `ml_add`, `ML_MAX_SIZE`, `MLConfig`. Module 11 §11.6's enum rule generalizes to the whole API.

6. Opaque types for anything non-trivial (Module 11 §11.8) — ABI stability, encapsulation, and fewer header dependencies all at once.

7. extern "C" guards if C++ code will consume your header:

```
#ifdef __cplusplus
extern "C" {
#endif
... declarations ...
#ifdef __cplusplus
}
#endif
```

Include-order convention

```

/* widget.c */
#include "widget.h"          /* 1. own header — proves it's self-contained */

#include <stdio.h>            /* 2. system headers */
#include <stdlib.h>

#include "log.h"             /* 3. project headers */
#include "util.h"

```

Tools: `include-what-you-use` (IWYU) analyzes and reports unnecessary includes. Worth running once on any codebase over ~50 files.

Module 20 rules

#	Rule
1	<code>-MMD -MP</code> in <code>CPPFLAGS</code> + <code>-include \$(DEPS)</code> — never hand-maintain header dependencies
2	Pattern rules, <code>.PHONY</code> , <code>?</code> for overridable variables, tabs not spaces
3	Avoid recursive Make
4	CMake with Ninja; target-based only — never <code>include_directories()</code>
5	Always out-of-source builds; always <code>CMAKE_EXPORT_COMPILE_COMMANDS</code>
6	Static for self-contained deployment, shared for updatability and memory sharing
7	<code>-fPIC</code> for shared libraries; <code>-fvisibility=hidden</code> + explicit API markers
8	Static libraries link left-to-right — put them after their users
9	<code>--start-group/--end-group</code> for circular dependencies
10	API breaks fail to compile; ABI breaks silently corrupt
11	Bump the soname on any ABI break; opaque structs prevent most of them
12	Ship a <code>.pc</code> file with every library
13	Headers must be self-contained; forward-declare instead of including
14	Include your own header first in the <code>.c</code>
15	Prefix every public identifier — C has no namespaces

Practice

1. **Build the Makefile.** Write the §20.1 Makefile for a 5-file project. Prove dependency tracking works: touch a `.c`, touch a header, delete a header. Then remove `-MP` and confirm the deleted-header failure appears.

2. **Add targets.** Extend it with `install`, `uninstall`, `format` (clang-format), `tidy` (clang-tidy using `compile_commands.json`), `coverage`, and `analyze`. Make `make -j8` work correctly.
3. **CMake conversion.** Convert the same project to CMake with a library target, an executable, install rules, and `ctest`. Build with both Make and Ninja generators and compare build times. Then add a `Debug` build with sanitizers via a generator expression.
4. **Static vs shared, measured.** Build the same library both ways. Compare: binary size, startup time (`perf stat` or a loop of 1000 executions), and what `ldd` reports. Then change one function in the library and measure what has to be rebuilt in each case.
5. **Visibility audit.** Take a library with 20 functions of which 5 are public. Count exported symbols with default visibility, with `-fvisibility=hidden`, and with hidden + explicit markers. Measure the `.so` size and the dynamic symbol table size (`readelf -sD | wc -l`) each time.
6. **Link order failure.** Build three static libraries with a chain of dependencies. Find an ordering that fails. Fix it with correct ordering, then with `--start-group`. Then make the dependency circular and confirm only `--start-group` works.
7. **Break an ABI on purpose.** Reproduce the §20.6 experiment. Then try four more breaks: reorder members, widen a member, change a parameter type, change an enum's underlying type. For each, record whether it crashes, corrupts silently, or happens to work.
8. **Fix it with opacity.** Rewrite the same library with an opaque `Config` type plus accessor functions. Repeat every ABI break from exercise 7 and confirm none of them affect the old binary.
9. **Version a library properly.** Build `libfoo.so.1.0.0` with a soname, link a program, then ship `1.1.0` (compatible) and `2.0.0` (incompatible). Verify the old binary picks up 1.1.0 automatically and refuses 2.0.0. Install `abidiff` and confirm it detects the break.
10. **Header hygiene pass.** Take a real project (yours or an open-source one). For each header: is it self-contained? Compile each one alone and fix what fails. Then run `include-what-you-use` and measure the build-time improvement from removing unnecessary includes.

Next: **Module 21 — Debugging & Analysis**, the module that most changes your day-to-day effectiveness: GDB properly (not just `break` and `print`), core dumps and post-mortem debugging, Valgrind's full tool suite, sanitizers in depth, `strace/ltrace/perf, rr` for reverse debugging, and the static analyzers that find bugs before you run anything.

21. Debugging & Analysis

- **GDB/LLDB**: breakpoints, watchpoints, backtraces, inspecting memory, TUI, scripting
- Core dumps and post-mortem debugging
- **Valgrind** (memcheck, callgrind, helgrind, massif)
- **Sanitizers**: ASan, UBSan, TSan, MSan, LeakSanitizer — non-negotiable in modern C
- `strace, ltrace, perf, ftrace, bpftrace`
- `rr` (record & replay debugging)
- Static analysis: `clang-tidy, cppcheck, scan-build`, Coverity, PVS-Studio
- Formal-ish tools: Frama-C, CBMC

Module 21 — Debugging & Analysis

The module that most changes your day-to-day effectiveness. Real GDB 15.1 / Valgrind 3.22 / cppcheck 2.13 / clang-tidy (LLVM 22) output throughout.

21.1 GDB

Build for debugging

```
gcc -Og -g3 -fno-omit-frame-pointer prog.c -o prog
```

- **-g3** includes macro definitions (so `print MY_MACRO` works), not just line tables
- **-Og** optimizes without destroying debuggability — better than **-O0** for large programs
- **-fno-omit-frame-pointer** keeps backtraces reliable

⚠ At **-O2**, variables get "optimized out" and line numbers jump around. My first watchpoint attempt at **-Og** failed with `value has been optimized out` — that's normal, and it's why you keep a debug build.

Batch mode — how you actually use GDB in scripts and CI

```
gdb -q -batch \
  -ex 'break average' -ex 'run' \
  -ex 'info args' -ex 'print *rs[0]' -ex 'bt' \
  ./buggy
```

```
Breakpoint 1, average (rs=0x5555555592a0, n=5) at buggy.c:21
21      for (size_t i = 0; i < n; i++) sum += rs[i]->score;
rs = 0x5555555592a0
n = 5
$2 = {id = 0, name = "alice\000\000\000\000\000\000\000\000\000\000", score = 0}
#0  average (rs=0x5555555592a0, n=5) at buggy.c:21
#1  0x0000555555555425 in main (argc=1) at buggy.c:41
```

No interactive session required. This form is scriptable, reproducible, and pasteable into a bug report.

The commands that matter

Category	Commands
Run	<code>run [args]</code> , <code>start</code> (break at main), <code>attach PID</code> , <code>run < in.txt</code>
Breakpoints	<code>break FILE:LINE</code> , <code>break FUNC</code> , <code>break FUNC if COND</code> , <code>tbreak</code> (temporary), <code>info break</code> , <code>delete</code> , <code>disable</code>

Category	Commands
Stepping	<code>next/n</code> (over), <code>step/s</code> (into), <code>finish</code> (out), <code>until</code> , <code>continue/c</code> , <code>advance</code> <code>LINE</code>
Inspect	<code>print</code> <code>EXPR</code> , <code>print/x</code> , <code>ptype</code> , <code>info locals</code> , <code>info args</code> , <code>display</code> <code>EXPR</code>
Memory	<code>x/16xb</code> <code>ptr</code> , <code>x/8dw</code> <code>arr</code> , <code>x/s</code> <code>str</code> , <code>x/4i</code> <code>\$pc</code>
Stack	<code>bt</code> , <code>bt full</code> , <code>frame</code> <code>N</code> , <code>up</code> , <code>down</code> , <code>info</code> <code>frame</code>
Watchpoints	<code>watch</code> <code>EXPR</code> , <code>rwatch</code> , <code>awatch</code>
Threads	<code>info</code> <code>threads</code> , <code>thread</code> <code>N</code> , <code>thread</code> <code>apply</code> <code>all</code> <code>bt</code>
Data	<code>set</code> <code>var</code> <code>x</code> = <code>5</code> , <code>call</code> <code>func(args)</code> , <code>return</code> <code>VALUE</code>

Conditional breakpoints — the biggest productivity win

```
gdb -q -batch -ex 'break make if id == 3' -ex run -ex 'info args' -ex finish -ex
'print *$' ./buggy0
```

```
Breakpoint 1, make (id=3, name=0x555555556014 "dave", score=7) at buggy.c:9
id = 3
name = 0x555555556014 "dave"
Value returned is $1 = (Rec *) 0x555555559360
$2 = {id = 3, name = "dave", '\000' <repeats 11 times>, score = 7}
```

Stopping on iteration 10,000 by hitting `continue` 10,000 times is how beginners use GDB. `break f if i == 10000` is how professionals do.

Related: `ignore` `N` `COUNT` (skip the first `COUNT` hits), `finish` (run to return and show the value), `$` (last value).

Breakpoint command lists — printf debugging without recompiling

```
break make
commands
  silent
  printf "make(id=%d, name=%s, score=%.1f)\n", id, name, score
continue
end
run
```

```
make(id=0, name=alice, score=0.0)
make(id=1, name=bob, score=6.0)
make(id=2, name=carol, score=1.0)
make(id=3, name=dave, score=7.0)
make(id=4, name=eve, score=2.0)
```

This is instrumented tracing with zero source changes and zero rebuilds. Enormously useful when the build takes minutes, or when the bug only reproduces in a deployed binary.

Watchpoints — "who changed my variable?"

```
break 35
run
watch rs[0]->score
continue
```

```
Hardware watchpoint 2: rs[0]->score
Old value = <unreadable>
New value = 0
0x00005555555554bb in main (...) at buggy.c:36
```

The single best tool for "this field has the wrong value and I don't know who wrote it." x86 gives you **4 hardware watchpoints** (fast); beyond that GDB falls back to software watchpoints (single-stepping — thousands of times slower).

rwatch = on read, **awatch** = on read or write.

TUI and other frontends

```
gdb -tui ./prog      # or Ctrl-X A inside gdb
```

Split-screen source + command. **layout src**, **layout asm**, **layout regs**, **layout split**. Ctrl-X 2 cycles layouts.

Better options for daily work: **gdb-dashboard** (a single **.gdbinit** file), **pwndbg/GEF** (security-focused, excellent memory views), your IDE's DAP integration (VS Code, CLion), or **gdbgui** in a browser.

.gdbinit — set it up once

```
set history save on
set history size 10000
set print pretty on
set print array on
set pagination off
set confirm off
set disassembly-flavor intel

define bta
```

```
thread apply all backtrace full
end
```

Project-local `.gdbinit` files need `add-auto-load-safe-path` in `~/.gdbinit`.

Python scripting

GDB embeds Python. **Pretty-printers** are the high-value use:

```
class RecPrinter:
    def __init__(self, val): self.val = val
    def to_string(self):
        return "Rec(id=%d, %s, %.1f)" % (self.val['id'],
self.val['name'].string(), self.val['score'])

def lookup(val):
    if str(val.type.strip_typedefs()) == 'Rec': return RecPrinter(val)
    return None
gdb.pretty_printers.append(lookup)
```

Now `print *rec` shows `Rec(id=3, dave, 7.0)` instead of a raw struct dump. **Ship pretty-printers with your data structures** — it transforms debugging a custom container.

LLDB

The LLVM debugger; default on macOS. Same concepts, different syntax:

GDB	LLDB
<code>break f</code>	<code>breakpoint set -n f / b f</code>
<code>info locals</code>	<code>frame variable</code>
<code>x/16xb p</code>	<code>memory read -c 16 -f x -s 1 p</code>
<code>bt</code>	<code>bt</code>

LLDB has better C++ support and a cleaner Python API; GDB has better C and embedded support. Learn whichever your platform defaults to.

21.2 Core dumps and post-mortem debugging

Enabling cores

```
ulimit -c unlimited
sudo sysctl -w kernel.core_pattern=/tmp/core.%e.%p
# systemd systems: coredumpctl list / coredumpctl gdb PID
```

Post-mortem GDB on a real core

```
gdb -q -batch -ex bt -ex 'frame 1' -ex 'info args' -ex 'info registers rip rsp'
./crash core.crash.608
```

```
Core was generated by `/home/claude/c_dbg/crash boom'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x000055740df1d179 in level3 (p=0x0) at crash.c:3
#1  0x000055740df1d199 in level2 (p=0x0) at crash.c:4
#2  0x000055740df1d1ba in level1 (p=0x0) at crash.c:5
#3  0x000055740df1d208 in main (argc=2) at crash.c:9

p = 0x0
rip  0x55740df1d199 <level2+28>
rsp  0x7ffcb1d67ea0
```

Full stack, full arguments, exact line — from a program that already died. This is why cores matter in production.

Requirements: the **exact same binary** (or the split debug file from Module 19), and matching shared libraries. Archive both with every release.

Backtraces from a signal handler

When you can't get a core (containers, embedded, restricted environments):

```
#include <execinfo.h>
static void handler(int sig)
{
    void *frames[32];
    int n = backtrace(frames, 32);
    fprintf(stderr, "*** signal %d, %d frames ***\n", sig, n);
    backtrace_symbols_fd(frames, n, STDERR_FILENO); /* async-signal-safe */
    _exit(128 + sig);
}
```

```
*** caught signal 11, 9 frames ***
/home/claude/c_dbg/bt(+0x1221)[0x558170396221]
/lib/x86_64-linux-gnu/libc.so.6(+0x45330)[0x7fc2d9045330]
/home/claude/c_dbg/bt(main+0x26)[0x5581703962f5]
```

Build with **-rdynamic** to get function names. Feed the offsets to **addr2line -e prog -f -C -i** (Module 19) to get file:line.

⚠ **backtrace_symbols** calls **malloc** and is not **async-signal-safe**. Use **backtrace_symbols_fd**, which isn't. And with PIE, subtract the load base before **addr2line**.

For production: **libunwind** (more robust), **Google Breakpad/Crashpad**, or **Sentry**.

21.3 Valgrind

Valgrind runs your program on a **synthetic CPU**, instrumenting every instruction. No recompilation needed — that's its superpower and the source of its 20–50× slowdown.

memcheck — the default

```
valgrind -q --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog
```

```
==627== Invalid read of size 4
==627==    at 0x109455: main (buggy.c:43)
==627==   Address 0x4a7c0b0 is 0 bytes inside a block of size 32 free'd
==627==    at free (vgpreload_memcheck-amd64-linux.so)
==627==   by 0x109450: main (buggy.c:43)
==627==   Block was alloc'd at
==627==    at malloc (vgpreload_memcheck-amd64-linux.so)
==627==   by 0x10929D: make (buggy.c:9)
==627==   by 0x109368: main (buggy.c:36)
```

Three stack traces: where the bad access happened, where it was freed, and where it was allocated.

That's the complete story of a use-after-free.

--track-origins=yes — the killer feature

```
static int compute(void){ int x; return x * 2; }
```

```
==636== Conditional jump or move depends on uninitialised value(s)
==636==    at __printf_buffer (vfprintf-process-arg.c:58)
==636==   by 0x1091D9: main (uninit.c:4)
==636==   Uninitialised value was created by a stack allocation
==636==    at 0x109189: compute (uninit.c:3)      ← THE ORIGIN
```

Valgrind traced the garbage back to the exact stack allocation that produced it. Module 16's table showed uninitialized reads are caught by *neither* ASan nor UBSan — this is the tool that catches them (along with Clang's MSan).

Costs ~2× extra slowdown. Worth it.

The tool suite

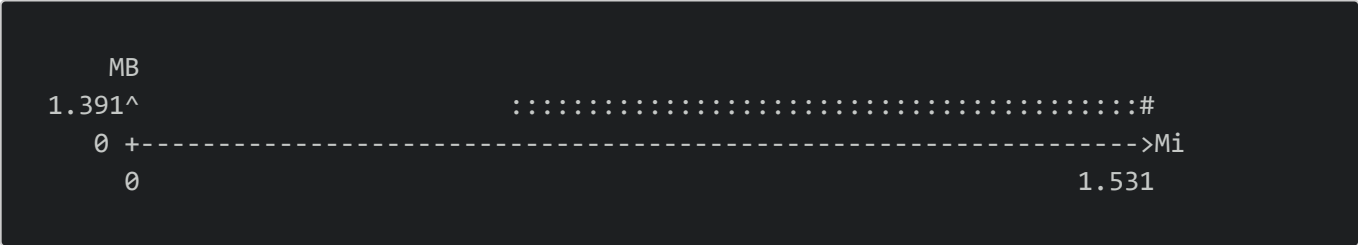
Tool	Purpose
memcheck (default)	Leaks, UAF, double free, uninitialized reads, invalid free, overlapping memcpy
helgrind	Data races, lock-order violations (deadlock potential)
drd	Alternative race detector, lower memory use
callgrind	Instruction-level profiling + call graph
cachegrind	Cache and branch-prediction simulation
massif	Heap profiler over time — peak usage and who allocated it
dhat	Heap access patterns: short-lived? read-only? over-sized?

callgrind:

```
Ir          file:function
5,601,209 (97.74%)  prof.c:main
 24,256 ( 0.42%)  dl-lookup.c:do_lookup_x
```

Deterministic instruction counts (unlike sampling profilers), so it's reproducible and great for regression tests. Visualize with **kcachegrind**.

massif:



ms_print ms.out gives you a peak-usage graph plus a tree of *which call sites* allocated the memory at peak. **The tool for "why does my server use 4 GB?"**

Practical notes

- **--error-exitcode=1** makes it usable in CI.
- **--gen-suppressions=all** writes suppression entries for known third-party noise.
- **--vgdb=yes --vgdb-error=1** lets GDB attach at the first error.
- Combine with **-O0/-O1** — at higher optimization the line info is less useful.
- ⚠ **Do not combine Valgrind with sanitizers.** They fight over memory management.

21.4 Sanitizers

Compile-time instrumentation. **Much faster than Valgrind, and they catch some things Valgrind can't** (stack overflows, global overflows, `_FORTIFY` interactions).

The ASan report

```
==703==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000040
READ of size 4 at 0x503000000040 thread T0
    #0 in main buggy.c:43
freed by thread T0 here:
    #0 in free
    #1 in main buggy.c:43
previously allocated by thread T0 here:
    #0 in malloc
    #1 in make buggy.c:9
    #2 in main buggy.c:36
```

Same three-trace structure as Valgrind, in a fraction of the time.

Measured overhead

```
baseline (no sanitizer)      0.001s
-fsanitize=undefined         0.002s   (~2x)
-fsanitize=address           0.014s   (~14x)
-fsanitize=address,undefined 0.016s
-fsanitize=thread            0.025s   (~25x)
valgrind memcheck            0.532s   (~530x)
```

(This workload is allocation-heavy, so ASan's overhead is at the high end of its usual 2–3×. The relative ordering is what matters: **UBSan is nearly free, ASan is cheap, Valgrind is expensive but needs no rebuild.**)

The suite

Sanitizer	Catches	Overhead	Notes
ASan	Heap/stack/global overflow, UAF, double free, UAR	2–3×	Includes LSan by default on Linux
LSan	Leaks	~free	Standalone via <code>-fsanitize=leak</code>
UBSan	Signed overflow, shifts, alignment, null, bounds, enum/bool range	~1.2×	Enable everywhere
TSan	Data races, lock-order inversions	5–15× + 5–10× memory	Module 17

Sanitizer	Catches	Overhead	Notes
MSan	Uninitialized reads	~3×	Clang only , needs all deps instrumented
CFI	Control-flow integrity violations	small	Clang, needs LTO

Incompatible pairs: ASan ⊕ TSan ⊕ MSan. Separate builds.

Runtime options

```
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1:strict_string_checks=1
:abort_on_error=1
UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1
TSAN_OPTIONS=halt_on_error=1:second_deadlock_stack=1
LSAN_OPTIONS=suppressions=lsan.supp
```

`detect_stack_use_after_return=1` is off by default and catches a real bug class (Module 9's dangling-stack pointers) — turn it on.

Which to use when

Sanitizers for your own code (needs a rebuild, fast enough for the whole test suite, best diagnostics for what they cover).

Valgrind for third-party binaries, uninitialized-memory hunting with `--track-origins`, heap profiling with massif, and deterministic profiling with callgrind.

Both in CI, on different jobs. They have different blind spots (Module 16's coverage table).

21.5 Tracing and profiling

`strace` — syscalls

```
strace -c -f ./prog          # summary table
strace -e trace=openat,read,write ./prog # filter
strace -f -o trace.log -p PID # attach, follow forks
strace -T -tt ./prog         # timestamps and durations
```

```
$ strace -c ./buggy
...
100.00  0.000000      0      34      1 total
```

`strace` answers questions the debugger can't: which file did it *actually* open? why did that `connect` fail? is it spinning on `futex`? where is it blocked?

```
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
write(1, "sorted by score:\n  alice  0.0\n  "..., 98)
```

First thing to run when a program mysteriously fails. The failing syscall and its `errno` are usually the whole answer.

`ltrace` does the same for library calls (`malloc`, `strcpy`) — less reliable with modern PLT hardening.

`perf` — the Linux profiler

```
perf stat ./prog                # cycles, instructions, IPC, cache misses,
                                # branch misses
perf record -g ./prog            # sample with call graphs
perf report                      # interactive browser
perf top                        # live, system-wide
perf record -e cache-misses ./prog # sample a specific hardware event
perf annotate                    # per-instruction attribution
```

Flame graphs are the standard visualization:

```
perf record -F 99 -g ./prog
perf script | stackcollapse-perf.pl | flamegraph.pl > out.svg
```

`perf stat` alone tells you enormous amounts — an IPC below ~1.0 means you're memory-bound, a high branch-miss rate means unpredictable control flow. (Module 25 covers this properly.)

Requires `perf_event_paranoid` permissions; often unavailable in containers (as here).

`ftrace` and `bpftrace`

ftrace — the kernel's built-in tracer, via `/sys/kernel/debug/tracing`. Function graphs of kernel execution.

bpftrace — eBPF, a scripting language for production tracing with near-zero overhead:

```
bpftrace -e 'uprobe:./prog:make { printf("make called with id=%d\n", arg0); }'
bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'
bpftrace -e 'uprobe:libc:malloc { @bytes = hist(arg0); }'
```

This is the modern production-debugging tool — attach to a running process, no restart, no instrumentation, no measurable slowdown. Brendan Gregg's *BPF Performance Tools* is the reference.

`rr` — record and replay

```
rr record ./prog
rr replay           # a GDB session where you can go BACKWARDS
```

Inside `rr replay`: `reverse-continue`, `reverse-step`, `reverse-next`, and **reverse watchpoints**.

This is a genuine superpower for hard bugs. You see corrupted memory; you set a watchpoint and `reverse-continue`; `rr` takes you directly to the write that corrupted it. No re-running, no guessing.

It also makes **non-deterministic bugs deterministic** — record once, replay identically forever. Perfect for race conditions and heisenbugs.

Overhead ~1.5–2×. Linux/x86-64 only, needs hardware performance counters (often unavailable in VMs and containers). When it works, it's the best debugging tool that exists.

21.6 Static analysis

Finds bugs **without running the program**, including on paths your tests never reach.

cppcheck

```
cppcheck --enable=all --inconclusive --std=c17 --quiet src/
```

```
analyze.c:9:11: style: Condition 'n=0' is always false [knownConditionTrueFalse]
    if (n = 0) return -1;
analyze.c:16:56: error: Memory leak: m [memleak]
```

Standalone, no build integration needed, low false-positive rate. Add `--suppress=missingIncludeSystem` to quiet the noise, and `--xml` for CI integration.

clang-tidy

```
clang-tidy src/*.c --checks='clang-analyzer-*,bugprone-*,cert-*' -p build/
```

```
analyze.c:8:5: warning: Call to function 'strcpy' is insecure as it does not
provide
    bounding of the memory buffer. CWE-119 [clang-analyzer-
security.insecureAPI strcpy]
analyze.c:9:11: warning: an assignment within an 'if' condition is bug-prone
    [bugprone-assignment-in-if-condition]
analyze.c:10:14: warning: Call to 'malloc' has an allocation size of 0 bytes
    [clang-analyzer-optin.portability.UnixAPI]
```

Note that last one — it followed `n = 0` through the (buggy) condition and deduced the allocation size. **That's path-sensitive analysis**, not pattern matching.

Uses `compile_commands.json` (Module 20's `CMAKE_EXPORT_COMPILE_COMMANDS`) so it sees your exact flags and includes. Check groups: `clang-analyzer-*` (deep), `bugprone-*`, `cert-*` (CERT C rules), `misc-*`, `performance-*`, `readability-*`.

Configure with a `.clang-tidy` file; `--fix` auto-applies many fixes.

The rest

Tool	Notes
scan-build	Wraps your build with the Clang Static Analyzer, produces an HTML report with path visualizations
GCC - fanalyzer	Built into GCC 10+. Interprocedural, catches double-free, UAF, leaks, null derefs. Free — just add the flag.
Coverity	Commercial, industry standard, free for open source (scan.coverity.com). Very low false-positive rate.
PVS-Studio	Commercial, excellent at copy-paste bugs. Free for open source.
Infer (Meta)	Separation-logic based, good at memory and null issues
CodeQL	GitHub. Query your code like a database; powers GitHub's security scanning

Use at least two. They have very different strengths, and static analysis is cheap.

Formal methods

Tool	What it does
Frama-C	ACSL contracts (pre/post-conditions, invariants) with deductive verification. Used in avionics and nuclear.
CBMC	Bounded model checker — exhaustively explores all paths up to a bound, produces a concrete counterexample
KLEE	Symbolic execution; generates test cases achieving high coverage
TIS-Interpreter	Interprets C detecting <i>all</i> UB, including cases no runtime tool catches
Astrée	Commercial sound static analyzer; proves absence of runtime errors. Airbus flight control.

These are for safety-critical code (DO-178C, ISO 26262) and for verifying small, critical functions — a parser, a crypto primitive, a state machine. **Not practical for a whole application**, but transformative for the 200 lines that must be correct.

21.7 The workflow

A debugging methodology

1. **Reproduce it reliably.** A bug you can't reproduce, you can't fix. Minimize the input (`creduce/delta` automate this).
2. **Run the sanitizers.** ASan+UBSan+LSan first, then TSan, then Valgrind with `--track-origins`. Most memory bugs die here in minutes.
3. **Run the static analyzers.** `-fanalyzer`, `cppcheck`, `clang-tidy`. Free, and they find things tests don't.
4. **Read the error carefully.** The three-trace ASan/Valgrind report usually is the answer.
5. **Bisect.** `git bisect run ./test.sh` finds the introducing commit automatically.
6. **Then reach for GDB** — with a conditional breakpoint or a watchpoint, not by stepping.
7. **rr for the truly hard ones.** Reverse-continue from the corruption to the corrupter.
8. **Write a regression test** before you fix it.

The CI matrix

```
# 1. Warnings as errors, two compilers
gcc -std=c17 -O2 $WARNINGS -Werror -c src/*.c
clang -std=c17 -O2 $WARNINGS -Werror -c src/*.c

# 2. ASan + UBSan + LSan
cmake -B b-asan -DCMAKE_C_FLAGS="-O1 -g -fsanitize=address,undefined"
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1 ctest --test-dir b-
asan

# 3. TSan (separate build)
cmake -B b-tsan -DCMAKE_C_FLAGS="-O1 -g -fsanitize=thread" && ctest --test-dir b-
tsan

# 4. Valgrind (catches what sanitizers don't)
valgrind --error-exitcode=1 --leak-check=full --track-origins=yes ./tests

# 5. Static analysis
gcc -fanalyzer -c src/*.c
cppcheck --enable=all --error-exitcode=1 src/
clang-tidy src/*.c -p build/ --warnings-as-errors='*'

# 6. Fuzzing (Module 22)
```

Every one of these has found bugs in every codebase I've seen it applied to.

Module 21 rules

#	Rule
1	Build debug with <code>-Og -g3 -fno-omit-frame-pointer</code> ; keep the debug build around
2	Conditional breakpoints and breakpoint command lists, not stepping 10,000 times
3	Watchpoints answer "who changed this variable?"

#	Rule
4	<code>gdb -batch -ex ...</code> is scriptable and pasteable into bug reports
5	Enable core dumps in production; archive the binary and its split debug file
6	<code>backtrace_symbols_fd</code> (not <code>backtrace_symbols</code>) in a signal handler; build <code>-rdynamic</code>
7	<code>valgrind --track-origins=yes</code> for uninitialized reads — sanitizers miss them
8	massif for "why is memory growing?", callgrind for deterministic profiles
9	ASan ~14×, UBSan ~2×, TSan ~25×, Valgrind ~500× on this workload
10	ASan ⊕ TSan ⊕ MSan — separate builds
11	<code>ASAN_OPTIONS=detect_stack_use_after_return=1</code> — off by default, catches real bugs
12	<code>strace</code> first when a program mysteriously fails
13	<code>rr</code> + reverse-continue for corruption bugs; it turns heisenbugs deterministic
14	<code>gcc -fanalyzer</code> is free — add it today
15	Run at least two static analyzers; they find different things
16	<code>git bisect run</code> before you open a debugger

Practice

1. **GDB fluency drill.** Take the `buggy.c` from this module. Without stepping once: (a) break on the 4th call to `make`, (b) print the whole `rs` array, (c) watch `rs[2]->score` and find every write, (d) call `average(rs, 3)` from the prompt, (e) change a score with `set var` and continue.
2. **Breakpoint tracing.** Write a `.gdb` script that traces every call to a function with its arguments and return value, without modifying the source. Compare the output to what you'd get from adding `printfs` — and compare the turnaround time.
3. **Pretty-printer.** Write a GDB Python pretty-printer for a linked list or hash table you've built. Make `print mylist` show the elements, not the raw pointers.
4. **Core dump round trip.** Enable cores, crash a program 5 frames deep, and extract from the core: the full backtrace, all locals in each frame, the faulting instruction, and the register state. Then strip the binary, split the debug info (Module 19), and prove the workflow still works.
5. **Sanitizer vs Valgrind matrix.** Take the 12-bug program from Module 16 §16.4 and add Valgrind and MSan columns to the table. Record what each tool catches, misses, and how long each takes.
6. **--track-origins hunt.** Write a program where an uninitialized value propagates through 3 function calls before being used. Confirm ASan and UBSan miss it, and that Valgrind names the exact origin.
7. **massif investigation.** Write a program with a slow memory leak in one of five allocation sites. Use massif to identify which one, without reading the source.

8. **strace diagnosis.** Write a program that fails for a non-obvious reason (permission denied on a config file, a missing library, a full disk via `/dev/full`). Have a colleague run it and diagnose it using only `strace`.
9. **Static analysis sweep.** Run `gcc -fanalyzer`, `cppcheck`, and `clang-tidy` on a real project. Count unique findings per tool and the overlap. Fix everything actionable, then add all three to CI with `--error-exitcode`.
10. **rr if you can install it.** Write a program that corrupts a struct field somewhere in a 1000-iteration loop. Find the corrupting write using `rr`: set a watchpoint on the field after the corruption, then `reverse-continue`. Time yourself against doing it with plain GDB.

Next: **Module 22 — Testing & Quality**, covering unit test frameworks for C, test doubles and link-time seams, coverage measurement, and **fuzzing** — which finds more real bugs in C than every other technique combined.

22. Testing & Quality

- Unit test frameworks: Unity, CMocka, Check, Criterion, greatest, `unit`
- Test doubles/mocks in C (link-time seams, function pointers)
- Code coverage: `gcov/lcov`, branch coverage
- **Fuzzing:** `libFuzzer`, `AFL++`, `honggfuzz` — huge in real C work
- Property-based and differential testing
- Regression suites, golden tests
- CI/CD: GitHub Actions, GitLab CI, building on multiple compilers/platforms

Module 22 — Testing & Quality

Real GCC 13.3 / `gcov` / `gcovr` output throughout. **Fuzzing (§22.4) finds more real bugs in C than every other technique combined** — if you take one thing from this module, take that.

22.1 Unit test frameworks

What a framework actually is

Here's a complete xUnit framework in ~40 lines. Understanding this demystifies every framework you'll meet:

```
static int mt_run = 0, mt_fail = 0, mt_asserts = 0;

#define TEST(name) static void name(void)

#define RUN_TEST(fn) do {
    mt_run++; int before = mt_fail; fn();
    printf("  %s %s\n", (mt_fail==before) ? "PASS" : "FAIL", #fn);
} while (0)

#define CHECK_EQ_INT(a,b) do { mt_asserts++;
```

```
        long a_=(long)(a), b_=(long)(b);                                \
        if (a_!=b_) { mt_fail++;                                         \
            printf("    %s:%d: %s == %s  (%ld != %ld)\n",                \
                __FILE__, __LINE__, #a, #b, a_, b_); return; } } while (0)

#define TEST_REPORT() (printf("\n%d tests, %d assertions, %d failures\n",    \
                               mt_run, mt_asserts, mt_fail), mt_fail ? 1 : 0)
```

Every technique from Module 12 is in there: `do{}while(0)`, `#x` stringification, `__FILE__`/`__LINE__`, and hygienic local names (`a_`).

```
running slist tests:
PASS test_new_is_empty
PASS test_push_pop
PASS test_growth_preserves_data
PASS test_null_safety
PASS test_out_of_range

5 tests, 2025 assertions, 0 failures
exit=0
```

And a failure:

```
    t_fail.c:5: slist_len(1) == 3  (2 != 3)
FAIL deliberately_broken
    t_fail.c:9: "hello" == "world"  ("hello" != "world")
FAIL str_and_float

2 tests, 2 assertions, 2 failures
exit=1          ← non-zero exit is what makes CI fail the build
```

Note the assertion messages print both the expressions and the values. That's the difference between a useful framework and `assert(x == y)`.

The real frameworks

Framework	Style	Notes
Unity	3 files, no deps	The embedded standard. Runs on bare metal. Pairs with CMock and Ceedling.
greatest	Single header, ~1000 lines	Zero dependencies, easy to drop in. Great default.
µnit (munit)	Single file	Parameterized tests, fixtures, seeded RNG for reproducibility
utest.h	Single header	Very minimal, auto-registers tests

Framework	Style	Notes
CMocka	Library	Best mocking support — <code>will_return/expect_value</code> , plus its own leak checker
Check	Library	Runs each test in a forked process , so a segfault fails one test instead of the run
Criterion	Library	Auto-registration, parallel execution, theories, nice output
cmocka/Unity + CTest	—	CMake's <code>add_test</code> wraps any of them

How to choose:

- **Embedded / no OS** → Unity
- **Need mocking** → CMocka
- **Crashes must not abort the suite** → Check (process isolation)
- **Just want something now** → greatest or utest.h (single header, copy and go)
- **Modern developer experience** → Criterion

Always run your tests under sanitizers:

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=address,undefined slist.c test_slist.c -o t && ./t
```

Half of what tests catch in C is memory corruption, and the sanitizers catch it *at the point of the bug* rather than three functions later.

What to test

The suite above covers the categories that matter:

Test	Category
<code>test_new_is_empty</code>	Initial state / boundary
<code>test_push_pop</code>	Happy path
<code>test_growth_preserves_data</code>	The realloc path (1000 elements forces many growths)
<code>test_null_safety</code>	Null arguments to every public function
<code>test_out_of_range</code>	Index 0, index n, index <code>(size_t)-1</code>

In C specifically, always test: NULL arguments, zero-length inputs, the maximum size, allocation failure, and every error return documented in the header (Module 15).

22.2 Test doubles and mocking

C has no runtime polymorphism, so mocking means **replacing a symbol** or **injecting a function pointer**. Three techniques, in increasing order of preference.

Technique 1 — Linker `--wrap`

```
gcc app.c net.c test.c -Wl,--wrap=net_send -o t
```

Every call to `net_send` is redirected to `__wrap_net_send`; the original is still reachable as `__real_net_send`.

```
static int mock_rc = 0, mock_calls = 0;
static char mock_host[64]; static size_t mock_len;

int __wrap_net_send(const char *host, const void *buf, size_t n)
{
    mock_calls++; mock_len = n;
    snprintf(mock_host, sizeof mock_host, "%s", host);
    return mock_rc;
}
```

```
linker --wrap mock:
called=1 host=example.com len=5 -> success path OK
injected failure -> upload returned -2 as designed
null host rejected without touching the network
```

Requires no source changes. But it's GNU-ld specific (also works with lld and gold), it can't wrap calls *within* the same translation unit (the compiler resolves them directly), and it's per-binary, not per-test.

Technique 2 — Weak symbols

```
__attribute__((weak))
int net_send(const char *host, const void *buf, size_t n) { /* real */ }
```

A strong definition in the test file overrides it:

```
int net_send(const char *h, const void *b, size_t n) { calls++; return 0; }
```

```
strong symbol won: mock called 1 time(s)
(without the test file, the weak real implementation is used)
[REAL weak net_send to prod.example, 1 bytes]
```

Simpler than `--wrap`, and it's the standard mechanism for **overridable defaults** in libraries (`__libc_start_main` hooks, embedded HAL stubs, `malloc` replacement). Same limitations otherwise.

Technique 3 — Dependency injection ☒ **preferred**

```
typedef int (*SendFn)(const char *host, const void *buf, size_t n, void *ctx);
typedef struct { SendFn send; void *ctx; } Transport;

static int upload2(const Transport *t, const char *host, const char *msg)
{
    if (!t || !t->send || !host || !msg) return -1;
    return t->send(host, msg, strlen(msg), t->ctx) == 0 ? 0 : -2;
}
```

Production passes the real implementation; tests pass a spy:

```
typedef struct { int calls; int rc; char last_host[64]; size_t last_len; } Spy;
static int spy_send(const char *h, const void *b, size_t n, void *c)
{ Spy *s = c; s->calls++; s->last_len = n; snprintf(s->last_host, 64, "%s", h);
  return s->rc; }
```

```
[REAL send to prod.example, 7 bytes]
spy: calls=1 host=test.local len=5
spy injected failure -> upload2 returned -2
no linker tricks, no rebuild, works with any test framework
```

This is Module 6's callback pattern with a `void *userdata` context, applied to testing. It's portable, per-test, works within a translation unit, composes with any framework, and — critically — **it makes the dependency visible in the type signature.**

Code that's hard to test is usually code with hidden dependencies. Injection isn't a testing trick; it's better design that happens to be testable.

CMocka's mocking (`will_return`, `expect_value`, `check_expected`) builds on the `--wrap` approach with a nicer API and automatic verification that expected calls happened.

22.3 Code coverage

```
gcc -O0 -g --coverage slist.c test_slist.c -o t_cov
./t_cov
gcov -b t_cov-slist.gcda
```

The measurement that matters

```
File 'slist.c'
Lines executed:100.00% of 20
Branches executed:100.00% of 24
Taken at least once:87.50% of 24
```

100% line coverage. 87.5% branch coverage. These are not the same thing, and the difference is where bugs hide.

```
$ gcovr --branches
File           Branches    Taken   Cover   Missing
slist.c         24          21     87%    15,23,28
```

Line 15 is:

```
    if (!p) return -1;
branch 0 taken 0% (fallthrough)    ← the realloc-failure path NEVER ran
```

The untested branches are the error paths — exactly the ones that are hardest to reach and most likely to be wrong. Module 15's fault-injection technique exists precisely to reach them.

The annotated source

```
function slist_push called 1012 returned 100% blocks executed 91%
1012: 10:int slist_push(SList *l, int v){
1012: 11:    if (!l) return -1;
branch 0 taken 1% (fallthrough)
1011: 12:    if (l->len == l->cap) {
branch 0 taken 1% (fallthrough)
13: 13:        size_t nc = l->cap ? l->cap*2 : 4;
13*: 15:        if (!p) return -1;
branch 0 taken 0% (fallthrough)    ← ← ←
```

Execution counts per line, hit rates per branch. * marks a line with an unexecuted branch.

Tools and interpretation

Tool	Purpose
<code>gcov -b -c</code>	Branch and call coverage, per file
<code>lcov + genhtml</code>	HTML reports with line highlighting
<code>gcovr</code>	Modern alternative; text/HTML/XML/JSON/Cobertura output for CI
<code>llvm-cov + llvm-profdata</code>	Clang's equivalent (<code>-fprofile-instr-generate -fcoverage-mapping</code>)

Tool	Purpose
Codecov / Coveralls	CI dashboards with per-PR deltas

Coverage types, weakest to strongest: statement → **branch** → condition → MC/DC (required by DO-178C Level A) → path (combinatorially infeasible).

How to think about coverage

- **Coverage is a floor, not a goal.** 100% coverage proves every line *ran*, not that it's correct. A test suite that executes everything and asserts nothing scores 100%.
- **Uncovered code is definitely untested.** That's the actionable signal.
- **Target branch coverage**, not line coverage.
- **Chase the uncovered error paths** with fault injection.
- Enforce a floor in CI (`gcovr --fail-under-branch 80`) and **ratchet it upward**, never down.
- ⚠ **Never ship a --coverage build** — it's slow and writes `.gcda` files at exit.

22.4 Fuzzing ⚠

This is the highest-value testing technique in C. OSS-Fuzz has found tens of thousands of bugs in critical infrastructure — OpenSSL, SQLite, curl, FFmpeg, the Linux kernel — most of which had extensive hand-written test suites.

The harness — one function, every fuzzer

```
#include <stdint.h>
#include <stddef.h>

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
{
    uint8_t out[64];
    parse_tlv(data, size, out, sizeof out);
    return 0;
}
```

That's the entire integration. **libFuzzer**, **AFL++**, **honggfuzz**, and **OSS-Fuzz** all use this exact signature.

Finding real bugs

A deliberately buggy TLV parser (`[type:1][len:2 BE][value:len]`) with three bugs: no input-length check, no output-length check, and a heap OOB write.

```
=== BUGGY parser under a naive RANDOM fuzzer + ASan ===
crashed: ERROR: AddressSanitizer: stack-buffer-overflow
(first iteration)

=== FIXED parser: 2,000,000 random inputs ===
```

```
survived 2000000 random inputs
exit=0 (clean under ASan + UBSan)
```

The fix:

```
if (len > n - 3) return -3;    /* input bound */
if (len > outcap) return -4;  /* output bound */
```

Even a **purely random** fuzzer found the bug immediately. Real fuzzers are **coverage-guided** — they instrument the binary, keep inputs that reach new code, and mutate those. That's orders of magnitude more effective at reaching deep states behind checksums, magic numbers, and nested structures.

The real fuzzers

```
# libFuzzer — in-process, fastest, part of Clang
clang -fsanitize=fuzzer,address,undefined parser.c harness.c -o fz
./fz corpus/ -max_total_time=300 -jobs=8

# AFL++ — out-of-process, best mutation engine, huge feature set
afl-clang-fast harness.c parser.c -o fz      # or afl-clang-lto for better
coverage
afl-fuzz -i seeds/ -o findings/ -- ./fz @@

# honggfuzz — good hardware-counter-based feedback
hfuzz-clang harness.c parser.c -o fz
honggfuzz -i seeds/ -- ./fz

# OSS-Fuzz — Google runs the same harness continuously, for free, on OSS projects
```

Getting it right

1. **Always fuzz with sanitizers on.** ASan+UBSan. Without them, the fuzzer only finds crashes; with them, it finds every memory bug and every UB (Module 16).
2. **Seed the corpus** with real, valid inputs. A fuzzer starting from "" may take hours to guess a file magic number that a seed gives it instantly.
3. **Use a dictionary** (`-dict=`) with your format's keywords, magic bytes, and field names.
4. **Minimize crashers** — `./fz -minimize_crash=1 crash-file` reduces a 4 KB input to the 6 bytes that matter.
5. **Keep the corpus in git** and re-run it as a regression test on every commit. This is the single cheapest way to keep fuzzing value.
6. **Make it deterministic** — no time, no randomness, no network in the harness.
7. **Fuzz the parsing boundary:** file formats, network protocols, config parsers, decompressors, image/font/media decoders, anything reachable from untrusted input.
8. **Structure-aware fuzzing** (libprotobuf-mutator, FuzzTest) when raw bytes rarely produce valid inputs.

If your C project parses anything an attacker can supply, and it isn't fuzzed, it has bugs. That's not rhetoric — it's the empirical finding of the last decade.

22.5 Property-based and differential testing

Property-based testing

Instead of "push(1), assert len == 1," assert **invariants over random operation sequences**:

```
for (int trial=0; trial<2000; trial++) {
    Slist *l = slist_new();
    size_t n = rnd() % 50;
    for (size_t i=0; i<n; i++) { ref[i]=rand16(); slist_push(l,ref[i]); }

    CHECK(slist_len(l) == n);
    for (size_t i=n; i-- > 0; ) {          /* PROPERTY: LIFO ordering */
        int v; slist_pop(l,&v);
        CHECK(v == ref[i]);
    }
    CHECK(slist_len(l)==0 && slist_pop(l,&d)==-1); /* PROPERTY: emptied == new
*/
}
```

property-based (2000 random sequences): PASS

Good properties in C: round-trips (`decode(encode(x)) == x`), invariants (a sorted array stays sorted; `len <= cap`), idempotence, commutativity, and "never crashes / never leaks" (which sanitizers verify for free).

C libraries: **theft**, **RapidCheck** (C++ but usable), or just hand-rolled loops as above.

Differential testing

Compare your implementation against a **trivially-correct reference model**:

```
typedef struct { int a[300000]; size_t n; } Model; /* obviously correct */
static void m_push(Model *m,int v){ if(m->n<300000) m->a[m->n++]=v; }
static int m_pop (Model *m,int *o){ if(!m->n) return -1; *o=m->a[--m->n]; return
0; }

for (int op=0; op<200000; op++) {
    if (rnd()%3 < 2) { slist_push(l,v); m_push(&m,v); }
    else { r1=slist_pop(l,&v1); r2=m_pop(&m,&v2);
        CHECK(r1==r2 && (r1!=0 || v1==v2)); }
    CHECK(slist_len(l) == m.n);
}
```

```
differential vs model (200k ops):      PASS
```

A useful thing happened while writing this. My first model capped at 4096 elements while the real list grows unbounded:

```
FAIL: len diverged at op 11962
```

The test found a divergence — and the bug was **in the reference model**, not the implementation. That's differential testing working exactly as intended: it doesn't tell you which side is wrong, it tells you they disagree.

⚠️ **A second lesson from that run:** the FAIL message didn't print at all until I added `setvbuf(stdout, NULL, _IONBF, 0)`. LeakSanitizer's abort discarded the buffered stdout — **exactly the Module 13 §13.6 bug**. Always run test binaries unbuffered.

Real-world differential testing: compare an optimized implementation against a naive one, a new version against the old, your parser against a reference parser, or two independent libraries against each other. `csmith` + differential testing is how compiler bugs get found.

22.6 Regression suites and golden tests

Regression tests

Every bug fix gets a test that fails before the fix and passes after.

This is the single most valuable testing discipline. It prevents the same bug returning, documents the bug's exact shape, and builds a suite anchored in reality rather than imagination.

Name them after the issue: `test_issue_142_overflow_on_empty_input`.

Fuzzer crashers are free regression tests — commit the minimized input to the corpus and replay it every build.

Golden / snapshot tests

Compare output against a stored expected file:

```
./mytool --format json input.txt > actual.json
diff -u expected/output.json actual.json || exit 1
```

```
test-golden:
  @for t in tests/cases/*.in; do \
    ./mytool < $$t > /tmp/out; \
    diff -u $${t%.in}.expected /tmp/out || exit 1; \
```



```
done
@echo "all golden tests pass"

update-golden:
@for t in tests/cases/*.in; do ./mytool < $$t > $$t%.in.expected; done
```

Excellent for: compilers, formatters, serializers, report generators, CLI tools — anything with complex output where hand-writing assertions is impractical.

⚠ **The failure mode:** blindly running `make update-golden` when a test fails. That converts every regression into an "accepted" change. **Review the diff.**

Related: `expect/pexpect` for interactive programs, and CLI testing tools like `bats`, `cram`, or `lit`.

22.7 CI/CD

The matrix that catches everything

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  build-test:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, macos-latest, windows-latest]
        compiler: [gcc, clang]
        std: [c17, c2x]
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/checkout@v4
      - run: cmake -B build -DCMAKE_C_COMPILER=${ matrix.compiler } \
        -DCMAKE_C_STANDARD=${ matrix.std } -DCMAKE_BUILD_TYPE=Release
      - run: cmake --build build
      - run: ctest --test-dir build --output-on-failure

  sanitizers:
    strategy:
      matrix:
        san: [address, undefined, thread]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: cmake -B b -DCMAKE_C_FLAGS="-O1 -g -fsanitize=${ matrix.san }"
      - run: cmake --build b
      - env:
          ASAN_OPTIONS:
            detect_leaks=1:detect_stack_use_after_return=1:abort_on_error=1
```

```

UBSAN_OPTIONS: print_stacktrace=1:halt_on_error=1
run: ctest --test-dir b --output-on-failure

valgrind:
  runs-on: ubuntu-latest
  steps: [...]
    - run: valgrind --error-exitcode=1 --leak-check=full --track-origins=yes
./b/tests

static-analysis:
  steps:
    - run: gcc -fanalyzer -c src/*.c
    - run: cppcheck --enable=all --error-exitcode=1 src/
    - run: clang-tidy src/*.c -p build/ --warnings-as-errors='*'

coverage:
  steps:
    - run: cmake -B bc -DCMAKE_C_FLAGS="--coverage -O0 -g" && cmake --build bc
    - run: ctest --test-dir bc
    - run: gcovr --fail-under-branch 80 --xml-pretty -o coverage.xml
    - uses: codecov/codecov-action@v4

fuzz:
  steps:
    - run: clang -fsanitize=fuzzer,address,undefined fuzz/*.c src/*.c -o fz
    - run: ./fz corpus/ -runs=100000 # regression: replay the committed
corpus

```

The principles

Principle	Why
Multiple compilers	GCC and Clang catch different bugs (Module 19)
Multiple platforms	Windows finds long -size and text-mode bugs; macOS finds different libc behavior
Multiple standards	-std=c17 and -std=c2x
fail-fast: false	See <i>all</i> failures, not just the first
Sanitizers in separate jobs	ASan ⊕ TSan ⊕ MSan (Module 21)
Fast feedback first	Compile+test in 2 minutes; fuzzing and Valgrind can take longer
Cache dependencies	actions/cache on the build directory
Coverage ratchet	--fail-under-branch that only goes up
Nightly deep runs	Long fuzzing sessions, full Valgrind, cross-compilation matrices

GitLab CI, Jenkins, Buildkite, and CircleCI all express the same matrix; the concepts transfer directly.

Also worth adding: `clang-format --dry-run --Werror` (formatting), a license check, an ABI check with `abidiff` (Module 20) if you ship a library, and a build-time regression alarm.

Module 22 rules

#	Rule
1	A test framework is ~40 lines — pick one and start, don't shop
2	Assertions must print both the expressions and the values
3	Always run tests under ASan+UBSan; run test binaries unbuffered
4	Test NULL args, zero length, max size, allocation failure, and every documented error return
5	Prefer dependency injection over <code>--wrap</code> and weak symbols
6	Hard-to-test code has hidden dependencies — fix the design, not the test
7	Branch coverage, not line coverage — 100% lines was only 87.5% branches here
8	The uncovered branches are the error paths; reach them with fault injection
9	Coverage is a floor, not a goal
10	Fuzz anything that parses untrusted input. Always with sanitizers on
11	Seed the corpus, use a dictionary, minimize crashers, commit the corpus
12	Properties over examples; differential tests against a simple model
13	Every bug fix gets a regression test named after the issue
14	Review golden-test diffs; never blindly regenerate
15	CI matrix: 2+ compilers × 3 platforms × sanitizers × Valgrind × static analysis × fuzz

Practice

1. **Build the framework.** Write `minitest.h` from §22.1 with `CHECK`, `CHECK_EQ_INT`, `CHECK_EQ_STR`, `CHECK_NEAR`, and a `SKIP` macro. Add setup/teardown support. Then write a 15-test suite for a data structure you built in an earlier module.
2. **Framework comparison.** Port that same suite to greatest and to Unity. Compare: lines of boilerplate, failure output quality, and how each handles a segfault mid-suite.
3. **All three mock techniques.** Take a function that calls `time()`, `malloc()`, and a network function. Mock each with `--wrap`, with weak symbols, and with injection. Then measure which lets you write the most tests with the least setup.
4. **Fault injection.** Add an injectable allocator (`static void *(*alloc_fn)(size_t) = malloc;`) to a library. Write a test that makes the Nth allocation fail for $N = 1..20$ and asserts LSan reports zero leaks each time. Measure the branch-coverage improvement.

5. **Coverage hunt.** Build any project with `--coverage`, run the suite, and generate a branch-coverage report. Find the 5 least-covered functions. Write tests for the uncovered branches. Report the before/after numbers.
6. **Fuzz a real parser.** Write a parser for something (INI, CSV with quoting, a binary header, base64). Write the `LLVMFuzzerTestOneInput` harness. Run it under ASan+UBSan with a random fuzzer for 10 million iterations. Fix everything it finds.
7. **Fuzz with a real fuzzer.** Install clang and libFuzzer (or AFL++). Fuzz the same parser for 10 minutes with a seed corpus and a dictionary. Compare the number of unique crashes and the coverage reached against your random fuzzer.
8. **Property tests.** For a hash table you write: encode/decode round-trip, "inserting N distinct keys gives count N", "lookup after insert always finds it", "delete then lookup always misses", "resizing preserves all entries." Run 10,000 random operation sequences under ASan.
9. **Differential test.** Implement a sorted-array set and a linked-list set with the same API. Fuzz both with the same random operation sequence and assert identical observable behavior at every step. Then deliberately introduce a bug in one and confirm the test catches it.
10. **Full CI pipeline.** Set up GitHub Actions for a project with: the compiler/platform matrix, three sanitizer jobs, Valgrind, three static analyzers, coverage with a ratcheting threshold, and a fuzz-corpus regression run. Make one job fail on purpose and verify you get a useful report.

Next: **Module 23 — Secure Coding**, which pulls the vulnerability classes from Modules 8, 10, and 16 into a systematic security framework: the CWE Top 25 as they appear in C, CERT C and MISRA, exploit mechanics (so you understand *why* the mitigations exist), hardening flags, and secure API design.

23. Secure Coding

- Buffer overflows (stack & heap), off-by-one, `strcpy` family abuse
- Format string vulnerabilities
- Integer overflow/underflow, signed/unsigned confusion, truncation
- Use-after-free, double free, type confusion
- TOCTOU races, unsafe temp files, `system()`/command injection
- Input validation, canonicalization
- **CERT C, MISRA C:2012/2023**, CWE Top 25
- Hardening: `-fstack-protector-strong`, `_FORTIFY_SOURCE=3`, PIE, RELRO, `-D_GLIBCXX_ASSERTIONS`, CFI
- Constant-time code for crypto, avoiding compiler-removed memset
- Threat modeling, secure API design (bounds-carrying interfaces)

Module 23 — Secure Coding

This module explains vulnerability *mechanisms* at the depth needed to write code that resists them, and demonstrates the defenses working. Real GCC 13.3 / glibc 2.39 output throughout.

The framing that matters: in C, most security bugs are the memory-safety and integer bugs from Modules 2, 8, 9, and 10 — reachable from attacker-controlled input. Security isn't a separate discipline bolted on; it's correctness on the paths an adversary chooses.

23.1 Buffer overflows

Why they matter — the stack frame

```
STACK FRAME LAYOUT:
&buf[0]      = 0x7fffe4e0f280
saved rbp    = 0x7fffe4e0f2a0    (+32 bytes from buf)
return addr  = 0x7fffe4e0f2a8    (+40 bytes from buf) <-- the target
return addr value = 0x55e46fdf828b
```

A 16-byte local buffer sits **40 bytes below the return address**. Writing past the end of the buffer reaches the saved frame pointer, then the return address. Controlling the return address controls where execution resumes — that is the entire mechanism, and it's why every buffer overflow is treated as potentially exploitable.

Heap overflows work differently but are equally serious: they corrupt the allocator's inline chunk metadata (Module 10 §10.5), so the *next malloc* can be steered to return an attacker-influenced address.

The *strcpy* family — CWE-120/121/122

Module 8 banned these. Here's the security framing:

Function	Why it's unsafe
<i>gets</i>	No size parameter at all. Removed from C11 . The Morris Worm (1988).
<i>strcpy, strcat</i>	Write until they find a NUL in the <i>source</i>
<i>sprintf, vsprintf</i>	Unbounded formatted output
<i>scanf("%s")</i>	Unbounded read (Module 13)
<i>strncpy</i>	Doesn't reliably terminate; not a "safe strcpy"
<i>strncat</i>	The <i>n</i> is bytes <i>appended</i> , not the buffer size
<i>alloca</i> , VLAs	Unchecked stack allocation from a length (Modules 7, 10)

Off-by-one — CWE-193

The most common overflow, and often exploitable:

```
char buf[N];
for (i = 0; i <= N; i++) buf[i] = ...;    /* writes buf[N] */
buf[strlen(s)] = '\0';                    /* if s is exactly N chars, writes
buf[N] */
```

```
strncat(dst, src, sizeof dst - strlen(dst)); /* forgot the -1 for the NUL */
memcpy(dst, src, sizeof dst);                /* if dst must also hold a NUL */
```

A single-byte overflow into the saved frame pointer or a heap chunk header has been the basis of numerous real exploits ("off-by-one-byte" / "poison NUL byte").

The defenses, measured

The same `strcpy` of a 64-byte string into a 16-byte buffer:

Build	Result
<code>-fno-stack-protector - U_FORTIFY_SOURCE</code>	<code>rc=139 Segmentation fault</code> — silent corruption, then a crash
<code>-fstack-protector-strong</code>	<code>rc=134 *** stack smashing detected ***: terminated</code>
<code>-D_FORTIFY_SOURCE=3</code>	<code>rc=134 *** buffer overflow detected ***: terminated</code>
<code>-fsanitize=address</code>	<code>ERROR: AddressSanitizer: stack-buffer-overflow</code> (with the exact line)

All three defenses turn "corrupt memory and continue" into "abort immediately." That converts a potential code-execution vulnerability into a denial of service — a large improvement.

Note that `_FORTIFY_SOURCE` catches it *at the call*, before any corruption, whereas the stack canary only detects it *at function return*, after the damage is done. Use both.

23.2 Format string vulnerabilities — CWE-134

```
printf(user_input);      /* VULNERABLE */
printf("%s", user_input); /* safe */
```

```
VULNERABLE - printf(user_input):
0x7ffc11f8d9a0 (nil) (nil)      ← leaked stack/register contents

SAFE - printf("%s", user_input):
%p %p %p                      ← printed literally
```

`%p/%s/%x` **read** arbitrary values from the argument registers and stack — an information leak that defeats ASLR and can dump secrets or stack canaries.

`%n` **writes**:

```
hello world
%n stored 5 into the variable
```

With a controlled format string, an attacker chooses both the value written (via field widths) and the target address (via stack-positional arguments) — **an arbitrary memory write primitive**.

Defenses

```
$ gcc -Wformat=2 -Wformat-security -c fmt.c
fmt.c:7:5: warning: format not a string literal and no format arguments [-Wformat-security]
```

```
$ gcc -O2 -D_FORTIFY_SOURCE=3 fmtdyn.c && ./fmtdyn
*** %n in writable segment detected ***
Aborted
```

glibc's `_FORTIFY_SOURCE` blocks `%n` when the format string lives in writable memory — which is exactly the attacker-controlled case.

Rules:

1. **Never pass non-literal data as a format string.** If you need a dynamic format, `-Wformat-nonliteral` will flag it; justify it explicitly.
2. `-Wformat=2 -Werror=format-security` in every build.
3. `__attribute__((format(printf, m, n)))` on every wrapper (Module 6).
4. This applies to `syslog`, `err/warn`, and every logging function too.

23.3 Integer bugs — CWE-190/191/197/195

The classic chain: overflow → undersized allocation → heap overflow

```
void *bad_alloc_array(size_t n, size_t elemsize)
{ return malloc(n * elemsize); }          /* n*elemsize can WRAP */
```

```
n = 4611686018427387905, elemsize = 4
n * elemsize (wrapped) = 4 bytes <-- TINY
bad_alloc_array -> SUCCEEDED (a 4-byte buffer for 4611686018427387905
elements!)
good_alloc_array -> NULL (rejected)
calloc(n, es)    -> NULL (rejected)
```

The allocation *succeeds* with 4 bytes. The subsequent loop writing `n` elements is an unbounded heap overflow. **This exact pattern is one of the most common serious CVEs in C.**

Fixes:

```

if (elemsize && n > SIZE_MAX / elemsize) return NULL; /* explicit */
p = calloc(n, elemsize); /* calloc checks
internally */
if (ckd_mul(&total, n, elemsize)) return NULL; /* C23 <stdckdint.h> */
if (__builtin_mul_overflow(n, elemsize, &total)) return NULL;

```

Signed/unsigned confusion — the length-check bypass

```

static int copy_bad(const char *src, int len) /* len from untrusted input */
{
    if (len > (int)sizeof dest) return -1; /* looks like a bounds check...
*/
    memcpy(dest, src, (size_t)len); /* ...but len can be NEGATIVE
*/
}

```

```

dest is 64 bytes. Attacker supplies len = -1.
copy_bad: 'len > (int)sizeof dest' -> -1 > 64 is FALSE -> check PASSES
        then memcpy(..., (size_t)-1) = 18446744073709551615 bytes
copy_good: rejects len < 0 first -> returns -1

```

And the mirror-image trap from Module 2:

```

int len = -1; size_t cap = 64;
actual result of (len < cap): FALSE ← -1 became 18446744073709551615
-> a 'len < capacity' check silently accepts every negative value

```

Rule: check for negative *first*, then convert to `size_t`.

```

if (len < 0 || (size_t)len > cap) return -1;

```

Truncation

```

size_t 0x100000001 -> unsigned short 1, int 1

```

A 4-gigabyte-plus length truncated into a 16-bit field becomes 1. Allocate 1 byte, copy 4 GB. Watch every place a length crosses a type boundary — `size_t` → `int`, `off_t` → `long`, network `uint32_t` → `int`.

Defenses: `-Wconversion` `-Wsign-conversion`, UBSan (`-fsanitize=integer` in Clang catches even *defined* unsigned wraparound), and `<stdckdint.h>`.

23.4 Lifetime bugs — CWE-416/415/843

From Module 10's taxonomy, with the security framing:

Bug	Why it's exploitable
Use-after-free (CWE-416)	The allocator reuses the block. An attacker who can trigger a well-timed allocation controls the contents — including function pointers in a freed object.
Double free (CWE-415)	Corrupts the free-list, letting a later <code>malloc</code> return an attacker-chosen address.
Type confusion (CWE-843)	Reading a union member that wasn't written, or casting between incompatible struct types, reinterprets attacker data as a pointer. Module 11's tagged-union discipline prevents this.
Dangling after <code>realloc</code>	Module 9 §9.8 — every alias becomes a UAF.
Uninitialized memory (CWE-457/908)	Leaks stack or heap contents. Heartbleed (CVE-2014-0160) was an over-read that returned 64 KB of uninitialized server memory per request.

Defenses: ASan + LSan in CI, `free(p); p = NULL;`, one owner per allocation with documented lifetime (Module 10 §10.6), opaque handles with a magic field (Module 11), and — where available — hardened allocators (glibc's tcache double-free detection, `MALLOC_CHECK_`, `scudo`, `hardened_malloc`).

23.5 TOCTOU, temp files, command injection

TOCTOU — CWE-367

```
/* BAD */
if (access(path, W_OK) == 0) {      /* CHECK */
    fd = open(path, O_WRONLY);      /* USE — attacker swaps the file in between */
}
```

The window between check and use lets an attacker replace `path` with a symlink to something else. `access()` is *especially* dangerous in `setuid` programs — it checks the *real* UID while `open()` uses the *effective* UID.

```
/* GOOD — operate on the object, not the name */
fd = open(path, O_WRONLY | O_NOFOLLOW | O_CLOEXEC);
if (fd < 0) return -1;
struct stat st;
fstat(fd, &st);                    /* same object, no race */
```

Also: `openat/fstatat` with a directory fd, `O_NOFOLLOW` to reject symlinks, and never `stat` then `open`.

Unsafe temp files — CWE-377

```
BAD : tmpnam()/mktemp()/tempnam() return a NAME -> race window
GOOD: mkstemp() creates AND opens atomically with mode 0600
      mkstemp -> /tmp/secureBr1rN mode=0600 (0600 = owner only)
```

```
char tmp1[] = "/tmp/secureXXXXXX";
mode_t old = umask(0077);           /* restrict permissions BEFORE creating */
int fd = mkstemp(tmp1);
umask(old);
unlink(tmp1);                       /* unlink immediately; the fd stays valid */
```

Also: `O_CREAT|O_EXCL` for a known filename, `mkdtemp` for directories, and `O_TMPFILE` (Linux) for a file with no name at all.

Command injection — CWE-78

```
/* BAD */
snprintf(cmd, sizeof cmd, "ls %s", user);
system(cmd);                       /* user = "; rm -rf ~" runs BOTH commands */
```

`system()` and `popen()` invoke `/bin/sh`. Every shell metacharacter — `;` `|` `&` `$` `\` `()` `<` `>` `*` `?` `\n` — is live. Escaping is a losing game.

```
/* GOOD — argument vector, no shell */
char *args[] = { "/bin/ls", (char*)user_path, NULL };
execve("/bin/ls", args, envp);
```

```
execve with a malicious 'filename':
; rm -rf /tmp/nothing; echo PWNED
^ passed through as a literal argument. No shell, no injection.
```

Also avoid: `exec1p/execvp` (they search `PATH`, which may be attacker-controlled), and any construction of a `sh -c` string. Use `posix_spawn` or `fork+execve` with an absolute path and a sanitized environment.

⚠ **In setuid programs, sanitize the entire environment** — `PATH`, `LD_PRELOAD`, `LD_LIBRARY_PATH`, `IFS`, locale variables. glibc strips some for setuid binaries, but not all, and not on every platform.

23.6 Input validation and canonicalization

Validate at the trust boundary, once, using an allow-list. Canonicalize before you validate.

The path traversal trap — CWE-22

```
snprintf(path, sizeof path, "/var/data/%s", user_file);
/* user_file = "../../../etc/passwd" */
```

Rejecting `".."` textually is not enough — attackers use `....//`, URL encoding (`%2e%2e`), overlong UTF-8 (Module 8), Unicode normalization, symlinks, and absolute paths.

The correct order:

1. **Canonicalize** with `realpath()` (or `openat` relative to a directory fd).
2. **Then** verify the result is under your base directory.
3. **Then** open it, with `O_NOFOLLOW`.

Better still: don't use user input as a filename at all. Map an opaque ID through a table you control.

Allow-list, not deny-list

```
/* BAD: deny-list - you will miss something */
if (strstr(input, "..") || strchr(input, '/')) reject();

/* GOOD: allow-list - enumerate what IS permitted */
for (const char *p = input; *p; p++)
    if (!isalnum((unsigned char)*p) && *p != '_' && *p != '-') return -1;
if (strlen(input) > MAX_NAME) return -1;
```

Note the `(unsigned char)` cast — Module 14's `ctype` trap is also a security issue on non-ASCII input.

The checklist for every untrusted input

- **Length** — bounded, before any copy
- **Type/range** — parsed with `strtoul` and all four checks (Module 8)
- **Character set** — allow-list
- **Encoding** — validate UTF-8, reject overlong forms and surrogates (Module 8 §8.6)
- **Structure** — for binary formats, validate every length field against the remaining input
- **Canonical form** — resolve before deciding

23.7 Standards: CERT C, MISRA, CWE

Scope	Use it when
-------	-------------

	Scope	Use it when
CERT C (SEI)	~120 rules + recommendations, each with non-compliant/compliant examples and a risk assessment	The best general secure-coding reference for C. Free online.
MISRA C:2012 / 2023	~175 rules for safety-critical embedded. Bans recursion, dynamic allocation, goto , most of the preprocessor, unions, VLAs	Automotive (ISO 26262), medical (IEC 62304), industrial (IEC 61508). Often contractually required.
CWE Top 25 (MITRE)	The 25 most dangerous weaknesses, ranked by real-world prevalence and impact	A prioritization list for what to test and review
AUTOSAR C++ / JSF AV C++	Adjacent standards you'll hear referenced	Aerospace/automotive C++
Annex K (__sfunctions)	Optional C11 bounds-checked functions	⚠ Effectively Microsoft-only ; glibc refused it; WG14 has proposed removal. Skip it.

The CWEs that are C-specific

CWE	Name	Module
CWE-787	Out-of-bounds Write	7, 10
CWE-125	Out-of-bounds Read	7, 10
CWE-416	Use After Free	10
CWE-476	NULL Pointer Dereference	9, 16
CWE-190	Integer Overflow or Wraparound	2
CWE-119/120/121/122	Buffer overflow family	8
CWE-134	Format String	13
CWE-415	Double Free	10
CWE-401	Memory Leak	10
CWE-457/908	Use of Uninitialized Memory	3, 21
CWE-367	TOCTOU	23
CWE-78	OS Command Injection	23
CWE-22	Path Traversal	23
CWE-362	Race Condition	17

Out-of-bounds write and read have topped the CWE Top 25 for years, and they are overwhelmingly C and C++ bugs.

Measured on the same source

```
-fstack-protector-strong -fstack-clash-protection -fcf-protection=full \
-D_FORTIFY_SOURCE=3 -ftrivial-auto-var-init=zero \
-fPIE"
LDFLAGS="-pie -Wl,-z,relro,-z,now,-z,noexecstack,-z,separate-code"
```

Verify with `checksec --file=./prog, hardening-check`, or the `readelf/nm` checks above. **Add a CI job that fails if any protection is missing.**

Beyond compiler flags

- **Sandboxing:** seccomp-bpf (restrict syscalls), Linux capabilities, namespaces, pledge/unveil (OpenBSD), AppArmor/SELinux
- **Privilege separation:** drop privileges early and permanently; split parsing into an unprivileged process (OpenSSH's model)
- **Hardened allocators:** scudo, hardened_malloc, `GLIBC_TUNABLES=glibc.malloc.check=3`
- **Runtime:** `MALLOC_PERTURB_` to fill freed memory with garbage in testing

23.9 Constant-time code

The timing side channel, measured

```
static int ct_equal(const void *a, const void *b, size_t n)
{
    const unsigned char *x = a, *y = b;
    unsigned char diff = 0;
    for (size_t i = 0; i < n; i++) diff |= (unsigned char)(x[i] ^ y[i]);
    return diff == 0;
    /* time depends only on n */
}
```

65536-byte buffers, 20,000 comparisons:

```
memcmp, differs at byte 0      :    0.1 ms
memcmp, differs at byte 65535 :   23.0 ms  <-- 328x SLOWER
ct_equal, differs at byte 0    :   39.9 ms
ct_equal, differs at byte 65535 :   40.0 ms  <-- 1.00x (constant)
```

A 328× timing difference that directly reveals how many bytes matched. An attacker submits candidate tokens byte by byte, keeping whichever prefix takes longest, and recovers the secret in $O(256 \times n)$ attempts instead of $O(256^n)$.

(At 32 bytes glibc's SIMD `memcmp` masked the effect in my first test — which is exactly why you must **never** rely on incidental timing behavior. It varies by libc, buffer size, and CPU.)

The rules for secret-handling code

1. **No branches on secret data.** No `if (secret_byte == x)`.

2. **No array indexing by secret data** — that leaks through the cache (this is how AES table-lookup implementations were broken).
3. **No division/modulo by secret values** — variable-latency on many CPUs.
4. **No early returns** from comparison loops.
5. Use branchless masking: `result = (mask & a) | (~mask & b)`.
6. **Zero secrets with `explicit_bzero` / `memset_explicit`** — Module 10 showed the compiler deleting a plain `memset` entirely, leaving the secret on the stack.
7. **`mlock` secret pages** so they're never swapped to disk; disable core dumps in processes holding keys.
8. **Use a vetted library.** `OPENSSL_memcmp`, `sodium_memcmp`, `CRYPTO_memcmp` exist because getting this right is hard.

Do not write your own crypto. Use libsodium, BoringSSL, or your platform's provider. The constant-time discipline above is for the code *around* the crypto — token comparison, HMAC verification, padding checks.

23.10 Secure API design

Bounds-carrying interfaces

The root cause of C's memory bugs is that a pointer doesn't know its own length. **Fix that at the type level:**

```
typedef struct { const uint8_t *p; size_t n; } Span;      /* read-only view */
typedef struct { uint8_t *p; size_t n; } MutSpan;      /* writable view */

static bool span_take(Span *s, size_t n, Span *out)
{
    if (n > s->n) return false;                          /* THE bounds check */
    if (out) *out = (Span){ s->p, n };
    s->p += n; s->n -= n;
    return true;
}

static bool span_u16(Span *s, uint16_t *v)
{ Span t; if (!span_take(s, 2, &t)) return false; *v = (uint16_t)((t.p[0]
<<8)|t.p[1]); return true; }

static bool span_copy(MutSpan dst, Span src)
{ if (src.n > dst.n) return false; memcpy(dst.p, src.p, src.n); return true; }
```

A parser written entirely against spans:

```
static bool parse_record(Span in, Record *r)
{
    if (!span_u8(&in, &r->type)) return false;
    if (!span_u16(&in, &r->len)) return false;
    Span body;
    if (!span_take(&in, r->len, &body)) return false;      /*
input bound */
    if (!span_copy((MutSpan){ r->value, sizeof r->value }, body)) return false; /*
```

```
output bound */
    r->vlen = body.n;
    return true;
}
```

```
BOUNDS-CARRYING API - the same parser, three inputs:
valid record      : OK  type=1 value="hello"
truncated (len>input): rejected
oversized (len>dest) : rejected

Every read and write is bounds-checked by construction.
There is no raw pointer arithmetic anywhere in the parser.
ASan exit=0
```

This is the single highest-leverage secure-coding technique in C. It's what Rust's slices give you for free, and you can have most of it with 30 lines of C.

The API design checklist

Principle	Implementation
Length travels with the pointer	Spans, or (ptr, len) pairs — never a bare char *
Fail closed	Default to reject; success must be explicit
No silent truncation	Return an error, don't quietly shorten (Module 8's snprintf check)
Ownership is documented	Module 10 §10.6
Opaque handles	Module 11 §11.8 — invariants can't be bypassed
[[nodiscard]]	Module 18 — ignoring an error becomes a compile warning
Validate at the boundary	Then assert internally (Module 15)
Minimize the attack surface	-fvisibility=hidden (Module 20); expose only what's needed
No global mutable state	Pass a context struct
Make the safe thing the easy thing	If your API is easy to misuse, it will be misused

Threat modeling

Before writing security-relevant code, answer four questions:

- 1. **What are you protecting?** (data, availability, integrity, keys)
- 2. **Who is the adversary and what can they control?** (network input, file contents, environment, local user, another process)
- 3. **What are the trust boundaries?** (every place untrusted data enters — that's where validation goes)

4. What happens if this component is compromised? (blast radius → does it need privilege separation?)

STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) is the standard checklist. Draw a data-flow diagram, mark the trust boundaries, and enumerate threats at each crossing.

Module 23 rules

#	Rule
1	Banned: <code>gets</code> , <code>strcpy</code> , <code>strcat</code> , <code>sprintf</code> , <code>scanf("%s")</code> , <code>alloca</code> , VLAs, <code>system</code> , <code>popen</code> , <code>mktemp</code> , <code>atoi</code>
2	Never pass non-literal data as a format string; <code>-Werror=format-security</code>
3	Every <code>n * size</code> computation gets an overflow check, or use <code>calloc/ckd_mul</code>
4	Check for negative first , then convert to <code>size_t</code>
5	Watch every place a length crosses a type boundary
6	<code>free(p); p = NULL;</code> — one owner, documented lifetime
7	Open first, then <code>fstat</code> — never <code>access/stat</code> then <code>open</code>
8	<code>mkstemp</code> + <code>umask(0077)</code> + immediate <code>unlink</code>
9	<code>execve</code> with an argument vector and an absolute path — never a shell
10	Canonicalize, then validate, with an allow-list
11	Ship with the full hardening flag set; verify it in CI with <code>checksec</code>
12	Constant-time comparison for every secret; <code>explicit_bzero</code> to wipe
13	Don't write your own crypto
14	Bounds-carrying spans instead of bare pointers
15	Threat model before you write; validate at the boundary, assert inside
16	Fuzz everything that parses untrusted input, with ASan + UBSan (Module 22)

Practice

1. **Stack layout.** Reproduce the §23.1 frame dump on your machine. Compute the exact distance from `buf[0]` to the return address. Then rebuild with `-fstack-protector-strong` and find the canary in the frame with GDB.
2. **Defense comparison.** Take one `strcpy` overflow and build it 5 ways: nothing, canary only, FORTIFY only, both, ASan. Record the exact diagnostic and exit code for each. Which one tells you the most? Which one costs the least?
3. **Format string.** Write a logging function that takes user input as a format string. Demonstrate the information leak with `%p`. Then add `-Wformat=2 -Werror=format-security` and confirm the build

fails. Fix it and confirm `_FORTIFY_SOURCE` also blocks the `%n` case.

4. **Integer overflow chain.** Write an `array_new(size_t n, size_t elemsize)` with no overflow check, then a loop that fills it. Find the `n` that produces a tiny allocation. Confirm ASan catches the heap overflow. Fix it three ways (explicit check, `calloc`, `__builtin_mul_overflow`) and verify each.
5. **Signed length bypass.** Write a "safe-looking" length check with a signed `int` that a negative value defeats. Prove it under ASan. Then fix it and add `-Wconversion -Wsign-conversion` to catch the whole class.
6. **TOCTOU.** Write the `access`-then-`open` pattern. Write a shell loop that races it by swapping the file for a symlink. Then rewrite with `open + fstat + O_NOFOLLOW` and confirm the race is closed.
7. **Command injection.** Write a "file lister" using `system()`. Break it with a crafted filename. Rewrite with `fork + execve` and an argument vector. Prove the same input is now harmless.
8. **Timing attack.** Reproduce the §23.9 measurement. Then write a program that recovers a 16-byte secret using only timing, one byte at a time, against a `memcmp`-based check. Then repeat against `ct_equal` and show it fails.
9. **Hardening audit.** Build a project with and without the full flag set. Verify with `checksec` (or the `readelf/nm` checks). Measure the performance and binary-size cost. Then write a CI check that fails if any protection is missing.
10. **Span library.** Implement a complete `Span/MutSpan` API: `span_take`, `span_u8/16/32/64` (both endiannesses), `span_bytes`, `span_find`, `span_split`, `span_copy`, `span_eq`. Rewrite a parser you've already written to use it exclusively. Fuzz both versions and compare crash counts.
11. **Threat model.** Take a program you've written. Draw the data-flow diagram, mark the trust boundaries, enumerate STRIDE threats at each crossing, and list the mitigation for each. Then check: does every untrusted input actually get validated at the boundary you drew?

Next: **Module 24 — Systems / POSIX Programming**, the largest module in Part 2: processes and `fork/exec`, signals done correctly, IPC, the file system API, sockets and networking, I/O multiplexing (`select/poll/epoll/io_uring`), and pthreads in practice.

24. Systems / POSIX Programming

- Syscalls vs library calls
- Processes: `fork`, `exec*`, `wait`, zombies, orphans, process groups
- **Signals:** handlers, async-signal-safety, `sigaction`, masks, `signalfd`
- IPC: pipes, FIFOs, UNIX sockets, shared memory, semaphores, message queues, `mmap`
- File system: paths, permissions, `stat`, directories, symlinks, `inotify`
- **Networking:** TCP/UDP sockets, `bind/listen/accept/connect`, byte order, non-blocking I/O
- Multiplexing: `select`, `poll`, `epoll`, `kqueue`, `io_uring`
- **pthreads:** `create/join/detach`, mutexes, condvars, rwlocks, barriers, TLS, thread pools
- Lock-free programming, ABA problem, memory barriers
- Daemons, resource limits, `getopt`, environment variables
- Dynamic loading: `dlopen/dlsym`, plugin architectures

- Windows counterparts if you target it: Win32 API, HANDLE model, IOCP

Module 24 — Systems / POSIX Programming

The largest module in Part 2. Real Linux 6.18 / glibc 2.39 output throughout.

24.1 Syscalls vs library calls

```
libc write() wrapper
direct syscall(SYS_write)
printf() -> buffered, may issue ZERO syscalls until flush
clock_gettime -> served from the vDSO, no context switch
```

Three layers:

```
your code → libc function → syscall wrapper → kernel
printf()      buffers      write(2)      sys_write
malloc()      free lists   brk/mmap     sys_brk
clock_gettime() —————→ vDSO (no kernel entry at all)
```

The measured cost

```
200000 iterations:
empty loop           :    0.00 ms
clock_gettime (vDSO) :    6.04 ms (30 ns/call)
getpid (real syscall) :   18.73 ms (94 ns/call)
```

A real syscall is ~94 ns; a vDSO call is ~30 ns; a function call is ~1 ns. That ~100× gap is why buffering exists (Module 13), why `epoll` beats `select`, and why `io_uring` was invented.

The **vDSO** is a small shared library the kernel maps into every process, providing `clock_gettime`, `gettimeofday`, `getcpu`, and `time` without a mode switch. Spectre/Meltdown mitigations pushed real syscall costs up substantially — measure on your target.

```
$ strace -c -e trace=write,clock_gettime,getpid ./sc
85.71  0.000012      4      3      write
14.29  0.000002      1      2      getpid
```

Note `clock_gettime` doesn't appear — it never reached the kernel.

Practical implications: batch syscalls (`writerv`, `readv`, `sendmmsg`), buffer aggressively, prefer `pread/pwrite` over `lseek+read` (one syscall, and thread-safe), and remember every syscall can fail with `EINTR` — always

retry (Module 13 §13.9).

24.2 Processes

fork

```
FORK: returns TWICE - 0 in the child, the child's pid in the parent
reaped 747: exited with 1
reaped 748: exited with 2
reaped 749: exited with 3
```

```
fflush(stdout);          /* ← CRITICAL: flush BEFORE fork */
pid_t p = fork();
if (p < 0) { perror("fork"); return -1; }
if (p == 0) { /* child */ _exit(code); }
/* parent */
```

Why the flush: `fork` copies the whole address space including stdio buffers. Unflushed output gets duplicated in both processes. This produces the classic "my printf appeared twice" bug.

The child gets: a copy of memory (copy-on-write), copies of all open file descriptors (**sharing the same file offsets**), and the same signal handlers. It does **not** inherit: pending signals, timers, or other threads.

⚠ **fork in a multithreaded program is dangerous.** Only the calling thread survives; a mutex held by another thread stays locked forever. **Between fork and exec in a threaded program, only async-signal-safe functions are legal** — so no `malloc`, no `printf`. Use `posix_spawn` or `vfork+exec`.

exec

```
EXEC: replaces the process image (same pid, new program)
before exec: pid=750
after exec: this is /bin/echo
```

```
char *args[] = { "/bin/echo", "text", NULL };
execve("/bin/echo", args, envp);
perror("execve");      /* only reached if exec FAILED */
_exit(127);
```

exec never returns on success. If it returns, it failed — always `_exit` after it.

Variant	Args	PATH search	Environment
execl/execlp/execle	list	p = yes	e = explicit

Variant	Args	PATH search	Environment
<code>execv/execvp/execve</code>	vector	<code>p</code> = yes	<code>e</code> = explicit

Use **execve** with an absolute path in security-sensitive code (Module 23). The `p` variants search an attacker-influenceable `PATH`.

FDs survive `exec` unless marked `FD_CLOEXEC` — **always open with `O_CLOEXEC`** to prevent descriptor leaks into children.

Zombies and orphans

```
ZOMBIE: a child that exited but was not reaped
    765 Z    zb
    ^ 'Z' = zombie: holds a pid slot until the parent wait()s
    after waitpid: reaped, slot released
```

A **zombie** is an exited child whose exit status hasn't been collected. It holds a pid slot. Enough zombies exhausts the pid table.

An **orphan** is a live child whose parent exited; it's re-parented to `init` (pid 1), which reaps it. That's how daemons detach.

Three ways to avoid zombies:

```
/* 1. Blocking wait */
waitpid(pid, &status, 0);

/* 2. SIGCHLD handler – the standard pattern */
static void reaper(int sig) {
    (void)sig;
    int saved = errno;                /* preserve errno (Module 15) */
    while (waitpid(-1, NULL, WNOHANG) > 0) { } /* reap ALL – signals aren't
queued */
    errno = saved;
}
struct sigaction sa = {0};
sa.sa_handler = reaper;
sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
sigaction(SIGCHLD, &sa, NULL);

/* 3. Ignore SIGCHLD – auto-reap, but you lose the exit status */
signal(SIGCHLD, SIG_IGN);
```

The **while loop in the handler is essential**: standard signals are not queued, so three children exiting simultaneously may deliver **one** `SIGCHLD`.

Status decoding

```
if (WIFEXITED(status))      code    = WEXITSTATUS(status);
else if (WIFSIGNALED(status)) signal = WTERMSIG(status);
else if (WIFSTOPPED(status)) signal = WSTOPSIG(status);
```

Process groups and sessions

```
parent pid=746 pgid=739 sid=0
```

Session → contains **process groups** → contains **processes**. A terminal signal (Ctrl-C → SIGINT) goes to the entire **foreground process group** — that's why Ctrl-C kills a whole pipeline.

`setsid()` creates a new session with no controlling terminal (the core of daemonizing). `setpgid()` creates a process group — useful for killing a whole subtree with `kill(-pgid, SIGTERM)`.

24.3 Signals

`sigaction`, not `signal`

```
signal()    : reset behaviour, restart behaviour, and masking are
              all IMPLEMENTATION-DEFINED. Do not use it.
sigaction(): defined semantics, a mask during the handler, SA_RESTART,
              SA_SIGINFO for siginfo_t, SA_NOCLDWAIT, SA_ONSTACK
```

```
struct sigaction sa = {0};
sa.sa_handler = on_term;
sigemptyset(&sa.sa_mask);
sigaddset(&sa.sa_mask, SIGUSR2);      /* additionally blocked during the handler */
sa.sa_flags = SA_RESTART;             /* auto-restart interrupted syscalls */
sigaction(SIGUSR1, &sa, NULL);
```

Flags worth knowing: `SA_RESTART`, `SA_SIGINFO` (gives you `siginfo_t` with the sender's pid, the faulting address for SIGSEGV, etc.), `SA_NOCLDSTOP`, `SA_NOCLDWAIT`, `SA_ONSTACK` (handle SIGSEGV on an alternate stack — the only way to survive a stack overflow), `SA_RESETHAND`.

⚠ Async-signal-safety

```
SAFE in a handler: write, _exit, signal, sigaction, kill, read,
                  sem_post, and assignment to volatile sig_atomic_t
NOT SAFE: printf, malloc, free, exit, any locking function
-> calling malloc from a handler can DEADLOCK (the heap lock)
```

A signal can arrive **in the middle of `malloc`**, while it holds the heap lock. Calling `malloc` from the handler then deadlocks. Same for `printf` (stdio locks), `exit` (runs `atexit` handlers), and anything touching `errno` without saving it.

POSIX lists ~180 async-signal-safe functions. In practice: `write`, `read`, `_exit`, `kill`, `sigaction`, `sem_post`, and the atomics.

The three correct patterns

1. Flag + main loop

```
static volatile sig_atomic_t g_stop = 0;
static void on_term(int sig) { (void)sig; g_stop = 1; } /* that's ALL */

while (!g_stop) { do_work(); }
```

```
after raise(SIGUSR1): g_stop = 1
```

⚠ `volatile sig_atomic_t` is the *only* type guaranteed safe here — and it's for signals, not threads (Module 17). In a threaded program use `_Atomic` with `sig_atomic_t`.

2. The self-pipe trick — turn signals into fd events

```
static void on_pipe(int sig) { char b = (char)sig; write(self_pipe[1], &b, 1); }

pipe2(self_pipe, O_CLOEXEC | O_NONBLOCK);
/* now poll/select/epoll on self_pipe[0] alongside your sockets */
```

```
poll() woke on the pipe: signal 12 delivered as an fd event
-> turns an async signal into an ordinary readable fd
```

This solves the fundamental event-loop problem: signals and I/O become the same kind of event, handled in one place with no races.

3. `signalfd` (Linux) — no handler at all

```
sigset_t mask; sigemptyset(&mask); sigaddset(&mask, SIGUSR1);
sigprocmask(SIG_BLOCK, &mask, NULL); /* MUST block it first */
int sfd = signalfd(-1, &mask, SFD_CLOEXEC);

struct signalfd_siginfo si;
read(sfd, &si, sizeof si); /* si.ssi_signo, si.ssi_pid, ... */
```

```
signalfd read: signo=10 from pid=794
-> no handler at all; signals become readable data
```

Cleanest of all — no async-signal-safety constraints, because there's no handler. BSD/macOS use `kqueue` with `EVFILT_SIGNAL`.

Masks and pending signals

```
sigprocmask(SIG_BLOCK): SIGINT is now PENDING, not lost
sigpending: SIGINT pending = 1
standard signals are NOT queued - N deliveries while blocked = 1 pending
(SIGRTMIN..SIGRTMAX real-time signals ARE queued)
```

Blocked signals are **pending**, not discarded. `sigsuspend` atomically unblocks and waits (avoiding the classic check-then-wait race). In threads use `pthread_sigmask` — the mask is per-thread, but handlers are per-process.

Signals you must handle in a server: `SIGPIPE` (ignore it, or a closed peer kills you), `SIGTERM`/`SIGINT` (graceful shutdown), `SIGCHLD` (reaping), `SIGHUP` (reload config).

24.4 IPC

```
=== PIPE (unidirectional, related processes) ===
parent read 16 bytes: "through the pipe"
pipe capacity: 65536 bytes (write blocks when full)
writing to a pipe with no reader -> SIGPIPE (or EPIPE if ignored)

=== UNIX SOCKETPAIR (bidirectional, can pass FDs) ===
ping -> "pong" (full duplex on one pair)
SCM_RIGHTS over a UNIX socket passes open FILE DESCRIPTORS between processes

=== SHARED MEMORY (mmap MAP_SHARED|MAP_ANONYMOUS) ===
4 processes x 10000 increments -> counter = 40000 (expected 40000)
the memory is the SAME PHYSICAL PAGES in every process
```

The mechanisms

Mechanism	Direction	Relationship	Notes
Pipe	one-way	related (fork)	Simplest. Writes $\leq$ <code>PIPE_BUF</code> (4096) are atomic.
FIFO (<code>mkfifo</code>)	one-way	unrelated	A pipe with a filesystem name
UNIX socket	two-way	unrelated	Passes file descriptors via <code>SCM_RIGHTS</code> ; supports datagrams with message boundaries

Mechanism	Direction	Relationship	Notes
socketpair	two-way	related	The best pipe replacement
Shared memory	n/a	either	Fastest — no copying. Needs explicit synchronization.
POSIX semaphore	n/a	either	sem_init(pshared=1) in shared memory, or sem_open("/name")
POSIX message queue	either	unrelated	mq_open/mq_send — prioritized, with message boundaries
System V IPC	—	—	shmget/semget/msgget. Legacy — awkward API, leaks on crash. Avoid.
eventfd/pidfd	—	—	Linux; lightweight event and process handles

Shared memory with a semaphore

```
struct Shared { sem_t sem; long counter; };
struct Shared *sh = mmap(NULL, sizeof *sh, PROT_READ|PROT_WRITE,
                          MAP_SHARED | MAP_ANONYMOUS, -1, 0);
sem_init(&sh->sem, 1, 1);          /* pshared=1 → usable across processes */
```

MAP_SHARED before fork gives you shared memory for free. For unrelated processes: shm_open("/name") + ftruncate + mmap.

⚠ **Shared memory has no synchronization.** All of Module 17 applies — you need semaphores, process-shared mutexes (PTHREAD_PROCESS_SHARED), or atomics. And **never put pointers in shared memory** — the same region can map at different addresses in different processes. Use offsets.

SCM_RIGHTS is worth knowing about: passing a live file descriptor between unrelated processes over a UNIX socket. It's the foundation of privilege separation (a privileged helper opens a file and hands the fd to an unprivileged worker) and of systemd socket activation.

24.5 File system

```
/etc/passwd: size=1111 mode=0644 uid=0 nlink=1 ino=26
stat follows symlinks; lstat does NOT; fstat takes an fd (TOCTOU-safe)
```

stat and friends

Call	Operates on
stat(path, &st)	The file, following symlinks
lstat(path, &st)	The symlink itself

Call	Operates on
<code>fstat(fd, &st)</code>	An open fd — race-free (Module 23)
<code>fstatat(dirfd, path, &st, flags)</code>	Relative to a directory fd

Key fields: `st_mode` (type + permissions), `st_size`, `st_uid/st_gid`, `st_nlink`, `st_ino+st_dev` (**together** they identify a file uniquely), `st_mtim/st_atim/st_ctim`.

Macros: `S_ISREG`, `S_ISDIR`, `S_ISLNK`, `S_ISFIFO`, `S_ISSOCK`, `S_ISCHR`, `S_ISBLK`.

Directories

```
DIR *d = opendir("/etc");
struct dirent *e;
while ((e = readdir(d)) != NULL) {
    if (!strcmp(e->d_name, ".") || !strcmp(e->d_name, "..")) continue;
    /* e->d_type may be DT_UNKNOWN — fall back to fstatat */
}
closedir(d);
```

```
rc2.d (type=4)      ← DT_DIR
papersize (type=8)  ← DT_REG
d_type may be DT_UNKNOWN on some filesystems -> fall back to fstatat
```

`readdir` is not thread-safe across the same `DIR*`; `readdir_r` is deprecated (use one `DIR*` per thread). For recursive walks, `nftw` or `fts` handle symlink loops and depth limits for you.

The `*at` family — race-free path operations

`openat`, `fstatat`, `unlinkat`, `renameat`, `mkdirat`, `linkat`, `symlinkat`, `faccessat`. Each takes a **directory fd** as the base, so the path can't be swapped out from under you. **Use these for anything security-sensitive** (Module 23 §23.5).

Permissions and ownership

`chmod/fchmod`, `chown/fchown`, `umask` (the bits *removed* from the requested mode), `access` (⚠ TOCTOU trap — use `faccessat` with `AT_EACCESS`, or just try the operation).

Also: hard links (`link`) share an inode; symlinks (`symlink`) are separate files containing a path; `realpath` canonicalizes; `readlink` reads a symlink target without following it.

`inotify` (Linux) — file system events

```
int fd = inotify_init1(IN_CLOEXEC | IN_NONBLOCK);
int wd = inotify_add_watch(fd, "/etc", IN_CREATE | IN_MODIFY | IN_DELETE |
```

```
IN_MOVED_TO);
/* then read() struct inotify_event records; add fd to your epoll set */
```

Watches are **not recursive** — you must add one per directory. Events can be coalesced or lost if you read too slowly (`IN_Q_OVERFLOW`). BSD/macOS: `kqueue` with `EVFILT_VNODE`, or `FSEvents`.

24.6 Networking

The TCP server sequence

```
int lfd = socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0);
int one = 1;
setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &one, sizeof one); /* avoid TIME_WAIT
bind failure */

struct sockaddr_in a = { .sin_family = AF_INET,
                        .sin_port   = htons(port),           /* NETWORK byte
order */
                        .sin_addr   = { htonl(INADDR_ANY) } };
bind(lfd, (struct sockaddr*)&a, sizeof a);
listen(lfd, 128);                                           /* backlog */
int cfd = accept4(lfd, NULL, NULL, SOCK_NONBLOCK | SOCK_CLOEXEC);
```

Client: `socket` → (optional `bind`) → `connect` → `read/write`.

Byte order

```
htons/htonl    /* host → network (big-endian) */
ntohs/ntohl    /* network → host */
```

Every multi-byte value on the wire is big-endian. x86 is little-endian, so these are real conversions.

Forgetting `htons` on a port is the single most common socket bug — and it "works" for palindromic ports, which makes it worse.

The options that matter

Option	Why
<code>SO_REUSEADDR</code>	Rebind while old connections are in <code>TIME_WAIT</code> . Always set it on servers.
<code>SO_REUSEPORT</code>	Multiple processes accept on the same port — kernel load balancing
<code>TCP_NODELAY</code>	Disable Nagle's algorithm. Essential for request/response protocols.
<code>SO_KEEPALIVE</code>	Detect dead peers
<code>SO_RCVTIMEO/SO_SNDTIMEO</code>	Timeouts on blocking sockets

Option	Why
<code>SO_LINGER</code>	Control <code>close()</code> behavior with pending data

The rules that bite people

1. **TCP is a byte stream, not a message stream.** One `write` may arrive as three `reads`, or three writes as one read. **You must frame your own messages** (length prefix or delimiter).
2. **read/write return short** — always loop (Module 13 §13.9).
3. **read returning 0 means orderly shutdown**, not an error.
4. **Ignore SIGPIPE** (`signal(SIGPIPE, SIG_IGN)`) or use `send(..., MSG_NOSIGNAL)` — otherwise a closed peer kills your process.
5. **Handle EINTR** on every blocking call.
6. **Use getaddrinfo**, not `gethostbyname` — it's thread-safe and handles IPv6.
7. **Write IPv6-capable code:** `AF_INET6` with `struct sockaddr_storage`, or dual-stack with `IPV6_V6ONLY=0`.

UDP

`sendto/recvfrom` (or `connect` a UDP socket to use `send/recv`). Datagrams preserve boundaries but are unreliable, unordered, and may be duplicated. **Keep them under the path MTU** (~1400 bytes safe) to avoid fragmentation.

24.7 I/O multiplexing

A working epoll echo server

```
int ep = epoll_create1(EPOOL_CLOEXEC);
struct epoll_event ev = { .events = EPOLLIN, .data.fd = lfd };
epoll_ctl(ep, EPOLL_CTL_ADD, lfd, &ev);

for (;;) {
    int n = epoll_wait(ep, evs, 16, timeout_ms);
    if (n < 0) { if (errno == EINTR) continue; break; }
    for (int i = 0; i < n; i++) {
        int fd = evs[i].data.fd;
        if (fd == lfd) {
            for (;;) {
                /* accept ALL pending */
                int c = accept4(lfd, NULL, NULL, SOCK_NONBLOCK|SOCK_CLOEXEC);
                if (c < 0) { if (errno==EAGAIN) break; else break; }
                struct epoll_event ce = { .events = EPOLLIN|EPOLLET, .data.fd = c
            };
            epoll_ctl(ep, EPOLL_CTL_ADD, c, &ce);
        }
        } else {
            ssize_t r;
            while ((r = read(fd, b, sizeof b)) > 0) { /* EDGE-TRIGGERED:
drain fully */
                ssize_t off = 0;
```

```
        while (off < r) { ssize_t w = write(fd, b+off, r-off); if (w<=0)
break; off += w; }
    }
    if (r == 0 || (r < 0 && errno != EAGAIN)) {
epoll_ctl(ep,Epoll_CTL_DEL,fd,NULL); close(fd); }
    }
}
```

```
echo server listening on 127.0.0.1:33123
server echoing 19 bytes
server echoing 19 bytes
server echoing 19 bytes
server done
```

Comparison

API	Complexity	Portability	Notes
select	O(n) per call	POSIX, everywhere	⚠ FD_SETSIZE is 1024 — fds above that corrupt the stack. Rebuilds the set every call.
poll	O(n) per call	POSIX	No fd limit. Still rescans everything each call.
epoll	O(1)	Linux	Kernel keeps the set. The Linux standard.
kqueue	O(1)	BSD, macOS	More general — also files, signals, timers, processes
IOCP	O(1)	Windows	Completion-based rather than readiness-based
io_uring	O(1), batched	Linux 5.1+	Shared submission/completion ring queues; near-zero syscalls . Truly asynchronous — works for regular file I/O, which epoll cannot.

Level-triggered vs edge-triggered

	Level-triggered (default)	Edge-triggered (EPOLLET)
Reports	Whenever data is available	Only when the state changes
You must	Nothing special	Drain until EAGAIN
Risk	More wakeups	Missed data if you don't drain

With **EPOLLET** you must loop reading until **EAGAIN**, and loop accepting until **EAGAIN**. Miss that and the connection hangs forever — the single most common epoll bug.

EPOLLONESHOT is essential for thread pools: the fd is disabled after one event, so two workers can't process the same connection concurrently.

Portable abstractions: libuv (Node.js), libevent, libev. Use one unless you need the last few percent.

24.8 pthreads

A production thread pool

```
typedef struct Task { void (*fn)(void*); void *arg; struct Task *next; } Task;
typedef struct {
    pthread_t *threads; size_t nthreads;
    Task *head, *tail;
    pthread_mutex_t mtx;
    pthread_cond_t work_ready, all_idle;
    size_t working; bool stop;
} Pool;

static void *worker(void *arg)
{
    Pool *p = arg;
    for (;;) {
        pthread_mutex_lock(&p->mtx);
        while (!p->head && !p->stop)
            pthread_cond_wait(&p->work_ready, &p->mtx); /* WHILE, not if */
        if (p->stop && !p->head) { pthread_mutex_unlock(&p->mtx); break; }
        Task *t = p->head; p->head = t->next; if (!p->head) p->tail = NULL;
        p->working++;
        pthread_mutex_unlock(&p->mtx);

        t->fn(t->arg); free(t); /* run OUTSIDE the
lock */

        pthread_mutex_lock(&p->mtx);
        if (--p->working == 0 && !p->head) pthread_cond_broadcast(&p->all_idle);
        pthread_mutex_unlock(&p->mtx);
    }
    return NULL;
}
```

```
thread pool: 4 workers, 200 tasks
all tasks complete, total = 59999000
clean shutdown, all threads joined
--- TSan ---
warnings: 0
```

Three design points: the predicate is checked in a **while** loop (Module 17), the task runs **outside** the lock (otherwise you've serialized everything), and shutdown is a flag + **broadcast** + **join**.

The API

Category	Functions
Threads	<code>pthread_create</code> , <code>pthread_join</code> , <code>pthread_detach</code> , <code>pthread_exit</code> , <code>pthread_self</code> , <code>pthread_equal</code>
Mutexes	<code>pthread_mutex_init/lock/trylock/timedlock/unlock/destroy</code> ; types <code>NORMAL</code> , <code>ERRORCHECK</code> (catches double-lock), <code>RECURSIVE</code> , <code>ADAPTIVE</code>
Condvares	<code>pthread_cond_init/wait/timedwait/signal/broadcast/destroy</code>
RW locks	<code>pthread_rwlock_rdlock/wrlock/unlock</code> — many readers or one writer
Barriers	<code>pthread_barrier_wait</code> — all N threads rendezvous
Spinlocks	<code>pthread_spin_lock</code> — only for very short critical sections on multicore
Once	<code>pthread_once</code> — thread-safe one-time init
TLS	<code>pthread_key_create</code> + <code>pthread_setspecific</code> (with a destructor), or just <code>_Thread_local</code>
Attributes	<code>pthread_attr_setstacksize</code> , <code>setdetachstate</code> , <code>setschedpolicy</code>

⚠ **Non-main thread stacks default to ~512 KB–8 MB** — set explicitly with `pthread_attr_setstacksize` if you recurse.

⚠ `pthread_cancel` is nearly unusable correctly (cancellation points, cleanup handlers, resource leaks). **Use a flag or a pipe for shutdown instead.**

Deadlock avoidance

1. **Global lock ordering** — always lock A before B, everywhere. Document it.
2. **Never call user callbacks while holding a lock** — they might call back into you.
3. `pthread_mutex_trylock` + **backoff** where ordering is impossible.
4. `PTHREAD_MUTEX_ERRORCHECK` in debug builds catches self-deadlock.
5. **TSan detects lock-order inversions**, not just races. Run it.

Lock-free and the ABA problem

Module 17 §17.5 covers this. The summary: CAS loops are the foundation, `atomic_compare_exchange_weak` in a loop and `_strong` outside one, and the **ABA problem** ($A \rightarrow B \rightarrow A$ between your read and your CAS) requires tagged pointers, hazard pointers, or epoch reclamation.

Memory barriers in the POSIX world: `__atomic_thread_fence`, `atomic_thread_fence` (C11), or the implicit ones in mutex lock/unlock. Prefer C11 atomics with explicit memory orders over hand-written barriers.

Use a mutex. Uncontended lock/unlock is ~20 ns. Lock-free algorithms are for measured hot paths, written by people who've read the papers.

24.9 Daemons, limits, options, environment

Daemonizing

```

/* classic double-fork */
if (fork() > 0) _exit(0);          /* 1. parent exits → child is orphaned */
setsid();                        /* 2. new session, no controlling terminal */
if (fork() > 0) _exit(0);          /* 3. second fork → can never reacquire a TTY */
chdir("/");                      /* 4. don't hold a mount busy */
umask(0);                        /* 5. explicit file modes */
/* 6. close/redirect fds 0,1,2 to /dev/null */

```

Under systemd, don't do any of this. Run in the foreground (`Type=simple` or `notify`), log to stdout/stderr, and let systemd handle supervision, restart, socket activation, and log capture. The double-fork dance is legacy.

Related: `sd_notify` for readiness, `LISTEN_FDS` for socket activation, and a PID file only if something requires one.

Resource limits

NOFILE (open fds)	soft=1024	hard=finite
STACK	soft=8388608	hard=unlimited
AS (address space)	soft=unlimited	hard=unlimited
NPROC	soft=15955	hard=finite
CORE	soft=0	hard=unlimited

RLIMIT_NOFILE = 1024 by default — a server handling more than ~1000 connections must raise it (`setrlimit`, or `LimitNOFILE=` in a systemd unit). This is the classic "my server stops accepting at 1000 clients" bug.

RLIMIT_CORE = 0 means **no core dumps** (Module 21) — raise it in development.

Also useful for testing: `RLIMIT_AS` to force allocation failures, `RLIMIT_CPU` and `RLIMIT_NPROC` as sandboxing measures.

getopt_long

```

static struct option opts[] = {
    {"verbose", no_argument,      0, 'v'},
    {"output",  required_argument, 0, 'o'},
    {"level",   optional_argument, 0, 'l'},
    {0,0,0,0}
};
while ((c = getopt_long(argc, argv, "vo:l:", opts, &idx)) != -1) { ... }
/* remaining positional args start at argv[optind] */

```

```

$ ./misc -vv --output=/tmp/x --level=3 extra1 extra2
--level=3

```



```
verbose=2 output=/tmp/x  remaining args from argv[4]
```

: = required argument, :: = optional (GNU). A leading + in the optstring stops at the first non-option; a leading : makes it return : for a missing argument instead of printing an error.

Environment

```
getenv("HOME");
setenv("VAR", "value", /*overwrite=*/1);
unsetenv("VAR");
extern char **environ;           /* enumerate everything */
```

⚠ **getenv returns attacker-controllable data.** In setuid programs, sanitize or clear PATH, LD_PRELOAD, LD_LIBRARY_PATH, IFS, and locale variables (Module 23). setenv is not thread-safe against concurrent getenv.

24.10 Dynamic loading and plugins

```
void *h = dlopen("./plugin.so", RTLD_NOW | RTLD_LOCAL);
if (!h) { fprintf(stderr, "dlopen: %s\n", dlerror()); return -1; }

dlerror();                               /* clear stale errors first */
union { void *o; ApplyFn f; } u;         /* ISO C doesn't bless void*↔fnptr */
u.o = dlsym(h, "plugin_apply");
const char *err = dlerror();
if (err) { ... }
```

```
loaded plugin "uppercase" v3
before: hello plugin world
after  : HELLO PLUGIN WORLD
dladdr: plugin_version from ./plugin.so
```

Flag	Meaning
RTLD_NOW	Resolve everything at load — fail fast
RTLD_LAZY	Resolve on first call — faster load, later failures
RTLD_LOCAL	Symbols not visible to later dlopens (default, safer)
RTLD_GLOBAL	Symbols available to subsequent loads

Design rules for a plugin ABI:

- 1. **Export one struct of function pointers**, not many symbols. It's versionable and it's a single dlsym.

2. Include a **version field** and check it on load.
3. Build plugins with `-fvisibility=hidden` (Module 20) so only the entry point is exported.
4. **`dLError()` must be cleared before `dlsym`** — a valid symbol can legitimately be `NULL`.
5. **Never `dllclose` while a callback from the library may still run**, or while any of its data is reachable.
6. `dldaddr` maps an address back to a symbol and library — useful for backtraces.
7. Note the `void*`↔function-pointer conversion isn't ISO C (Module 9); POSIX requires it and the union is the conventional workaround.

24.11 Windows counterparts

POSIX	Win32
<code>fd</code> (int)	<code>HANDLE</code> (opaque) — not an int, and not a small integer
<code>open/read/write/close</code>	<code>CreateFile/ReadFile/WriteFile/CloseHandle</code>
<code>fork + exec</code>	<code>CreateProcess</code> (no fork equivalent)
<code>pthread_create</code>	<code>CreateThread</code> / <code>_beginthreadex</code>
<code>pthread_mutex_t</code>	<code>CRITICAL_SECTION</code> (in-process) / <code>Mutex</code> (cross-process)
<code>pthread_cond_t</code>	<code>CONDITION_VARIABLE</code>
<code>epoll/kqueue</code>	IOCP (I/O Completion Ports) — completion-based, not readiness-based
<code>mmap</code>	<code>CreateFileMapping</code> + <code>MapViewOfFile</code>
signals	Structured Exception Handling, <code>SetConsoleCtrlHandler</code>
<code>dlopen/dlsym</code>	<code>LoadLibrary/GetProcAddress</code>
<code>errno</code>	<code>GetLastError()</code>
pipes	Named pipes (much richer — they're bidirectional and network-capable)

The deep difference: POSIX is **readiness-based** ("tell me when I can read"), Windows IOCP is **completion-based** ("do this read and tell me when it's done"). `io_uring` brings the completion model to Linux.

For portability: libuv (the best cross-platform abstraction), or an internal platform layer with `#ifdef _WIN32`. Don't scatter `#ifdefs` through your logic — isolate them in one file.

Module 24 rules

#	Rule
1	A syscall is ~100× a function call — buffer and batch
2	<code>fflush</code> before <code>fork</code> , or your output duplicates
3	<code>exec</code> returning means it failed — always <code>_exit</code> after it
4	Reap children with <code>while (waitpid(-1, NULL, WNOHANG) > 0)</code> — signals aren't queued

#	Rule
5	<code>sigaction</code> , never <code>signal</code>
6	In a handler: set a <code>volatile sig_atomic_t</code> , <code>write()</code> , <code>_exit()</code> . Nothing else.
7	Self-pipe or <code>signalfd</code> to unify signals with your event loop
8	Never put pointers in shared memory — use offsets
9	Use the <code>*at</code> family for race-free path operations
10	Every multi-byte network value goes through <code>htons/htonl</code>
11	TCP is a byte stream — frame your own messages
12	<code>SO_REUSEADDR</code> on every server; ignore <code>SIGPIPE</code> ; loop on short reads/writes and <code>EINTR</code>
13	<code>EPOLLET</code> requires draining until <code>EAGAIN</code> — for both <code>read</code> and <code>accept</code>
14	<code>RLIMIT_NOFILE</code> defaults to 1024 — raise it for servers
15	Thread pools: <code>while</code> on the predicate, run tasks outside the lock, join on shutdown
16	Global lock ordering; never call callbacks holding a lock; run TSan
17	Under systemd, don't daemonize — run in the foreground
18	Plugins export one versioned struct of function pointers
19	<code>O_CLOEXEC/SOCK_CLOEXEC</code> on everything

Practice

1. **Syscall cost.** Measure `getpid`, `clock_gettime`, `read` of 1 byte, and `write` of 1 byte in a loop. Then measure `write` of 4096 bytes. Compute the per-call and per-byte costs and explain why buffering exists.
2. **Process supervisor.** Write a program that forks N children running a given command, restarts any that exit, reaps with a `SIGCHLD` handler, and shuts them all down cleanly on `SIGTERM` using a process group. Verify no zombies with `ps`.
3. **Signal-safe logging.** Write a signal handler that logs the signal number, the faulting address (via `SA_SIGINFO`), and a backtrace — using only async-signal-safe functions. Test it on `SIGSEGV` with `SA_ONSTACK` and an alternate stack so it survives a stack overflow.
4. **Three signal patterns.** Implement graceful shutdown three ways: flag+loop, self-pipe+poll, and `signalfd`+`epoll`. Compare the code and the race characteristics of each.
5. **IPC benchmark.** Send 1 million 64-byte messages between two processes using: a pipe, a UNIX socketpair, a POSIX message queue, and shared memory with a semaphore. Measure throughput and latency for each.
6. **FD passing.** Write a privileged parent that opens files and passes the fds to an unprivileged child over a UNIX socket with `SCM_RIGHTS`. This is the OpenSSH privilege-separation model.

7. **Complete TCP server.** Build an epoll-based server with: length-prefixed message framing, per-connection state, edge-triggered I/O with proper draining, write buffering for slow clients, idle timeouts, graceful shutdown via `signalfd`, and `SO_REUSEADDR`. Load-test it with 10,000 concurrent connections (raise `RLIMIT_NOFILE` first).
8. **select vs poll vs epoll.** Implement the same server three ways. Benchmark with 10, 100, 1000, and 10000 idle connections plus 10 active ones. Plot the per-event cost and explain the $O(n)$ vs $O(1)$ difference.
9. **Edge-triggered bug.** Write an `EPOLLET` handler that reads only once per event. Send a large message and observe the hang. Fix it with the drain loop. Do the same for `accept`.
10. **Thread pool extension.** Take the §24.8 pool and add: a bounded queue with backpressure, per-task futures (`pthread_cond` + result slot), work stealing, and graceful drain-then-stop. Verify with TSan and run 1M tasks.
11. **Deadlock and detection.** Write two threads that lock $A \rightarrow B$ and $B \rightarrow A$. Confirm it deadlocks. Confirm TSan reports the lock-order inversion. Fix with global ordering, then with `trylock` + backoff.
12. **Plugin system.** Define a plugin ABI as a versioned struct of function pointers. Write a loader that scans a directory, `dlopen`s each `.so`, validates the version, and registers it. Write three plugins. Handle load failure, version mismatch, and safe unload.
13. **File watcher.** Build a recursive directory watcher with `inotify` integrated into an epoll loop, handling `IN_CREATE` on directories by adding new watches, and `IN_Q_OVERFLOW` by rescanning.

Next: **Module 25 — Performance Engineering**, where the cache-layout effects you measured in Module 11 get a systematic treatment: the memory hierarchy, branch prediction, profiling methodology, SIMD, and — most importantly — how to know whether an optimization actually helped.

25. Performance Engineering

- Big-O in practice vs constants and memory
- **CPU caches, cache lines, locality, prefetching, false sharing**
- Branch prediction, branchless techniques
- Data-oriented design, SoA vs AoS
- Profiling: `perf`, `gprof`, `callgrind`, VTune, flamegraphs
- Benchmarking methodology (warmup, variance, avoiding compiler elision)
- SIMD: SSE/AVX/NEON intrinsics, auto-vectorization
- Inline assembly, compiler barriers
- `restrict`, `likely/unlikely`, alignment, memory pools
- Understanding what the optimizer does and doesn't do

Module 25 — Performance Engineering

Every number below is measured on this box: **Intel Xeon @ 2.10 GHz, 1 vCPU, L1d 48 KB, L2 2 MB, L3 260 MB, 64-byte cache lines**. Your numbers will differ; the *shapes* won't.

25.1 Big-O vs constants and memory

BIG-O: linear $O(n)$ vs binary $O(\log n)$ search			
n	linear	binary	
4	0.56 ms	0.47 ms	
64	5.05 ms	1.01 ms	
4096	197.16 ms	10.16 ms	

Asymptotics won here — but that's not the general lesson. **The general lesson is that asymptotic complexity is a statement about the limit, and real code often lives well below it.**

Cases where the "worse" algorithm wins in practice:

Situation	Why
Small n ($< \sim 64$)	Linear scan of a contiguous array beats a tree: no pointer chasing, perfect prefetching, no branches
Sorted-array binary search vs hash table	The array has better locality; hashing costs a computation plus a random access
insertion sort inside qsort	Every production sort switches to insertion sort below ~ 16 elements
Array vs linked list	An $O(n)$ array scan often beats an $O(1)$ list insert, because the list traversal is a random-latency pointer chase

The constant factor in C is dominated by memory access patterns, which is what the rest of this module is about. A cache miss is $\sim 100\times$ an L1 hit — that dwarfs the difference between n and $\log n$ for small n .

Know your n . Optimizing for $n = 10^9$ when production n is 20 is a common and expensive mistake.

25.2 The memory hierarchy ⚠

Measured latencies (random pointer chase — pure latency, prefetcher defeated)

WORKING SET	ns/access	
8 KB	1.61	← L1d (48 KB)
32 KB	1.63	← L1d
128 KB	3.94	← L2
512 KB	5.54	← L2 (2 MB)
2 MB	22.13	← L2/L3 boundary
8 MB	57.03	← L3
32 MB	66.46	← L3 (260 MB)
128 MB	148.81	← RAM

~ 1.6 ns to ~ 149 ns — a $92\times$ range. Every plateau is a cache level; every jump is where the working set stopped fitting.

At 2.1 GHz, one cycle is 0.48 ns. So:

Level	Latency	Cycles
Register	—	0
L1	1.6 ns	~3
L2	5.5 ns	~12
L3	60 ns	~125
RAM	149 ns	~310

A main-memory access costs about 310 cycles. In that time a modern core could have retired ~1200 instructions. **This is why layout beats algorithms in most real optimization work.**

Cache lines

```
CACHE LINE EFFECT: touch one byte every STRIDE bytes (same access count)
stride      time
1           0.003 ms (multiple accesses share a line)
32          0.003 ms
64          0.003 ms <- one access per 64-byte cache LINE
128         0.005 ms
256         0.027 ms
512         0.084 ms <- 28x slower than stride 1
```

Memory moves in 64-byte lines, not bytes. Below stride 64, several accesses share a line — nearly free. At and above 64, every access needs its own line. The cost keeps climbing past 64 because larger strides also defeat the prefetcher and blow out the TLB.

Practical consequences:

- **Iterate in memory order.** Row-major for 2D arrays (Module 7 §7.2) — this was a 5–10× effect.
- **Pack hot data together.** Module 11's field-reordering halved a struct and gave a **7.17×** speedup on a key-scanning loop.
- **A 64-byte struct means one object per cache line.** A 16-byte struct means four.

Prefetching

The hardware prefetcher detects **sequential and constant-stride** patterns and fetches ahead. It cannot follow pointer chains or random indices — which is exactly why the pointer-chase benchmark above measures true latency.

```
__builtin_prefetch(&data[i + 16], 0, 3); /* 0=read, 3=high temporal locality */
```

Software prefetching rarely helps unless you have a genuinely irregular but *predictable-ahead* pattern (hash-table probing, tree traversal where you know the next node early). Measure; it usually does nothing or

hurts.

False sharing

```
addresses (shared): 0x...180 0x...188 0x...190 0x...198    ← all within 64 bytes
addresses (padded): 0x...080 0x...0c0 0x...100 0x...140    ← 64 bytes apart
```

Two threads writing to *different* variables that share a cache line cause the line to ping-pong between cores.

No data is shared logically — the cache line is.

⚠ **This box has 1 vCPU, so I could not measure the effect honestly here.** On a real multicore machine the difference is typically **3–10x**. The fix is Module 11's:

```
struct alignas(64) Counter { long v; };
struct Counter counters[NTHREADS];
```

Detect it with `perf c2c` (cache-to-cache) on Linux.

25.3 Branch prediction

The classic experiment — and a lesson about modern compilers

```
for (int i = 0; i < N; i++) if (data[i] >= 128) s += data[i];
```

```
branchy, RANDOM data      :    28.8 ms
branchy, SORTED data      :    28.9 ms   (1.00x – no difference?!)
BRANCHLESS, random       :    18.7 ms   (1.54x faster)
```

Sorting made no difference. Why? Look at the assembly:

```
sum_ge:
...
    cmovg    rdx, rax    ← conditional MOVE, not a branch
```

GCC already converted the branch to a `cmov` at -O2. There was no branch left to mispredict.

Force a real branch (a `noinline` call the compiler can't flatten):

```
branchy w/ real call, RANDOM :    281.5 ms
branchy w/ real call, SORTED :    30.9 ms   (9.12x faster)
```

9.12x — from sorting the input. Identical code, identical arithmetic, identical memory access pattern. The only difference is whether the branch is predictable.

Why it costs so much

A modern CPU is a deep pipeline (~15–20 stages) executing speculatively. A mispredict flushes everything in flight: **~15–20 cycles wasted**, every time. At 50% mispredict rate on a hot loop, that dominates.

Modern predictors (TAGE) are extremely good at repeating patterns, loops, and correlated branches. They are helpless against genuinely random data.

Branchless techniques

```
/* conditional add via a mask */
int m = -(v >= 128);          /* 0 or -1 (all bits set) */
s += v & m;

/* branchless min/max/abs */
int min = b ^ ((a ^ b) & -(a < b));
int mask = v >> 31; int abs = (v + mask) ^ mask;

/* branchless select */
result = (mask & a) | (~mask & b);
```

⚠ **Branchless is not automatically faster.** It always does the work of both branches. It wins when the branch is unpredictable, loses when it's predictable (a well-predicted branch is nearly free, and skipping expensive work is a real win). **Measure.**

Also: **cmov** creates a **data dependency** where a predicted branch does not, which can hurt in a dependency-chain-limited loop. And branchless code is what you *must* write for constant-time crypto (Module 23 §23.9) regardless of speed.

Branch hints

```
#define likely(x)    __builtin_expect(!(x), 1)
#define unlikely(x) __builtin_expect(!(x), 0)
```

```
BRANCH HINTS (0.1% of elements take the rare path)
plain          :    31.02 ms
__builtin_expect :    21.00 ms  (1.48x)
```

Hints work mainly through code layout, not the hardware predictor: the hot path falls through, the cold path is moved out of line, and the i-cache holds more useful instructions. C23 spells this `[[likely]]`/`[[unlikely]]` (as attributes on statements, though GCC 13 doesn't have them for C yet).

Profile-guided optimization does this automatically and far better:

```
gcc -O2 -fprofile-generate prog.c -o prog && ./prog <typical workload>
gcc -O2 -fprofile-use prog.c -o prog
```

PGO typically yields 5–20% with zero source changes. Use it before hand-annotating.

25.4 Data-oriented design

Module 11 §11.3 measured **7.17×** from layout alone. The systematic version:

```
/* AoS – natural OOP-ish layout */
struct Particle { float x,y,z, vx,vy,vz; float mass; int id; };
struct Particle p[N];

/* SoA – every field is a dense contiguous array */
struct Particles {
    float x[N], y[N], z[N];
    float vx[N], vy[N], vz[N];
    float mass[N]; int id[N];
};
```

	AoS wins	SoA wins
Access pattern	You touch most fields of one object	You touch one or two fields across many objects
Examples	Per-entity game logic, tree nodes, linked structures	Physics integration, filters, aggregations, column stores
Vectorization	Hard (strided access)	Trivial — contiguous, aligned
Cache efficiency	Wastes the line if you read one field	Every byte loaded is used

Hybrid: AoSoA (array of structs of small arrays) — group 8 or 16 objects' worth of each field into SIMD-width blocks. Best of both, common in physics and rendering engines.

The other layout techniques

- **Hot/cold splitting** — frequently-accessed fields in one struct, the rest behind a pointer (Module 11)
- **Field reordering** by decreasing alignment — often halves the struct
- **Indices instead of pointers** — 4-byte `uint32_t` indices instead of 8-byte pointers halves your node size and survives `realloc`
- **Arena/pool allocation** (Module 10) — sequential allocation means sequential memory. Measured there: **14×** for arena, **5×** for pool vs `malloc`.

- **Struct-of-arrays for hash tables** — separate key, hash, and value arrays; probe the hash array (dense, cache-friendly) before touching keys

25.5 Benchmarking methodology ⚠

Most published C benchmarks are wrong. Here are the three failures, measured:

Mistake 1 — dead-code elimination

```
result discarded :    0.000 ms    <- the loop was DELETED
result consumed  :   14.525 ms    <- real work
```

If nothing consumes the result, the optimizer deletes the computation. Fixes:

```
volatile double sink; sink = result;           /* forces a store */
__asm__ volatile("" :: "r"(result) : "memory"); /* compiler barrier */
if (result == 0.0) printf("");                 /* opaque consumer */
```

Google Benchmark's `benchmark::DoNotOptimize` is the packaged version.

Mistake 2 — no warmup

```
run 1 :    1.347 ms    <- cold
run 10:    1.405 ms    <- warm
```

(Small here because the array is only 16 MB and was just written. On larger data or with a cold page cache, the first run can be **10–100×** slower.) The first run pays for page faults, cold caches, lazy PLT binding, and CPU frequency ramp-up.

Mistake 3 — reporting the mean

```
min=1.368 median=1.421 mean=1.426 max=1.592 ms
spread: 16.3%  -> report MIN or MEDIAN, never the mean
```

The maximum is **contamination** — scheduling, interrupts, another process. It's not information about your code. **Report the minimum** (the run least disturbed by noise) or the **median**. The mean is dragged by outliers in one direction only.

The checklist

Do

Why

Do	Why
Consume every result	Defeat DCE
Warm up, then measure	Page faults, cold caches, frequency scaling
Repeat ≥ 9 times, report min or median	Noise is one-sided
Report the spread too	If min and max differ by 3 $\times$, your number is meaningless
Use <code>CLOCK_MONOTONIC</code>	Module 14 — never <code>time()</code> or <code>CLOCK_REALTIME</code>
Pin the CPU (<code>taskset</code>) and disable turbo/scaling	Removes the largest noise source
Benchmark the whole realistic workload	Microbenchmarks lie about cache and branch behavior
Compare against a baseline in the same run	Absolute numbers aren't comparable across machines or days
Randomize the order of the variants	Removes systematic drift

⚠ **Microbenchmarks are systematically optimistic.** In a microbenchmark your data is in cache, the branch predictor is trained on your exact pattern, and the i-cache holds only your loop. In production none of that holds. **Always validate with an end-to-end measurement.**

25.6 Profiling

Never optimize without profiling. Programmer intuition about where time goes is famously bad.

`perf` — the Linux profiler

```
perf stat ./prog                # cycles, IPC, cache misses, branch misses
perf record -g ./prog && perf report  # sampling profile with call graphs
perf top                        # live, system-wide
perf record -e cache-misses,LLC-load-misses ./prog
perf annotate                    # per-instruction attribution
perf c2c record ./prog          # false sharing detection
perf stat -e task-clock,cycles,instructions,branch-misses,cache-misses ./prog
```

`perf stat` alone tells you the shape of your problem:

Signal	Diagnosis
IPC < 1.0	Memory-bound — fix layout
IPC > 2.5	Compute-bound — consider SIMD
High cache-misses	Layout or working-set problem
High branch-misses	Unpredictable branches (§25.3)

Signal	Diagnosis
High task-clock vs wall time	Multi-threaded, or spinning

Flame graphs are the standard visualization:

```
perf record -F 99 -g ./prog
perf script | stackcollapse-perf.pl | flamegraph.pl > out.svg
```

Width = time spent, stacked by call depth. One glance tells you where the time is.

(**perf** requires `perf_event_paranoid` permissions and hardware counters — often unavailable in containers, as here.)

callgrind — deterministic, reproducible

```
Ir                                file:function
5,601,209 (97.74%)  prof.c:main
```

Exact instruction counts rather than samples. **Reproducible**, so it works in CI as a regression test. Costs ~50× runtime. Visualize with **kcachegrind**. **cachegrind** adds cache and branch-prediction simulation.

The rest

Tool	Notes
gprof	-pg , instrumentation-based. Legacy — high overhead, distorts inlining, misses shared libraries.
Intel VTune	Best-in-class on Intel: microarchitecture analysis, memory-access analysis, threading. Free.
AMD µProf	The AMD equivalent
bpfttrace	Production tracing with near-zero overhead (Module 21)
massif / heaptrack	Memory profiling over time
Tracy, Perfetto	Frame-based tracing for games and real-time systems

Method

- 1. **Measure end-to-end first.** Know the total before you optimize a part.
- 2. **Profile to find the hot spots.** Usually 3 functions account for 80%.
- 3. **Understand why they're hot** — **perf stat** for IPC/misses, then **perf annotate**.
- 4. **Fix the biggest one.** Algorithm → layout → micro-optimization, in that order.
- 5. **Re-measure end-to-end.** Amdahl's law: making a function that's 5% of runtime infinitely fast gains 5%.

6. Keep the benchmark in CI so regressions are caught.

25.7 SIMD

```
DOT PRODUCT, 4194304 floats (32 MB touched)
  scalar (-O2)           :    3.06 ms
  restrict (-O2)         :    3.03 ms  (1.01x)
  AVX2 intrinsics + FMA :    1.34 ms  (2.28x)
```

And auto-vectorization:

```
-O2                3.15 ms
-O3                3.09 ms
-O3 -march=native  3.13 ms
-O3 -march=native -ffast-math  1.36 ms  ← 2.3x, matching hand-written
```

Why `-ffast-math` was required

Floating-point addition is **not associative** (Module 2). A vectorized reduction computes 8 partial sums and combines them, which changes the order — and therefore the result:

```
values agree: scalar=75122080 avx=75497432
```

They don't agree exactly. That 0.5% difference is real reassociation error. The compiler refuses to vectorize a float reduction unless you explicitly permit reassociation via `-ffast-math` (or the safer `-fassociative-math`, or `#pragma omp simd reduction(+:s)`).

For integer reductions, no such permission is needed — integer addition *is* associative.

The intrinsics

```
#include <immintrin.h>

__m256 acc = _mm256_setzero_ps();
for (int i = 0; i + 8 <= n; i += 8) {
    __m256 vx = _mm256_loadu_ps(x + i);
    __m256 vy = _mm256_loadu_ps(y + i);
    acc = _mm256_fmadd_ps(vx, vy, acc);    /* fused multiply-add */
}
float tmp[8]; _mm256_storeu_ps(tmp, acc);
float s = tmp[0]+tmp[1]+...+tmp[7];      /* horizontal reduction */
for (; i < n; i++) s += x[i] * y[i];     /* scalar tail */
```

ISA	Width	Header
SSE2	128-bit (4 floats)	<code><emmintrin.h></code> — baseline on x86-64
AVX/AVX2	256-bit (8 floats)	<code><immintrin.h></code>
AVX-512	512-bit (16 floats)	<code><immintrin.h></code> — ⚠ can cause frequency throttling
NEON	128-bit	<code><arm_neon.h></code> — baseline on ARM64
SVE	scalable	ARM, vector-length agnostic

Practical guidance

1. **Try auto-vectorization first.** `-O3 -march=... -ffast-math`, check with `-fopt-info-vec` and `-fopt-info-vec-missed`.
2. **Help the vectorizer:** `restrict` (Module 2), aligned data, simple counted loops, no early exits, no function calls in the body, contiguous access (SoA!).
3. **Hand-write intrinsics only where it measurably matters.** They're unreadable and non-portable.
4. **Always keep a scalar fallback** and handle the tail.
5. **Runtime dispatch** for distributed binaries — `__builtin_cpu_supports("avx2")` plus `__attribute__((target("avx2")))` function multiversioning. **Never ship `-march=native`** (Module 19).
6. **Alignment matters** — use `aligned_alloc(64, ...)` and the aligned load/store variants where you can guarantee it.
7. **Consider a library first:** OpenBLAS, Eigen, simdjson, xsimd, Highway — they've done the work.

25.8 The optimizer: what it does and doesn't do

What it does well

Constant folding and propagation, dead-code elimination, common subexpression elimination, inlining, loop-invariant code motion, strength reduction (`x*8` $\rightarrow$ `x<<3`), register allocation, instruction scheduling, tail-call optimization (Module 6), auto-vectorization, branch-to-`cmov` conversion (§25.3), and exploiting UB (Module 16).

What it cannot do

Can't	Because
Change your algorithm	$O(n^2)$ stays $O(n^2)$
Change your data layout	AoS $\rightarrow$ SoA is a source change
Reorder across possible aliases	Hence <code>restrict</code>
Reassociate floats	Not associative — hence <code>-ffast-math</code>
Optimize across TU boundaries	Unless you enable LTO (Module 19)
Optimize through indirect calls	Function pointers, <code>dlsym</code> , virtual dispatch

Can't	Because
Remove volatile accesses	By definition (Module 2)
Know your runtime data	Hence PGO
Fix cache misses	It can't move memory

The hints that actually help

```

/* restrict – no aliasing (Module 2). Measured: enables vectorization. */
void axpy(float *restrict y, const float *restrict x, float a, int n);

/* alignment */
float *p = aligned_alloc(64, n * sizeof *p);
__builtin_assume_aligned(p, 64);

/* branch layout */
if (unlikely(error)) { ... }

/* purity – enables CSE and reordering */
__attribute__((const)) int f(int x); /* no memory access at all */
__attribute__((pure)) int g(const T *t); /* reads memory, no side effects */

/* force/prevent inlining after measuring */
__attribute__((always_inline)) / __attribute__((noinline))

/* compiler barrier – prevents reordering across this point, emits nothing */
__asm__ __volatile__("" ::: "memory");

```

Inline assembly

```

uint64_t rdtsc(void) {
    uint32_t lo, hi;
    __asm__ __volatile__("rdtsc" : "=a"(lo), "=d"(hi));
    return ((uint64_t)hi << 32) | lo;
}

```

Rarely justified. Intrinsic cover almost everything, are readable, and let the optimizer schedule around them. Reserve inline asm for: instructions with no intrinsic (**rdtsc**, **cuid**, privileged instructions), atomic sequences the compiler won't generate, and bare-metal startup code.

If you must: get the constraints exactly right, list every clobber including **"memory"** and **"cc"**, and prefer **__asm__ __volatile__** when the instruction has side effects.

The order of operations

1. Better algorithm → 2. Better data layout → 3. Better memory access → 4. Better branches → 5. SIMD → 6. Inline assembly.

Most real wins come from steps 1–3. Most wasted effort goes into steps 5–6.

Module 25 rules

#	Rule
1	Know your actual n — asymptotics may not apply
2	RAM is $\sim 92\times$ L1 on this machine (1.6 ns → 149 ns). Layout is the dominant constant.
3	Memory moves in 64-byte lines — iterate in memory order
4	A mispredicted branch costs $\sim 15\text{--}20$ cycles; sorting data gave 9.12$\times$
5	Check the assembly before concluding a branch exists — cmov may already be there
6	Branchless wins on unpredictable branches, loses on predictable ones
7	alignas(64) on per-thread data to kill false sharing
8	SoA when you touch few fields across many objects; AoS when you touch many fields of one
9	Consume every benchmark result, or the compiler deletes your loop
10	Warm up; report min or median, never the mean; report the spread
11	Microbenchmarks are systematically optimistic — validate end-to-end
12	perf stat first: IPC < 1 means memory-bound
13	Try auto-vectorization before intrinsics; restrict and SoA are what enable it
14	Float reductions need -ffast-math to vectorize — and the result changes
15	Never ship -march=native ; use runtime dispatch
16	PGO before hand-annotating branches
17	Algorithm → layout → memory → branches → SIMD → asm, in that order

Practice

- Map your own hierarchy.** Run the pointer-chase benchmark across working sets from 4 KB to 1 GB. Plot it. Compare the plateaus to **lscpu**'s reported cache sizes. Then repeat with *sequential* access and explain why the curve flattens.
- Stride experiment.** Measure the cost of touching one byte every 1, 2, 4 ... 8192 bytes. Find the knee. Confirm it matches **getconf LEVEL1_DCACHE_LINESIZE**.
- Branch prediction.** Reproduce the sorted/unsorted experiment. First confirm GCC flattens it to **cmov** at **-O2**. Then force a real branch three ways (a **noinline** call, **-O0**, a **volatile** condition) and measure the speedup from sorting each time.

4. **Branchless suite.** Implement branchless `min`, `max`, `abs`, `clamp`, and conditional-swap. Benchmark against the branchy versions on (a) random data, (b) sorted data, (c) all-same data. Build a table of when each wins.
5. **False sharing.** On a real multicore machine: 4 threads incrementing adjacent `longs` vs `alignas(64)` padded ones. Measure. Then use `perf c2c` to see the cache-line contention directly.
6. **AoS vs SoA.** Build a particle system with 8 fields and 1M particles. Implement position-integration (touches 6 fields) and a "count particles above y=0" query (touches 1 field) in both layouts. Measure all four combinations and explain the pattern.
7. **Benchmark harness.** Write a reusable harness with: warmup, N repetitions, min/median/max/stddev reporting, DCE prevention, and CPU pinning. Use it to re-measure something you benchmarked earlier in this course and see whether your original number was honest.
8. **Profile something real.** Take a program over 500 lines. Profile with `perf record -g` and generate a flame graph. Identify the top 3 hot functions. Run `perf stat` and diagnose from IPC and miss rates whether it's memory- or compute-bound. Optimize the top one. Re-measure end-to-end and compute the actual speedup vs the theoretical Amdahl limit.
9. **Vectorization.** Write a `saxpy` (`y[i] += a*x[i]`). Measure at `-O2`, `-O3`, `-O3 -march=native`, with and without `restrict`, and with hand-written AVX2. Use `-fopt-info-vec-missed` to find out why each version did or didn't vectorize.
10. **Float reassociation.** Sum 10 million floats scalar and vectorized. Compare the results bit-for-bit. Quantify the error. Then implement Kahan summation and compare accuracy and speed against both.
11. **PGO.** Take any nontrivial program. Build with `-O2`, then with `-fprofile-generate` / representative run / `-fprofile-use`. Measure the difference. Then check whether hand-adding `likely/unlikely` on top of PGO helps or hurts.
12. **Optimizer limits.** For each of the 9 "can't do" items in §25.8, write a small program demonstrating it, dump the assembly, and show the source change that fixes it.

Next: **Module 26 — Portability**, covering the endianness, word-size, and alignment assumptions that break when your code leaves your machine; feature-test macros and the `_POSIX_C_SOURCE` maze you've hit repeatedly in this course; freestanding vs hosted implementations; and how to build the abstraction layer that keeps `#ifdef` out of your logic.

26. Portability

- Endianness, word size, `LP64` vs `LLP64`
- Alignment and unaligned access on strict architectures
- Feature test macros (`_POSIX_C_SOURCE`, `_GNU_SOURCE`, `_DEFAULT_SOURCE`)
- Platform detection macros, compiler-specific extensions
- Freestanding vs hosted implementations
- Toolchain differences and abstraction layers

Module 26 — Portability

Real GCC 13.3 / glibc 2.39 on x86-64 Linux (LP64). This module is about the assumptions that break when your code leaves your machine.

26.1 Endianness

```

=== DETECTION ===
__BYTE_ORDER__          : little
runtime union check     : little

=== HOW IT LOOKS IN MEMORY ===
uint32_t 0x12345678 in memory: 78 56 34 12
little-endian: 78 56 34 12   big-endian: 12 34 56 78

```

Little-endian stores the least-significant byte first (x86, ARM in practice, RISC-V). **Big-endian** stores the most-significant byte first — also called *network byte order* (SPARC, older PowerPC, and every network protocol).

Detection, best to worst

```

/* 1. C23 – the standard way */
#if __STDC_ENDIAN_NATIVE__ == __STDC_ENDIAN_LITTLE__

/* 2. GCC/Clang – universally available in practice */
#if __BYTE_ORDER__ == __ORDER_LITTLE_ENDIAN__

/* 3. Runtime union – always works, no #ifdef, optimizes to a constant */
union { uint32_t u; uint8_t b[4]; } e = { .u = 1 };
bool little = (e.b[0] == 1);

```

Byte swapping

```

__builtin_bswap16(0x1234)      = 0x3412
__builtin_bswap32(0x12345678) = 0x78563412
__builtin_bswap64              = 0xEFCDAB8967452301

```

Portable names: `htons/htonl/ntohs/ntohl` (POSIX), or glibc's `<endian.h>` family (`htobe32`, `le64toh`, ...).

⚠ The right answer: don't detect at all

```

static void put_u32be(uint8_t *p, uint32_t v)
{ p[0]=(uint8_t)(v>>24); p[1]=(uint8_t)(v>>16); p[2]=(uint8_t)(v>>8); p[3]=
(uint8_t)v; }

static uint32_t get_u32be(const uint8_t *p)

```

```
{ return ((uint32_t)p[0]<<24)|((uint32_t)p[1]<<16)|((uint32_t)p[2]<<8)|
(uint32_t)p[3]; }
```

```
put_u32be(0xDEADBEEF) -> DE AD BE EF
get_u32be()           -> 0xDEADBEEF  round-trip OK: 1
```

This code produces identical bytes on every machine. No `#ifdef`, no detection, no byte-swap builtin, no alignment requirement. And GCC recognizes the pattern and compiles it to a single `bswap` on little-endian.

Never `fwrite` a struct or `memcpy` an integer into a wire buffer (Modules 11, 13). Serialize field by field with explicit shifts. That single discipline eliminates endianness, padding, and alignment portability problems at once.

Also note: **bitfield order** is implementation-defined (Module 11), so bitfields are never portable across the wire either.

26.2 Word size and data models

```
=== THIS PLATFORM: x86_64 / LP64 ===
char 1  short 2  int 4  long 8  long long 8
void* 8  size_t 8  ptrdiff_t 8  time_t 8  wchar_t 4  long double 16
CHAR_BIT = 8
```

The models

Model	short	int	long	ptr	Where
ILP32	16	32	32	32	32-bit Linux, Win32
LP64	16	32	64	64	Linux, macOS, BSD (64-bit)
LLP64	16	32	32	64	Windows 64-bit ← the odd one
ILP64	16	64	64	64	Rare (some Cray, SPARC64)

The bugs this causes

```
long p = (long)ptr;           -> TRUNCATES on Win64 (long is 32-bit)
printf("%ld", (long)size)     -> wrong width on Win64
sizeof(long)==sizeof(void*)   -> FALSE on Win64
int used as an array index    -> breaks past 2 GB on all 64-bit models
32-bit time_t                 -> Y2038
```

long is the least portable type in C. It's the only integer type whose size differs between two mainstream 64-bit platforms.

The fixes

Need	Use
Pointer ↔ integer	<code>intptr_t/uintptr_t</code> — never <code>long</code>
Sizes, counts, indices	<code>size_t</code> , print with <code>%zu</code>
Pointer differences	<code>ptrdiff_t</code> , print with <code>%td</code>
Exact widths (protocols, files)	<code>int32_t/uint64_t</code> + <code>PRId32/PRIu64</code>
File offsets	<code>off_t</code> + <code>-D_FILE_OFFSET_BITS=64</code>
Time on 32-bit systems	<code>-D_TIME_BITS=64</code> (Y2038)

Assert your assumptions (Module 15's cheapest tool):

```
_Static_assert(sizeof(void*) == sizeof(uintptr_t), "uintptr_t must hold a
pointer");
_Static_assert(CHAR_BIT == 8, "octet-addressable platform required");
_Static_assert(sizeof(uint32_t) == 4, "exact-width type required");
```

Other size assumptions that bite

Assumption	Reality
<code>CHAR_BIT == 8</code>	True everywhere you'll meet, but some DSPs use 16 or 32
<code>sizeof(int) == 4</code>	2 on some 16-bit embedded targets
<code>char</code> is signed	Unsigned on ARM, PowerPC, s390 (Module 2 — measured)
Plain <code>int</code> bitfield is signed	Implementation-defined (Module 11 — measured <code>-1</code>)
<code>long double</code> is 80-bit	16 bytes on x86-64 Linux, 8 on MSVC, 16-byte quad on ARM64
<code>wchar_t</code> is 4 bytes	2 on Windows (Module 8)
<code>time_t</code> is 64-bit	32-bit on older 32-bit systems
Two's complement	Guaranteed since C23 , not before

26.3 Alignment on strict architectures

Module 9 §9.9 measured this. The portability framing:

Architecture	Unaligned access
x86/x86-64	Works, slower; faults for some SSE/AVX instructions

Architecture	Unaligned access
ARM64	Works for normal loads; faults for LDXR/STXR and some SIMD
ARMv5 and earlier	Silently returns rotated garbage — the worst failure mode
SPARC, MIPS (older), some DSPs	SIGBUS
RISC-V	Implementation-defined; may trap and be emulated (extremely slow)

```
/* NOT portable – misalignment AND strict aliasing (Module 2) */
uint32_t v = *(const uint32_t *)(buf + 1);

/* Portable, correct everywhere, and free at -O2 */
uint32_t v;
memcpy(&v, buf + 1, sizeof v);
```

unaligned store/load at offset 1: 0xDEADBEEF ← via memcpy, ASan/UBSan clean

memcpy is the universal answer — it handles alignment and aliasing simultaneously, and compiles to a single instruction where the hardware allows it.

Related: **alignas** for stronger alignment, **aligned_alloc** for SIMD/DMA buffers, **__attribute__((packed))** creates misaligned members (Module 11 — **-Waddress-of-packed-member**), and structure layout differs between ABIs so **never fwrite a struct**.

26.4 Feature test macros ⚠

You've hit this repeatedly throughout this course. Here's the systematic explanation.

glibc's headers hide symbols by default. Which ones you get depends entirely on feature test macros:

```
### -std=c17 (STRICT: no POSIX at all) ###
error: 'strdup' undeclared
error: 'strtok_r' undeclared
error: 'gmtime_r' undeclared

### -std=gnu17 (the DEFAULT) ###
_POSIX_C_SOURCE = 200809L, _DEFAULT_SOURCE defined
strdup, strtok_r, gmtime_r : all visible

### -std=c17 -D_POSIX_C_SOURCE=200809L ###
all visible

### -std=c17 -D_GNU_SOURCE ###
all visible
```

The macros

Macro	Exposes
(nothing) with <code>-std=cXX</code>	ISO C only. No POSIX.
<code>_DEFAULT_SOURCE</code>	POSIX 2008 + BSD + SVID extensions. The default with <code>-std=gnuXX</code>.
<code>_POSIX_C_SOURCE=200809L</code>	POSIX.1-2008. The portable choice.
<code>_POSIX_C_SOURCE=199309L</code>	POSIX.1b — real-time (<code>clock_gettime</code> , <code>nanosleep</code>)
<code>_XOPEN_SOURCE=700</code>	XSI = POSIX 2008 + extras (<code>strdup</code> , <code>crypt</code> , <code>sync</code>)
<code>_GNU_SOURCE</code>	Everything , including GNU-only (<code>asprintf</code> , <code>memmem</code> , <code>pipe2</code> , <code>qsort_r</code> , <code>strchrnul</code>)
<code>_FILE_OFFSET_BITS=64</code>	64-bit <code>off_t</code> on 32-bit systems
<code>_TIME_BITS=64</code>	64-bit <code>time_t</code> (Y2038)
<code>_LARGEFILE64_SOURCE</code>	The <code>*64</code> variants (<code>lseek64</code>)

⚠ The ordering rule

```
/* WRONG */
#include <string.h>      /* string.h is processed FIRST */
#define _GNU_SOURCE      /* ...too late */
```

```
warning: implicit declaration of function 'strdup'
```

```
/* RIGHT */
#define _GNU_SOURCE      /* FIRST LINE, before ANY include */
#include <string.h>
```

```
correct order works: x
```

Feature test macros must appear before every `#include`, including transitively. In practice: **define them on the command line** (`-D_POSIX_C_SOURCE=200809L`) or in your build system, never in source files. That way you can't get the order wrong.

Why this whole system exists

Standards evolve. `strdup` was BSD, then POSIX 2001, then POSIX 2008, and is now **ISO C23** (Module 18). Feature test macros let one set of headers serve strict-conformance builds, POSIX builds, and GNU builds —

with different symbol visibility for each.

The practical policy

```
# Portable POSIX code – recommended default
CPPFLAGS += -D_POSIX_C_SOURCE=200809L -D_FILE_OFFSET_BITS=64

# Or, for Linux-only code with GNU extensions
CPPFLAGS += -D_GNU_SOURCE

# Or just use -std=gnu17 (GCC's default), which sets _DEFAULT_SOURCE
```

Pick one, set it globally, document why. Mixing them per-file is how you get "it compiles in one directory and not another."

⚠ **Non-glibc systems differ.** `musl` is more permissive; `macOS` uses `_DARWIN_C_SOURCE`; `BSDs` have their own conventions; `MSVC` has none of this. Test on your actual targets.

26.5 Platform and capability detection

```
=== COMPILER ===
gcc 13.3.0
NOTE: clang defines __GNUC__ too -> check __clang__ FIRST

=== OS ===          linux, __unix__ also defined
=== ARCHITECTURE === x86-64
```

The macros

Category	Macros
Compiler	<code>__GNUC__</code> , <code>__clang__</code> , <code>_MSC_VER</code> , <code>__INTEL_LLVM_COMPILER</code> , <code>__TINYC__</code>
OS	<code>_WIN32</code> (also on Win64), <code>_WIN64</code> , <code>__linux__</code> , <code>__APPLE__</code> , <code>__FreeBSD__</code> , <code>__unix__</code>
Arch	<code>__x86_64__</code> / <code>_M_X64</code> , <code>__i386__</code> , <code>__aarch64__</code> / <code>_M_ARM64</code> , <code>__arm__</code> , <code>__riscv</code> (+ <code>__riscv_xlen</code>)
Model	<code>__LP64__</code> , <code>_LP64</code> , <code>__ILP32__</code>

Three traps:

- **Clang defines `__GNUC__`** — check `__clang__` first.
- **`_WIN32` is defined on 64-bit Windows too** — use it for "Windows," `_WIN64` for "64-bit."
- **`__APPLE__` covers macOS and iOS** — use `<TargetConditionals.h>` to distinguish.

☒ Prefer capability probes over version checks

```
__has_include(<threads.h>)    = 1
__has_include(<stdbit.h>)     = 1
__has_include(<sys/epoll.h>)  = 1
__has_builtin(add_overflow)   = 1
__has_attribute(nonnull)      = 1
```

```
#if defined(__has_include) && __has_include(<stdckdint.h>)
# include <stdckdint.h>
# define mul_ovf(r,a,b) ckd_mul((r),(a),(b))
#elif defined(__has_builtin) && __has_builtin(__builtin_mul_overflow)
# define mul_ovf(r,a,b) __builtin_mul_overflow((a),(b),(r))
#else
# define mul_ovf(r,a,b) mul_ovf_fallback((r),(a),(b))
#endif
```

`__has_include`, `__has_builtin`, `__has_attribute`, and `__has_c_attribute` ask "can you do this?" rather than "which version are you?" That's far more robust — backported features, vendor forks, and partial implementations all work correctly.

C11 optional-feature markers:

```
threads.h supported / stdatomic.h supported / VLAs supported
```

`__STDC_NO_THREADS__`, `__STDC_NO_ATOMICS__`, `__STDC_NO_VLA__`, `__STDC_NO_COMPLEX__`.

Build-mode detection:

```
ASan instrumentation detected    ← __SANITIZE_ADDRESS__
debug build (asserts active)    ← !NDEBUG
```

Useful for skipping timing-sensitive tests under sanitizers, or adjusting timeouts.

Reference: sourceforge.net/p/predef/wiki has the exhaustive list.

26.6 Freestanding vs hosted

The C standard defines two conformance levels:

	Hosted	Freestanding
Entry point	<code>main</code>	Implementation-defined (<code>_start</code> , a reset vector, ...)

	Hosted	Freestanding
Standard library	All of it	Only: <code><float.h></code> <code><iso646.h></code> <code><limits.h></code> <code><stdalign.h></code> <code><stdarg.h></code> <code><stdbool.h></code> <code><stddef.h></code> <code><stdint.h></code> <code><stdnoreturn.h></code>
OS	required	none
Examples	Applications	Kernels, bootloaders, bare-metal firmware

A real freestanding program

```
#include <stdint.h>
#include <stddef.h>

/* No libc – provide our own; the compiler may still CALL these */
void *memset(void *d, int c, size_t n){ unsigned char *p=d; while(n--) *p++=(unsigned char)c; return d; }
void *memcpy(void *d, const void *s, size_t n){ ... }

static long sys_write(int fd, const void *buf, size_t n)
{   long r;
    __asm__ volatile ("syscall" : "=a"(r) : "a"(1L), "D"((long)fd), "S"(buf), "d"(n)
                        : "rcx", "r11", "memory");
    return r; }

/* _start, not main: there is no C runtime to call main for us */
__attribute__((noreturn)) void _start(void) { ...; sys_exit(0); }
```

```
gcc -ffreestanding -nostdlib -nostartfiles -static -O2 free.c -o freestanding
```

```
freestanding: no libc, no crt, direct syscalls
value = 42

freestanding      :   9368 bytes
hosted (dyn)      :  15960 bytes
hosted (static): 785360 bytes    ← 84× larger

freestanding: not a dynamic executable
```

What you lose and must provide

- `memset`, `memcpy`, `memmove`, `memcpy` — GCC emits calls to these even with `-ffreestanding`, so you must supply them
- Startup code: zeroing `.bss`, copying `.data` from flash to RAM, setting up the stack (Module 27)
- `malloc`, `printf`, all I/O, all math, `errno`
- Static initializers and `atexit`

The flags

Flag	Effect
- <code>ffreestanding</code>	The implementation is freestanding; don't assume libc semantics for <code>main</code> , <code>printf</code> , etc.
<code>-nostdlib</code>	Don't link libc or the startup files
<code>-nostartfiles</code>	Don't link <code>crt1.o/crti.o</code>
- <code>nodefaultlibs</code>	Don't link default libraries but keep the startup files
<code>-fno-builtin</code>	Don't recognize standard function names as builtins
<code>-static</code>	No dynamic loader

This is exactly how the Linux kernel, U-Boot, FreeRTOS applications, and every bare-metal firmware image is built (Module 27).

26.7 Abstraction layers

The single most important portability practice: put every `#ifdef` in one file.

Application code should be plain, portable C. Platform differences live in a `plat.h` + `plat_posix.c` / `plat_win32.c` pair.

```
/* plat.h – the ONE header containing every #ifdef in the project */

/* 1. compiler attributes */
#if defined(__clang__) || defined(__GNUC__)
# define PLAT_PRINTF(m,n) __attribute__((format(printf,m,n)))
# define PLAT_UNLIKELY(x) __builtin_expect(!(x),0)
# define PLAT_EXPORT __attribute__((visibility("default")))
#elif defined(_MSC_VER)
# define PLAT_PRINTF(m,n)
# define PLAT_UNLIKELY(x) (x)
# define PLAT_EXPORT __declspec(dllexport)
#else
# define PLAT_PRINTF(m,n)
# define PLAT_UNLIKELY(x) (x)
# define PLAT_EXPORT
#endif

/* prefer C23 attributes where they exist */
#if defined(__has_c_attribute) && __has_c_attribute(nodiscard)
# define PLAT_NODISCARD [[nodiscard]]
#elif defined(__GNUC__)
# define PLAT_NODISCARD __attribute__((warn_unused_result))
#else
# define PLAT_NODISCARD
```

```

#endif

/* 2. checked arithmetic: C23 → GNU → fallback */
#if defined(__has_include) && __has_include(<stdckdint.h>)
# include <stdckdint.h>
# define plat_mul_ovf(r,a,b) ckd_mul((r),(a),(b))
#elif defined(__GNUC__)
# define plat_mul_ovf(r,a,b) __builtin_mul_overflow((a),(b),(r))
#else
# define plat_mul_ovf(r,a,b) plat_mul_ovf_fn((r),(a),(b))
#endif

/* 3. byte order: EXPLICIT, no detection */
static inline void      plat_put_u32be(uint8_t *p, uint32_t v) { ... }
static inline uint32_t plat_get_u32be(const uint8_t *p)      { ... }

/* 4. alignment-safe access */
static inline uint32_t plat_load_u32(const void *p)
{ uint32_t v; memcpy(&v,p,sizeof v); return v; }

/* 5. platform services – implemented per-OS */
PLAT_NODISCARD int      plat_mkdir(const char *path);
PLAT_NODISCARD int      plat_sleep_ms(unsigned ms);
PLAT_NODISCARD uint64_t plat_mono_ns(void);

/* 6. assert your assumptions */
_Static_assert(sizeof(void*) == sizeof(uintptr_t), "uintptr_t must hold a
pointer");
_Static_assert(CHAR_BIT == 8, "octet-addressable platform required");

```

The application then contains **no `#ifdef` at all**:

```

#include "plat.h"
int main(void)
{
    uint64_t t0 = plat_mono_ns();
    if (plat_sleep_ms(20) != 0) return 1;
    printf("slept %.1f ms\n", (double)(plat_mono_ns()-t0)/1e6);

    uint8_t wire[8];
    plat_put_u32be(wire, 0xCAFEBAFE);
    plat_store_u32(buf + 1, 0xDEADBEEF);          /* misaligned – still correct */

    size_t total;
    if (plat_mul_ovf(&total, (size_t)1<<40, (size_t)1<<40)) { /* overflow */ }
}

```

```

platform layer: posix
slept 20.1 ms

```

```
serialized: CA FE BA BE -> 0xCAFEBAFE
unaligned store/load at offset 1: 0xDEADBEEF
overflow correctly detected
ASan/UBSan exit=0
```

Why this matters

Scattered <code>#ifdef</code>	One abstraction layer
2 ⁿ programs, you test one	One program, n small backends
Every configuration rots independently	Only the backends can rot
Unreadable business logic	Logic reads as plain C
Adding a platform touches every file	Adding a platform means one new <code>.c</code>

Prefer runtime `if` to `#ifdef` where possible (Module 12) — all configurations stay compiled and type-checked.

Existing abstraction layers worth using

libuv (event loop, files, processes, threads — the best general one), **SDL** (windowing, input, audio), **APR** (Apache Portable Runtime), **glib** (data structures + platform layer), **PDCurses/ncurses**, **mimalloc**.

Don't write your own if one of these fits.

Module 26 rules

#	Rule
1	Serialize with explicit shifts — no endianness detection needed
2	Never <code>fwrite/memcpy</code> a struct into a file or socket
3	<code>long</code> is the least portable type — it's 32-bit on Win64
4	<code>intptr_t</code> for pointers, <code>size_t/%zu</code> for sizes, <code>intN_t/PRId32</code> for wire formats
5	<code>_Static_assert</code> every size and alignment assumption
6	<code>memcpy</code> for all unaligned and type-punned access
7	Feature test macros go on the command line , before every include
8	Pick one FTM policy per project and document it
9	Check <code>__clang__</code> before <code>__GNUC__</code> ; <code>_WIN32</code> is defined on Win64 too
10	Prefer <code>__has_include/</code> <code>__has_builtin/</code> <code>__has_attribute</code> over version checks
11	Freestanding gives you 9 headers and no <code>main</code> — you supply <code>memcpy</code> and startup
12	One file holds every <code>#ifdef</code> ; application code is plain C

#	Rule
13	Prefer runtime <code>if</code> to <code>#ifdef</code> so all configurations stay compiled
14	Test on real targets — cross-compile + QEMU in CI (Module 19)

Practice

1. **Indian round-trip.** Write `put_u{16,32,64}{be,le}` and matching getters. Serialize a struct with 6 fields, dump the bytes with `xxd`, and confirm they're identical when cross-compiled and run under `qemu-aarch64` (or a big-endian target like `qemu-mips`).
2. **Find the long bugs.** Take any C project and grep for `(long)`, `%ld`, and `sizeof(long)`. For each, decide whether it would break on Win64. Fix the ones that would.
3. **Size assertion header.** Write `assumptions.h` containing `_Static_asserts` for every size, alignment, and signedness assumption your code makes. Include it everywhere. Then compile with `-m32` and see which ones fire.
4. **Alignment fault.** Cross-compile a program doing `*(uint32_t*)(buf+1)` for a strict-alignment target (`qemu-mips` or `qemu-sparc`) and observe the SIGBUS. Then fix it with `memcpy` and confirm it works. Compare the generated assembly on both x86-64 and the strict target.
5. **FTM archaeology.** Take one source file using `strdup`, `strtok_r`, `nanosleep`, `asprintf`, and `pipe2`. Find the minimum feature test macro that exposes each. Then find the single macro that exposes all of them, and determine which of them are non-portable off Linux.
6. **The ordering bug.** Reproduce the "define `_GNU_SOURCE` after the first include" failure. Then find a header in a real project that defines a FTM in the wrong place.
7. **Capability probe header.** Write `compat.h` providing `MY_NODISCARD`, `MY_FALLTHROUGH`, `MY_LIKELY`, `ckd_add`, `secure_zero`, and `mono_ns`, using `__has_include`/`__has_builtin`/`__has_c_attribute` throughout — with no compiler-version checks at all. Verify it builds under `-std=c17` and `-std=c2x`, with GCC and Clang.
8. **Freestanding hello.** Build the §26.6 program. Then extend it: implement `memcmp` and a minimal `printf` supporting `%d` and `%s`. Measure the final binary size. Then try it with `-Os -ffunction-sections -Wl,--gc-sections` (Module 19).
9. **Build the platform layer.** Implement `plat.h` + `plat_posix.c` covering: monotonic time, sleep, mkdir, file existence, path separator, temp directory, thread creation, and mutex. Then write `plat_win32.c` (compile-test it with `x86_64-w64-mingw32-gcc` if available). Confirm the application file contains zero `#ifdef`.
10. **Cross-compile CI.** Add a CI job that cross-compiles your project for `aarch64-linux-gnu` and runs the test suite under `qemu-aarch64-static`. Then add a big-endian target. Record every bug it finds.

Next: **Module 27 — Embedded / Bare-Metal C**, the largest employer of C developers: memory-mapped I/O and correct `volatile` usage, interrupts and reentrancy, linker scripts and startup code, no-dynamic-allocation

strategies, fixed-point arithmetic, the peripheral protocols (UART/SPI/I2C/DMA), RTOS concepts, and the safety standards that govern automotive, medical, and aerospace C.

27. Embedded / Bare-Metal C (huge C job market)

- MCU architecture, memory maps, registers
- **Memory-mapped I/O and correct volatile usage**
- Interrupts, ISRs, vector tables, latency, reentrancy
- **Linker scripts**, startup code, .data/.bss init, stack setup
- No dynamic allocation / static allocation strategies
- Fixed-point arithmetic, no-FPU environments
- Peripherals & protocols: GPIO, UART, SPI, I2C, CAN, ADC, PWM, DMA, timers, watchdog
- RTOS: FreeRTOS, Zephyr, ThreadX — tasks, queues, priorities, priority inversion
- Low-power modes, boot loaders, firmware update / OTA
- Debug hardware: JTAG/SWD, OpenOCD, logic analyzers, oscilloscopes
- MISRA C compliance, safety standards (ISO 26262, DO-178C, IEC 61508)
- CMSIS, HAL vs register-level programming

Module 27 — Embedded / Bare-Metal C

The largest employer of C developers. Everything below is built and inspected with a real arm-none-eabi-gcc 13.2 toolchain targeting Cortex-M4.

27.1 MCU architecture, memory maps, registers

A microcontroller is a CPU core plus memory plus peripherals on one die. A typical Cortex-M4 memory map:

```
0x00000000 - Code (aliased flash)
0x08000000 - FLASH      512 KB  ← program + constants + .data initial values
0x20000000 - SRAM       128 KB  ← .data, .bss, heap, stack
0x40000000 - Peripherals      ← GPIO, UART, SPI, timers... memory-mapped
0xE0000000 - Cortex-M private ← NVIC, SysTick, SCB, debug
0xFFFFFFFF
```

Everything is memory. Configuring a UART means writing to an address. Reading a pin means loading from an address. There is no I/O instruction, no driver model, no OS.

Registers come in flavors that C's type system doesn't express:

Kind	Behavior
Read-write	Ordinary
Read-only	Status registers — hardware changes them under you
Write-only	Reading returns garbage

Kind	Behavior
Write-1-to-clear	Writing 1 <i>clears</i> a flag
Read-to-clear	The act of reading has a side effect
Set/Reset pairs (BSRR)	Atomic bit manipulation without read-modify-write

That last one matters: `GPIOA->ODR |= (1<<5)` is a **read-modify-write** — an interrupt between the read and the write loses concurrent changes. `GPIOA->BSRR = (1<<5)` is a single atomic store. **Use set/reset registers where the hardware provides them.**

27.2 Memory-mapped I/O and **volatile** ⚠

```
#define GPIOA_BASE 0x40020000u
typedef struct {
    volatile uint32_t MODER, OTYPER, OSPEEDR, PUPDR;
    volatile uint32_t IDR;      /* input data — READ ONLY, changes by hardware */
    volatile uint32_t ODR;      /* output data */
    volatile uint32_t BSRR;     /* bit set/reset — WRITE ONLY, atomic */
} GPIO_TypeDef;
#define GPIOA ((GPIO_TypeDef *)GPIOA_BASE)
```

A struct overlaid on a hardware address is the standard idiom — CMSIS device headers are thousands of lines of exactly this.

Why **volatile** is not optional

```
void blink(void) {
    GPIOA->ODR |= (1u << 5); /* on */
    GPIOA->ODR &= ~(1u << 5); /* off */
    GPIOA->ODR |= (1u << 5); /* on */
}
```

With **volatile** — three real read-modify-writes:

```
ldr  r2, [r3, #20]    orr r2, r2, #32    str r2, [r3, #20]
ldr  r2, [r3, #20]    bic r2, r2, #32    str r2, [r3, #20]
ldr  r2, [r3, #20]    orr r2, r2, #32    str r2, [r3, #20]
```

Without **volatile** — the compiler collapses it to the net effect:

```
ldr  r3, [r2, #20]    orr r3, r3, #32    str r3, [r2, #20]
bx   lr
```

One store. The LED would never blink.

And a status-polling loop:

```
while ((GPIOA->IDR & 1u) == 0) { }
```

With **volatile** — re-reads every iteration:

```
.L9:    ldr    r3, [r2, #16]    lsls r3, r3, #31    bpl .L9
```

Without **volatile** — reads once and spins forever:

```
        ldr    r0, [r3, #16]    lsls r3, r0, #31    bmi .L14
.L16:    b      .L16            ← infinite loop, hardware never re-read
```

This is **volatile's actual purpose** (Module 2). It is not thread safety (Module 17) — it is "this memory changes outside the program's control, so emit every access exactly as written."

What **volatile** does *not* give you

- **No ordering with non-volatile accesses.** Use a compiler barrier: `__asm__ __volatile__("" ::: "memory")`.
- **No CPU memory ordering.** On Cortex-M with a write buffer or on multi-master systems you need `__DSB()`, `__DMB()`, `__ISB()`. A classic bug: write to a peripheral, immediately sleep — the write is still buffered. `__DSB()` before `__WFI()`.
- **No atomicity** for anything wider than one word.
- **Cache coherence** — on Cortex-A / cached MCUs, DMA buffers need explicit clean/invalidate.

27.3 Interrupts, ISRs, reentrancy

The vector table

On Cortex-M the CPU loads **two words from address 0** on reset:

```
Contents of section .isr_vector:
80000000 00000220 55000008 51000008 51000008 ...
```

```
word 0 = 0x20020000 = initial STACK POINTER (RAM top: 0x20000000 + 128K)
word 1 = 0x08000055 = Reset_Handler (Thumb bit set → odd address)
```


The rest is a table of function pointers, one per exception and IRQ:

```
__attribute__((section(".isr_vector"), used))
void (* const g_vectors[])(void) = {
    (void (*)(void))&_estack, /* [0] stack pointer */
    Reset_Handler,           /* [1] reset */
    NMI_Handler, HardFault_Handler, MemManage_Handler,
    BusFault_Handler, UsageFault_Handler,
    0, 0, 0, 0,               /* reserved */
    SVC_Handler, 0, 0, PendSV_Handler, SysTick_Handler,
    /* device-specific IRQs from here */
    WWDG_IRQHandler, EXTI0_IRQHandler, TIM2_IRQHandler, USART1_IRQHandler,
};
```

Weak aliases let a driver override only the handlers it implements:

```
#define WEAK_ALIAS __attribute__((weak, alias("Default_Handler")))
void TIM2_IRQHandler(void) WEAK_ALIAS;
void Default_Handler(void){ for(;;){} } /* trap unhandled interrupts */
```

Module 22's weak-symbol technique, applied to hardware.

⚠ Sharing data with an ISR

32-bit reads are atomic on ARM (one instruction):

```
read_tick_ok: ldr r3, .L7      ldr r0, [r3]      bx lr
```

64-bit reads are not:

```
read_micros_bad: ldrd r0, [r3, #8]      ← two words; an ISR between them tears the
value
```

Two correct fixes:

```
/* 1. Lock-free double-read (works when the ISR only increments) */
uint64_t read_micros_ok(void)
{
    uint64_t a, b;
    do { a = g_micros; b = g_micros; } while (a != b);
    return b;
}

/* 2. Critical section */
```

```
static inline uint32_t irq_save(void) {
    uint32_t p;
    __asm__ volatile ("mrs %0, primask\n cpsid i" : "=r"(p) :: "memory");
    return p;
}
static inline void irq_restore(uint32_t p) {
    __asm__ volatile ("msr primask, %0" :: "r"(p) : "memory");
}
```

```
read_micros_critical:
    mrs r3, primask      ← save the interrupt-enable state
    cpsid i              ← disable interrupts
    ldrd r0, [r2, #8]
    msr primask, r3      ← RESTORE (not blindly re-enable)
    bx lr
```

Save and restore, never blindly `cpsie i` — the caller might already have had interrupts disabled.

The ISR rules

1. **Keep it short.** Set a flag, push to a queue, return. Do the work in the main loop.
2. **volatile on every variable shared with an ISR.**
3. **No malloc, no printf, no blocking, no floating point** (unless you've enabled FPU context saving).
4. **Never wait for anything** — no polling loops, no semaphore-take-with-timeout.
5. **Clear the interrupt flag**, or you'll re-enter immediately. On some MCUs the write must complete (`__DSB()`) before returning, or you get a spurious re-entry.
6. **Nesting:** Cortex-M NVIC supports priority-based preemption. Higher-priority ISRs interrupt lower ones — so shared data needs protection between *ISRs*, not just *ISR-vs-main*.
7. **Latency** = interrupt assertion → first handler instruction. On Cortex-M it's typically **12 cycles** (hardware stacks the registers). Disabling interrupts adds directly to worst-case latency — keep critical sections tiny.
8. **Stack:** ISRs run on the main stack (or the process stack under an RTOS). Nested interrupts each need frame space. **Budget for the deepest nesting.**

27.4 Linker scripts and startup code ⚠

This is where "freestanding" (Module 26) becomes concrete. **You write the C runtime.**

The linker script

```
ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
```

```

}

_estack = ORIGIN(RAM) + LENGTH(RAM);      /* stack grows DOWN from the top */

SECTIONS
{
    .isr_vector : { KEEP(*(.isr_vector)) } > FLASH /* KEEP: --gc-sections must not
remove it */

    .text : { *(.text*) *(.rodata*)  . = ALIGN(4); _etext = .; } > FLASH

    /* .data lives in RAM but its INITIAL VALUES are stored in FLASH */
    _sdata = LOADADDR(.data);
    .data : { . = ALIGN(4); _sdata = .;  *(.data*)  . = ALIGN(4); _edata = .; } >
RAM AT> FLASH

    .bss  : { . = ALIGN(4); _sbss  = .;  *(.bss*) *(COMMON)  . = ALIGN(4); _ebss =
.; } > RAM

    /* fail the LINK if there is no room left for heap + stack */
    ._check : { . = ALIGN(8); . = . + _min_heap + _min_stack; . = ALIGN(8); } > RAM
}

```

The measured result:

Idx	Name	Size	VMA	LMA	File off
0	.isr_vector	00000050	08000000	08000000	
1	.text	000000c8	08000050	08000050	
2	.data	00000004	20000000	08000118	← VMA=RAM, LMA=FLASH
3	.bss	00000008	20000004	0800011c	

> **RAM AT> FLASH is the key line.** **.data runs** in RAM (VMA) but is **stored** in flash (LMA). The startup code copies it. This is the mechanism behind Module 3's **.data** vs **.bss** distinction, made explicit.

The startup code

```

extern uint32_t _sdata, _edata, _sbss, _ebss; /* linker-defined */

void Reset_Handler(void)
{
    uint32_t *src = &_sdata, *dst = &_sdata;
    while (dst < &_edata) *dst++ = *src++;          /* 1. copy .data FLASH →
RAM */
    for (dst = &_sbss; dst < &_ebss; ) *dst++ = 0; /* 2. zero .bss (Module
3!) */
    /* 3. __libc_init_array() - C++ ctors, __attribute__((constructor)) */
    main();                                         /* 4. finally, main()
*/
    for (;;) { }                                  /* main must never return

```

```
*/
}
```

The real disassembly:

```
08000054 <Reset_Handler>:
 8000054: push {r3, lr}
 8000056: ldr r2, [pc,#40]      ← _sidata (0x080000f0, in FLASH)
 8000058: ldr r3, [pc,#40]      ← _sdata (0x20000000, in RAM)
 800005c: cmp r3, r1
 800005e: bcc 8000070
 8000070: ldr.w r0, [r2], #4    ← the .data copy loop
 8000074: str.w r0, [r3], #4
 8000066: cmp r3, r2
 800007a: str.w r1, [r3], #4    ← the .bss zeroing loop
 800006a: bl 80000a4 <main>
 800006e: b.n 800006e          ← trap if main returns
```

Module 3's "statics are zero-initialized" guarantee is these six instructions. On a hosted system `crt0.o` does it; here you do.

The build and the flashable image

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -Os -g \
  -ffreestanding -nostdlib -nostartfiles \
  -ffunction-sections -fdata-sections -Wl,--gc-sections \
  -T link.ld startup.c main.c -o fw.elf -Wl,-Map=fw.map

arm-none-eabi-size fw.elf
  text    data    bss     dec     hex
    280      4   3084   3368    d28

arm-none-eabi-objcopy -O binary fw.elf fw.bin    # 284 bytes – raw flash image
arm-none-eabi-objcopy -O ihex   fw.elf fw.hex    # Intel HEX
```

```
:0200000040800F2
:10000000000000220550000085100000851000008BF
```

A complete, flashable firmware image in 284 bytes. Compare to Module 26's 785 KB statically-linked hosted hello-world.

Note `-Wl, -Map=fw.map` — **always generate a map file.** When you're 200 bytes over your flash budget, it tells you exactly what's consuming space.

27.5 Static allocation

Most embedded codebases ban `malloc` entirely. MISRA C forbids it (Rule 21.3); DO-178C and IEC 61508 effectively do too.

Why:

- **Fragmentation** — a device running for years will eventually fail an allocation
- **Non-deterministic timing** — unacceptable in hard real-time
- **No recovery** — what does a thermostat do when `malloc` returns NULL?
- **Unbounded memory** — you can't prove the worst case
- **Heap corruption is unrecoverable** with no MMU

The strategies

```
/* 1. Static pools – allocate everything at compile time */
#define MAX_CONNECTIONS 16
static Connection g_conns[MAX_CONNECTIONS];
static uint32_t g_conn_used; /* a bitmask, or a free list */

/* 2. Fixed-size block allocator (Module 10's pool, statically backed) */
static uint8_t g_pool[MAX_BLOCKS][BLOCK_SIZE] __attribute__((aligned(8)));

/* 3. Arena reset per cycle (Module 10) */
static uint8_t g_arena[8192];
/* ... allocate during one control cycle, arena_reset() at the end ... */

/* 4. Caller-provides-storage APIs */
int parser_init(Parser *p, uint8_t *buf, size_t buflen); /* no allocation at all */

/* 5. Ring buffers for streaming */
static uint8_t g_uart_rx[256]; /* power of 2 → masking, not modulo */
```

The discipline this forces

- **Size everything at compile time**, then `_Static_assert` the totals fit.
- **Bound every input.** No VLAs, no `alloca` (Modules 7, 10).
- **Compute worst-case stack usage** — `-fstack-usage` emits per-function `.su` files; tools like `punclover` and `StackAnalyzer` roll them up. Add ISR nesting depth.
- **Fill unused RAM with a pattern at startup** and check the high-water mark at runtime — the standard stack-usage measurement.

Recursion is banned too (MISRA Rule 17.2) for the same reason: unbounded stack.

27.6 Fixed-point arithmetic

Many MCUs have **no FPU**, or only single-precision. Measured on Cortex-M4:

```

typedef int32_t q16_t;                                /* Q16.16: 16 integer, 16 fractional
bits */
#define Q16_ONE    (1 << 16)
#define Q16(x)     ((q16_t)((x) * (double)Q16_ONE))    /* compile-time only */

static inline q16_t q16_mul(q16_t a, q16_t b){ return (q16_t)((((int64_t)a * b) >>
16); }
static inline q16_t q16_div(q16_t a, q16_t b){ return (q16_t)((((int64_t)a) << 16)
/ b); }

```

The same computation, $x * 3.5 + 1.25$:

Fixed-point — 6 inline integer instructions, no library:

```

calc_fixed:
    mov.w r3, #229376          @ 3.5 in Q16.16
    smull r0, r3, r0, r3       @ 64-bit multiply
    lsrs r0, r0, #16
    orr.w r0, r0, r3, lsl #16
    add.w r0, r0, #81920       @ 1.25 in Q16.16
    bx lr

```

Float with soft-float — two library calls:

```

calc_float:
    push {r3, lr}
    bl __aeabi_fmul            ← software floating point
    bl __aeabi_fadd
    pop {r3, pc}

```

```

soft-float library pulled in:
__aeabi_dadd __aeabi_dmul __aeabi_fadd __aeabi_fmul

```

Float with a hardware FPU — 4 VFP instructions, one fused multiply-add:

```

calc_float:
    vmov.f32 s14, #12          @ 3.5
    vmov.f32 s15, #116         @ 1.25
    vfma.f32 s15, s0, s14      ← fused multiply-add, single instruction
    bx lr

```

The rules

Rule	Why
Never use <code>double</code> on a 32-bit MCU	<code>-mfloat-abi=hard -mfpv4-sp-d16</code> is single-precision only — every <code>double</code> silently falls back to software. <code>3.14</code> is a <code>double</code> ; write <code>3.14f</code> .
<code>-Wdouble-promotion</code>	Catches accidental promotion (Module 19)
Use <code>float</code> variants	<code>sinf</code> , <code>sqrtf</code> , <code>fabsf</code> — not <code>sin</code> , <code>sqrt</code> , <code>fabs</code>
Q-format for sensor math	Deterministic timing, no library, exact for the range you chose
Intermediate in 64-bit	<code>((int64_t)a * b) >> 16</code> — otherwise <code>q16_mul</code> overflows immediately
Watch saturation	Fixed-point overflows silently; add saturating helpers where inputs are unbounded
FPU + ISRs	If an ISR uses the FPU, the FPU context must be stacked (Cortex-M lazy stacking). Getting this wrong corrupts float state non-deterministically.

27.7 Peripherals and protocols

Peripheral	Key points
GPIO	Mode (in/out/alt/analog), pull-up/down, speed, open-drain. Use <code>BSRR</code> for atomic set/clear.
UART	Async, no clock. Both ends must agree on baud/parity/stop bits. Errors: framing, parity, overrun (you were too slow). Use DMA + idle-line detection for framing.
SPI	Sync, master/slave, full-duplex, chip-select per device. Fast (tens of MHz). CPOL/CPHA modes 0–3 must match the device datasheet — a mismatch gives silent garbage.
I²C	Two wires, addressed, multi-drop, open-drain (needs pull-ups). Slow (100 k/400 k/1 M). Prone to bus lockup — a slave holding SDA low requires a recovery sequence (clock 9 pulses). Always implement timeouts.
CAN	Multi-master, differential, arbitration by ID, built-in error handling. Automotive standard. CAN-FD for higher rates.
ADC	Resolution (12-bit typical), sampling time, reference voltage. Needs calibration, oversampling for noise, and often DMA + timer triggering.
PWM	Timer with a compare register. Duty = compare/period. Motor control, LED dimming, DAC emulation.
DMA	Moves data without the CPU. ⚠ Buffers must be in DMA-accessible memory, correctly aligned, and cache-maintained on cached parts. The buffer must be <code>volatile</code> and must not be a stack local that goes out of scope.
Timers	Free-running counters, input capture (measure pulse widths), output compare, encoder mode.

Peripheral	Key points
Watchdog	Reset the MCU if not "kicked" in time. ⚠️ Kick it from the main loop only after verifying all tasks are alive — a watchdog kicked from a timer ISR protects nothing.

The universal peripheral patterns

Polling — simplest, wastes CPU, fine for init and low-rate. **Interrupt-driven** — one interrupt per byte/word. Good up to moderate rates. **DMA** — zero CPU per byte. Essential above ~1 Mbit/s. Combine with double-buffering (half-transfer + transfer-complete interrupts) for continuous streams.

Always implement timeouts. A peripheral that never sets its ready bit will hang your firmware forever:

```
uint32_t deadline = now_ms() + 100;
while (!(SPI1->SR & SPI_SR_TXE)) {
    if (time_after(now_ms(), deadline)) return -ETIMEDOUT;
}
```

27.8 RTOS

RTOS	Notes
FreeRTOS	The most widely deployed. Small, MIT-licensed, AWS-maintained. Tasks, queues, semaphores, mutexes, timers, event groups, stream buffers.
Zephyr	Linux Foundation. Much larger scope: drivers, networking, Bluetooth, device tree, Kconfig. The modern choice for connected devices.
ThreadX (Azure RTOS)	Microsoft, safety-certified (IEC 61508 SIL 4, ISO 26262 ASIL D, DO-178C).
RT-Thread, NuttX, ChibiOS, µC/OS	Regional and niche strengths

The concepts

Task — a function with its own stack, run by the scheduler. **Statically allocate task stacks** and size them from `-fstack-usage` analysis.

Preemptive priority scheduling — the highest-priority ready task runs. Equal priorities may round-robin.

Queues — the primary inter-task mechanism. Copy-by-value (safer than pointers), blocking with timeout, ISR-safe variants (`xQueueSendFromISR`).

Semaphores vs mutexes — a semaphore signals (ISR → task); a **mutex** provides mutual exclusion **with priority inheritance**.

⚠️ Priority inversion


```
Low-priority task L takes mutex M.
High-priority task H blocks waiting on M.
Medium-priority task Mp preempts L (it's higher priority).
→ H is now effectively blocked by Mp, which it outranks. Unbounded delay.
```

This is not hypothetical — it nearly lost the Mars Pathfinder mission in 1997. The rover kept resetting because a low-priority meteorological task held a mutex needed by a high-priority bus task, while medium-priority communications tasks ran. The fix was uploaded from Earth: enable priority inheritance.

Solutions: priority inheritance (FreeRTOS mutexes do this; semaphores do **not**), priority ceiling, or — best — don't share resources between tasks of different priorities.

RTOS rules

1. **...FromISR variants in ISRs**, always, plus the `xHigherPriorityTaskWoken` yield dance.
2. **Never block in an ISR.**
3. **Size every task stack** and enable stack-overflow checking in development.
4. **Watch for deadlock** — the same global lock ordering rule as Module 24.
5. **Prefer message passing to shared state.** Queues eliminate most synchronization bugs.
6. **Measure worst-case response time**, not average. Real-time means *deadlines*, not speed.
7. **You may not need an RTOS.** A superloop with a state machine and interrupt-driven I/O is simpler, smaller, and easier to verify. Add an RTOS when you genuinely have concurrent activities with different deadlines.

27.9 Low power, bootloaders, OTA

Low power

```
__DSB();      /* flush pending writes BEFORE sleeping */
__WFI();      /* Wait For Interrupt – core clock stops until an IRQ */
```

Modes range from *sleep* (core off, peripherals on, wake in μ s) through *stop* (most clocks off, RAM retained) to *standby* (RAM lost, wake resets the MCU).

The technique: be asleep by default. Wake on interrupt, do the minimum, sleep again. A well-written sensor node runs at >99% sleep and lasts years on a coin cell. Measure with a current probe or a Nordic PPK — **average current is what matters**, and it's dominated by how long you're awake, not how fast you compute.

Also: clock gating unused peripherals, dynamic frequency scaling, and configuring unused GPIOs (floating inputs leak current).

Bootloaders and OTA

A minimal bootloader lives at the start of flash, validates the application image, and jumps to it:

```
uint32_t app_sp      = *(volatile uint32_t*)APP_BASE;
uint32_t app_reset   = *(volatile uint32_t*)(APP_BASE + 4);
__set_MSP(app_sp);
SCB->VTOR = APP_BASE;           /* relocate the vector table */
((void(*) (void))app_reset)();
```

A production OTA update must be:

Requirement	Mechanism
Atomic	A/B dual-bank flash — write to the inactive bank, swap on success
Verified	Cryptographic signature checked <i>before</i> jumping (secure boot chain)
Rollback-capable	Boot the new image with a "trial" flag; a watchdog reset reverts to the known-good bank
Interruptible	Power loss mid-update must leave a bootable device
Anti-rollback	Monotonic version counter in one-time-programmable fuses

Never write a bootloader from scratch for production. Use **MCUboot** (the de facto standard, used by Zephyr) or your vendor's certified equivalent.

27.10 Debug hardware

Tool	Purpose
SWD / JTAG	The debug port. SWD is 2 wires (SWDIO/SWCLK) plus ground; JTAG is 4–5. Gives full memory access, breakpoints, and single-stepping — even with the core halted.
Probes	ST-Link, J-Link (best software, SWO+RTT), CMSIS-DAP/DAPLink (open), Black Magic Probe (GDB server built in)
OpenOCD / pyOCD	Open-source GDB servers: <code>openocd -f board.cfg</code> then <code>arm-none-eabi-gdb -ex 'target remote :3333'</code>
SWO / ITM	Single-wire trace output — <code>printf</code> -style debugging at MHz rates without a UART
SEGGER RTL/RTT	Ring buffer in RAM read by the probe. Very fast, no dedicated pin.
ETM trace	Full instruction trace. Needed for hard timing bugs and certification coverage.
Logic analyzer	Saleae, or a \$10 clone + sigrok. Indispensable for SPI/I²C/UART protocol bugs — decodes the bus and shows you what actually went on the wire.
Oscilloscope	Analog reality: signal integrity, ringing, rise times, glitches, power rails.
Current probe	Power profiling (Nordic PPK2, Joulescope)

The debugging skill that distinguishes embedded engineers: when the code looks right, look at the signal. Half of "firmware bugs" are a floating pin, a missing pull-up, a wrong SPI mode, a brown-out, or a noisy power rail.

Also: **hard fault handlers**. Capture the stacked registers and decode `SCB->CFSR/HFSR/BFAR` — that turns "it hangs" into "unaligned access at 0x0800_1234."

27.11 MISRA C and safety standards

MISRA C:2012 / 2023

~175 rules for safety-critical embedded C. Categories: **Mandatory** (never violate), **Required** (violate only with a documented deviation), **Advisory**.

Representative rules:

Rule	Content
8.13	Pointers should point to <code>const</code> wherever possible
17.2	No recursion
21.3	No <code>malloc/free</code>
15.5	Single point of exit from a function
15.6	Compound statements (braces) required for every <code>if/loop</code>
14.4	The controlling expression of <code>if</code> shall be essentially boolean
10.x	The "essential type" model — heavy restrictions on implicit conversions (Module 2's traps, systematized)
11.x	Restrictions on pointer casts
20.x	Heavy restrictions on the preprocessor
1.3	No undefined or critical unspecified behaviour (Module 16)

Deviations are permitted but must be documented, justified, and approved. MISRA is not "no exceptions" — it's "no *undocumented* exceptions."

⚠ **MISRA is a coding standard, not a correctness proof.** Compliant code can still be wrong. And some rules (single exit, no `goto`) arguably make code worse — hence the deviation process.

Tools: PC-lint Plus, Helix QAC, LDRA, Polyspace, Parasoft, Coverity, and `clang-tidy`'s `cert-*` checks for partial coverage.

The certification standards

Standard	Domain	Levels
ISO 26262	Automotive	ASIL A–D

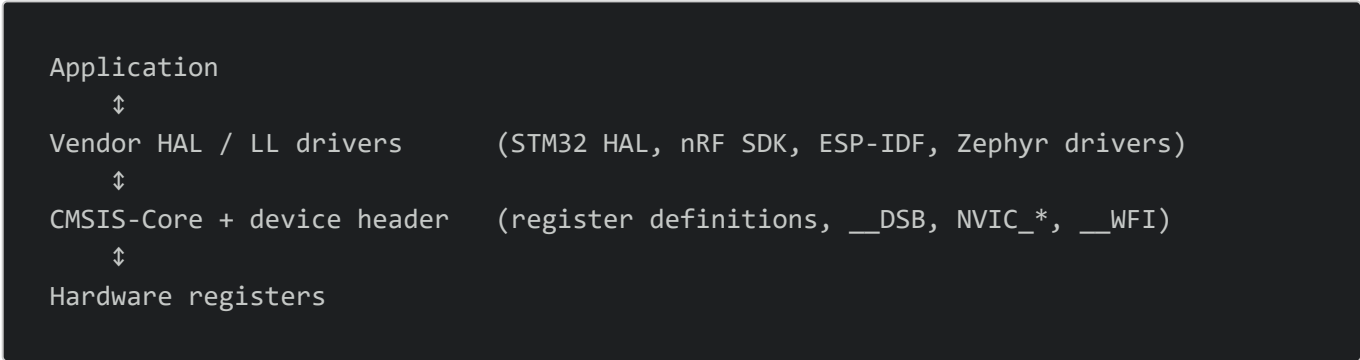
Standard	Domain	Levels
DO-178C	Avionics	DAL A–E; Level A requires MC/DC coverage
IEC 61508	Industrial functional safety	SIL 1–4
IEC 62304	Medical devices	Class A/B/C
EN 50128	Railway	SIL 0–4

What certification demands: full requirements traceability (every line traces to a requirement, every requirement to a test), structural coverage (statement → branch → **MC/DC**), a qualified toolchain, documented deviations, independent verification, and a configuration-management trail for everything.

This is why safety-critical C moves slowly and costs what it does — and why the roles pay accordingly.

27.12 CMSIS, HAL, and register-level code

The layers



CMSIS (ARM's Cortex Microcontroller Software Interface Standard) provides: **CMSIS-Core** (register definitions, intrinsics, NVIC access — vendor-neutral), **CMSIS-DSP** (optimized filters, transforms, matrix math), **CMSIS-NN**, **CMSIS-Driver**, and **CMSIS-Pack**.

HAL vs registers

	Vendor HAL	Register-level
Time to first blink	Minutes	Hours
Code size	Large (often 10–50× for the same function)	Minimal
Determinism	Hidden state machines, hidden timeouts	Exactly what you wrote
Portability	Within one vendor's family	Rewrite per part
Debuggability	Layers of abstraction to step through	Direct
Correctness	Vendor HALs have a poor bug reputation	Yours to get right

The pragmatic position

Use the HAL for: initialization boilerplate (clock trees are genuinely hard), prototyping, and complex stacks (USB, TCP/IP, Bluetooth) you'd never write yourself.

Drop to registers for: hot paths, ISRs, anything with tight timing, anything you need to certify, and anything where the HAL's behavior is unclear.

Regardless: always use the CMSIS device header for register definitions rather than hand-written `#defines` — it's generated from the vendor's SVD file and it's correct.

Read the reference manual. The single most important embedded skill is being able to open a 2000-page reference manual, find the register description, and understand what each bit does. No abstraction layer removes that need — it only delays it until something breaks.

Module 27 rules

#	Rule
1	Every hardware register is <code>volatile</code> — without it the compiler deletes your I/O
2	Use atomic set/reset registers (<code>BSRR</code>) instead of read-modify-write on <code>ODR</code>
3	<code>volatile</code> gives no ordering, atomicity, or cache coherence — add <code>__DSB/__DMB</code> and barriers
4	ISRs: short, <code>volatile</code> shared data, no <code>malloc/printf/blocking</code> , clear the flag
5	32-bit accesses are atomic on ARM; wider ones need a critical section or double-read
6	Save and restore <code>PRIMASK</code> — never blindly re-enable interrupts
7	<code>KEEP()</code> the vector table so <code>--gc-sections</code> can't remove it
8	<code>> RAM AT> FLASH</code> — <code>.data</code> runs in RAM, is stored in flash, copied by <code>Reset_Handler</code>
9	You write the C runtime: copy <code>.data</code> , zero <code>.bss</code> , then call <code>main</code> — which must never return
10	Always generate a map file and a <code>-fstack-usage</code> report
11	Ban <code>malloc</code> , VLAs, <code>alloca</code> , and recursion — static allocation with compile-time sizing
12	Never <code>double</code> on a 32-bit MCU; <code>-Wdouble-promotion</code> , and <code>f</code> -suffixed literals and functions
13	Fixed-point needs 64-bit intermediates; watch saturation
14	Every peripheral wait needs a timeout
15	Kick the watchdog from the main loop after verifying all tasks are alive
16	RTOS mutexes have priority inheritance; semaphores do not — Pathfinder
17	Prefer message passing to shared state; you may not need an RTOS at all
18	OTA must be atomic, signed, rollback-capable, and power-loss safe — use MCUboot
19	When the code looks right, look at the signal
20	MISRA deviations are allowed but must be documented and approved

#	Rule
21	Read the reference manual

Practice

1. **Prove `volatile` matters.** Install `gcc-arm-none-eabi`. Write a GPIO toggle with and without `volatile`. Dump both at `-O2` and count the memory accesses. Then do the same for a status-polling loop and find the infinite loop.
2. **Build the firmware.** Reproduce the complete §27.4 build: linker script, startup code, vector table, `main`. Verify: word 0 of the vector table is the stack top, word 1 is `Reset_Handler | 1`, `.data`'s VMA is RAM and its LMA is flash. Read the `Reset_Handler` disassembly and identify both loops.
3. **Break the linker script.** (a) Remove `KEEP()` from `.isr_vector` and build with `--gc-sections` — what happens? (b) Set RAM `LENGTH` to 4K and confirm the `._check` section fails the link. (c) Forget `AT> FLASH` on `.data` and explain what the image would do.
4. **Run it.** Install `qemu-system-arm` and run your firmware on a `-machine lm3s6965evb` or `netduinoplus2` target. Add a semihosting or UART `putchar` and get "hello" out. Then attach GDB (`-s -S`) and single-step `Reset_Handler`.
5. **Torn read.** Write a 64-bit tick counter incremented by a simulated ISR and read from the main loop. Prove the tearing with a test that detects a non-monotonic sequence. Fix it three ways: double-read loop, `PRIMASK` critical section, and splitting into two 32-bit halves with a sequence counter.
6. **Stack budget.** Build a firmware with `-fstack-usage`. Sum the worst-case call chain by hand. Then fill RAM with `0xAA` at startup and add a runtime high-water-mark reporter. Compare the static estimate to the measured value.
7. **Fixed-point library.** Implement Q16.16 with add, sub, mul, div, sqrt, sin (via a lookup table + interpolation), and saturating variants. Test against `double` for accuracy across the range. Compare code size and cycle counts against soft-float and hard-float on Cortex-M4.
8. **The `double` trap.** Write `float x = y * 3.14;` (no `f` suffix) and build with `-mfloat-abi=hard -mfpu=fpv4-sp-d16`. Find the `__aeabi_d*` calls in the disassembly. Then enable `-Wdouble-promotion` and fix it.
9. **Ring buffer + ISR.** Implement a lock-free single-producer/single-consumer ring buffer for UART RX, with the ISR as producer and the main loop as consumer. Prove it needs no critical section when the size is a power of two and the indices are word-sized. Verify with a stress test.
10. **Priority inversion.** In FreeRTOS (or a simulation), reproduce the Pathfinder scenario with three tasks and a shared resource. Measure the high-priority task's delay. Then switch from a binary semaphore to a mutex and re-measure.
11. **MISRA audit.** Take a driver you've written and check it against 10 MISRA rules (17.2, 21.3, 15.5, 15.6, 14.4, 8.13, 11.3, 20.7, 10.3, 1.3). Document each violation with either a fix or a written deviation justification.

12. **Hard fault decoder.** Write a `HardFault_Handler` in assembly that captures the stacked frame and passes it to a C function that prints PC, LR, PSR, `CFSR`, `HFSR`, and `BFAR`. Trigger it with an unaligned access, a null dereference, and a divide by zero, and confirm you can identify each.

Next: **Module 28 — Software Design in C**, which pulls the opaque handles, function-pointer vtables, callbacks, and intrusive structures you've met into a coherent architectural toolkit: how to build modules, plugin systems, state machines, and generic containers in a language with no classes, no templates, and no namespaces.

28. Software Design in C

- Modularity: one module = header + implementation, clear boundaries
- **Opaque handle APIs** (`typedef struct Db Db; Db *db_open(...)`)
- Polymorphism via function-pointer vtables
- Callbacks with context/user-data pointers
- Object lifecycle: `init/deinit`, `create/destroy`, ownership rules
- State machines (table-driven, switch-based)
- Plugin systems, dependency injection
- Generic containers: `void*`, macro templates, intrusive lists
- Layering, avoiding global state, testability
- API design: naming, versioning, deprecation, error reporting, thread-safety documentation

Module 28 — Software Design in C

C has no classes, no templates, no namespaces, no destructors, and no modules. **Everything in this course's architecture toolkit is built from four primitives: structs, function pointers, opaque types, and the linker.** All output below is from real, ASan-clean builds.

28.1 Modularity

One module = one header + one implementation. The header is the contract; the `.c` is the secret.

```
widget.h    ← the ONLY thing consumers see
widget.c    ← everything else: static helpers, private structs, globals
```

Rule	Why
<code>static</code> everything by default	Remove it only when a symbol must be shared (Module 3)
Prefix every public identifier	C has no namespaces: <code>widget_create</code> , <code>WIDGET_MAX</code> , <code>WidgetKind</code>
Include your own header first in the <code>.c</code>	Proves it's self-contained (Module 20)
Forward-declare rather than include in headers	<code>struct Db;</code> instead of <code>#include "db.h"</code>

Rule	Why
One responsibility per module	If the header has 40 unrelated functions, split it
No circular dependencies	A depends on B depends on A means they're one module
-fvisibility=hidden for shipped libraries	Only the marked API is exported (Module 20)

The test: can you describe the module in one sentence, and does its header fit on one screen? If not, split it.

28.2 Opaque handles

Module 11 §11.8 established the mechanism. Here's the design guidance.

```
/* db.h – the consumer sees only this */
typedef struct Db Db;                                /* incomplete type */

Db *db_open(const char *path);
void db_close(Db *db);                               /* accepts NULL */
int db_query(Db *db, const char *sql, Row **out);

/* db.c – the layout is private */
struct Db { unsigned magic; int fd; Cache cache; Stmt *stmts; size_t nstmts; };
```

What you get:

Benefit	Consequence
ABI stability	Add, remove, or reorder fields without recompiling consumers (Module 20 measured what happens without this)
Real invariants	State changes only through your functions
Fewer header dependencies	The header needn't include the types your fields use
Faster builds	Layout changes don't rebuild the world

The cost: one allocation per object, and a function call per field access. When that's unacceptable — embedded (Module 27), hot inner loops — expose the struct but document that only the API may touch it. That's the weaker guarantee `pthread_mutex_t` and `FILE` make.

Every serious C library works this way: `FILE*`, `sqlite3*`, `SSL_CTX*`, `png_structp`, `curl` handles, `pthread_t`.

28.3 Polymorphism via vtables

This is exactly what a C++ compiler generates for virtual functions. Doing it by hand costs 30 lines and buys you full control.

```
/* stream.h – the interface */
typedef struct Stream Stream;

typedef struct StreamVTable {
    const char *name;
    ptrdiff_t (*read)(Stream *s, void *buf, size_t n);
    ptrdiff_t (*write)(Stream *s, const void *buf, size_t n);
    int (*flush)(Stream *s);
    void (*close)(Stream *s);          /* frees the object itself */
} StreamVTable;

struct Stream { const StreamVTable *vt; }; /* every implementation embeds this
FIRST */

/* dispatch */
static inline ptrdiff_t stream_write(Stream *s, const void *b, size_t n)
{ return s->vt->write(s, b, n); }
static inline int stream_flush(Stream *s)
{ return s->vt->flush ? s->vt->flush(s) : 0; } /* optional methods → NULL
check */
```

An implementation embeds `Stream` as its **first member**, so `(Stream*) ↔ (MemStream*)` is a valid cast:

```
typedef struct {
    Stream base; /* MUST be first */
    char *buf; size_t len, cap, pos;
} MemStream;

static ptrdiff_t mem_write(Stream *s, const void *b, size_t n) {
    MemStream *m = (MemStream*)s; /* downcast */
    ...
}

static const StreamVTable MEM_VT = { "memory", mem_read, mem_write, NULL,
mem_close };

Stream *memstream_create(size_t cap) {
    MemStream *m = calloc(1, sizeof *m);
    ...
    m->base.vt = &MEM_VT;
    return &m->base;
}
```

The client knows nothing about the implementations:

```
static void write_report(Stream *out, const char *title)
{
    stream_write(out, "=== ", 4);
    stream_write(out, title, strlen(title));
    ...
    stream_flush(out);
}
```

And decorators compose for free:

```
Stream *tee = teestream_create(file_stream, mem_stream); /* one write → two
sinks */
write_report(tee, "to file AND memory");
stream_close(tee); /* closes both children
*/
```

```
memstream (memory) captured 45 bytes:
=== to memory ===
    item 1 / item 2 / item 3

tee (tee) -> mirrored copy is 54 bytes

file contents:
=== to file AND memory ===
    item 1 / item 2 / item 3

ASan exit=0
```

The rules

1. **The base struct must be the first member.** That's what makes the cast valid (offset 0 is guaranteed).
2. **The vtable is `static const`** — one instance per type, in `.rodata`, shared by every object.
3. **NULL entries mean "not supported"** — check before calling, or provide a default.
4. **`close/destroy` lives in the vtable** so the caller doesn't need the concrete type.
5. **Cost:** one pointer per object, one indirect call per operation. That indirect call defeats inlining — measure before using this on a hot path.
6. **Alternative for a fixed, small set of types:** a tagged union (Module 11) with a `switch`. Faster, but not extensible.

Real examples: the Linux kernel's `struct file_operations`, `struct net_device_ops`; SQLite's VFS layer; GObject; libcurl's protocol handlers.

28.4 Callbacks with context

Module 6 §6.6 established this. The design rule:

Every callback signature takes a `void *userdata`. No exceptions.

```
typedef void (*VisitFn)(const Item *item, void *ctx);
void tree_walk(Tree *t, VisitFn fn, void *ctx);
```

C has no closures. The context pointer is the only way a caller can attach state — omit it and callers must use globals, which destroys thread safety and testability.

qsort gets this wrong (Module 14) — which is why `qsort_r/qsort_s` exist as incompatible platform extensions. Learn from the mistake.

Additional guidance:

- **Context last** by convention, so it reads like a trailing parameter.
- **Document reentrancy:** may the callback modify the container? Call back into the API?
- **Never invoke a callback while holding a lock** (Module 24) — the callback might take the same lock.
- **Return a value from the callback** if the caller might want to stop early: `bool visit(...)` where `false` means abort.
- **A struct of function pointers plus one context** is better than five separate callbacks — that's an interface (§28.7).

28.5 Object lifecycle

Two patterns, both useful

Pattern A — `init/deinit`: the caller owns the storage.

```
typedef struct { char *buf; size_t len, cap; } Buffer;

bool buffer_init(Buffer *b, size_t cap);    /* does NOT allocate the struct */
void buffer_deinit(Buffer *b);             /* does NOT free the struct    */
```

Buffer on the stack, `cap=32`. Embeddable in another struct, usable with no allocator at all (Module 27).

Use when: the type may live on the stack, be embedded in another struct, or be used where there's no heap (embedded).

Pattern B — `create/destroy`: the module owns everything.

```
Session *session_create(const char *user);
void      session_destroy(Session *s);    /* MUST accept NULL */
Session *session_retain(Session *s);     /* shared ownership */
const char *session_user(const Session *s); /* BORROWED - do not free */
```

```

session_create("alice") -> user=alice refcount=1
after retain: refcount=2
after 1st destroy: refcount=1 (object still alive)
after 2nd destroy: freed
session_destroy(NULL) is a safe no-op
ASan exit=0

```

They compose: `Session` embeds a `Buffer` by value and calls `buffer_init`/`buffer_deinit` in its own create/destroy.

The six rules

```

static Session *session_create(const char *user)
{
    Session *s = calloc(1, sizeof *s);
    if (!s) return NULL;
    s->user = strdup(user);
    if (!s->user)                { free(s); return NULL; }          /* unwind
(Module 5) */
    if (!buffer_init(&s->io, 64)) { free(s->user); free(s); return NULL; }
    s->magic = SESSION_MAGIC; s->refcount = 1;
    return s;
}

static void session_destroy(Session *s)
{
    if (!s) return;                                     /* 1. accept NULL */
    if (s->magic != SESSION_MAGIC) { /* UAF or type confusion */ return; } /* 3.
*/
    if (--s->refcount > 0) return;                        /* shared ownership */
    buffer_deinit(&s->io);
    free(s->user);
    s->magic = 0;                                         /* 4. poison BEFORE free */
    free(s);
}

```

1. **destroy()** always accepts **NULL** — makes cleanup chains trivial.
2. Every failure path in **create()** unwinds what it acquired.
3. A magic field catches use-after-free and type confusion in debug builds.
4. Poison the magic *before* **free**, not after.
5. **Getters return borrowed pointers** — document the lifetime in the header.
6. **Reference counting only when shared ownership is genuinely required** — and it must be atomic if threads are involved (Module 17).

28.6 State machines

Table-driven

```
typedef struct { State next; void (*action)(void*); } Transition;
#define KEEP { S_COUNT, NULL } /* S_COUNT = "no transition" */

static const Transition TABLE[S_COUNT][E_COUNT] = {
[S_CLOSED]      = {{S_LISTEN, NULL}, KEEP, KEEP, KEEP, KEEP, KEEP},
[S_LISTEN]      = {KEEP, {S_SYN_RCVD, act_send_syn_ack}, KEEP, KEEP, KEEP,
{S_CLOSED, act_cleanup}},
[S_ESTABLISHED] = {KEEP, KEEP, {S_ESTABLISHED, act_deliver}, {S_FIN_WAIT, NULL},
KEEP, {S_CLOSED, act_cleanup}},
...
};

static bool fsm_table(Conn *c, Event e)
{
    const Transition *t = &TABLE[c->st][e];
    if (t->next == S_COUNT) return false; /* not accepted in this state */
    if (t->action) t->action(c);
    c->st = t->next;
    return true;
}
```

The entire engine is 6 lines. States and events come from X-Macros (Module 12) with `_Static_asserted` name tables:

```
STATE MACHINE: 6 states x 6 events

CLOSED      + OPEN      -> LISTEN
LISTEN      + SYN       -> SYN_RCVD
SYN_RCVD    + ACK       -> ESTABLISHED
ESTABLISHED + FIN       -> FIN_WAIT
FIN_WAIT    + ACK       -> TIME_WAIT
TIME_WAIT   + TIMEOUT   -> CLOSED
```

Switch-based

```
switch (c->st) {
case S_LISTEN:
    if (e == E_SYN) { act_send_syn_ack(c); c->st = S_SYN_RCVD; break; }
    if (e == E_RESET) { act_cleanup(c); c->st = S_CLOSED; break; }
    return false;
...
}
```

Measured comparison

```

differential test (200k random events): IDENTICAL
table-driven : 73.1 ms
switch-based : 73.6 ms

```

Performance is identical at 20 million transitions. The choice is entirely about maintainability.

Table-driven	Switch-based
Adding a state/event edits data	Adding a state/event edits code
The whole machine is visible at a glance	Logic is spread across cases
Verifiable/generatable — you can check completeness, generate a diagram, or emit the table from a spec	Easier to add per-case logic and guards
<code>-Werror=switch</code> doesn't help	<code>-Werror=switch</code> enforces exhaustiveness (Module 11)
Memory: <code>states × events × sizeof(Transition)</code>	No table

Use a table for protocol machines, parsers, and anything with many states and a specification to match. **Use a switch** for small machines with complex per-transition logic.

Note the differential test — Module 22's technique applied to verify two implementations agree on 200,000 random inputs. That's how you refactor a state machine safely.

Hierarchical state machines (a state can have substates and inherit transitions) are worth knowing for complex UIs and protocols — see the UML statechart model and libraries like QP/C.

28.7 Dependency injection and plugins

Every external dependency becomes an explicit interface passed in. No globals, no direct syscalls, no hidden state.

```

typedef struct { uint64_t (*now_ms)(void *ctx); void *ctx; } Clock;
typedef struct { void (*log)(void *ctx, const char *lvl, const char *msg); void *ctx; } Logger;
typedef struct { int (*send)(void *ctx, const char *dst, const char *body); void *ctx; } Mailer;

typedef struct { Clock clock; Logger logger; Mailer mailer; } Deps;

static int alerter_fire(Alerter *a, const Deps *d, const char *who, const char *msg)
{
    uint64_t t = d->clock.now_ms(d->clock.ctx);
    if (a->last_sent_ms && t - a->last_sent_ms < COOLDOWN_MS) {
        d->logger.log(d->logger.ctx, "DEBUG", "suppressed: cooldown");
        return 1;
    }
}

```

```

    }
    if (d->mailer.send(d->mailer.ctx, who, msg) != 0) { ...; return -1; }
    ...
}

```

Production wiring:

```
Deps prod = { {real_now,NULL}, {real_log,NULL}, {real_send,NULL} };
```

Tests wire fakes — **no mocking framework, no linker tricks, no `#ifdef TEST`:**

```

FakeClock clk = {1000};
SpyMail ml = {0};
Deps test = { {fake_now,&clk}, {spy_log,&lg}, {spy_send,&ml} };

```

```

=== TESTS - no globals, no mocking framework, no time.h ===
test 1: first alert is sent
test 2: a second alert within the cooldown is suppressed
test 3: after the cooldown it sends again
test 4: a mailer failure is reported

```

ALL PASS (0 failures)

Note: test 2 required advancing time by 30 SECONDS.
With a hard-coded `clock()` call the test would have to SLEEP.

That last line is the whole argument. With a hard-coded `clock_gettime`, testing the cooldown means a 60-second `sleep()` in your test suite — or not testing it. With injection, `clk.t += 30000`; and it's instant and deterministic.

Plugin systems

A plugin is the same idea, loaded at runtime (Module 24 §24.10):

```

/* the ABI: ONE versioned struct of function pointers */
#define PLUGIN_ABI_VERSION 3
typedef struct {
    uint32_t abi_version;
    const char *name;
    int (*init)(void **state, const Config *cfg);
    int (*process)(void *state, const Frame *in, Frame *out);
    void (*shutdown)(void *state);
} PluginAPI;

```

```
/* the ONE exported symbol */
PLUGIN_EXPORT const PluginAPI *plugin_entry(void);
```

Design rules:

1. **One exported symbol** returning a versioned struct — one `dlsym`, one thing to version.
2. **Check `abi_version` before using anything.**
3. **Per-instance state via an opaque `void *`** returned by `init`.
4. **Build plugins `-fvisibility=hidden`** so only the entry point is exported.
5. **Never `dllclose` while any plugin state or callback is reachable.**

28.8 Generic containers

Four approaches, all demonstrated working:

```
=== 1. void* + size (runtime generic, NO type safety) ===
0 1 2 3 4
cost: acast at every access; wrong esz = silent corruption

=== 2. macro template (COMPILE-TIME generic, full type safety) ===
IntVec: 0 1 4 9 16
RecVec: 1:alpha 2:beta
IntVec_push(&iv, "str") -> COMPILE ERROR (type-checked)

=== 3. intrusive list (ZERO allocations for the nodes) ===
disk(10) net(20) cpu(30)
offsetof(Item,link)=24 -> no node allocation, one object in N lists

=== 4. _Generic dispatch over the templates ===
vec_push worked on both types: iv.len=6 rv.len=3
```

The macro template

```
#define VEC_DECLARE(NAME, T) \
    typedef struct { T *data; size_t len, cap; } NAME; \
    static inline void NAME##_init(NAME *v){ *v = (NAME){0}; } \
    static inline bool NAME##_push(NAME *v, T e){ \
        if (v->len == v->cap) { \
            size_t nc = v->cap ? v->cap*2 : 8; \
            T *p = realloc(v->data, nc*sizeof *p); if (!p) return false; \
            v->data = p; v->cap = nc; } \
        v->data[v->len++] = e; return true; } \
    static inline T NAME##_at(const NAME *v, size_t i){ return v->data[i]; } \
    static inline void NAME##_free(NAME *v){ free(v->data); *v=(NAME){0}; }

VEC_DECLARE(IntVec, int)
VEC_DECLARE(RecVec, Rec)
```


Token pasting (Module 12) generates a complete, type-safe, fully-inlined container per type.

Trade-offs

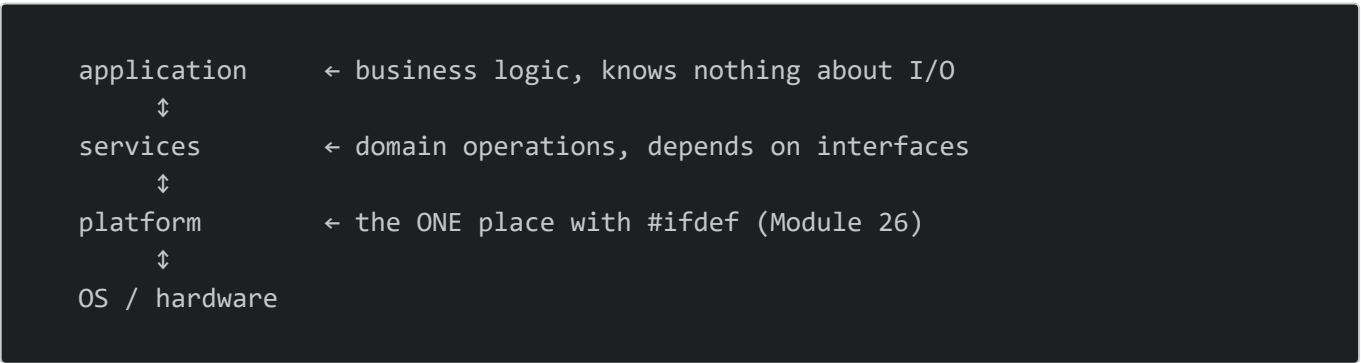
Approach	Type safety	Speed	Code size	Errors
<code>void*</code> + <code>size</code>	✗ none	indirection + memcpy per access	one copy	runtime corruption
Macro template	☑ full	inlined, zero overhead	one copy per type	awful macro-expansion messages
Intrusive	partial	best (zero alloc, best locality)	tiny	—
<code>_Generic</code> facade	☑ full	dispatches to any of the above	—	good

⚠ **A real trap I hit building this:** `vec_push(&rv, (Rec){3, "gamma"})` fails to compile — the comma inside the compound literal is an **argument separator to the preprocessor** (Module 12 §12.4). Protect commas with parentheses, or use a named temporary.

Recommendation: intrusive for internal data structures where you own the type; macro templates for reusable containers; `void*` only for genuinely dynamic cases (like `qsort`). Or use an existing library: **stb_ds**, **klib**, **sc**, **uthash**.

28.9 Layering, global state, testability

Layering



Dependencies point in one direction only. An upper layer may call down; a lower layer talks back only through **callbacks or interfaces injected from above**.

The test: can you compile and test the application layer with the platform layer replaced by fakes? If not, you have a leak.

Global state

Globals destroy: thread safety (shared mutable state), testability (tests interfere), reentrancy, and initialization-order predictability (Module 3).

```
/* BAD */
static Config g_config;
static Logger g_logger;
int process(const char *input);           /* hidden dependencies */

/* GOOD */
typedef struct { const Config *cfg; Logger log; Stats *stats; } Context;
int process(Context *ctx, const char *input);
```

Where globals are defensible: genuinely process-wide immutable data (a lookup table, `const` config read once at startup), and a single logger sink — and even then, thread-safe and initialized via `call_once`.

⚠ **Function-local `static` has the same problems** plus non-reentrancy. It's why `strtok`, `gmtime`, and `strerror` need `_r` variants (Module 14).

Testability is a design property

Hard to test	Why	Fix
Reads the clock directly	Can't control time	Inject a <code>Clock</code>
Calls <code>malloc</code> directly	Can't test OOM	Inject an allocator, or fault-inject
Reads a global	Tests interfere	Pass a context
Does I/O in the middle of logic	Needs real files/sockets	Separate pure logic from I/O
Big function with many branches	Combinatorial test count	Split it
No error return	Can't verify failure handling	Return <code>err_t</code> (Module 15)

Code that's hard to test has hidden dependencies. Making it testable makes it better. The DI example above is the proof: the same change that made time controllable also made the cooldown logic explicit and the mailer swappable.

28.10 API design

Naming

```
/* module_noun_verb, consistently */
widget_create / widget_destroy
widget_get_name / widget_set_name
widget_list_append / widget_list_remove

/* types: PascalCase or module_t – pick one */
typedef struct Widget Widget;
```

```
/* constants: MODULE_SCREAMING */
#define WIDGET_MAX_NAME 64
enum { WIDGET_FLAG_HIDDEN = 1u << 0 };
```

Prefix everything. C has no namespaces, and an unprefixd `min/max/list_add` in a public header will collide (Module 12).

Parameter conventions

```
/* object first, output last, context last */
int widget_render(Widget *w, const Canvas *c, Rect *out_bounds, void *userdata);

/* const on everything you don't modify */
size_t widget_serialize(const Widget *w, uint8_t *buf, size_t buflen);

/* lengths travel with pointers (Module 23) */
int widget_set_data(Widget *w, const uint8_t *data, size_t len);
```

Error reporting

Module 15's design: an error code return, out-parameters for values, `[[nodiscard]]` (Module 18) on everything fallible, and **every error code a function can return documented in the header**.

Documentation that must be in the header

```
/**
 * Render the widget onto `canvas`.
 *
 * @param w      Must be non-NULL and initialized.
 * @param canvas Not modified. Must outlive this call.
 * @param out    On success, receives the bounds. Untouched on failure.
 * @return ERR_OK, ERR_INVALID (null args), ERR_NOMEM, ERR_IO.
 *
 * Thread safety: NOT thread-safe. Callers must serialize access to `w`.
 * Reentrancy:    Must not be called from a widget callback.
 * Allocation:    Does not allocate.
 */
[[nodiscard]] err_t widget_render(Widget *w, const Canvas *canvas, Rect *out);
```

Four things every public function's documentation must state:

1. **Ownership** — who frees what (Module 10)
2. **Lifetime** — how long returned pointers stay valid
3. **Thread safety** — safe / not safe / safe if different objects
4. **Every error return**

Versioning and deprecation

Concern	Approach
Semantic versioning	MAJOR = ABI break, MINOR = additions, PATCH = fixes (Module 20)
ABI stability	Opaque handles; bump the soname on any break
Deprecation	<code>[[deprecated("use widget_render2()")]]</code> — warn for at least one major cycle before removing
Feature detection	<code>MYLIB_VERSION</code> macros + a runtime <code>mylib_version(); MYLIB_HAS_FEATURE_X</code>
Extensibility	Options structs with a <code>size</code> field, so new fields can be added compatibly
Compile-time checks	<code>_Static_assert(sizeof(PublicStruct) == 32, "ABI changed")</code>

The design test

Before shipping an API, ask:

- Can it be **misused**? (If misuse is easy, redesign — Module 23)
- Is the **safe path the easy path**?
- Can a caller **forget to clean up**? (Make `destroy` accept NULL, provide a cleanup chain)
- Is every **error distinguishable**?
- Can it be **tested** without the real world?
- Can you **change the implementation** without breaking callers?
- Does it **compose** with itself? (The `tee` stream is the test)

Module 28 rules

#	Rule
1	One module = header + implementation; <code>static</code> by default; prefix everything
2	Opaque handles for every non-trivial type — ABI stability and real invariants
3	Vtable = <code>static const</code> struct of function pointers; base struct first
4	Every callback signature takes <code>void *userdata</code>
5	<code>destroy()</code> always accepts NULL; poison the magic before <code>free</code>
6	<code>init/deinit</code> when the caller owns storage; <code>create/destroy</code> when the module does
7	Every <code>create</code> failure path unwinds what it acquired
8	Table-driven FSM for protocols; switch for small machines with complex guards
9	Differential-test two FSM implementations against each other
10	Inject every external dependency — clock, allocator, I/O, logger

#	Rule
11	Plugins export one versioned struct; check the version before use
12	Intrusive for internal, macro templates for reusable, <code>void*</code> only when genuinely dynamic
13	Protect commas inside macro arguments
14	Dependencies point one direction; the application layer must be testable with fakes
15	No global mutable state — pass a context
16	Document ownership, lifetime, thread safety, and every error return in the header
17	If it's hard to test, the design is wrong

Practice

1. **Refactor to modules.** Take a single-file program over 500 lines and split it into 4+ modules with clean headers. Make everything `static` that can be, prefix all public names, and verify with `nm` that each `.o` exports only what its header declares.
2. **Opaque conversion.** Take a struct-exposing API and convert it to an opaque handle. Prove with a test file that `sizeof` and member access are now compile errors. Then change the internal layout and confirm consumers don't need recompiling (relink only).
3. **Build the stream hierarchy.** Implement the §28.3 `Stream` interface with memory, file, and null backends. Then add: a `gzip` decorator, a counting decorator (tracks bytes written), and a buffering decorator. Verify with ASan that closing the outermost stream closes the whole chain exactly once.
4. **Lifecycle audit.** For every type in a project you've written, classify it as init/deinit or create/destroy. Add NULL-safety to every destroy, magic fields to every opaque type, and verify every create failure path with fault injection + LSan (Module 22).
5. **Reference counting.** Add atomic reference counting to an opaque type. Write a test with 8 threads randomly retaining and releasing. Verify with TSan (Module 17) that there are no races and with LSan that nothing leaks.
6. **State machine, both ways.** Implement a real protocol FSM (HTTP request parsing, or a vending machine with coins and change) as both a table and a switch. Differential-test them on 1M random event sequences. Then add a new state to each and compare the diff size.
7. **FSM generator.** Write a Python script that reads a state-transition spec (CSV or YAML) and emits the C table, the enums, the name arrays, and the `_Static_asserts`. Verify the generated code compiles and matches your hand-written version.
8. **Full DI refactor.** Take a program that calls `time()`, `malloc()`, `fopen()`, and a network function directly. Convert every one to an injected interface. Then write a test suite that runs with zero real I/O and zero sleeps, covering: timeouts, allocation failure, partial reads, and network errors. Measure the coverage improvement.

9. **Plugin system.** Define a versioned plugin ABI. Write a loader that scans a directory, validates versions, and manages per-plugin state. Write three plugins with different behaviors. Handle: load failure, version mismatch, `init` failure, and safe unload while state is live.
10. **Container comparison.** Implement a dynamic array four ways (`void*`, macro template, intrusive-adjacent, and a `_Generic` facade). For each measure: lines of code, binary size for 3 element types, per-operation time, and the quality of the error message when you pass the wrong type.
11. **API review.** Take a public header you've written. For every function, check the §28.10 test: can it be misused? Is cleanup forgettable? Are errors distinguishable? Is thread safety documented? Rewrite the three worst functions and write the documentation comments in full.

Next: **Module 29 — Data Structures & Algorithms in C**, where you'll implement the working set — dynamic arrays, intrusive and non-intrusive lists, ring buffers, hash tables with both collision strategies, balanced trees, heaps, tries, and bitsets — using every design technique from this module, with sanitizer-verified correctness and measured performance.

30. Interoperability

- `extern "C"` and C++ interop
- FFI: Python (`ctypes`, `cffi`, C extensions), Rust (`bindgen`, `cbindgen`), Go cgo, Java JNI, Lua, Node N-API
- WebAssembly via Emscripten/WASI
- Designing C APIs meant to be bound from other languages

Module 30 — Interoperability

C is the lingua franca of computing. Every language can call C, and almost none can call each other directly — so C's ABI is the universal interop layer. All output below is from real builds: GCC 13.3, G++ 13.3, Python 3.12 with `ctypes` and `cffi` 2.0.

30.1 The library everyone will bind

One library, bound from C++, Python/`ctypes`, Python/`cffi` ABI, and Python/`cffi` API. **Every design choice here exists because of FFI.**

```
/* mathlib.h */

/* ---- 1. C++ GUARD ---- */
#ifdef __cplusplus
extern "C" {
#endif

/* ---- 2. EXPORT MACRO (Module 20) ---- */
#if defined(_WIN32)
#  ifdef MATHLIB_BUILD
#    define ML_API __declspec(dllexport)
#  else
#    define ML_API __declspec(dllimport)
```

```
#   define ML_API __declspec(dllimport)
#   endif
#elif defined(__GNUC__)
#   define ML_API __attribute__((visibility("default")))
#   endif

/* ---- 3. FFI-FRIENDLY SIGNATURES ---- */
typedef enum { ML_OK=0, ML_EINVAL=1, ML_ERANGE=2, ML_ENOMEM=3 } ml_status;

ML_API const char *ml_version(void);
ML_API const char *ml_strerror(int32_t status);

ML_API int32_t ml_add(int32_t a, int32_t b);          /* scalar in/out */
ML_API int64_t ml_sum(const int32_t *data, int32_t n); /* array + LENGTH */
ML_API int32_t ml_divide(int32_t a, int32_t b, int32_t *out); /* status + out-
param */

typedef struct ml_stats ml_stats;                    /* OPAQUE HANDLE */
ML_API ml_stats *ml_stats_create(void);
ML_API void      ml_stats_destroy(ml_stats *s);      /* accepts NULL */
ML_API int32_t   ml_stats_add(ml_stats *s, double v);
ML_API double    ml_stats_mean(const ml_stats *s);

ML_API int32_t ml_format(const ml_stats *s, char *buf, int32_t buflen); /*
CALLER's buffer */

typedef int32_t (*ml_visit_fn)(double value, void *userdata);          /* callback
+ ctx */
ML_API int32_t ml_stats_foreach(const ml_stats *s, ml_visit_fn fn, void
*userdata);

#ifdef __cplusplus
}
#endif
```

Built with `-fvisibility=hidden`, exporting exactly 15 symbols:

```
$ nm -D --defined-only libmathlib.so | grep " T "
ml_add ml_divide ml_format ml_hypot ml_stats_add ml_stats_count
ml_stats_create ml_stats_destroy ml_stats_foreach ml_stats_mean
ml_stats_stddev ml_strerror ml_sum ml_version ml_version_major
```

30.2 extern "C" and C++

Why it's needed — name mangling, measured

```
int mangled_function(int a, double b);
```

```

symbol the C code looks for      : U mangled_function
symbol the C++ code looks for    : U _Z16mangled_functionid
demangled                        : mangled_function(int, double)

```

C++ encodes the parameter types into the symbol name (that's how overloading works). Without `extern "C"`, C++ looks for `_Z16mangled_functionid` and the C library exports `mangled_function` — a link error.

`extern "C"` tells the C++ compiler: *use C linkage and the C calling convention for these declarations*. The `#ifdef __cplusplus` guard means one header serves both languages.

The idiomatic C++ wrapper — RAI over an opaque handle

```

#include "mathlib.h"          // the extern "C" guard makes this work

class Stats {
    struct Deleter { void operator()(ml_stats *s) const noexcept {
ml_stats_destroy(s); } };
    std::unique_ptr<ml_stats, Deleter> h_;
public:
    Stats() : h_(ml_stats_create()) { if (!h_) throw std::bad_alloc(); }

    void add(double v) {
        if (int32_t rc = ml_stats_add(h_.get(), v); rc != ML_OK)
            throw std::runtime_error(ml_strerror(rc));    // C error code -> C++
    }
    exception
    }
    std::string format() const {
        char buf[128];
        if (ml_format(h_.get(), buf, sizeof buf) != ML_OK) throw
std::runtime_error("format");
        return buf;
    }
    template <class F> void for_each(F f) const {          // C++ lambda -> C
callback
        auto trampoline = [](double v, void *ud) -> int32_t {
            (*static_cast<F*>(ud))(v); return 0;
        };
        ml_stats_foreach(h_.get(), trampoline, &f);
    }
};

```

```

C library version 1.4.2 called from C++
ml_add(20,22) = 42
ml_sum      = 15
ml_divide(7,0) = out of range
RAII wrapper : n=8 mean=5.0000 sd=2.1381
lambda callback: total = 40
(destructor calls ml_stats_destroy automatically)

```


The trampoline pattern is the key technique. A capturing C++ lambda isn't a function pointer — but a *captureless* lambda is. Pass the capturing lambda through the `void *userdata` slot and use a captureless trampoline to call it. This works for closures in every language.

The C++ interop rules

Rule	Why
Never let an exception cross into C	C has no unwinding; the C frames leak. Wrap every callback in <code>try { ... } catch(...) { return ERROR; }</code>
extern "C" applies to linkage, not the body	An <code>extern "C"</code> function can be written in C++ internally
Only one overload can be extern "C"	No mangling means no overloading
No C++ types in the C header	No <code>std::string</code> , no templates, no references, no default arguments
extern "C" on function pointer types too	<code>extern "C" typedef void (*Cb)(int);</code> — technically the calling convention differs
Union punning is legal in C, UB in C++ (Module 11)	Use <code>memcpy</code> in shared headers
static_cast, not C casts, on the C++ side	Better diagnostics
C++ can't see C's restrict	It's not a C++ keyword (<code>__restrict</code> is a common extension)

30.3 Python

`ctypes` — no compiler, runtime binding

```
import ctypes
lib = ctypes.CDLL("./libmathlib.so")

# YOU MUST DECLARE argtypes AND restype.
# Without them ctypes assumes int arguments and an int return -> silent
corruption.
lib.ml_version.restype = ctypes.c_char_p
lib.ml_add.argtypes = [ctypes.c_int32, ctypes.c_int32]
lib.ml_add.restype = ctypes.c_int32
lib.ml_sum.argtypes = [ctypes.POINTER(ctypes.c_int32), ctypes.c_int32]
lib.ml_sum.restype = ctypes.c_int64
lib.ml_stats_create.restype = ctypes.c_void_p # opaque handle
VISIT = ctypes.CFUNCTYPE(ctypes.c_int32, ctypes.c_double, ctypes.c_void_p)
```

```

=== PYTHON ctypes ===
ml_version() = 1.4.2
ml_add(20,22) = 42
ml_hypot(3,4) = 5.0
ml_sum([1..5])= 15
ml_divide(7,0)= out of range
opaque handle : n=8 mean=5.0000 sd=2.1381
callback total: 40.0
destroyed cleanly

```

⚠ **Forgetting `restype` is the #1 ctypes bug.** A function returning `double` read as `int` gives garbage; a function returning a 64-bit pointer read as a 32-bit `int` gives a truncated pointer that segfaults later.

Declare both, always.

⚠ **Keep the `CFUNCTYPE` object alive** for as long as C might call it. If Python garbage-collects the wrapper mid-call, you get a crash.

`cffi` ABI mode — paste the real declarations

```

from cffi import FFI
ffi = FFI()
ffi.cdef("""
    const char *ml_version(void);
    int64_t ml_sum(const int32_t *data, int32_t n);
    typedef struct ml_stats ml_stats;          /* opaque - cffi handles it */
    ml_stats *ml_stats_create(void);
    void      ml_stats_destroy(ml_stats *s);
    typedef int32_t (*ml_visit_fn)(double value, void *userdata);
    int32_t ml_stats_foreach(const ml_stats *s, ml_visit_fn fn, void *userdata);
""")
C = ffi.dlopen("./libmathlib.so")

h = ffi.gc(C.ml_stats_create(), C.ml_stats_destroy) # ties C lifetime to Python
GC

@ffi.callback("int32_t(double, void*)")           # a real C function pointer
def visit(v, ud): acc.append(v); return 0

```

```

=== PYTHON cffi (ABI mode) ===
ml_version() = 1.4.2
ml_sum      = 15
ml_divide(7,0)= out of range
opaque + ffi.gc: n=8 mean=5.0000 sd=2.1381
callback total : 40.0

```

cffi parses real C declarations — you copy-paste from the header instead of hand-writing type tables. And `ffi.gc()` attaches the destructor to Python's garbage collector, so cleanup is automatic.

cffi API mode — compiles a real extension

```
ffibuilder.set_source("_mathlib_api",
    '#include "mathlib.h"',
    include_dirs=[here], libraries=["mathlib", "m"])
ffibuilder.compile()
```

```
=== cffi API MODE (compiled extension) ===
version      : 1.4.2
ml_add       : 42
mean         : 5.0
```

API mode compiles against the real header, so type errors are caught at build time, macros and inline functions work, and calls are much faster. The cost is needing a C compiler at install time (or shipping wheels).

⚠ The measured cost of crossing the boundary

```
=== FFI CALL OVERHEAD (2,000,000 calls of ml_add) ===
pure python add          167.5 ms (   84 ns/call)
ctypes                   1518.4 ms (  759 ns/call)
cffi ABI mode            767.7 ms (   384 ns/call)
cffi API mode            423.7 ms (   212 ns/call)
```

The C function itself takes ~1 ns. Everything above is boundary cost — argument marshalling, type conversion, GIL handling, and the call machinery. **ctypes is 759× the cost of the work.**

The consequence: batch across the boundary

```
=== BATCHING (2,000,000 elements) ===
pure python sum()          17.5 ms  result=1999999000000
ONE C call over an array    1.3 ms  result=1999999000000  <- 13x faster
ONE C call PER element    813.6 ms  result=1999999000000  <- 47x SLOWER
```

Same C code. The only difference is how many times you cross the boundary.

Calling C once per element made it **47× slower than pure Python**. Calling it once over the whole array made it **13× faster**.

This is the single most important FFI design principle: expose *bulk* operations, not scalar ones.

`process_array(ptr, n)`, not `process_one(x)` in a loop.

Python binding options ranked

Approach	Speed	Type safety	Needs a compiler
ctypes	slowest	none (runtime)	✗
cffi ABI	~2× ctypes	parses C decls	✗
cffi API	~4× ctypes	build-time	☑
Cython	fastest	build-time	☑
pybind11 / nanobind	fast	build-time	☑ (C++)
Hand-written CPython C API	fastest	manual	☑
NumPy buffer protocol	zero-copy for arrays	—	—

For numeric work, support the buffer protocol / NumPy arrays — that gives you a raw pointer to contiguous data with no copy at all.

⚠ **Release the GIL** around long C calls (`Py_BEGIN_ALLOW_THREADS`, or `ffi.def_extern` with the right flags), or you serialize the whole interpreter.

30.4 The other languages

Rust

```
// bindgen generates Rust declarations FROM your C header
// build.rs:
bindgen::Builder::default().header("mathlib.h").generate()?;

// Then wrap it safely:
pub struct Stats(*mut ml_stats);
impl Stats {
    pub fn new() -> Option<Self> {
        let p = unsafe { ml_stats_create() };
        if p.is_null() { None } else { Some(Stats(p)) }
    }
    pub fn add(&mut self, v: f64) -> Result<(), Error> {
        match unsafe { ml_stats_add(self.0, v) } { 0 => Ok(()), e => Err(Error(e)) }
    }
}
impl Drop for Stats { fn drop(&mut self) { unsafe { ml_stats_destroy(self.0) } } }
```

- **bindgen**: C header → Rust FFI declarations
- **cbindgen**: Rust `#[no_mangle] extern "C"` functions → a C header (the other direction)
- Everything crossing the boundary is **unsafe**; the idiom is a thin **unsafe** layer wrapped in a safe API with **Drop**.
- ⚠ **Rust panics must not unwind into C** — use `catch_unwind` in every `extern "C"` function, or `panic = "abort"`.

Go (cgo)

```
/*
#cgo LDFLAGS: -L. -lmathlib -lm
#include "mathlib.h"
*/
import "C"
import "unsafe"

func Sum(data []int32) int64 {
    return int64(C.ml_sum((*C.int32_t)(unsafe.Pointer(&data[0])),
C.int32_t(len(data))))
}
```

- ⚠ **cgo calls are expensive** (~50–200 ns) — the goroutine must move to a system thread.
- ⚠ **Go pointers may not be stored in C memory** across a call (the GC moves and tracks them). Pass C-allocated memory, or pin.
- Go callbacks into C need `//export` and a separate file.
- **purego** is a newer alternative that avoids cgo entirely.

Java (JNI / Panama)

```
public class MathLib {
    static { System.loadLibrary("mathlibjni"); }
    public static native int add(int a, int b);
}
```

```
JNIEXPORT jint JNICALL Java_MathLib_add(JNIEnv *env, jclass cls, jint a, jint b)
{ return ml_add(a, b); }
```

- JNI is verbose, error-prone, and requires a hand-written shim per function.
- ⚠ Every `Get*ArrayElements` needs a matching `Release*`, or you leak or pin the heap.
- **Project Panama** (`java.lang.foreign`, **JDK 22+**) **replaces JNI** — no C shim at all, `jextract` generates bindings from headers. Use it for new work.

Node.js

- **N-API / node-addon-api** — ABI-stable across Node versions (unlike the old V8 API). The right choice for native addons.
- **ffi-napi** — runtime binding like ctypes; convenient, slow.
- **WebAssembly** — often the best option: no native build per platform, and it runs in browsers too.

Lua

The cleanest C API of any scripting language — a stack-based interface designed for embedding:

```
static int l_add(lua_State *L) {
    lua_pushinteger(L, ml_add((int32_t)luaL_checkinteger(L,1),
(int32_t)luaL_checkinteger(L,2)));
    return 1;                                /* number of return values */
}
```

LuaJIT's FFI is remarkable — it parses C declarations at runtime and JIT-compiles the calls, often to zero overhead.

30.5 WebAssembly

Emscripten — C to the browser

```
emcc mathlib.c -O3 -o mathlib.js \
-
sEXPORTED_FUNCTIONS=['["_ml_add","_ml_sum","_ml_stats_create","_malloc","_free"]' \
-sEXPORTED_RUNTIME_METHODS=['ccall","cwrap","getValue","setValue","HEAP32"]' \
-sMODULARIZE=1
```

```
const Module = await createModule();
const add = Module.cwrap('ml_add', 'number', ['number','number']);
console.log(add(20, 22));                                // 42

// arrays: allocate in the WASM heap, get a pointer
const n = 5;
const ptr = Module._malloc(n * 4);
Module.HEAP32.set([1,2,3,4,5], ptr >> 2);
const sum = Module._ml_sum(ptr, n);
Module._free(ptr);
```

Key points:

- Exported symbols get a leading underscore.
- **Pointers are offsets into a single `ArrayBuffer`** (the WASM linear memory) — JS reads it through typed-array views (`HEAP8`, `HEAP32`, `HEAPF64`).
- Emscripten emulates a POSIX-ish environment: a virtual filesystem, `emscripten_set_main_loop` instead of a blocking loop, pthreads via Web Workers + `SharedArrayBuffer`.
- ⚠ **No blocking, no infinite loops** — the browser's event loop must run. Use `ASYNCIFY` if you must block.

WASI — WebAssembly outside the browser

```
clang --target=wasm32-wasi -O2 --sysroot=$WASI_SYSROOT prog.c -o prog.wasm
wasmtime prog.wasm
```

A standardized syscall interface for WASM: files, clocks, random, args, env. **Capability-based security** — a module gets access only to what you explicitly grant (`--dir=.`). Runtimes: wasmtime, wasmer, WasmEdge, wazero.

Why WASM matters for C: one binary runs everywhere, sandboxed by construction (a buffer overflow can't escape the linear memory), no per-platform native builds, and near-native speed.

⚠ **WASM is 32-bit by default** (`wasm32`): `sizeof(void*) == 4`, `size_t` is 32-bit. Code assuming 64-bit pointers breaks. And `long` is 32-bit — Module 26's data-model problem, again.

30.6 Designing C APIs for FFI

Every design decision in §30.1 exists for a reason.

The rules

Rule	Why bindings need it
Fixed-width types only (<code>int32_t</code> , <code>int64_t</code> , <code>double</code>)	<code>long</code> is 32-bit on Win64 and WASM (Module 26); binding generators guess wrong
No <code>size_t</code> in the ABI where you can avoid it	Its width varies; <code>int32_t/int64_t</code> is unambiguous
No unions, bitfields, or packed structs	Layout is implementation-defined (Module 11); no binding generator handles them reliably
No varargs	<code>printf</code> -style functions are nearly unbindable — every language needs a shim
No function-like macros in the API	The preprocessor doesn't exist in the binding language. Provide real functions.
Opaque handles, not exposed structs	Layout can change without breaking bindings; no struct marshalling
Always pass array lengths	The binding must know how much to marshal (and it's a security requirement, Module 23)
Caller-supplied buffers	Avoids allocator mismatch — the caller's <code>free</code> may not match your <code>malloc</code>
If you must return allocated memory, provide the matching free	<code>ml_free_string()</code> , never "call <code>free()</code> "
Status return + out-parameters	Every language can express "int return, pointer out-param"
Callbacks always take <code>void *userdata</code>	The only way to attach a closure (Module 6/28)

Rule	Why bindings need it
No exceptions/panics/longjmp across the boundary	C frames don't unwind
Thread safety documented per function	GIL, goroutine scheduling, and JVM threading all interact
extern "C" guard + export macro	C++ consumers and Windows DLLs
ml_version() + a version macro	Runtime compatibility checking

The shape that binds cleanly from every language

```
/* Every language can express all four of these */
int32_t  fn(int32_t a, double b);           /* scalars           */
int32_t  fn(const uint8_t *data, int32_t len); /* array + length    */
int32_t  fn(Handle *h, int32_t *out_value); /* status + out-param*/
int32_t  fn(Handle *h, Callback cb, void *userdata); /* callback + ctx    */
```

Bulk operations, not scalar ones

The §30.3 measurement is the argument. **Design for one call over N items, not N calls over one item.**

```
/* BAD for FFI: N boundary crossings */
double transform_one(double x);

/* GOOD: one crossing */
int32_t transform_array(const double *in, double *out, int32_t n);
```

A binding-friendly checklist

```
/* Ship all of these: */
mathlib.h           /* extern "C" guarded, fixed-width, no macros in the API */
mathlib.pc          /* pkg-config (Module 20) */
libmathlib.so.1.4.2 /* versioned soname (Module 20) */
ml_version() / ML_VERSION_MAJOR /* runtime + compile-time version */
```

Plus: document ownership, lifetime, thread safety, and every error code (Module 28) — **binding authors cannot read your implementation.**

Module 30 rules

#	Rule
1	#ifdef __cplusplus extern "C" { in every public header

#	Rule
2	Never let an exception, panic, or <code>longjmp</code> cross into C
3	Fixed-width types only in the ABI — no <code>long</code> , no bitfields, no unions, no varargs
4	Opaque handles; expose no struct layouts
5	Always pass array lengths
6	Caller-supplied buffers, or ship the matching free function
7	Status return + out-parameters is the universal FFI shape
8	Every callback takes <code>void *userdata</code> — that's how closures cross
9	Use the trampoline pattern for capturing closures
10	ctypes: declare <code>argtypes</code> AND <code>restype</code>, always
11	Keep callback objects alive for the duration of the C call
12	Batch across the boundary — measured 47× slower per-element, 13× faster batched
13	Design bulk operations, not scalar ones
14	Release the GIL / yield the scheduler around long C calls
15	Go pointers may not be stored in C memory; Rust <code>extern "C"</code> needs <code>catch_unwind</code>
16	WASM is 32-bit — check your pointer and <code>long</code> assumptions
17	Ship a <code>.pc</code> file, a versioned soname, and a runtime version function

Practice

1. **Build the library and bind it four ways.** Reproduce §30.1's `mathlib`, then bind it from C++ (with a RAI wrapper), ctypes, cffi ABI, and cffi API. Verify all four produce identical results under ASan.
2. **Break ctypes deliberately.** Omit `restype` on `ml_hypot` (returns `double`) and on `ml_stats_create` (returns a pointer). Observe the garbage and the eventual crash. Then add the declarations and confirm the fix.
3. **Measure your own boundary cost.** Reproduce the §30.3 benchmark on your machine. Add Cython and a hand-written CPython extension. Then find the array size at which the batched C call beats pure Python.
4. **Fix a chatty API.** Take an API with a scalar operation called in a loop. Add a bulk variant. Measure the speedup from Python. Report the number of boundary crossings before and after.
5. **Callbacks in both directions.** Extend the library with a function that takes a callback and calls it 1M times. Measure the cost from ctypes, cffi, and C++. Then invert it: have the library *store* a callback and invoke it later — and work out how each language keeps the callback object alive.
6. **C++ exception safety.** Write a C API that takes a callback, and call it from C++ with a callback that throws. Observe the resource leak (or `std::terminate`). Then wrap the callback body in

`try/catch(...)` returning an error code, and verify with LSan that nothing leaks.

- 7. **Name mangling.** Compile the same header from C and C++ without `extern "C"`. Compare the symbols with `nm` and `c++filt`. Then add the guard and confirm they match.
- 8. **Rust binding** (if you can install Rust). Use `bindgen` to generate FFI declarations for `mathlib.h`, write a safe wrapper with `Drop`, and add `catch_unwind` to an `extern "C"` callback. Test that a panic in the callback doesn't unwind into C.
- 9. **WASM** (if you can install Emscripten or a WASI SDK). Compile `mathlib.c` to WASM, call `ml_sum` from JavaScript with a typed array, and verify the result. Then check `sizeof(void*)` and `sizeof(long)` inside WASM and find any code that breaks.
- 10. **API audit for bindability.** Take a public C header (yours, or `zlib.h`, `sqlite3.h`, `libcurl`). Score every function against the §30.6 table. Identify which ones would be painful to bind and redesign the three worst.

Otherwise, next is **Module 31 — Professional Practice**: Git workflows for C (including `git bisect`, which is the single most underused debugging tool), coding standards and `clang-format`, Doxygen and architecture documentation, release engineering and semantic versioning, licensing implications for C libraries, and how to read and contribute to a large unfamiliar C codebase.

31. Professional Practice

- Git: branching, rebasing, bisect (`git bisect` is gold for C bugs), code review
- Coding standards: Linux kernel style, GNU, Google, MISRA; `clang-format`, `.editorconfig`
- Documentation: Doxygen, README, man pages, architecture docs
- Semantic versioning, changelogs, release engineering
- Licensing: GPL/LGPL/MIT/BSD/Apache and the implications for C libraries
- Reading and contributing to large C codebases
- Debugging production issues, logging, telemetry, crash reporting

Module 31 — Professional Practice

The skills that make you employable rather than merely capable. Real git 2.43 / clang-format 22 / GCC 13.3 output throughout.

31.1 Git — and `git bisect` especially

The workflow

Concept	Practice
Branching	Short-lived feature branches off <code>main</code> . Trunk-based development beats long-lived branches for C — merge conflicts in headers are painful.
Rebase vs merge	Rebase your <i>local</i> work to keep history linear; never rebase a branch others have pulled . <code>git pull --rebase</code> as the default.

Concept	Practice
Commit hygiene	One logical change per commit. It must build and pass tests. This is what makes bisect work.
Commit messages	50-char summary, blank line, body explaining why (the diff already shows <i>what</i>). Reference the issue. Conventional Commits (feat: , fix: , perf:) enables automated changelogs.
Code review	Small PRs (< 400 lines). Review for correctness, not style — clang-format handles style.

⚠ **git bisect** — the highest-leverage debugging tool in C

A real experiment: a ring buffer with a regression buried 4 commits deep among 8.

```
$ git log --oneline
bebf5bf feat: add ring_space()
b3b76a0 feat: add ring_clear()
6d744a8 docs: expand README
2a36088 feat: add ring_cap()
a62be7e perf: replace modulo with a mask in the hot path
cdca2e1 feat: add ring_full()
16f9040 feat: add ring_empty()
a59ada0 docs: add README
a00c6c2 feat: initial ring buffer

$ make -s check
FAIL test.c:11 v == (uint8_t)(cycle*n+i)
```

The test script — **the only thing you write:**

```
#!/bin/sh
# Exit 0 = good, 1 = bad, 125 = SKIP (this commit cannot build)
make -s clean >/dev/null 2>&1
make -s test >/dev/null 2>&1 || exit 125
./test >/dev/null 2>&1
```

```
$ git bisect start HEAD a00c6c2
$ git bisect run ./bisect.sh

Bisecting: 3 revisions left to test after this (roughly 2 steps)
[a62be7e] perf: replace modulo with a mask in the hot path
running './bisect.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
[16f9040] feat: add ring_empty()
running './bisect.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[cdca2e1] feat: add ring_full()
```

```
running './bisect.sh'

a62be7e9320820d1ada0eb954d1a8838bdbb9ed4 is the first bad commit
perf: replace modulo with a mask in the hot path
```

The culprit:

```
-   r->tail = (r->tail + 1) % r->cap;
+   r->tail = (r->tail + 1) & (r->cap - 1);
```

A "performance optimization" that's only valid when `cap` is a power of two. **Three automated tests found it exactly**, with no debugger and no reading of code.

Why it scales

```
8 commits -> 3 tests
100 commits -> 7 tests
1000 commits -> 10 tests
10000 commits -> 14 tests
100000 commits -> 17 tests
```

Binary search: $O(\log n)$. A regression introduced 10,000 commits ago is found in 14 automated builds.

Bisect in practice

```
git bisect start
git bisect bad           # current is broken
git bisect good v1.2.0   # this release worked
git bisect run ./test.sh # fully automatic
git bisect reset

git bisect skip          # this commit doesn't build (or use exit 125)
git bisect log > bisect.log # save the session
git bisect replay bisect.log # resume it
git bisect start -- src/parser.c # restrict to paths that touched one file
```

For C specifically: make the test script a **sanitizer build** (`-fsanitize=address,undefined`) and it will bisect memory bugs that don't even crash the plain build. That combination — bisect + sanitizers — finds regressions that no amount of reading will.

Requirements: every commit must build and be testable. That's the practical argument for commit hygiene.

The other tools

```
git log -S"ring_push" --oneline      # commits that ADDED or REMOVED this string
("pickaxe")
git log -L :ring_push:ring.c        # the full history of ONE function
git blame -w -C -C ring.c           # ignore whitespace, follow code moved
between files
git diff --word-diff                # better for prose and dense C
git worktree add ../bugfix          # a second checkout – no stashing
git rerere                          # remember conflict resolutions across
rebases
```

`git blame` answers "who wrote this?"; `git log -S` answers "when did this change?" — usually the more useful question.

31.2 Coding standards

The major C styles

Style	Indent	Braces	Notes
Linux kernel	8-wide tabs	K&R (function brace on its own line)	8 columns is deliberate: "if you need more than 3 levels of indentation, you're screwed anyway"
GNU	2 spaces, braces indented	Own line, indented	<code>func (a, b)</code> with a space. Distinctive, widely disliked outside GNU.
Google	2 spaces	K&R	<code>int* p</code> pointer binding
BSD/Allman	4/8	Braces on own lines	
MISRA	—	Mandates braces, single exit, no recursion	Module 27; it's a <i>safety</i> standard, not a formatting one

The same file under four styles:

```
--- LLVM ---
static int cmp(const void *a) {
    const Rec *x = a;
    ...
}

--- Google ---
static int cmp(const void* a) {
    ...
}

--- GNU ---
static int
cmp (const void *a)
{
    const Rec *x = a;
}

--- Mozilla ---
static int
cmp(const void* a)
{
```

Note `int *p` vs `int* p` — Module 9 explained why the C convention is `int *p` (the `*` binds to the declarator, not the type). Google's C++ style says otherwise; both exist.

`.clang-format` — kernel style

```
BasedOnStyle: LLVM
IndentWidth: 8
UseTab: Always
TabWidth: 8
BreakBeforeBraces: Linux      # functions on a new line, everything else K&R
AllowShortIfStatementsOnASingleLine: false
AllowShortFunctionsOnASingleLine: None
IndentCaseLabels: false
ColumnLimit: 100
PointerAlignment: Right      # `int *p` (Module 9)
SortIncludes: false          # C include ORDER often matters (Module 26
                              # FTMs!)
AlignConsecutiveMacros: true
```

```
static int cmp(const void *a, const void *b)
{
    const Rec *x = a, *y = b;
    return (x->score > y->score) - (x->score < y->score);
}
int process(Rec *recs, size_t n, int flags)
{
    if (!recs || n == 0)
        return -1;
```

⚠ **SortIncludes: false matters in C.** Feature test macros must precede every include (Module 26), and some headers must come in a specific order. Auto-sorting silently breaks builds.

Enforce it in CI

```
$ clang-format --dry-run --Werror messy.c
messy.c:2:9: error: code should be clang-formatted [-Wclang-format-violations]
#include<stdlib.h>
    ^
(exit code 1 = would fail the build)
```

Plus a pre-commit hook, and `git-clang-format` to format only what you changed.

The point of a formatter is to end style discussion. Any consistent style beats an inconsistent one. Pick one, commit `.clang-format`, and never review whitespace again.

⚠️ **Apply it in one "reformat everything" commit** and add that commit's hash to `.git-blame-ignore-revs` so `git blame` skips it.

`.editorconfig` — what every editor understands

```
root = true

[*]
end_of_line = lf
insert_final_newline = true
trim_trailing_whitespace = true

[*.{c,h}]
indent_style = tab
indent_size = 8
max_line_length = 100

[{Makefile,*.mk}]
indent_style = tab      # Make REQUIRES real tabs (Module 20)
```

Universally supported, no plugin needed in most editors, and it prevents the "my editor converted all your tabs to spaces" PR.



31.3 Documentation

The four kinds, and who reads them

Document	Audience	Answers
README	Someone deciding whether to use it	What is it, how do I build it, show me 10 lines that work
API reference (Doxygen)	Someone writing code against it	What does this function do, what are the error returns
Architecture doc	Someone changing it	Why is it structured this way, what were the alternatives
man page	Someone running the tool	Synopsis, options, exit codes, files, examples

README structure that works

```
# projectname
One sentence: what it does and for whom.

## Quick start
git clone ... && cd ... && make && ./example
[10 lines of code that actually work]
```

```
## Installation      # package managers, then from source
## Documentation    # link to the full reference
## Requirements     # C standard, compilers, platforms, dependencies
## Contributing     # link to CONTRIBUTING.md
## License          # SPDX identifier + link
```

The single most common README failure is not showing working code in the first screen.

Doxygen

```
/**
 * @brief Render the widget onto a canvas.
 *
 * @param[in] w      Widget. Must be non-NULL and initialized.
 * @param[in] canvas Not modified. Must outlive this call.
 * @param[out] bounds On success, receives the bounds. Untouched on failure.
 *
 * @return ::ERR_OK on success.
 * @retval ERR_INVALID A required argument was NULL.
 * @retval ERR_NOMEM Allocation failed.
 *
 * @note Thread safety: NOT thread-safe. Callers must serialize access to @p w.
 * @warning Must not be called from a widget callback.
 * @see widget_measure()
 *
 * @code
 * Rect r;
 * if (widget_render(w, canvas, &r) == ERR_OK) { ... }
 * @endcode
 */
[[nodiscard]] err_t widget_render(Widget *w, const Canvas *canvas, Rect *bounds);
```

The four things every public function must document (Module 28): ownership, lifetime, thread safety, and every error return. **Binding authors and callers cannot read your implementation.**

Doxyfile essentials: `EXTRACT_ALL=NO` (so undocumented functions are visible as gaps), `WARN_IF_UNDOCUMENTED=YES`, `WARN_AS_ERROR=YES` in CI. Alternatives: Sphinx+Breathe, Natural Docs, or just clean headers with comments.

Architecture documentation

ADRs (Architecture Decision Records) — one short markdown file per significant decision:

```
# ADR-0007: Use an arena allocator for per-request memory

## Status
Accepted (2026-03-14)

## Context
```



```
Profiling showed malloc/free at 18% of request-handling time...

## Decision
Per-request arena, reset at the end of each request.

## Consequences
+ 14x faster allocation (measured)
+ Zero leak risk within a request
- Cannot free individual objects
- ASan cannot instrument arena allocations without manual poisoning
```

This is the documentation people actually miss. Six months later nobody remembers *why*, and someone "fixes" it back. `docs/adr/` costs 20 minutes per decision and saves days.

Also worth having: a `docs/ARCHITECTURE.md` with a module diagram and the dependency direction (Module 28), and `CONTRIBUTING.md` with build, test, and review expectations.

man pages

```
.TH MYTOOL 1 "2026-07-29" "mytool 1.4.2"
.SH NAME
mytool \- process widget files
.SH SYNOPSIS
.B mytool
[\fB\ -v \fR] [\fB\ -o \fR \fI FILE \fR] \fI INPUT \fR...
.SH EXIT STATUS
.TP
0 Success
.TP
1 Invalid arguments
```

Write them in Markdown and convert with **pandoc** or **scdoc** — raw roff is miserable. And **document exit codes** (Module 1) — scripts depend on them.

31.4 Versioning and release engineering

Semantic versioning for C

`MAJOR.MINOR.PATCH`:

Bump	When
MAJOR	ABI break or API removal (Module 20 measured what an ABI break does)
MINOR	New API, backward compatible
PATCH	Bug fixes only

For C libraries the ABI is the contract, not the API. Bump the soname on any MAJOR (Module 20):

```
libfoo.so      -> libfoo.so.2      (dev symlink)
libfoo.so.2    -> libfoo.so.2.1.0  (SONAME – what binaries record)
```

Verify with **abidiff** (libabigail) in CI — it compares two builds and reports ABI differences automatically.

Expose the version three ways

```
#define FOO_VERSION_MAJOR 1
#define FOO_VERSION_MINOR 4
#define FOO_VERSION_PATCH 2
#define FOO_VERSION_NUM ((1*10000) + (4*100) + 2) /* for #if comparisons */
#define FOO_VERSION_STR "1.4.2"

const char *foo_version(void); /* RUNTIME – may differ from compile time! */
int         foo_version_num(void);
```

Compile-time and runtime versions can differ — that's the whole point of shared libraries. Check both.

CHANGELOG

Follow keepachangelog.com: **Added** / **Changed** / **Deprecated** / **Removed** / **Fixed** / **Security**, newest first, with an **[Unreleased]** section.

```
## [1.4.2] - 2026-07-29
### Fixed
- Ring buffer corruption with non-power-of-two capacities (#142)
### Security
- Integer overflow in `foo_alloc_array()` could undersize the allocation (CVE-2026-XXXX)
```

Write it for users, not from git log. "Fixed ring buffer corruption" is useful; "perf: replace modulo with a mask" is not.

Conventional Commits + [git-cliff/semantic-release](https://github.com/git-cliff/semantic-release) can generate the skeleton, but a human must edit it.

Release checklist

```
[ ] CHANGELOG updated, version bumped everywhere (header, build files, .pc, spec)
[ ] Full CI green: all compilers, all platforms, all sanitizers, Valgrind, static analysis
[ ] Fuzz corpus replayed clean (Module 22)
[ ] abidiff clean, or the soname bumped
[ ] Documentation regenerated; examples still compile
[ ] Reproducible build verified (no __DATE__/__TIME__ – Module 12)
```

```
[ ] Signed tag: git tag -s v1.4.2 -m "Release 1.4.2"
[ ] Release tarball + signature + SHA256SUMS
[ ] Debug symbols archived (Module 19's split debug info)
```

Archive the unstripped binary or .debug file with every release — without it, production crash addresses are undecodable (§31.7).

31.5 Licensing

I'm not a lawyer, and this is not legal advice. License choice has real legal consequences — consult counsel for anything that matters commercially. Here is the factual landscape as it's generally understood.

License	Type	Effect on your code
MIT	Permissive	Use freely; keep the copyright notice
BSD-2/3-Clause	Permissive	Same; 3-clause adds a no-endorsement clause
Apache-2.0	Permissive	Same + an explicit patent grant and a patent-retaliation clause
ISC	Permissive	MIT-equivalent, shorter
LGPL-2.1/3.0	Weak copyleft	Changes <i>to the library</i> must be shared; dynamic linking generally lets your application stay proprietary
GPL-2.0/3.0	Strong copyleft	Linking generally makes the whole work GPL when distributed
AGPL-3.0	Network copyleft	GPL + triggers on network use , not just distribution
MPL-2.0	File-level copyleft	Modified <i>files</i> stay MPL; the rest of your project doesn't
Public domain / CC0 / Unlicense	None	⚠ Not valid in every jurisdiction; some companies ban it

What actually matters for C

Static linking is the sharp edge. With GPL/LGPL, statically linking generally incorporates the library into your binary in a way that triggers the license's conditions. LGPL specifically contemplates dynamic linking as the way to keep your application separate; static LGPL linking imposes additional obligations (typically providing object files so users can relink).

This is why:

- glibc is **LGPL**, so proprietary Linux programs can link it dynamically.
- The Linux kernel is **GPL-2.0** with an explicit syscall exception, so userspace programs aren't derived works.
- musl is **MIT**, which is why it's popular for static linking and containers.

- Many C libraries dual-license (GPL for open source, commercial for proprietary) — Qt's historical model.

Practical guidance:

- **A library you want widely adopted:** MIT, BSD, or Apache-2.0. Apache-2.0 if patents are a concern.
- **A library you want to stay open:** LGPL (dynamic linking) or GPL.
- **An application:** anything you like.
- **Corporate use:** most companies have a list of approved licenses. **Check before adding a dependency,** not after.

Compliance hygiene

```
// SPDX-License-Identifier: Apache-2.0
// Copyright (c) 2026 Example Corp
```

SPDX identifiers are machine-readable and now standard practice (the Linux kernel uses them throughout). Add them to every file.

Also: ship the full **LICENSE** file; maintain a **NOTICE/THIRD_PARTY_LICENSES** file listing dependencies; scan with **FOSSA**, **ScanCode**, **licensecheck**, or GitHub's dependency graph; and **⚠ check transitive dependencies** — a permissive library that vendors a GPL component carries the GPL obligation to you.

31.6 Reading a large C codebase

You will spend far more time reading C than writing it. A repeatable method:

1. Orient before reading

```
cloc .                # size and language breakdown
ls -la               # README, CONTRIBUTING, docs/,
ARCHITECTURE.md
cat README.md CONTRIBUTING.md docs/*.md
git log --oneline | wc -l  # project age and activity
git shortlog -sn | head   # who to ask
find . -name '*.h' | head -30 # the API surface lives in headers
```

Read the headers first. They're the contract. A 200,000-line project might have 3,000 lines of public header — that's the part you must understand.

2. Follow the entry point

```
grep -rn "int main" --include=*.c
```

Trace **main** → initialization → the main loop. Don't read breadth-first; **follow one execution path all the way through**.

3. Build it and instrument it

```
make                                # or cmake -B build && cmake --build
build
gdb -q -batch -ex 'break main' -ex run -ex bt ./prog
strace -c ./prog                    # what does it ACTUALLY do? (Module 21)
ltrace ./prog
```

Running it under a debugger and a tracer teaches you more in 10 minutes than an hour of reading.

4. Navigate mechanically

```
ctags -R .                          # then Ctrl-] in vim
cscope -Rbq                         # "who calls this?" – the killer feature
bear -- make                         # generate compile_commands.json
# then clangd in your editor: real go-to-definition, find-references, rename
```

Set up `compile_commands.json` and `clangd` first. Grep-based navigation in C is unreliable because of macros and the preprocessor; `clangd` understands the actual translation units.

5. Read history, not just code

```
git log --oneline -20 -- src/parser.c # what's been changing here?
git log -S"suspicious_function"      # when did this appear?
git log -L :parse_header:parser.c    # this function's whole history
git blame -w -C src/parser.c
```

The commit that introduced a strange line usually explains it.

6. Change something small

Fix a typo. Add a test. Improve an error message. Submitting one trivial PR teaches you the build system, the test suite, the review process, and the maintainers' expectations — all the things documentation doesn't tell you.

Codebases worth reading

Project	What it teaches
SQLite	Testing discipline (100% MC/DC branch coverage), a single-file amalgamation build
redis	Clean modern C, excellent data structures, readable event loop

Project	What it teaches
curl	Portability across everything, protocol implementation
musl libc	Minimal, standards-exact, extremely readable stdlib
Lua	Small, elegant, embeddable; the best C API design in existence
nginx	Event-driven architecture, memory pools
Linux kernel	Subsystem design, intrusive data structures, the goto cleanup pattern at scale
FFmpeg	Performance work, SIMD
stb single-header libs	API design under extreme constraints
zstd / lz4	Optimization craft

31.7 Production debugging, logging, telemetry, crash reporting

In production you can't attach a debugger. Everything must be built in beforehand.

Structured logging

```
void log_emit(LogLevel lvl, const char *file, int line, const char *func,
               const char *fmt, ...) __attribute__((format(printf, 5, 6)));

#define LOG_AT(lvl, ...) log_emit((lvl), __FILE__, __LINE__, __func__,
__VA_ARGS__)
#define LOGI(...) LOG_AT(LOG_INFO, __VA_ARGS__)
```

Human-readable:

```
2026-07-29T16:16:56.349Z INFO [-] app.c:52 main(): service starting version=1.4.2
pid=937
2026-07-29T16:16:56.349Z DEBUG [req-a1b2c3] app.c:36 handle_request(): request
received size=512
2026-07-29T16:16:56.349Z WARN [req-d4e5f6] app.c:38 handle_request(): negative
size, rejecting
2026-07-29T16:16:56.349Z ERROR [req-g7h8i9] app.c:39 handle_request(): size 99999
exceeds limit 1024
```

JSON for aggregation (Loki, ELK, Datadog):

```
{ "ts": "2026-07-29T16:16:56.352Z", "level": "DEBUG", "trace": "req-a1b2c3",
  "src": "app.c:36", "fn": "handle_request", "msg": "request received size=512" }
```

Every technique from earlier modules is in there:

Element	Module
<code>__FILE__, __LINE__, __func__</code>	12
<code>__attribute__((format(printf,...)))</code>	6 — catches format bugs at compile time
<code>vsnprintf</code> into a bounded buffer	8
<code>_Thread_local</code> trace ID	3, 15 — like <code>errno</code> , per-thread
<code>gmtime_r</code> , not <code>gmtime</code>	14 — thread safety
<code>setvbuf(stderr, _IOLBF)</code>	13 — so logs survive a crash
Early return when filtered	Near-zero cost for disabled levels

Logging rules:

- 1. **Levels, and honor them.** `LOGD` in a hot loop must cost ~one comparison.
- 2. **Structured, not prose.** `size=99999 limit=1024`, not "the size was too big."
- 3. **Correlation IDs.** A trace ID threaded through every log line is what makes distributed debugging possible.
- 4. **Never log secrets.** Passwords, tokens, keys, PII — logs are the most commonly leaked artifact.
- 5. **stderr for logs, stdout for data** (Module 13).
- 6. **Bounded.** Rate-limit, rotate, and cap. A log-flooding bug takes down the disk.
- 7. **Line-buffered or unbuffered** so the last lines before a crash survive.

Metrics

```
--- metrics ---
requests_total           3
bytes_processed          512
requests_rejected        1
requests_failed           1
```

Counters are the cheapest telemetry that exists — an increment, and you can answer "how often does this happen in production?" Export them over an HTTP endpoint (Prometheus format) or push them periodically.

The standard types: **counters** (monotonic), **gauges** (current value), **histograms** (latency distributions — always report **percentiles**, never averages).

Crash reporting

```
static void crash_handler(int sig)
{
    void *fr[32];
    int n = backtrace(fr, 32);
    /* async-signal-safe only: write(), _exit() (Module 24) */
```

```

    write(STDERR_FILENO, "\n*** FATAL SIGNAL - backtrace ***\n", 34);
    backtrace_symbols_fd(fr, n, STDERR_FILENO);
    _exit(128 + sig);
}

static void install_crash_handlers(void)
{
    static char altstack[64*1024];
    stack_t ss = { .ss_sp = altstack, .ss_size = sizeof altstack };
    sigaltstack(&ss, NULL); /* survive a STACK OVERFLOW */
    struct sigaction sa = {0};
    sa.sa_handler = crash_handler;
    sa.sa_flags = SA_ONSTACK | SA_RESETHAND;
    sigaction(SIGSEGV,&sa,NULL); sigaction(SIGBUS,&sa,NULL);
    sigaction(SIGFPE,&sa,NULL);  sigaction(SIGILL,&sa,NULL);
    sigaction(SIGABRT,&sa,NULL);
}

```

```

*** FATAL SIGNAL - backtrace ***
./app(+0x195d)[0x5555b7f8095d]
/lib/x86_64-linux-gnu/libc.so.6(+0x45330)[0x7fdf3e445330]
./app(main+0x11c)[0x5555b7f80d62]

```

And decoding it (Module 19):

```

$ addr2line -e app -f -C -i 0x195d
crash_handler
/home/claude/c_prof/obs/app.c:14

```

Three details that make this work:

- **SA_ONSTACK with an alternate stack** — otherwise a stack overflow can't run the handler.
- **backtrace_symbols_fd, not backtrace_symbols** — the latter calls `malloc` and is not async-signal-safe.
- **-rdynamic** at build time, so function names appear.

⚠ With PIE, subtract the load base (from `/proc/self/maps`) before `addr2line`.

For real production: Google **Breakpad/Crashpad**, **Sentry**, or **systemd-coredump** + `coredumpctl`. Plus: enable core dumps (`RLIMIT_CORE`), archive the split debug file with every release (Module 19), and have a symbol server.

The production debugging kit

Situation	Tool
Process is hung	<code>gdb -p PID -batch -ex 'thread apply all bt'</code>

Situation	Tool
What is it doing?	<code>strace -p PID -c</code> , <code>perf top -p PID</code>
Memory growing?	<code>/proc/PID/status</code> , <code>pmap</code> , <code>heaptrack</code> , <code>massif</code>
Need data from a live process	bpftrace — attach with near-zero overhead, no restart
Crash after the fact	Core dump + <code>addr2line</code> + the archived debug file
Non-deterministic	<code>rr</code> record/replay (Module 21)

Build in observability before you need it. A production incident is not the time to discover you have no logs, no metrics, and no symbols.

Module 31 rules

#	Rule
1	Every commit builds and passes tests — that's what makes bisect work
2	<code>git bisect run</code> with a sanitizer build finds regressions in $O(\log n)$
3	<code>git log -S</code> answers "when did this change?" better than blame
4	One <code>.clang-format</code> , enforced in CI; <code>SortIncludes: false</code> in C
5	Ignore the reformat commit in <code>.git-blame-ignore-revs</code>
6	README shows working code in the first screen
7	Document ownership, lifetime, thread safety, and every error return
8	Write ADRs — the <i>why</i> is what gets lost
9	For C libraries the ABI is the contract; bump the soname, verify with <code>abidiff</code>
10	Expose the version at compile time and runtime
11	Static linking is the sharp edge of GPL/LGPL — check licenses before adding a dependency
12	Add SPDX identifiers to every file
13	Read headers first, follow one path, set up clangd, then change something small
14	Logs: leveled, structured, correlated, bounded, on <code>stderr</code> , never containing secrets
15	<code>backtrace_symbols_fd</code> + <code>SA_ONSTACK</code> + <code>-rdynamic</code> for crash handlers
16	Archive the debug file with every release — without it, crashes are undecodable
17	Build in observability before you need it

Practice

1. **Bisect a real regression.** Reproduce §31.1: create a repo with 10 commits, one containing a subtle bug, and find it with `git bisect run`. Then make the test script a sanitizer build and plant a memory bug that doesn't crash — confirm bisect still finds it.
2. **Bisect an actual project.** Clone a large open-source C project, pick a behavior, and bisect to find when it was introduced. Time yourself. Then try to find it by reading.
3. **Formatting rollout.** Take an inconsistently formatted project. Write a `.clang-format`, apply it in one commit, add that hash to `.git-blame-ignore-revs`, add a CI check, and add a pre-commit hook. Verify `git blame` still shows the real authors.
4. **Document a module.** Take a header you've written. Add complete Doxygen for every public function covering the four required elements. Generate the HTML. Then hand the header (only) to someone and see if they can use the API correctly.
5. **Write three ADRs.** For three design decisions in a project you've built, write ADRs with Status/Context/Decision/Consequences. Include the alternatives you rejected and why.
6. **Release a library.** Take something you've built and do a full release: version macros + runtime function, soname, `.pc` file, CHANGELOG, signed tag, tarball with SHA256SUMS, split debug file. Then make an ABI-breaking change, verify `abidiff` catches it, and do a MAJOR release.
7. **License audit.** For a project with 5+ dependencies, list every direct and transitive dependency with its license. Identify anything that would prevent proprietary distribution. Add SPDX headers and a `THIRD_PARTY_LICENSES` file.
8. **Read an unfamiliar codebase.** Pick one from the §31.6 table. Spend two hours: `cloc`, read the headers, find `main`, trace one execution path in GDB, run it under `strace`, set up `compile_commands.json` + `clangd`, then write a one-page summary of the architecture. Finally, submit one small PR.
9. **Build the observability layer.** Implement the §31.7 logger with levels, JSON output, thread-local trace IDs, and the `format` attribute. Add counters, gauges, and a latency histogram with p50/p95/p99. Verify with TSan that it's thread-safe.
10. **Crash-to-line pipeline.** Build a program with a crash handler, split debug info (Module 19), and PIE. Crash it, capture the backtrace, and write a script that computes the load base from `/proc/PID/maps` and pipes the addresses to `addr2line`. Confirm you get exact file:line from a stripped binary.
11. **Production drill.** Take a running program and, using only `gdb -p`, `strace -p`, `perf top -p`, and `/proc`, determine without stopping it: which functions are hot, what syscalls it's making, how much memory it's using, and where each thread is blocked.

That completes **Part 2 — the industry toolchain (Modules 19–28, 30, 31)**.

Remaining from the original map:

- **Module 29 — Data Structures & Algorithms in C** (you skipped it)
- **Module 32 — Study These Real Codebases**, a guided reading plan: what to look at in each, in what order, and what specific technique each one will teach you

Say which you'd like, or ask for anything you want to go deeper on.

